

The benchmark inventory

86 cases: titles, categories, kinds, status, sources, tags, Datalog and Whiel encodings

Generated by Benchmark/report/build_report.py

Contents

1	Overview	2
2	Tag vocabulary and index	7
3	Example0001: Transitive closure by naive iteration under the weak transitivity specification	10
4	Example0013: Transitive closure by naive iteration under a false specification ($T \subseteq E$)	11
5	Example0106: Prefix-equivalence: naive tc_left computes lfp of tc_nonlinear	12
6	Example0107: Prefix-equivalence: naive tc_nonlinear computes lfp of tc_left	13
7	Example0108: Prefix-equivalence: naive tc_right computes lfp of tc_left	15
8	Example0109: Containment: red-blue alternating paths $\subseteq TC(R \cup B)$	16
9	Example0115: Prefix-equivalence: naive all_paths_via_even_odd (linear) computes lfp of tc_left	18
10	Example0117: Prefix-equivalence: naive all_paths_via_even_odd (non-linear) computes lfp of tc_left	19
11	Example0119: Prefix-equivalence: naive all_paths_via_even_odd (linear) computes lfp of tc_nonlinear	21
12	Example0123: Prefix-equivalence: naive even_odd_nonlinear computes lfp of even_odd_linear	22
13	Example0125: Prefix-equivalence: naive even_odd_linear computes lfp of even_odd_nonlinear	24
14	Example0128: Containment: naive red_blue_nonlinear outputs $\subseteq$ pre-fixpoint of $TC(R \cup B)$	26
15	Example0130: Containment: even/odd/ambiguous-depth path-system program $\subseteq$ every set closed under the basic path-system rules	27
16	Example0132: Odd and even paths: linear versus non-linear program (two compiled loops)	29

17 Example0133: All paths via odd/even paths: linear versus non-linear program (two compiled loops)	31
18 Example0134: Red/blue alternating paths: linear versus non-linear program (two compiled loops)	33
19 Example0136: Right-linear transitive closure versus the non-linear odd/even path program (two compiled loops)	35
20 Example0137: Non-linear transitive closure versus the linear odd/even path program (two compiled loops)	37
21 Example0138: Joint-loop equivalence: <code>tc_nonlinear</code> vs <code>all_paths_via_even_odd</code> (non-linear, naive)	38
22 Example0162: Two-player game reachability mixing existential and universal successor steps	40
23 Example0163: Greatest-fixpoint safe set: shrinking iteration bounded below by <code>Rb</code>	41
24 Example1030: Foreign-key support core for sharded materialized rows	42
25 Example1061: Equivalence of computed same-generation with <code>lfp</code> of the same-generation rules (pre-fixpoint)	43
26 Example1081: Audited delegation contained in broad authorization closure	45
27 Example1088: Equivalent forward dependency closure and transposed reverse resolution	47
28 Example1090: Equivalent taint reachability with ternary modes and projected frontiers	49
29 Example1091: Delegation reachability bounded by a transitively closed policy view	51
30 Example1093: Lineage triples contained in a closed ternary cover	53
31 Example1094: Routing table contained in an administratively closed distance view	55
32 Example2004: Magic-set transformation: demanded answers equal the original program's answers (Soufflé)	57
33 Example2005: Restricted-edge reachability is contained in full-edge reachability (edge-monotonicity)	59
34 Example2006: Policy-filtered connectivity is contained in raw topological connectivity	60
35 Example2007: $E[f \cup g]$ is contained in $EF\ g$	61
36 Example2010: Semi-naive evaluation $\equiv$ naive/declarative transitive closure	62
37 Example2011: Incremental view maintenance $\equiv$ from-scratch recomputation	64
38 Example2012: Frontier (grey-set) marking $\equiv$ naive mark propagation	65

39 Example3001: Audited delegation alternation loses an uncertified reviewed hop	66
40 Example3009: Certified delegation closure exceeds audit-bound authorization	68
41 Example3014: Auditor trust diverges from resource trust under recursive elevation	70
42 Example4001: Directed triangle query versus its path-view reformulation	72
43 Example4002: Transitive closure versus its length-two path-view reformulation	73
44 Example4003: Containment of the path-view reformulation in transitive closure	75
45 Example4004: Path-view reformulation versus the even-length path query	76
46 Example4034: Containment of a reformulation over sound odd/even views in transitive closure	78
47 Example4035: Good-interior path query versus its begin-good and end-good edge-view reformulation	80
48 Example4036: Airline reachability query versus its three-view reformulation, forward containment	81
49 Example4037: Airline reachability query versus its three-view reformulation, reverse containment	83
50 Example4040: Closure of red and blue edges versus its reformulation over Datalog path views	84
51 Example4041: Transitive closure versus its reformulation over exact odd/even views	86
52 Example5001: PostgreSQL recursive UNION versus naive seeded closure	88
53 Example5002: Datafrog semi-naive delta evaluation versus naive evaluation	89
54 Example5003: Worklist breadth-first reachability versus naive evaluation	90
55 Example5004: Materialized role-hierarchy closure with a join versus a recursive has-role query	92
56 Example5005: Incremental maintenance of a reachability view versus recomputation from scratch	94
57 Example5006: Semi-naive closure with an appended delta versus a prepended delta	95
58 Example5007: Transitive closure by repeated squaring versus linear closure	97
59 Example5008: Soufflé tc benchmark: naive versus semi-naive evaluation	98
60 Example5009: Soufflé family benchmark: naive versus semi-naive evaluation	100
61 Example5010: Soufflé orbits benchmark: naive versus semi-naive evaluation	102
62 Example5011: Soufflé topological_ordering benchmark: naive versus semi-naive evaluation	103

63 Example5012: Casbin depth-bounded role resolution contained in the unbounded role closure	105
64 Example5013: Casbin depth-bounded role resolution versus the unbounded role closure	107
65 Example5014: W3C SPARQL 1.1 ALP evaluation of p^* and p^+ versus the Datalog closure with reflexive pairs over nodes(G)	108
66 Example5015: W3C SPARQL 1.1 ALP evaluation of p^* versus closure plus identity over the p endpoints (invalid twin of Example5014)	110
67 Example5016: DRed deletion maintenance of a materialised transitive closure versus recomputation	111
68 Example5017: Overdeletion without rederivation versus recomputation (invalid twin of Example5016)	113
69 Example5018: Chase order-independence on the full-TGD subset of ChaseBench correctness/tgds: dependency-at-a-time versus parallel chase	115
70 Example5019: One pass over the dependencies versus the parallel chase on ChaseBench correctness/tgds (invalid twin of Example5018)	117
71 Example5020: RDFS closure by the rho-df rules versus the full W3C RDFS rule set: equal up to the reflexive subPropertyOf/subClassOf triples	118
72 Example5021: rho-df versus full RDFS with the reflexive subClassOf triples forgotten (invalid twin of Example5020)	121
73 Example5022: SQL recursion limits (MySQL <code>cte_max_recursion_depth</code> , SQL Server <code>MAXRECURSION</code>): completion within the limit implies the closure	123
74 Example5023: Win-move: the alternating fixpoint of the well-founded semantics (nested loops) versus the attractor computation	124
75 Example5024: Win-move: alternating fixpoint versus attractor, claiming the odd limit is the win set (invalid twin of Example5023)	126
76 Example5030: NetworkX per-source transitive closure versus PostgreSQL recursive-union rounds	128
77 Example5031: NetworkX per-source search with cutoff 2 versus PostgreSQL recursive-union rounds (invalid twin of Example5030)	130
78 Example5032: Delta-driven incremental maintenance of a transitive closure under insertions versus PostgreSQL recomputation on the updated edges	131
79 Example5033: Soufflé magic-set demand-driven evaluation versus PostgreSQL bulk evaluation restricted to the demanded sources	133
80 Example5034: Warshall's intermediate-vertex sweep (boolean Floyd-Warshall) versus PostgreSQL recursive-union rounds	135

-
- 81 **Example5035:** Warshall's sweep truncated to the first two vertices versus PostgreSQL recursive-union rounds (invalid twin of Example5034) 137
- 82 **Example5036:** NetworkX DAG transitive closure by a reverse topological sweep versus PostgreSQL recursive-union rounds 139
- 83 **Example5037:** Go runtime frontier marking versus the materialised Soufflé closure of the pointer graph joined with the roots 141
- 84 **Example5038:** OpenFGA Check by top-down userset dispatch versus OpenFGA ListObjects by reverse expansion from the user 143
- 85 **Example5039:** Permission exclusion applied at every object (SpiceDB/OpenFGA) is contained in the exclusion applied once to the finished closure 145
- 86 **Example5040:** Permission exclusion applied at every object versus once to the finished closure, claimed equal (invalid twin of Example5039) 147
- 87 **Example5041:** PostgreSQL foreign-key cascade as a trigger worklist with the surviving-reference RESTRICT check versus the declarative delete closure 149
- 88 **Example5042:** The over-approximating RESTRICT check of legacy Example2024 versus PostgreSQL's surviving-reference check, claimed equal (invalid twin of Example5041) 151

1 Overview

The folder **Benchmark** holds 86 cases. Each case is one section below with its title, category, kind, status, sources, tags, the Datalog program(s) it encodes and the Whiel Hoare triple as written in **Input.lean**. The tag vocabulary is in Section 2.

Status. The status of a case is read off its folder and off nothing else: a case is certified only when the certificate emitter wrote the certificate that stands beside its input, and solved when the frozen record a search writes is there but the certificate built from it is not. A solved case has an answer and a record of it, not a kernel-checked proof. What the metadata expects of a case is reported next to the status, never in place of it, so an expectation can never be mistaken for a checked result.

Status	Cases	Read off
certified valid	69	Certificate/Valid.lean, written by the certificate emitter
certified invalid	17	Certificate/Invalid.lean, written by the certificate emitter
solved valid, not yet certified	0	Core.json, the frozen invariant record; no certificate built from it yet
solved invalid, not yet certified	0	Counterexample.json, the frozen counterexample record; no certificate built from it yet
open	0	no certificate and no frozen record in the case folder

of which outside the library build: 10 (counted as certified above; the kernel check runs on its own, under a raised memory limit, and each such case says so in its own section).

Status against expectation. certified invalid / expected invalid: 17; certified valid / expected valid: 68; certified valid / expected valid (expected; strengthened claim): 1.

Categories. llm: 11; production: 7; production-pair: 22; souffle-benchmark: 4; specification: 11; synthetic: 21; verification-task: 10.

Kinds. conditional claim: 1; containment: 18; correctness: 2; equivalence: 64; implication: 1.

Case	Title	Category	Kind	Status	Tags
Example0001	Transitive closure by naive iteration under the weak transitivity specification	synthetic	correctness	certified valid	prophecy
Example0013	Transitive closure by naive iteration under a false specification ($T \subseteq E$)	synthetic	correctness	certified invalid	
Example0106	Prefix-equivalence: naive tc_left computes lfp of tc_nonlinear	synthetic	equivalence	certified valid	fixpoint-witness linear-vs-nonlinear nonlinear-recursion prophecy
Example0107	Prefix-equivalence: naive tc_nonlinear computes lfp of tc_left	synthetic	equivalence	certified valid	fixpoint-witness linear-vs-nonlinear nonlinear-recursion
Example0108	Prefix-equivalence: naive tc_right computes lfp of tc_left	synthetic	equivalence	certified valid	fixpoint-witness linear-vs-nonlinear
Example0109	Containment: red-blue alternating paths $\subseteq TC(R \cup B)$	synthetic	containment	certified valid	fixpoint-witness mutual-recursion
Example0115	Prefix-equivalence: naive all_paths_via_even_odd computes lfp of tc_left (linear)	synthetic	equivalence	certified valid	fixpoint-witness mutual-recursion

Case	Title	Category	Kind	Status	Tags
Example0117	Prefix-equivalence: naive all_paths_via_even_odd (non-linear) computes lfp of tc_left	synthetic	equivalence	certified valid	fixpoint-witness linear-vs-nonlinear mutual-recursion nonlinear-recursion
Example0119	Prefix-equivalence: naive all_paths_via_even_odd (linear) computes lfp of tc_nonlinear	synthetic	equivalence	certified valid	fixpoint-witness linear-vs-nonlinear mutual-recursion nonlinear-recursion
Example0123	Prefix-equivalence: naive even_odd_nonlinear computes lfp of even_odd_linear	synthetic	equivalence	certified valid	prophecy fixpoint-witness linear-vs-nonlinear mutual-recursion nonlinear-recursion
Example0125	Prefix-equivalence: naive even_odd_linear computes lfp of even_odd_nonlinear	synthetic	equivalence	certified valid	fixpoint-witness linear-vs-nonlinear mutual-recursion nonlinear-recursion prophecy
Example0128	Containment: naive red_blue_nonlinear outputs $\subseteq$ pre- fixpoint of TC(R $\cup$ B)	synthetic	containment	certified valid	fixpoint-witness mutual-recursion nonlinear-recursion
Example0130	Containment: even/odd/ambiguous- depth path-system program $\subseteq$ every set closed under the basic path-system rules	synthetic	containment	certified valid	fixpoint-witness mutual-recursion nonlinear-recursion
Example0132	Odd and even paths: linear versus non- linear program (two compiled loops)	synthetic	equivalence	certified valid	linear-vs-nonlinear mutual-recursion nonlinear-recursion prophecy
Example0133	All paths via odd/even paths: linear versus non-linear program (two com- piled loops)	synthetic	equivalence	certified valid	linear-vs-nonlinear mutual-recursion nonlinear-recursion prophecy
Example0134	Red/blue alternating paths: linear ver- sus non-linear program (two compiled loops)	synthetic	equivalence	certified valid	linear-vs-nonlinear mutual-recursion nonlinear-recursion prophecy
Example0136	Right-linear transitive closure versus the non-linear odd/even path program (two compiled loops)	synthetic	equivalence	certified valid	linear-vs-nonlinear mutual-recursion nonlinear-recursion prophecy
Example0137	Non-linear transitive closure versus the linear odd/even path program (two compiled loops)	synthetic	equivalence	certified valid	linear-vs-nonlinear mutual-recursion nonlinear-recursion prophecy
Example0138	Joint-loop equivalence: tc_nonlinear vs all_paths_via_even_odd (non-linear, naive)	synthetic	equivalence	certified valid	mutual-recursion nonlinear-recursion prophecy
Example0162	Two-player game reachability mixing existential and universal successor steps	synthetic	equivalence	certified valid	fixpoint-witness negation
Example0163	Greatest-fixpoint safe set: shrinking it- eration bounded below by Rb	synthetic	equivalence	certified valid	fixpoint-witness greatest-fixpoint negation

Case	Title	Category	Kind	Status	Tags
Example1030	Foreign-key support core for sharded materialized rows	llm	equivalence	certified valid	fixpoint-witness greatest-fixpoint integrity-constraints negation
Example1061	Equivalence of computed same-generation with lfp of the same-generation rules (pre-fixpoint)	llm	equivalence	certified valid	fixpoint-witness
Example1081	Audited delegation contained in broad authorization closure	llm	containment	certified valid	dependencies mutual-recursion nonlinear-recursion prophecy
Example1088	Equivalent forward dependency closure and transposed reverse resolution	llm	equivalence	certified valid	dependencies mutual-recursion nonlinear-recursion
Example1090	Equivalent taint reachability with ternary modes and projected frontiers	llm	equivalence	certified valid	dependencies mutual-recursion nonlinear-recursion
Example1091	Delegation reachability bounded by a transitively closed policy view	llm	containment	certified valid	dependencies mutual-recursion nonlinear-recursion
Example1093	Lineage triples contained in a closed ternary cover	llm	containment	certified valid	dependencies mutual-recursion nonlinear-recursion
Example1094	Routing table contained in an administratively closed distance view	llm	containment	certified valid	dependencies mutual-recursion nonlinear-recursion
Example2004	Magic-set transformation: demanded answers equal the original program's answers (Soufflé)	production	equivalence	certified valid	database-engine magic-sets prophecy
Example2005	Restricted-edge reachability is contained in full-edge reachability (edge-monotonicity)	production	containment	certified valid	fixpoint-witness negation
Example2006	Policy-filtered connectivity is contained in raw topological connectivity	production	containment	certified valid	fixpoint-witness negation
Example2007	$E[f \cup g]$ is contained in $EF \ g$	production	containment	certified valid	fixpoint-witness semi-naive
Example2010	Semi-naive evaluation $\equiv$ naive/declarative transitive closure	production	equivalence	certified valid	database-engine evaluation-strategies fixpoint-witness semi-naive
Example2011	Incremental view maintenance $\equiv$ from-scratch recomputation	production	equivalence	certified valid	evaluation-strategies fixpoint-witness view-maintenance
Example2012	Frontier (grey-set) marking $\equiv$ naive mark propagation	production	equivalence	certified valid	evaluation-strategies fixpoint-witness semi-naive
Example3001	Audited delegation alternation loses an uncertified reviewed hop	llm	containment	certified invalid	dependencies mutual-recursion nonlinear-recursion
Example3009	Certified delegation closure exceeds audit-bound authorization	llm	containment	certified invalid	dependencies mutual-recursion nonlinear-recursion
Example3014	Auditor trust diverges from resource trust under recursive elevation	llm	containment	certified invalid	dependencies mutual-recursion nonlinear-recursion
Example4001	Directed triangle query versus its path-view reformulation	verification-task	equivalence	certified valid	views

Case	Title	Category	Kind	Status	Tags
Example4002	Transitive closure versus its length-two path-view reformulation	verification-task	equivalence	certified invalid	in-views
Example4003	Containment of the path-view reformulation in transitive closure	verification-task	containment	certified valid	prophecy views
Example4004	Path-view reformulation versus the even-length path query	verification-task	equivalence	certified valid	mutual-recursion views prophecy
Example4034	Containment of a reformulation over sound odd/even views in transitive closure	verification-task	conditional claim	certified valid	mutual-recursion views prophecy
Example4035	Good-interior path query versus its begin-good and end-good edge-view reformulation	verification-task	equivalence	certified valid	prophecy views
Example4036	Airline reachability query versus its three-view reformulation, forward containment	verification-task	containment	certified valid	prophecy views
Example4037	Airline reachability query versus its three-view reformulation, reverse containment	verification-task	containment	certified invalid	in-views
Example4040	Closure of red and blue edges versus its reformulation over Datalog path views	verification-task	equivalence	certified valid	linear-vs-nonlinear nonlinear-recursion prophecy views
Example4041	Transitive closure versus its reformulation over exact odd/even views	verification-task	equivalence	certified valid	mutual-recursion views prophecy
Example5001	PostgreSQL recursive UNION versus naive seeded closure	production-pair	equivalence	certified valid	database-engine evaluation-strategies prophecy semi-naive
Example5002	Datafrog semi-naive delta evaluation versus naive evaluation	production-pair	equivalence	certified valid	database-engine evaluation-strategies prophecy semi-naive
Example5003	Worklist breadth-first reachability versus naive evaluation	production-pair	equivalence	certified valid	evaluation-strategies prophecy semi-naive
Example5004	Materialized role-hierarchy closure with a join versus a recursive has-role query	production-pair	equivalence	certified valid	access-control evaluation-strategies prophecy
Example5005	Incremental maintenance of a reachability view versus recomputation from scratch	production-pair	equivalence	certified valid	evaluation-strategies prophecy view-maintenance
Example5006	Semi-naive closure with an appended delta versus a prepended delta	production-pair	equivalence	certified valid	evaluation-strategies linear-vs-nonlinear prophecy semi-naive
Example5007	Transitive closure by repeated squaring versus linear closure	production-pair	equivalence	certified valid	evaluation-strategies linear-vs-nonlinear nonlinear-recursion prophecy
Example5008	Soufflé tc benchmark: naive versus semi-naive evaluation	souffle-benchmark	equivalence	certified valid	database-engine evaluation-strategies nonlinear-recursion prophecy semi-naive
Example5009	Soufflé family benchmark: naive versus semi-naive evaluation	souffle-benchmark	equivalence	certified valid	database-engine evaluation-strategies negation prophecy semi-naive

Case	Title	Category	Kind	Status	Tags
Example5010	Soufflé orbits benchmark: naive versus semi-naive evaluation	souffle-benchmark	equivalence	certified valid	database-engine evaluation-strategies negation nonlinear-recursion prophecy semi-naive
Example5011	Soufflé topological_ordering benchmark: naive versus semi-naive evaluation	souffle-benchmark	equivalence	certified invalid	database-engine evaluation-strategies negation nonlinear-recursion semi-naive
Example5012	Casbin depth-bounded role resolution contained in the unbounded role closure	production-pair	containment	certified valid	access-control recursion-limit prophecy
Example5013	Casbin depth-bounded role resolution versus the unbounded role closure	production-pair	equivalence	certified invalid	access-control recursion-limit
Example5014	W3C SPARQL 1.1 ALP evaluation of p* and p+ versus the Datalog closure with reflexive pairs over nodes(G)	specification	equivalence	certified valid	evaluation-strategies semi-naive
Example5015	W3C SPARQL 1.1 ALP evaluation of p* versus closure plus identity over the p endpoints (invalid twin of Example5014)	specification	equivalence	certified invalid	evaluation-strategies semi-naive
Example5016	DRed deletion maintenance of a materialised transitive closure versus recomputation	specification	equivalence	certified valid	evaluation-strategies negation prophecy semi-naive view-maintenance
Example5017	Overdeletion without rederivation versus recomputation (invalid twin of Example5016)	specification	equivalence	certified invalid	evaluation-strategies negation semi-naive view-maintenance
Example5018	Chase order-independence on the full-TGD subset of ChaseBench correctness/tgds: dependency-at-a-time versus parallel chase	specification	equivalence	certified valid	chase evaluation-strategies mutual-recursion
Example5019	One pass over the dependencies versus the parallel chase on ChaseBench correctness/tgds (invalid twin of Example5018)	specification	equivalence	certified invalid	chase evaluation-strategies mutual-recursion
Example5020	RDFS closure by the rho-df rules versus the full W3C RDFS rule set: equal up to the reflexive subPropertyOf/subClassOf triples	specification	equivalence	certified valid	mutual-recursion nonlinear-recursion
Example5021	rho-df versus full RDFS with the reflexive subClassOf triples forgotten (invalid twin of Example5020)	specification	equivalence	certified invalid	mutual-recursion nonlinear-recursion
Example5022	SQL recursion limits (MySQL cte_max_recursion_depth, SQL Server MAXRECURSION): completion within the limit implies the closure	specification	implication	certified valid	database-engine recursion-limit prophecy
Example5023	Win-move: the alternating fixpoint of the well-founded semantics (nested loops) versus the attractor computation	specification	equivalence	certified valid	evaluation-strategies greatest-fixpoint negation prophecy
Example5024	Win-move: alternating fixpoint versus attractor, claiming the odd limit is the win set (invalid twin of Example5023)	specification	equivalence	certified invalid	evaluation-strategies greatest-fixpoint negation

Case	Title	Category	Kind	Status	Tags
Example5030	NetworkX per-source transitive closure versus PostgreSQL recursive-union rounds	production-pair	equivalence	certified valid	database-engine evaluation-strategies prophecy semi-naive
Example5031	NetworkX per-source search with cut-off 2 versus PostgreSQL recursive-union rounds (invalid twin of Example5030)	production-pair	equivalence	certified invalid	database-engine evaluation-strategies recursion-limit semi-naive
Example5032	Delta-driven incremental maintenance of a transitive closure under insertions versus PostgreSQL recomputation on the updated edges	production-pair	equivalence	certified valid	database-engine evaluation-strategies prophecy semi-naive view-maintenance
Example5033	Soufflé magic-set demand-driven evaluation versus PostgreSQL bulk evaluation restricted to the demanded sources	production-pair	equivalence	certified valid	database-engine evaluation-strategies magic-sets prophecy semi-naive
Example5034	Warshall’s intermediate-vertex sweep (boolean Floyd-Warshall) versus PostgreSQL recursive-union rounds	production-pair	equivalence	certified valid	database-engine evaluation-strategies prophecy semi-naive
Example5035	Warshall’s sweep truncated to the first two vertices versus PostgreSQL recursive-union rounds (invalid twin of Example5034)	production-pair	equivalence	certified invalid	database-engine evaluation-strategies recursion-limit semi-naive
Example5036	NetworkX DAG transitive closure by a reverse topological sweep versus PostgreSQL recursive-union rounds	production-pair	equivalence	certified valid	database-engine evaluation-strategies prophecy semi-naive
Example5037	Go runtime frontier marking versus the materialised Soufflé closure of the pointer graph joined with the roots	production-pair	equivalence	certified valid	evaluation-strategies nonlinear-recursion prophecy semi-naive
Example5038	OpenFGA Check by top-down user set dispatch versus OpenFGA ListObjects by reverse expansion from the user	production-pair	equivalence	certified valid	access-control evaluation-strategies magic-sets prophecy
Example5039	Permission exclusion applied at every object (SpiceDB/OpenFGA) is contained in the exclusion applied once to the finished closure	production-pair	containment	certified valid	access-control negation
Example5040	Permission exclusion applied at every object versus once to the finished closure, claimed equal (invalid twin of Example5039)	production-pair	equivalence	certified invalid	access-control negation
Example5041	PostgreSQL foreign-key cascade as a trigger worklist with the surviving-reference RESTRICT check versus the declarative delete closure	production-pair	equivalence	certified valid	database-engine evaluation-strategies integrity-constraints negation prophecy semi-naive
Example5042	The over-approximating RESTRICT check of legacy Example2024 versus PostgreSQL’s surviving-reference check, claimed equal (invalid twin of Example5041)	production-pair	equivalence	certified invalid	database-engine evaluation-strategies integrity-constraints negation semi-naive

2 Tag vocabulary and index

A short vocabulary of what makes a program interesting to database people. The three recursion tags are computed from the Datalog rules and the Whiel text by the build script; the others are assigned by hand in **Benchmark/**

report/curated.json; the vocabulary lives in Benchmark/report/tags.json. Selecting cases by several tags at once is supported by the HTML page and the spreadsheet next to this report.

rules

nonlinear-recursion (31) A recursive rule joins two recursive relations ($T(x,y) :- T(x,z), T(z,y)$), or a hand-written loop composes a relation with itself (squaring): recursion beyond the linear recursion that SQL recursive CTEs support.

0106, 0107, 0117, 0119, 0123, 0125, 0128, 0130, 0132, 0133, 0134, 0136, 0137, 0138, 1081, 1088, 1090, 1091, 1093, 1094, 3001, 3009, 3014, 4040, 5007, 5008, 5010, 5011, 5020, 5021, 5037

mutual-recursion (30) Two or more relations defined in terms of each other (odd/even paths, RDFS classes and properties), which SQL recursive CTEs do not express directly.

0109, 0115, 0117, 0119, 0123, 0125, 0128, 0130, 0132, 0133, 0134, 0136, 0137, 0138, 1081, 1088, 1090, 1091, 1093, 1094, 3001, 3009, 3014, 4004, 4034, 4041, 5018, 5019, 5020, 5021

negation (16) Negation with semantic content: negated body atoms (stratified, or unstratified as in win-move), set difference in the claim, or a program whose meaning is an exclusion or a deletion (policy exclusion, edge filtering, overdeletion, a surviving-reference check). The set differences of frontier bookkeeping ($\Delta \setminus \text{Acc}$, a cursor over an order) do not count.

0162, 0163, 1030, 2005, 2006, 5009, 5010, 5011, 5016, 5017, 5023, 5024, 5039, 5040, 5041, 5042

query

linear-vs-nonlinear (15) Two formulations of one recursive query compared: linear versus nonlinear ($T \circ T$), or left- versus right-linear, the classic rewriting question for recursive SQL.

0106, 0107, 0108, 0117, 0119, 0123, 0125, 0132, 0133, 0134, 0136, 0137, 4040, 5006, 5007

views (10) Query reformulation over views (answering queries using views): LAV views, exact views stated as precondition equations, sound views stated as an implication in the claim, Datalog-defined views. A view that only links two schemas inside a claim does not count.

4001, 4002, 4003, 4004, 4034, 4035, 4036, 4037, 4040, 4041

magic-sets (3) Magic-set / demand transformation of a recursive query.

2004, 5033, 5038

dependencies (9) Two programs compared under tuple-generating dependencies between their input relations, stated as the precondition, including existential ones (a witness must exist): containment or equivalence under constraints, the setting of the chase.

1081, 1088, 1090, 1091, 1093, 1094, 3001, 3009, 3014

fixpoint-witness (21) The claim is stated through a free witness relation, never assigned by the command and occurring in the postcondition, that ranges over all pre-fixpoints (or post-fixpoints) of a rule set (RBound, Ra, Closed, ...): the executed program is compared with the least (or greatest) fixpoint of the rules, the equ.pdf method, instead of with a second executed program.

0106, 0107, 0108, 0109, 0115, 0117, 0119, 0123, 0125, 0128, 0130, 0162, 0163, 1030, 1061, 2005, 2006, 2007, 2010, 2011, 2012

evaluation

evaluation-strategies (33) Two evaluation algorithms of one recursive query are compared, or a non-naive evaluation algorithm is verified against the declarative meaning of its rules: naive versus semi-naive, worklist/BFS, per-source search, squaring, Warshall or topological sweeps, delta-driven or recomputing maintenance, chase orders, top-down versus bottom-up, materialize-then-join versus recursive plan. A naive iteration verified against the least fixpoint of its own rules does not count.

2010, 2011, 2012, 5001, 5002, 5003, 5004, 5005, 5006, 5007, 5008, 5009, 5010, 5011, 5014, 5015, 5016, 5017, 5018, 5019, 5023, 5024, 5030, 5031, 5032, 5033, 5034, 5035, 5036, 5037, 5038, 5041, 5042

semi-naive (25) One side evaluates the recursion by the semi-naive (delta/frontier) technique of Datalog engines and SQL recursive CTE executors: only the facts new in the last round are joined and the loop stops on an empty frontier, whatever the source calls it (PostgreSQL working table, Datafrog delta, GC grey set, BFS worklist, trigger queue, SPARQL visited set, DRed rounds). Computed from the command.

2007, 2010, 2012, 5001, 5002, 5003, 5006, 5008, 5009, 5010, 5011, 5014, 5015, 5016, 5017, 5030, 5031, 5032, 5033, 5034, 5035, 5036, 5037, 5041, 5042

view-maintenance (5) Incremental maintenance of a materialized recursive view (insertions, deletions with DRed, delta-driven) against recomputation from scratch.

2011, 5005, 5016, 5017, 5032

recursion-limit (5) One side truncates the recursion: SQL recursion limits (MySQL `cte_max_recursion_depth`, SQL Server `MAXRECURSION`), a depth bound (Casbin `maxHierarchyLevel`), a search cutoff.

5012, 5013, 5022, 5031, 5035

semantics

chase (2) The chase with tuple-generating dependencies (data exchange, ChaseBench scenarios).

5018, 5019

greatest-fixpoint (4) The meaning is a greatest fixpoint or an alternating fixpoint: safe sets, integrity cores, the well-founded semantics of win-move.

0163, 1030, 5023, 5024

integrity-constraints (3) Foreign keys, ON DELETE CASCADE / RESTRICT, referential-integrity cores.

1030, 5041, 5042

systems

database-engine (18) The program is a production database or Datalog engine's own mechanism, read from its source or documentation: PostgreSQL recursive union and RI triggers, Soufflé's magic-set transformation and semi-naive translation, Datafrog, MySQL / SQL Server recursion limits. A system that is only the setting of the claim (the streaming engines behind an insert-only restart) or a paper algorithm (DRed) does not count.

2004, 2010, 5001, 5002, 5008, 5009, 5010, 5011, 5022, 5030, 5031, 5032, 5033, 5034, 5035, 5036, 5041, 5042

access-control (6) Relationship-based access control and role hierarchies as production systems compute them (Zanzibar-style OpenFGA/SpiceDB, Casbin RBAC): Datalog-like reachability in production authorization databases. The LLM-generated delegation stories carry dependencies instead.

5004, 5012, 5013, 5038, 5039, 5040

verification

prophecy (41) The one verification tag: known or expected to need a prophecy-level invariant, formerly 'obstructed'; the case page says whether this is established or only expected.

0001, 0106, 0119, 0125, 0132, 0133, 0134, 0136, 0137, 0138, 1081, 2004, 4003, 4004, 4034, 4035, 4036, 4040, 4041, 5001, 5002, 5003, 5004, 5005, 5006, 5007, 5008, 5009, 5010, 5012, 5016, 5022, 5023, 5030, 5032, 5033, 5034, 5036, 5037, 5038, 5041

3 Example0001: Transitive closure by naive iteration under the weak transitivity specification

Category	synthetic (syntactic generation from the classic Datalog programs); family: Core.
Kind	correctness (closure/safety conditions on one program).
Contributors	Jesse Comer, Fangzhu Shen.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	prophecy.
Input	Benchmark/Example0001/Input.lean (canonical notation).
Summary	Transitive closure by naive iteration, under the weak specification that its result is transitively closed.
Sources	legacy source record hard-lowerbound-spec of the retired classic-source collection
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).
Note	Carries the kernel-checked certificate for this triple. The classic Example0160 stated the same triple and was removed as a duplicate of this case.

Datalog program(s). *Datalog reading of the executed loop (written for this report; right-linear transitive closure, $S = E \cup E \circ T$, $T := S$) — hand-written reading (this report).*

```
T(x, y) :- E(x, y).
T(x, y) :- E(x, z), T(z, y).
```

Reading. The loop performs naive evaluation of the right-linear transitive-closure program $T(x,y):-E(x,y)$, $T(x,y):-E(x,z),T(z,y)$, iterating $S:=E \cup (E \circ T)$ until $S=T$, the classical way to compute reachability (transitive closure) in a directed graph E . The Hoare triple states only a weak postcondition: that the computed T is transitively closed ($T \circ T \subseteq T$), with no assertion pinning T from above, so it does not claim T is the least or exact transitive closure, only that it is closed under composition with itself.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\top$

The precondition is simply true, imposing no constraint on the input relations.

POST: $\forall x y. ((\exists z. T(x,z) \wedge T(z,y)) \rightarrow T(x,y))$

The postcondition asserts that the computed relation T is transitively closed, i.e., composing T with itself never leaves T : $(T \circ T) \subseteq T$.

Whiel encoding (Hoare triple).

```
SCHEMA
  E/2, T/2, S/2

PRE
  true

CMD
  T :=  $\emptyset$ ;
  S :=  $(E \cup (\pi[0, 3] (\sigma[\#1 = \#2] (E \times T))))$ ;
  WHILE (S  $\neq$  T) DO
    T := S;
    S :=  $(E \cup (\pi[0, 3] (\sigma[\#1 = \#2] (E \times T))))$ 
  END

POST
   $((\pi[0, 3] (\sigma[\#1 = \#2] (T \times T))) \subseteq T)$ 
```

4 Example0013: Transitive closure by naive iteration under a false specification ($T \subseteq E$)

Category	synthetic (syntactic generation from the classic Datalog programs); family: Core.
Kind	correctness (closure/safety conditions on one program).
Contributors	Jesse Comer, Fangzhu Shen.
Status	certified invalid (Certificate/Invalid.lean, written by the certificate emitter); expected invalid.
Tags	.
Input	Benchmark/Example0013/Input.lean (canonical notation).
Summary	Transitive closure by naive iteration, under a false specification that a two-edge instance refutes.
Sources	none recorded.
Note	The refutable twin of Example0001: the same schema, precondition and command under the false postcondition $T \subseteq E$, refuted by the two-edge instance $E = \{(0, 1), (1, 2)\}$.

Datalog program(s). *Datalog reading of the executed loop (written for this report; right-linear transitive closure, $S = E \cup E \circ T$, $T := S$) — hand-written reading (this report).*

```
T(x, y) :- E(x, y).
T(x, y) :- E(x, z), T(z, y).
```

Reading. The program and schema are identical to Example0001 — naive right-linear evaluation of transitive closure $T(x,y):-E(x,y), T(x,y):-E(x,z),T(z,y)$ — but the postcondition is replaced by the false claim that the computed closure adds nothing beyond the base edges, $T \subseteq E$. The Hoare triple is invalid: on the two-edge instance $E=\{(0,1),(1,2)\}$ the loop computes $T=\{(0,1),(1,2),(0,2)\}$, and the derived pair $(0,2)$ is not itself an edge, refuting $T \subseteq E$.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in Input.lean remains the authority.

PRE: $\top$

The precondition is simply true, imposing no constraint on the input relations.

POST: $\forall x y. (T(x,y) \rightarrow E(x,y))$

The postcondition claims the computed transitive closure T is contained in the base edge relation E , i.e., that transitive closure adds no new pairs.

Whiel encoding (Hoare triple).

```
SCHEMA
  E/2, T/2, S/2

PRE
  true

CMD
  T :=  $\emptyset$ ;
  S :=  $(E \cup (\pi[0, 3] (\sigma[\#1 = \#2] (E \times T))))$ ;
  WHILE (S  $\neq$  T) DO
    T := S;
    S :=  $(E \cup (\pi[0, 3] (\sigma[\#1 = \#2] (E \times T))))$ 
  END

POST
  (T  $\subseteq$  E)
```

5 Example0106: Prefix-equivalence: naive tc_left computes lfp of tc_nonlinear

Category	synthetic (syntactic generation from the classic Datalog programs); family: Synthetic.
Kind	equivalence (with a least fixpoint, via a pre-fixpoint witness).
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	fixpoint-witness linear-vs-nonlinear nonlinear-recursion prophecy.
Input	Benchmark/Example0106/Input.lean (canonical notation).
Summary	A left-linear closure loop, specified as the least fixpoint of the non-linear closure program.
Sources	retired-corpus source record synthetic_6 of the Synthetic family retired-corpus input record synthetic_6 of the Synthetic family classic Datalog program tc-left of the retired generator suite classic Datalog program tc-nonlinear of the retired generator suite
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).

Datalog program(s). *tc_left* — legacy inventory: source programs with one executed WHIEL program and one formal PRE/POST reference (not executed); sources: tc-left.dl, tc-nonlinear.dl.

```
-- tc_left
-- Executed WHIEL program
tc1(x, y) :- base(x, y).
tc1(x, y) :- tc1(x, z), base(z, y).
```

tc_nonlinear — legacy inventory: source programs with one executed WHIEL program and one formal PRE/POST reference (not executed); sources: tc-left.dl, tc-nonlinear.dl.

```
-- tc_nonlinear
-- Formal PRE/POST reference only; not executed by the WHIEL command
tc(x, y) :- base(x, y).
tc(x, y) :- tc(x, z), tc(z, y).
```

Reading. The executed loop naively evaluates the left-linear transitive-closure program $T(x,y) \text{ :- Base}(x,y), T(x,y) \text{ :- } T(x,z), \text{Base}(z,y)$, while the reference program against which it is measured is the ordinary non-linear transitive closure $Tc(x,y) \text{ :- Base}(x,y), Tc(x,y) \text{ :- } Tc(x,z), Tc(z,y)$. The Hoare triple is the standard least-fixpoint introduction obligation: given any relation $RBound$ that is already a pre-fixpoint of the non-linear closure program, the naively computed T must itself be a pre-fixpoint of that same program and lie below $RBound$, establishing that the left-linear loop computes the least pre-fixpoint (i.e. the transitive closure) of the non-linear program.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\forall x y. (\text{Base}(x,y) \vee (\exists z. RBound(x,z) \wedge RBound(z,y)) \rightarrow RBound(x,y))$

The precondition states that $RBound$ is a pre-fixpoint of the non-linear closure program: $\text{Base} \cup (RBound \circ RBound) \subseteq RBound$.

POST: $(\forall x y. (\text{Base}(x,y) \vee (\exists z. T(x,z) \wedge T(z,y)) \rightarrow T(x,y))) \wedge \forall u v. (T(u,v) \rightarrow RBound(u,v))$

The postcondition states that the computed T is itself a pre-fixpoint of the non-linear closure program, $\text{Base} \cup (T \circ T) \subseteq T$, and that $T \subseteq RBound$.

Whiel encoding (Hoare triple).

```

SCHEMA
  Base/2, T/2, T_aux/2, RBound/2

PRE
  ((Base  $\cup$  ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (RBound  $\times$  RBound))))
   $\subseteq$  RBound)

CMD
  T_aux :=  $\emptyset$ ;
  T := Base;
  WHILE T  $\neq$  T_aux DO
    T_aux := T;
    T :=
      (Base  $\cup$ 
       ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (T_aux  $\times$  Base))))
  END

POST
  (((Base  $\cup$  ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (T  $\times$  T))))  $\subseteq$  T)
   $\wedge$  (T  $\subseteq$  RBound))

```

6 Example0107: Prefix-equivalence: naive tc_nonlinear computes lfp of tc_left

Category	synthetic (syntactic generation from the classic Datalog programs); family: Synthetic.
Kind	equivalence (with a least fixpoint, via a pre-fixpoint witness).
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	<code>fixpoint-witness</code> <code>linear-vs-nonlinear</code> <code>nonlinear-recursion</code> .
Input	Benchmark/Example0107/Input.lean (canonical notation).
Summary	Canonical form of the earlier single-level encoding of this case: the same schema, precondition, command and postcondition; the snapshot relation T_2 is now T_aux.
Sources	retired-corpus source record synthetic_7 of the Synthetic family retired-corpus input record synthetic_7 of the Synthetic family classic Datalog program tc-left of the retired generator suite classic Datalog program tc-nonlinear of the retired generator suite

Datalog program(s). *tc_left* — legacy inventory: source programs with one executed WHIEL program and one formal PRE/POST reference (not executed); sources: tc-left.dl, tc-nonlinear.dl.

```

-- tc_left
-- Formal PRE/POST reference only; not executed by the WHIEL command
tcl(x, y) :- base(x, y).
tcl(x, y) :- tcl(x, z), base(z, y).

```

tc_nonlinear — legacy inventory: source programs with one executed WHIEL program and one formal PRE/POST reference (not executed); sources: tc-left.dl, tc-nonlinear.dl.

```

-- tc_nonlinear
-- Executed WHIEL program
tc(x, y) :- base(x, y).
tc(x, y) :- tc(x, z), tc(z, y).

```

Reading. The executed loop naively evaluates the non-linear transitive-closure program by squaring, $T := \text{Base} \cup (T \circ T)$, while the reference program is the left-linear closure $T(x,y) := \text{Base}(x,y)$, $T(x,y) := T(x,z), \text{Base}(z,y)$. The Hoare triple is the least-fixpoint introduction obligation for that reference: given any RBound already a pre-fixpoint of the left-linear program, the naively computed non-linear T must itself be a pre-fixpoint of the left-linear program and lie below RBound , i.e. the non-linear loop computes the least pre-fixpoint of the left-linear program, which is again the transitive closure of Base .

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in `Input.lean`. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\forall x y. (\text{Base}(x,y) \vee (\exists z. \text{RBound}(x,z) \wedge \text{Base}(z,y)) \rightarrow \text{RBound}(x,y))$

The precondition states that RBound is a pre-fixpoint of the left-linear closure program: $\text{Base} \cup (\text{RBound} \circ \text{Base}) \subseteq \text{RBound}$.

POST: $(\forall x y. (\text{Base}(x,y) \vee (\exists z. T(x,z) \wedge \text{Base}(z,y)) \rightarrow T(x,y))) \wedge \forall u v. (T(u,v) \rightarrow \text{RBound}(u,v))$

The postcondition states that the computed T is itself a pre-fixpoint of the left-linear closure program, $\text{Base} \cup (T \circ \text{Base}) \subseteq T$, and that $T \subseteq \text{RBound}$.

Whiel encoding (Hoare triple).

```

SCHEMA
  Base/2, T/2, T_aux/2, RBound/2

PRE
  ((Base ∪ (π[0, 3] (σ[#1 = #2] (RBound × Base)))) ⊆ RBound)

CMD
  T_aux := ∅;
  T := Base;
  WHILE T ≠ T_aux DO
    T_aux := T;
    T := (Base ∪ (π[0, 3] (σ[#1 = #2] (T_aux × T_aux))))
  END

POST
  (((Base ∪ (π[0, 3] (σ[#1 = #2] (T × Base)))) ⊆ T) ∧ (T ⊆ RBound))

```

7 Example0108: Prefix-equivalence: naive tc_right computes lfp of tc_left

Category	synthetic (syntactic generation from the classic Datalog programs); family: Synthetic.
Kind	equivalence (with a least fixpoint, via a pre-fixpoint witness).
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	fixpoint-witness linear-vs-nonlinear.
Input	Benchmark/Example0108/Input.lean (canonical notation).
Summary	Canonical form of the earlier single-level encoding of this case: the same schema, precondition, command and postcondition; the snapshot relation T_2 is now T_aux.
Sources	retired-corpus source record synthetic_8 of the Synthetic family retired-corpus input record synthetic_8 of the Synthetic family classic Datalog program tc-left of the retired generator suite classic Datalog program tc-right of the retired generator suite

Datalog program(s). *tc_left* — legacy inventory: source programs with one executed WHIEL program and one formal PRE/POST reference (not executed); sources: tc-left.dl, tc-right.dl.

```
-- tc_left
-- Formal PRE/POST reference only; not executed by the WHIEL command
tcl(x, y) :- base(x, y).
tcl(x, y) :- tcl(x, z), base(z, y).
```

tc_right — legacy inventory: source programs with one executed WHIEL program and one formal PRE/POST reference (not executed); sources: tc-left.dl, tc-right.dl.

```
-- tc_right
-- Executed WHIEL program
tcr(x, y) :- base(x, y).
tcr(x, y) :- base(x, z), tcr(z, y).
```

Reading. The executed loop naively evaluates the right-linear transitive-closure program $T(x,y) \text{ :- Base}(x,y), T(x,y) \text{ :- Base}(x,z), T(z,y)$, while the reference program is the left-linear closure $T(x,y) \text{ :- Base}(x,y), T(x,y) \text{ :- T}(x,z), \text{Base}(z,y)$. The Hoare triple is the least-fixpoint introduction obligation for the left-linear reference: given any RBound already a pre-fixpoint of the left-linear program, the naively computed right-linear T must itself be a pre-fixpoint of the left-linear program and lie below RBound, i.e. right-linear naive evaluation computes the least pre-fixpoint of the left-linear program (both compute the transitive closure of Base).

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in *Input.lean* remains the authority.

PRE: $\forall x y. (\text{Base}(x,y) \vee (\exists z. \text{RBound}(x,z) \wedge \text{Base}(z,y)) \rightarrow \text{RBound}(x,y))$

The precondition states that RBound is a pre-fixpoint of the left-linear closure program: $\text{Base} \cup (\text{RBound} \circ \text{Base}) \subseteq \text{RBound}$.

POST: $(\forall x y. (\text{Base}(x,y) \vee (\exists z. T(x,z) \wedge \text{Base}(z,y)) \rightarrow T(x,y))) \wedge \forall u v. (T(u,v) \rightarrow \text{RBound}(u,v))$

The postcondition states that the computed T is itself a pre-fixpoint of the left-linear closure program, $\text{Base} \cup (T \circ \text{Base}) \subseteq T$, and that $T \subseteq \text{RBound}$.

Whiel encoding (Hoare triple).

```

SCHEMA
  Base/2, T/2, T_aux/2, RBound/2

PRE
  ((Base  $\cup$  ( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (RBound  $\times$  Base))))  $\subseteq$  RBound)

CMD
  T_aux :=  $\emptyset$ ;
  T := Base;
  WHILE T  $\neq$  T_aux DO
    T_aux := T;
    T := (Base  $\cup$  ( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (Base  $\times$  T_aux))))
  END

POST
  (((Base  $\cup$  ( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (T  $\times$  Base))))  $\subseteq$  T)  $\wedge$  (T  $\subseteq$  RBound))

```

8 Example0109: Containment: red-blue alternating paths $\subseteq$ TC(R $\cup$ B)

Category	synthetic (syntactic generation from the classic Datalog programs); family: Synthetic.
Kind	containment (via a pre-fixpoint witness).
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	fixpoint-witness mutual-recursion.
Input	Benchmark/Example0109/Input.lean (canonical notation).
Summary	Canonical form of the earlier single-level encoding of this case: the same schema, precondition, command and postcondition; the snapshot relations RR_2 is now RR_aux, BB_2 is now BB_aux, RB_2 is now RB_aux, BR_2 is now BR_aux.
Sources	retired-corpus source record synthetic_9 of the Synthetic family retired-corpus input record synthetic_9 of the Synthetic family

Datalog program(s). *red_blue_linear* — legacy inventory: `catalog/source` intent plus actual left-recursive formal PRE/POST reference-operator reconstruction; sources: `red_blue_linear.dl`, `tc_of_union.dl`, `Benchmark/Example0109/Input.lean`.

```

-- red_blue_linear
-- Executed WHIEL program
RR(x,y) :- R(x,y).
BB(x,y) :- B(x,y).
RB(x,y) :- RR(x,z), B(z,y).
BR(x,y) :- BB(x,z), R(z,y).
RR(x,y) :- R(x,z), BR(z,y).
BB(x,y) :- B(x,z), RB(z,y).

```

tc_of_union — legacy inventory: `catalog/source` intent plus actual left-recursive formal PRE/POST reference-operator reconstruction; sources: `red_blue_linear.dl`, `tc_of_union.dl`, `Benchmark/Example0109/Input.lean`.

```

-- tc_of_union
-- Catalog/source target intent only; not executed by the WHIEL command
E(x,y) :- R(x,y).
E(x,y) :- B(x,y).
T(x,y) :- E(x,y).
T(x,y) :- E(x,z), T(z,y).
-- Actual formal PRE/POST reference only; not executed (left-recursive T composed with R union B)
E_union(x,y) :- R(x,y).

```

```

E_union(x,y) :- B(x,y).
T_left(x,y) :- E_union(x,y).
T_left(x,z) :- T_left(x,y), E_union(y,z).

```

Reading. The loop naively evaluates a red/blue alternating-path program: RR and BB collect same-colour-start walks and RB, BR collect colour-switching walks, computed by mutual recursion matching `red_blue_linear` (e.g. $RR(x,y) \text{:-} R(x,z), BR(z,y)$, $RB(x,y) \text{:-} RR(x,z), B(z,y)$, etc.), so together the four relations characterize walks in a two-coloured graph that strictly alternate colour. The Hoare triple is a containment claim rather than an equivalence: it asserts that the union of these four alternating-path relations is contained in any pre-fixpoint bound `RBound` of the plain transitive closure of the underlying uncoloured graph `RUB`, since every colour-alternating walk is in particular an ordinary walk in `RUB`.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in `Input.lean`. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\forall x y. (R(x,y) \vee B(x,y) \vee (\exists z. RBound(x,z) \wedge (R(z,y) \vee B(z,y)))) \rightarrow RBound(x,y)$

The precondition states that `RBound` is a pre-fixpoint of the transitive-closure program on the union graph `RUB`: $(RUB) \cup (RBound \circ (RUB)) \subseteq RBound$.

POST: $\forall x y. (RR(x,y) \vee BB(x,y) \vee RB(x,y) \vee BR(x,y) \rightarrow RBound(x,y))$

The postcondition states that the union of the four computed red/blue relations, `RRUBBUBRUBR`, is contained in `RBound`.

Whiel encoding (Hoare triple).

```

SCHEMA
  R/2, B/2, RR/2, BB/2, RB/2, BR/2, RR_aux/2, BB_aux/2, RB_aux/2, BR_aux/2, RBound/2

PRE
  (((R ∪ B) ∪ (π[0, 3] (σ[#1 = #2] (RBound × (R ∪ B))))) ⊆ RBound)

CMD
  RR_aux := ∅;
  BB_aux := ∅;
  RB_aux := ∅;
  BR_aux := ∅;
  RR := R;
  BB := B;
  RB := (π[0, 3] (σ[#1 = #2] (R × B)));
  BR := (π[0, 3] (σ[#1 = #2] (B × R)));
  WHILE ((RR ≠ RR_aux) ∨ (BB ≠ BB_aux)) ∨ ((RB ≠ RB_aux) ∨ (BR ≠ BR_aux)) DO
    RR_aux := RR;
    BB_aux := BB;
    RB_aux := RB;
    BR_aux := BR;
    RB := (RB ∪ (π[0, 3] (σ[#1 = #2] (RR_aux × B))));
    BR := (BR ∪ (π[0, 3] (σ[#1 = #2] (BB_aux × R))));
    RR := (RR ∪ (π[0, 3] (σ[#1 = #2] (R × BR_aux))));
    BB := (BB ∪ (π[0, 3] (σ[#1 = #2] (B × RB_aux))));
  END

POST
  (((RR ∪ BB) ∪ RB) ∪ BR) ⊆ RBound

```

9 Example0115: Prefix-equivalence: naive all_paths_via_even_odd (linear) computes lfp of tc_left

Category	synthetic (syntactic generation from the classic Datalog programs); family: Synthetic.
Kind	equivalence (with a least fixpoint, via a pre-fixpoint witness).
Contributors	Fangzhu Shen, Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	fixpoint-witness mutual-recursion.
Input	Benchmark/Example0115/Input.lean (canonical notation).
Summary	Canonical form of the earlier single-level encoding of this case: the same schema, precondition, command and postcondition.
Sources	legacy source record v2__prefix_equivalence__tc_left__all_paths_via_even_odd_linear__naive of the retired classic-source collection

Datalog program(s). *all_paths_via_even_odd_linear* — legacy inventory: `catalog/source` intent plus actual right-recursive formal PRE/POST reference-operator reconstruction; sources: `all_paths_via_even_odd.dl`, `tc-left.dl`, `Benchmark/Example0115/Input.lean`.

```
-- all_paths_via_even_odd_linear
-- Executed WHIEL program
OL(x,y) :- E(x,y).
OL(x,y) :- E(x,z), EL(z,y).
EL(x,y) :- E(x,z), OL(z,y).
T(x,y) :- EL(x,y).
T(x,y) :- OL(x,y).
```

tc_left — legacy inventory: `catalog/source` intent plus actual right-recursive formal PRE/POST reference-operator reconstruction; sources: `all_paths_via_even_odd.dl`, `tc-left.dl`, `Benchmark/Example0115/Input.lean`.

```
-- tc_left
-- Catalog/source target intent only; not executed by the WHIEL command
tcl(x, y) :- base(x, y).
tcl(x, y) :- tcl(x, z), base(z, y).
-- Actual formal PRE/POST reference only; not executed (right-recursive E composed with T)
T_right(x,y) :- E(x,y).
T_right(x,y) :- E(x,z), T_right(z,y).
```

Reading. The loop naively evaluates the `all_paths_via_even_odd` program: T_a and T_b are computed by mutual recursion matching the even/odd path relations ($T_a(x,y) :- E(x,z), T_b(z,y)$, $T_b(x,y) :- E(x,y)$ or $E(x,z), T_a(z,y)$), and $T_c := T_a \cup T_b$ collects all paths of length one or more. Although the title names `tc_left` as the reference, the precondition and postcondition actually encode the right-linear closure $T(x,y) :- E(x,y)$, $T(x,y) :- E(x,z), T(z,y)$ (as the recorded Datalog reference for this case acknowledges), so the Hoare triple is the least-fixpoint introduction obligation stating that the naively computed union T_c of even and odd paths is itself a pre-fixpoint of that right-linear closure program and lies below any given pre-fixpoint R — i.e. the even/odd decomposition computes the transitive closure, since every nonempty path has either even or odd length.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in `Input.lean`. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\forall x y. (E(x,y) \vee (\exists z. E(x,z) \wedge R(z,y)) \rightarrow R(x,y))$

The precondition states that R is a pre-fixpoint of the right-linear closure program: $E \cup (E \circ R) \subseteq R$.

POST: $(\forall x y. (E(x,y) \vee (\exists z. E(x,z) \wedge T_c(z,y)) \rightarrow T_c(x,y))) \wedge \forall u v. (T_c(u,v) \rightarrow R(u,v))$

The postcondition states that the computed union T_c is itself a pre-fixpoint of the right-linear closure program, $E \cup (E \circ T_c) \subseteq T_c$, and that $T_c \subseteq R$.

Whiel encoding (Hoare triple).

```

SCHEMA
  E/2, R/2, Ta/2, Sa/2, Tb/2, Sb/2, Tc/2, Sc/2

PRE
  ((E  $\cup$  ( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (E  $\times$  R))))  $\subseteq$  R)

CMD
  Ta :=  $\emptyset$ ;
  Tb :=  $\emptyset$ ;
  Tc :=  $\emptyset$ ;
  Sa := ( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (E  $\times$  Tb)));
  Sb := (E  $\cup$  ( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (E  $\times$  Ta))));
  Sc := (Ta  $\cup$  Tb);
  WHILE  $\neg$ ((Sa = Ta)  $\wedge$  ((Sb = Tb)  $\wedge$  (Sc = Tc))) DO
    Ta := Sa;
    Tb := Sb;
    Tc := Sc;
    Sa := ( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (E  $\times$  Tb)));
    Sb := (E  $\cup$  ( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (E  $\times$  Ta))));
    Sc := (Ta  $\cup$  Tb)
  END

POST
  (((E  $\cup$  ( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (E  $\times$  Tc))))  $\subseteq$  Tc)  $\wedge$  (Tc  $\subseteq$  R))

```

10 Example0117: Prefix-equivalence: naive all_paths_via_even_odd (non-linear) computes lfp of tc_left

Category	synthetic (syntactic generation from the classic Datalog programs); family: Synthetic.
Kind	equivalence (with a least fixpoint, via a pre-fixpoint witness).
Contributors	Fangzhu Shen, Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	fixpoint-witness linear-vs-nonlinear mutual-recursion nonlinear-recursion.
Input	Benchmark/Example0117/Input.lean (canonical notation).
Summary	Canonical form of the earlier single-level encoding of this case: the same schema, precondition, command and postcondition.
Sources	legacy source record v2__prefix_equivalence__tc_left__all_paths_via_even_odd_nonlinear__naive of the retired classic-source collection

Datalog program(s). *all_paths_via_even_odd_nonlinear* — legacy inventory: `catalog/source` intent plus actual right-recursive formal PRE/POST reference-operator reconstruction; sources: `all_paths_via_even_odd_nonlinear.dl`, `tc-left.dl`, `Benchmark/Example0117/Input.lean`.

```

-- all_paths_via_even_odd_nonlinear
-- Executed WHIEL program
OL(x,y) :- E(x,y).
EL(x,y) :- E(x,z), E(z,y).
OL(x,y) :- OL(x,z), EL(z,y).
OL(x,y) :- EL(x,z), OL(z,y).
EL(x,y) :- EL(x,z), EL(z,y).
EL(x,y) :- OL(x,z), OL(z,y).
T(x,y) :- EL(x,y).
T(x,y) :- OL(x,y).

```

tc_left — legacy inventory: `catalog/source` intent plus actual right-recursive formal PRE/POST reference-operator reconstruction; sources: `all_paths_via_even_odd_nonlinear.dl`, `tc-left.dl`, `Benchmark/Example0117/Input.lean`.

```
-- tc_left
-- Catalog/source target intent only; not executed by the WHIEL command
tcl(x, y) := base(x, y).
tcl(x, y) := tcl(x, z), base(z, y).
-- Actual formal PRE/POST reference only; not executed (right-recursive E composed with T)
T_right(x,y) := E(x,y).
T_right(x,y) := E(x,z), T_right(z,y).
```

Reading. The loop naively evaluates the non-linear `all_paths_via_even_odd` program, where T_a (even-length paths) and T_b (odd-length paths) are each computed by squaring/composing T_a and T_b against each other and against E (T_a gets contributions from $E \circ E$, $T_a \circ T_a$ and $T_b \circ T_b$), and $T_c := T_a \cup T_b$ collects all paths of length one or more. As in the related Example0115, although the title names `tc_left`, the precondition and postcondition actually encode the right-linear closure $T(x,y) :- E(x,y)$, $T(x,y) :- E(x,z), T(z,y)$; the Hoare triple is the least-fixpoint introduction obligation stating that the naively computed union T_c is itself a pre-fixpoint of that right-linear program and lies below any given pre-fixpoint R , again because every nonempty path has even or odd length.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in `Input.lean`. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\forall x y. (E(x,y) \vee (\exists z. E(x,z) \wedge R(z,y)) \rightarrow R(x,y))$

The precondition states that R is a pre-fixpoint of the right-linear closure program: $E \cup (E \circ R) \subseteq R$.

POST: $(\forall x y. (E(x,y) \vee (\exists z. E(x,z) \wedge T_c(z,y)) \rightarrow T_c(x,y))) \wedge \forall u v. (T_c(u,v) \rightarrow R(u,v))$

The postcondition states that the computed union T_c is itself a pre-fixpoint of the right-linear closure program, $E \cup (E \circ T_c) \subseteq T_c$, and that $T_c \subseteq R$.

Whiel encoding (Hoare triple).

```
SCHEMA
  E/2, R/2, Ta/2, Sa/2, Tb/2, Sb/2, Tc/2, Sc/2

PRE
  ((E ∪ (π[0, 3] (σ[#1 = #2] (E × R)))) ⊆ R)

CMD
  Ta := ∅;
  Tb := ∅;
  Tc := ∅;
  Sa := (((π[0, 3] (σ[#1 = #2] (E × E))) ∪ (π[0, 3] (σ[#1 = #2] (Ta × Ta)))) ∪ (π[0, 3] (σ[#1 = #2] (Tb × Tb))));
  Sb := ((E ∪ (π[0, 3] (σ[#1 = #2] (Tb × Ta)))) ∪ (π[0, 3] (σ[#1 = #2] (Ta × Tb))));
  Sc := (Ta ∪ Tb);
  WHILE ¬(((Sa = Ta) ∧ ((Sb = Tb) ∧ (Sc = Tc)))) DO
    Ta := Sa;
    Tb := Sb;
    Tc := Sc;
    Sa := (((π[0, 3] (σ[#1 = #2] (E × E))) ∪ (π[0, 3] (σ[#1 = #2] (Ta × Ta)))) ∪ (π[0, 3] (σ[#1 = #2] (Tb × Tb))));
    Sb := ((E ∪ (π[0, 3] (σ[#1 = #2] (Tb × Ta)))) ∪ (π[0, 3] (σ[#1 = #2] (Ta × Tb))));
    Sc := (Ta ∪ Tb)
  END

POST
  (((E ∪ (π[0, 3] (σ[#1 = #2] (E × Tc)))) ⊆ Tc) ∧ (Tc ⊆ R))
```

11 Example0119: Prefix-equivalence: naive all_paths_via_even_odd (linear) computes lfp of tc_nonlinear

Category	synthetic (syntactic generation from the classic Datalog programs); family: Synthetic.
Kind	equivalence (with a least fixpoint, via a pre-fixpoint witness).
Contributors	Fangzhu Shen, Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	<code>fixpoint-witness</code> <code>linear-vs-nonlinear</code> <code>mutual-recursion</code> <code>nonlinear-recursion</code> <code>prophecy</code> .
Input	Benchmark/Example0119/Input.lean (canonical notation).
Summary	All paths, computed through mutually recursive odd and even halves, specified as the least fixpoint of the non-linear closure program.
Sources	legacy source record v2__prefix_equivalence__tc_nonlin__all_paths_via_even_odd_linear__naive of the retired classic-source collection classic Datalog program all_paths_via_even_odd of the retired generator suite classic Datalog program tc-nonlinear of the retired generator suite
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).

Datalog program(s). *all_paths_via_even_odd_linear* — legacy inventory: source programs with one executed WHIEL program and one formal PRE/POST reference (not executed); sources: *all_paths_via_even_odd.dl*, *tc-nonlinear.dl*.

```
-- all_paths_via_even_odd_linear
-- Executed WHIEL program
OL(x,y) :- E(x,y).
OL(x,y) :- E(x,z), EL(z,y).
EL(x,y) :- E(x,z), OL(z,y).
T(x,y) :- EL(x,y).
T(x,y) :- OL(x,y).
```

tc_nonlin — legacy inventory: source programs with one executed WHIEL program and one formal PRE/POST reference (not executed); sources: *all_paths_via_even_odd.dl*, *tc-nonlinear.dl*.

```
-- tc_nonlin
-- Formal PRE/POST reference only; not executed by the WHIEL command
tc(x, y) :- base(x, y).
tc(x, y) :- tc(x, z), tc(z, y).
```

Reading. The loop naively evaluates the *all_paths_via_even_odd* program (Tb the odd-length paths via $Tb := E \cup (E \circ Ta)$, Ta the even-length paths via $Ta := E \circ Tb$, and $Tc := Ta \cup Tb$ their union), matching the linear *all_paths_via_even_odd.dl* program. The Hoare triple is the least-fixpoint introduction obligation for the non-linear transitive-closure program $T(x,y) :- E(x,y)$, $T(x,y) :- T(x,z), T(z,y)$: given any R that is already a pre-fixpoint of that non-linear program, the computed Tc must itself be a pre-fixpoint of it and lie below R, so the even/odd decomposition computes the (non-linear) transitive closure, since every path has odd or even length.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in *Input.lean* remains the authority.

PRE: $\forall x y. (E(x,y) \vee (\exists z. R(x,z) \wedge R(z,y)) \rightarrow R(x,y))$

The precondition states that R is a pre-fixpoint of the non-linear transitive-closure program: $E \cup (R \circ R) \subseteq R$.

POST: $(\forall x y. (E(x,y) \vee (\exists z. Tc(x,z) \wedge Tc(z,y)) \rightarrow Tc(x,y))) \wedge \forall u v. (Tc(u,v) \rightarrow R(u,v))$

The postcondition states that the computed Tc is itself a pre-fixpoint of the non-linear transitive-closure program, $E \cup (Tc \circ Tc) \subseteq Tc$, and that $Tc \subseteq R$.

Whiel encoding (Hoare triple).

```

SCHEMA
  E/2, R/2, Ta/2, Sa/2, Tb/2, Sb/2, Tc/2, Sc/2

PRE
  ((E ∪ (π[0, 3] (σ[#1 = #2] (R × R)))) ⊆ R)

CMD
  Ta := ∅;
  Tb := ∅;
  Tc := ∅;
  Sa := (π[0, 3] (σ[#1 = #2] (E × Tb)));
  Sb := (E ∪ (π[0, 3] (σ[#1 = #2] (E × Ta))));
  Sc := (Ta ∪ Tb);
  WHILE
    ¬((Sa = Ta) ∧
      ((Sb = Tb) ∧ (Sc = Tc))) DO
    Ta := Sa;
    Tb := Sb;
    Tc := Sc;
    Sa := (π[0, 3] (σ[#1 = #2] (E × Tb)));
    Sb :=
      (E ∪ (π[0, 3] (σ[#1 = #2] (E × Ta))));
    Sc := (Ta ∪ Tb)
  END

POST
  ((E ∪ (π[0, 3] (σ[#1 = #2] (Tc × Tc)))) ⊆ Tc)
  ∧ (Tc ⊆ R)

```

12 Example0123: Prefix-equivalence: naive even_odd_nonlinear computes lfp of even_odd_linear

Category	synthetic (syntactic generation from the classic Datalog programs); family: Synthetic.
Kind	equivalence (with a least fixpoint, via a pre-fixpoint witness).
Contributors	Fangzhu Shen, Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	fixpoint-witness linear-vs-nonlinear mutual-recursion nonlinear-recursion.
Input	Benchmark/Example0123/Input.lean (canonical notation).
Summary	Canonical form of the earlier single-level encoding of this case: the same schema, precondition, command and postcondition.
Sources	legacy source record v2__prefix_equivalence__even_odd_linear__even_odd_nonlinear__naive of the retired classic-source collection classic Datalog program even_odd_linear of the retired generator suite classic Datalog program even_odd_nonlinear of the retired generator suite

Datalog program(s). *even_odd_linear* — legacy inventory: source programs with one executed WHIEL program and one formal PRE/POST reference (not executed); sources: even_odd_linear.dl, even_odd_nonlinear.dl.

```

-- even_odd_linear
-- Formal PRE/POST reference only; not executed by the WHIEL command
OL(x,y) :- E(x,y).
OL(x,y) :- E(x,z), EL(z,y).
EL(x,y) :- E(x,z), OL(z,y).

```

even_odd_nonlinear — legacy inventory: source programs with one executed WHIEL program and one formal PRE/POST reference (not executed); sources: even_odd_linear.dl, even_odd_nonlinear.dl.

```
-- even_odd_nonlinear
-- Executed WHIEL program
OL(x,y) :- E(x,y).
EL(x,y) :- E(x,z), E(z,y).
OL(x,y) :- OL(x,z), EL(z,y).
OL(x,y) :- EL(x,z), OL(z,y).
EL(x,y) :- EL(x,z), EL(z,y).
EL(x,y) :- OL(x,z), OL(z,y).
```

Reading. The loop naively evaluates the non-linear `even_odd_nonlinear` program, computing even-length paths Ta and odd-length paths Tb by mutual squaring recursion (Ta from $E \circ E$, $Ta \circ Ta$ and $Tb \circ Tb$; Tb from E , $Tb \circ Ta$ and $Ta \circ Tb$), matching `even_odd_nonlinear.dl`. The Hoare triple is the least-fixpoint introduction obligation for the linear `even_odd_linear` recursion ($Ta(x,y) :- E(x,z), Tb(z,y)$; $Tb(x,y) :- E(x,y)$ or $E(x,z), Ta(z,y)$): given any pair Ra, Rb already a pre-fixpoint of that linear recursion, the computed Ta, Tb must themselves form a pre-fixpoint of it and lie below Ra, Rb respectively, i.e. the non-linear program computes the least pre-fixpoint pair of the linear even/odd program.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in `Input.lean`. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $(\forall x y. ((\exists z. E(x,z) \wedge Rb(z,y)) \rightarrow Ra(x,y))) \wedge \forall u v. (E(u,v) \vee (\exists w. E(u,w) \wedge Ra(w,v)) \rightarrow Rb(u,v))$

The precondition states that the pair Ra, Rb is a pre-fixpoint of the linear even/odd recursion: $E \circ Rb \subseteq Ra$ and $EU(E \circ Ra) \subseteq Rb$.

POST: $(\forall x y. ((\exists z. E(x,z) \wedge Tb(z,y)) \rightarrow Ta(x,y))) \wedge (\forall u v. (E(u,v) \vee (\exists w. E(u,w) \wedge Ta(w,v)) \rightarrow Tb(u,v))) \wedge (\forall x_1 x_2. (Ta(x_1, x_2) \rightarrow Ra(x_1, x_2))) \wedge \forall x_3 x_4. (Tb(x_3, x_4) \rightarrow Rb(x_3, x_4))$

The postcondition states that the computed pair Ta, Tb is itself a pre-fixpoint of the linear even/odd recursion, $E \circ Tb \subseteq Ta$ and $EU(E \circ Ta) \subseteq Tb$, and that $Ta \subseteq Ra$ and $Tb \subseteq Rb$.

Whiel encoding (Hoare triple).

```
SCHEMA
  E/2, Ra/2, Rb/2, Ta/2, Sa/2, Tb/2, Sb/2

PRE
  ((( $\pi[0, 3] (\sigma[\#1 = \#2] (E \times Rb))) \subseteq Ra) \wedge ((E \cup (\pi[0, 3] (\sigma[\#1 = \#2] (E \times Ra)))) \subseteq Rb))

CMD
  Ta :=  $\emptyset$ ;
  Tb :=  $\emptyset$ ;
  Sa := ((( $\pi[0, 3] (\sigma[\#1 = \#2] (E \times E))) \cup (\pi[0, 3] (\sigma[\#1 = \#2] (Ta \times Ta)))) \cup (\pi[0, 3] (\sigma[\#1 = \#2] (Tb \times Tb))))$ ;
  Sb := (( $E \cup (\pi[0, 3] (\sigma[\#1 = \#2] (Tb \times Ta)))$ )  $\cup (\pi[0, 3] (\sigma[\#1 = \#2] (Ta \times Tb)))$ );
  WHILE  $\neg((Sa = Ta) \wedge (Sb = Tb))$  DO
    Ta := Sa;
    Tb := Sb;
    Sa := ((( $\pi[0, 3] (\sigma[\#1 = \#2] (E \times E))) \cup (\pi[0, 3] (\sigma[\#1 = \#2] (Ta \times Ta)))) \cup (\pi[0, 3] (\sigma[\#1 = \#2] (Tb \times Tb))))$ ;
    Sb := (( $E \cup (\pi[0, 3] (\sigma[\#1 = \#2] (Tb \times Ta)))$ )  $\cup (\pi[0, 3] (\sigma[\#1 = \#2] (Ta \times Tb)))$ );
  END

POST
  ((( $\pi[0, 3] (\sigma[\#1 = \#2] (E \times Tb))) \subseteq Ta) \wedge (((E \cup (\pi[0, 3] (\sigma[\#1 = \#2] (E \times Ta)))) \subseteq Tb) \wedge ((Ta \subseteq Ra) \wedge (Tb \subseteq Rb))))$$ 
```

13 Example0125: Prefix-equivalence: naive even_odd_linear computes lfp of even_odd_nonlinear

Category	synthetic (syntactic generation from the classic Datalog programs); family: Synthetic.
Kind	equivalence (with a least fixpoint, via a pre-fixpoint witness).
Contributors	Fangzhu Shen, Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	fixpoint-witness linear-vs-nonlinear mutual-recursion nonlinear-recursion prophecy.
Input	Benchmark/Example0125/Input.lean (canonical notation).
Summary	Odd and even paths computed by a linear program, specified as the least fixpoint of the non-linear odd/even program.
Sources	legacy source record v2__prefix_equivalence__even_odd_nonlinear__even_odd_linear__naive of the retired classic-source collection classic Datalog program even_odd_linear of the retired generator suite classic Datalog program even_odd_nonlinear of the retired generator suite
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).

Datalog program(s). *even_odd_linear* — legacy inventory: source programs with one executed WHIEL program and one formal PRE/POST reference (not executed); sources: even_odd_linear.dl, even_odd_nonlinear.dl.

```
-- even_odd_linear
-- Executed WHIEL program
OL(x,y) :- E(x,y).
OL(x,y) :- E(x,z), EL(z,y).
EL(x,y) :- E(x,z), OL(z,y).
```

even_odd_nonlinear — legacy inventory: source programs with one executed WHIEL program and one formal PRE/POST reference (not executed); sources: even_odd_linear.dl, even_odd_nonlinear.dl.

```
-- even_odd_nonlinear
-- Formal PRE/POST reference only; not executed by the WHIEL command
OL(x,y) :- E(x,y).
EL(x,y) :- E(x,z), E(z,y).
OL(x,y) :- OL(x,z), EL(z,y).
OL(x,y) :- EL(x,z), OL(z,y).
EL(x,y) :- EL(x,z), EL(z,y).
EL(x,y) :- OL(x,z), OL(z,y).
```

Reading. The loop naively evaluates the linear *even_odd_linear* program, computing even-length paths T_a and odd-length paths T_b by the mutual recursion $T_a(x,y) :- E(x,z), T_b(z,y)$ and $T_b(x,y) :- E(x,y)$ or $E(x,z), T_a(z,y)$. The Hoare triple is the least-fixpoint introduction obligation for the non-linear *even_odd_nonlinear* recursion, which computes the same even/odd relations by squaring (e.g. T_a from $E \circ E$, $T_a \circ T_a$ and $T_b \circ T_b$): given any pair R_a, R_b already a pre-fixpoint of that non-linear program, the computed T_a, T_b must themselves form a pre-fixpoint of it and lie below R_a, R_b respectively, so the linear program computes the least pre-fixpoint pair of the non-linear one.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in *Input.lean* remains the authority.

PRE: $(\forall x y. ((\exists z. E(x,z) \wedge E(z,y)) \vee (\exists u. R_a(x,u) \wedge R_a(u,y)) \vee (\exists v. R_b(x,v) \wedge R_b(v,y)) \rightarrow R_a(x,y))) \wedge \forall w x1. (E(w,x1) \vee (\exists x2. R_b(w,x2) \wedge R_a(x2,x1)) \vee (\exists x3. R_a(w,x3) \wedge R_b(x3,x1)) \rightarrow R_b(w,x1))$

The precondition states that the pair Ra, Rb is a pre-fixpoint of the non-linear even/odd recursion: $(E \circ E) \cup (Ra \circ Ra) \cup (Rb \circ Rb) \subseteq Ra$, and $E \cup (Rb \circ Ra) \cup (Ra \circ Rb) \subseteq Rb$.

POST: $(\forall x y. ((\exists z. E(x, z) \wedge E(z, y)) \vee (\exists u. Ta(x, u) \wedge Ta(u, y)) \vee (\exists v. Tb(x, v) \wedge Tb(v, y)) \rightarrow Ta(x, y))) \wedge (\forall w x1. (E(w, x1) \vee (\exists x2. Tb(w, x2) \wedge Ta(x2, x1)) \vee (\exists x3. Ta(w, x3) \wedge Tb(x3, x1)) \rightarrow Tb(w, x1))) \wedge (\forall x4 x5. (Ta(x4, x5) \rightarrow Ra(x4, x5))) \wedge \forall x6 x7. (Tb(x6, x7) \rightarrow Rb(x6, x7))$

The postcondition states that the computed pair Ta, Tb is itself a pre-fixpoint of the non-linear even/odd recursion, $(E \circ E) \cup (Ta \circ Ta) \cup (Tb \circ Tb) \subseteq Ta$ and $E \cup (Tb \circ Ta) \cup (Ta \circ Tb) \subseteq Tb$, and that $Ta \subseteq Ra$ and $Tb \subseteq Rb$.

Whiel encoding (Hoare triple).

```

SCHEMA
  E/2, Ra/2, Rb/2, Ta/2, Sa/2, Tb/2, Sb/2

PRE
  ((((( $\pi[0, 3] (\sigma[\#1 = \#2] (E \times E)) \cup$ 
    ( $\pi[0, 3] (\sigma[\#1 = \#2] (Ra \times Ra))) \cup$ 
    ( $\pi[0, 3] (\sigma[\#1 = \#2] (Rb \times Rb))) \subseteq Ra$ )
     $\wedge ((E \cup$ 
      ( $\pi[0, 3] (\sigma[\#1 = \#2] (Rb \times Ra))) \cup$ 
      ( $\pi[0, 3] (\sigma[\#1 = \#2] (Ra \times Rb))) \subseteq Rb$ ))

CMD
  Ta :=  $\emptyset$ ;
  Tb :=  $\emptyset$ ;
  Sa := ( $\pi[0, 3] (\sigma[\#1 = \#2] (E \times Tb))$ );
  Sb := ( $E \cup (\pi[0, 3] (\sigma[\#1 = \#2] (E \times Ta)))$ );
  WHILE  $\neg((Sa = Ta) \wedge (Sb = Tb))$  DO
    Ta := Sa;
    Tb := Sb;
    Sa := ( $\pi[0, 3] (\sigma[\#1 = \#2] (E \times Tb))$ );
    Sb :=
      ( $E \cup (\pi[0, 3] (\sigma[\#1 = \#2] (E \times Ta)))$ )
  END

POST
  ((((( $\pi[0, 3] (\sigma[\#1 = \#2] (E \times E)) \cup$ 
    ( $\pi[0, 3] (\sigma[\#1 = \#2] (Ta \times Ta))) \cup$ 
    ( $\pi[0, 3] (\sigma[\#1 = \#2] (Tb \times Tb))) \subseteq Ta$ )
     $\wedge (((E \cup$ 
      ( $\pi[0, 3] (\sigma[\#1 = \#2] (Tb \times Ta))) \cup$ 
      ( $\pi[0, 3] (\sigma[\#1 = \#2] (Ta \times Tb))) \subseteq Tb$ )
       $\wedge ((Ta \subseteq Ra) \wedge (Tb \subseteq Rb)))$ 

```

14 Example0128: Containment: naive red_blue_nonlinear outputs $\subseteq$ pre-fixpoint of TC(R $\cup$ B)

Category	synthetic (syntactic generation from the classic Datalog programs); family: Synthetic.
Kind	containment (via a pre-fixpoint witness).
Contributors	Fangzhu Shen, Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	fixpoint-witness mutual-recursion nonlinear-recursion.
Input	Benchmark/Example0128/Input.lean (canonical notation).
Summary	Canonical form of the earlier single-level encoding of this case: the same schema, precondition, command and postcondition.
Sources	legacy source record v2__containment__tc_of_union__red_blue_nonlinear__naive of the retired classic-source collection classic Datalog program red_blue_nonlinear of the retired generator suite classic Datalog program tc_of_union of the retired generator suite

Datalog program(s). *red_blue_nonlinear* — legacy inventory: source programs with one executed WHIEL program and one formal PRE/POST reference (not executed); sources: red_blue_nonlinear.dl, tc_of_union.dl.

```
-- red_blue_nonlinear
-- Executed WHIEL program
RR(x,y) :- R(x,y).
BB(x,y) :- B(x,y).
RB(x,y) :- R(x,z), B(z,y).
BR(x,y) :- B(x,z), R(z,y).
RR(x,y) :- RR(x,z), BR(z,y).
BB(x,y) :- BB(x,z), RB(z,y).
RB(x,y) :- RR(x,z), BB(z,y).
BR(x,y) :- BB(x,z), RR(z,y).
```

tc_of_union — legacy inventory: source programs with one executed WHIEL program and one formal PRE/POST reference (not executed); sources: red_blue_nonlinear.dl, tc_of_union.dl.

```
-- tc_of_union
-- Formal PRE/POST reference only; not executed by the WHIEL command
E(x,y) :- R(x,y).
E(x,y) :- B(x,y).
T(x,y) :- E(x,y).
T(x,y) :- E(x,z), T(z,y).
```

Reading. The loop naively evaluates the non-linear *red_blue_nonlinear* program, computing four colour-alternating path relations (same-colour-start relations analogous to RR/BB and colour-switching relations analogous to RB/BR) by mutual recursion over a red edge relation *R* and a blue edge relation *B*, matching *red_blue_nonlinear.dl*. The Hoare triple is a containment claim: it asserts that each of the four computed relations is contained in any relation *R* that is already a pre-fixpoint of the (uncoloured) transitive-closure program on $R \cup B$, since every colour-alternating walk is in particular an ordinary walk in the union graph.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in *Input.lean* remains the authority.

PRE: $\forall x y. (RX(x,y) \vee B(x,y) \vee (\exists z. RX(x,z) \wedge R(z,y)) \vee (\exists u. B(x,u) \wedge R(u,y)) \rightarrow R(x,y))$

The precondition states that *R* is a pre-fixpoint of the transitive-closure program on the union graph $R \cup B$: $(R \cup B) \cup (R \circ R) \cup (B \circ R) \subseteq R$.

POST: $(\forall x y. (Ta(x,y) \rightarrow R(x,y))) \wedge (\forall z u. (Tb(z,u) \rightarrow R(z,u))) \wedge (\forall v w. (Tc(v,w) \rightarrow R(v,w))) \wedge \forall x1 x2. (Td(x1,x2) \rightarrow R(x1,x2))$

The postcondition states that all four computed relations Ta, Tb, Tc, Td are each contained in R.

Whiel encoding (Hoare triple).

```

SCHEMA
  B/2, RX/2, R/2, Ta/2, Sa/2, Tb/2, Sb/2, Tc/2, Sc/2, Td/2, Sd/2

PRE
  (((RX  $\cup$  B)  $\cup$  ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (RX  $\times$  R))))  $\cup$  ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (B  $\times$  R))))  $\subseteq$  R

CMD
  Ta :=  $\emptyset$ ;
  Tb :=  $\emptyset$ ;
  Tc :=  $\emptyset$ ;
  Td :=  $\emptyset$ ;
  Sa := (B  $\cup$  ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (Ta  $\times$  Tc))));
  Sb := (( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (B  $\times$  RX)))  $\cup$  ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (Ta  $\times$  Td))));
  Sc := (( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (RX  $\times$  B)))  $\cup$  ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (Td  $\times$  Ta))));
  Sd := (RX  $\cup$  ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (Td  $\times$  Tb))));
  WHILE  $\neg$ ((Sa = Ta)  $\wedge$  ((Sb = Tb)  $\wedge$  ((Sc = Tc)  $\wedge$  (Sd = Td)))) DO
    Ta := Sa;
    Tb := Sb;
    Tc := Sc;
    Td := Sd;
    Sa := (B  $\cup$  ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (Ta  $\times$  Tc))));
    Sb := (( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (B  $\times$  RX)))  $\cup$  ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (Ta  $\times$  Td))));
    Sc := (( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (RX  $\times$  B)))  $\cup$  ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (Td  $\times$  Ta))));
    Sd := (RX  $\cup$  ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (Td  $\times$  Tb))));
  END

POST
  ((Ta  $\subseteq$  R)  $\wedge$  ((Tb  $\subseteq$  R)  $\wedge$  ((Tc  $\subseteq$  R)  $\wedge$  (Td  $\subseteq$  R))))

```

15 Example0130: Containment: even/odd/ambiguous-depth path-system program $\subseteq$ every set closed under the basic path-system rules

Category	synthetic (syntactic generation from the classic Datalog programs); family: Synthetic.
Kind	containment.
Contributors	Fangzhu Shen, Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	<code>fixpoint-witness</code> <code>mutual-recursion</code> <code>nonlinear-recursion</code> .
Input	Benchmark/Example0130/Input.lean (canonical notation).
Summary	Path systems (Cook): unary axioms A and ternary rules $R(x,y,z)$, read as "if y and z are accessible then x is accessible". The basic program $T(x) :- A(x). T(x) :- R(x,y,z), T(y), T(z).$ computes the accessible nodes T.
Sources	legacy source record v2__containment__path_systems_basic__path_systems_height__naive of the retired classic-source collection classic Datalog program path_systems_basic of the retired generator suite classic Datalog program path_systems_height of the retired generator suite

Datalog program(s). *rules as stated in the header comment of Input.lean* — header comment.

```

T(x) :- A(x).
T(x) :- R(x,y,z), T(y), T(z).
EH(x) :- A(x). x has a derivation whose leaves all lie at even depth

```

```

OH(x) :- R(x,y,z), EH(y), EH(z). ... all at odd depth
EH(x) :- R(x,y,z), OH(y), OH(z).
AH(x) :- R(x,y,z), Y(y), Z(z) for every other pair (Y,Z) over EH/OH/AH:

```

Reading. The Datalog programs classify the nodes a Cook path system (unary axioms A, ternary rules $R(x,y,z)$ meaning x is accessible if y and z both are) makes accessible, by the parity of their derivation trees' leaf depths: EH, OH and AH hold for a node with a derivation whose leaves are all at even depth, all at odd depth, or mixed, respectively. The precondition says a free relation Closed already contains every axiom and is closed under the path-system rule (a pre-fixpoint); the postcondition says the three depth-parity outputs are each contained in Closed. Since Closed ranges over every such closed set and the accessible-node relation is the least one, this is the standard least-fixpoint-containment argument, applied to three refinements of the accessible set at once.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\forall x. (A(x) \vee (\exists y z. R(x,y,z) \wedge \text{Closed}(y) \wedge \text{Closed}(z)) \rightarrow \text{Closed}(x))$

PRE says Closed contains every axiom $A(x)$ and is closed under the path-system rule: whenever $R(x,y,z)$ holds and y and z are both already in Closed, x is too.

POST: $(\forall x. (AH(x) \rightarrow \text{Closed}(x))) \wedge (\forall y. (EH(y) \rightarrow \text{Closed}(y))) \wedge \forall z. (OH(z) \rightarrow \text{Closed}(z))$

POST says the even-depth, odd-depth and mixed-depth output relations AH, EH and OH are each contained in Closed.

Whiel encoding (Hoare triple).

```

SCHEMA
  A/1, Closed/1, AH/1, EH/1, OH/1, AHNext/1, EHNext/1, OHNext/1, R/3

PRE
  ((A ∪ (π[0] (σ[#2 = #4] (σ[#1 = #3] ((R × Closed) × Closed)))))) ⊆ Closed

CMD
  AH := ∅;
  EH := ∅;
  OH := ∅;
  AHNext :=
    ((((((π[0] (σ[#2 = #4] (σ[#1 = #3] ((R × EH) × OH)))) -- AH(x) :- R(x,y,z), EH(y), OH(z)
    ∪ (π[0] (σ[#2 = #4] (σ[#1 = #3] ((R × OH) × EH)))) -- AH(x) :- R(x,y,z), OH(y), EH(z)
    ∪ (π[0] (σ[#2 = #4] (σ[#1 = #3] ((R × AH) × OH)))) -- AH(x) :- R(x,y,z), AH(y), OH(z)
    ∪ (π[0] (σ[#2 = #4] (σ[#1 = #3] ((R × OH) × AH)))) -- AH(x) :- R(x,y,z), OH(y), AH(z)
    ∪ (π[0] (σ[#2 = #4] (σ[#1 = #3] ((R × AH) × EH)))) -- AH(x) :- R(x,y,z), AH(y), EH(z)
    ∪ (π[0] (σ[#2 = #4] (σ[#1 = #3] ((R × EH) × AH)))) -- AH(x) :- R(x,y,z), EH(y), AH(z)
    ∪ (π[0] (σ[#2 = #4] (σ[#1 = #3] ((R × AH) × AH))))); -- AH(x) :- R(x,y,z), AH(y), AH(z)
  EHNext := (A ∪ (π[0] (σ[#2 = #4] (σ[#1 = #3] ((R × OH) × OH))))); -- EH(x) :- A(x). EH(x) :- R(x,y,z), OH(y), OH(z)
  OHNext := (π[0] (σ[#2 = #4] (σ[#1 = #3] ((R × EH) × EH))))); -- OH(x) :- R(x,y,z), EH(y), EH(z)
  WHILE ¬((AHNext = AH) ∧ (EHNext = EH) ∧ (OHNext = OH)) DO
    AH := AHNext;
    EH := EHNext;
    OH := OHNext;
    AHNext :=
      ((((((π[0] (σ[#2 = #4] (σ[#1 = #3] ((R × EH) × OH)))) -- AH(x) :- R(x,y,z), EH(y), OH(z)
      ∪ (π[0] (σ[#2 = #4] (σ[#1 = #3] ((R × OH) × EH)))) -- AH(x) :- R(x,y,z), OH(y), EH(z)
      ∪ (π[0] (σ[#2 = #4] (σ[#1 = #3] ((R × AH) × OH)))) -- AH(x) :- R(x,y,z), AH(y), OH(z)
      ∪ (π[0] (σ[#2 = #4] (σ[#1 = #3] ((R × OH) × AH)))) -- AH(x) :- R(x,y,z), OH(y), AH(z)
      ∪ (π[0] (σ[#2 = #4] (σ[#1 = #3] ((R × AH) × EH)))) -- AH(x) :- R(x,y,z), AH(y), EH(z)
      ∪ (π[0] (σ[#2 = #4] (σ[#1 = #3] ((R × EH) × AH)))) -- AH(x) :- R(x,y,z), EH(y), AH(z)
      ∪ (π[0] (σ[#2 = #4] (σ[#1 = #3] ((R × AH) × AH))))); -- AH(x) :- R(x,y,z), AH(y), AH(z)
    EHNext := (A ∪ (π[0] (σ[#2 = #4] (σ[#1 = #3] ((R × OH) × OH))))); -- EH(x) :- A(x). EH(x) :- R(x,y,z), OH(y), OH(z)
    OHNext := (π[0] (σ[#2 = #4] (σ[#1 = #3] ((R × EH) × EH))))); -- OH(x) :- R(x,y,z), EH(y), EH(z)
  END

POST
  ((AH ⊆ Closed) ∧ (EH ⊆ Closed) ∧ (OH ⊆ Closed))

```

16 Example0132: Odd and even paths: linear versus non-linear program (two compiled loops)

Category	synthetic (syntactic generation from the classic Datalog programs); family: Obstructed.
Kind	equivalence.
Contributors	Fangzhu Shen.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	<code>linear-vs-nonlinear</code> <code>mutual-recursion</code> <code>nonlinear-recursion</code> <code>prophecy</code> .
Input	Benchmark/Example0132/Input.lean (canonical notation).
Summary	Odd and even paths, computed twice: by a linear program and by a non-linear one.
Sources	legacy source record v2__equiv_pairs__even_odd__linear_vs_nonlinear__naive of the retired classic-source collection classic Datalog program even_odd_linear of the retired generator suite classic Datalog program even_odd_nonlinear of the retired generator suite
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).

Datalog program(s). *programX — Odd and even paths, linearly.* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
TbX(x1, y1) :- E(x1, y1).
TbX(x1, y1) :- E(x1, z1), TaX(z1, y1).
TaX(x1, y1) :- E(x1, z1), TbX(z1, y1).
```

programY — Odd and even paths, non-linearly. — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
TaY(x1, y1) :- E(x1, z1), E(z1, y1).
TaY(x1, y1) :- TaY(x1, z1), TaY(z1, y1).
TaY(x1, y1) :- TbY(x1, z1), TbY(z1, y1).
TbY(x1, y1) :- E(x1, y1).
TbY(x1, y1) :- TbY(x1, z1), TaY(z1, y1).
TbY(x1, y1) :- TaY(x1, z1), TbY(z1, y1).
```

Reading. The command runs two independently compiled Datalog programs in sequence — a linear even/odd program producing TaX (even-length paths) and TbX (odd-length paths) via naive accumulation, and a non-linear even/odd program producing TaY and TbY via squaring — and the tool's preprocessor merges the two into a single verified loop. The postcondition states that the two computations agree relation by relation, TaX=TaY and TbX=TbY, i.e. the Hoare triple is an equivalence claim between the linear and non-linear encodings of the even/odd path decomposition, both of which converge to the same least fixpoints since the underlying immediate-consequence operators are monotone.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in Input.lean remains the authority.

PRE: $\top$

The precondition is simply true, imposing no constraint on the input relations.

POST: $(\forall x y. (TaX(x,y) \leftrightarrow TaY(x,y))) \wedge \forall z u. (TbX(z,u) \leftrightarrow TbY(z,u))$

The postcondition asserts that the linear program's outputs equal the non-linear program's outputs: TaX = TaY and TbX = TbY.

Whiel encoding (Hoare triple).

```

SCHEMA
E/2, TaX/2, TbX/2, TaY/2, TbY/2, TaX_aux/2, TbX_aux/2, TaY_aux/2, TbY_aux/2

PRE
true

CMD
TbX_aux := {}[2];
TaX_aux := {}[2];
TbX :=
  (E  $\cup$  ( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (E  $\times$  TaX_aux)))));
TaX := ( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (E  $\times$  TbX_aux)));
WHILE
  ( $\neg$  ((TbX = TbX_aux)  $\wedge$  (TaX = TaX_aux))) DO
    TbX_aux := TbX;
    TaX_aux := TaX;
    TbX :=
      (TbX  $\cup$ 
        (E  $\cup$ 
          ( $\pi$ [0, 3]
            ( $\sigma$ [#1 = #2] (E  $\times$  TaX_aux)))));
    TaX :=
      (TaX  $\cup$ 
        ( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (E  $\times$  TbX_aux)))));
END;
TaY_aux := {}[2];
TbY_aux := {}[2];
TaY :=
  (( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (E  $\times$  E)))  $\cup$ 
   ( $\pi$ [0, 3]
    ( $\sigma$ [#1 = #2] (TaY_aux  $\times$  TaY_aux)))  $\cup$ 
   ( $\pi$ [0, 3]
    ( $\sigma$ [#1 = #2] (TbY_aux  $\times$  TbY_aux)))));
TbY :=
  (E  $\cup$ 
   (( $\pi$ [0, 3]
    ( $\sigma$ [#1 = #2] (TbY_aux  $\times$  TaY_aux)))  $\cup$ 
    ( $\pi$ [0, 3]
    ( $\sigma$ [#1 = #2] (TaY_aux  $\times$  TbY_aux)))));
WHILE
  ( $\neg$  ((TaY = TaY_aux)  $\wedge$  (TbY = TbY_aux))) DO
    TaY_aux := TaY;
    TbY_aux := TbY;
    TaY :=
      (TaY  $\cup$ 
        (( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (E  $\times$  E)))  $\cup$ 
         ( $\pi$ [0, 3]
          ( $\sigma$ [#1 = #2]
            (TaY_aux  $\times$  TaY_aux)))  $\cup$ 
          ( $\pi$ [0, 3]
            ( $\sigma$ [#1 = #2]
              (TbY_aux  $\times$  TbY_aux))))));
    TbY :=
      (TbY  $\cup$ 
        (E  $\cup$ 
          (( $\pi$ [0, 3]
            ( $\sigma$ [#1 = #2]
              (TbY_aux  $\times$  TaY_aux)))  $\cup$ 
            ( $\pi$ [0, 3]
              ( $\sigma$ [#1 = #2]
                (TaY_aux  $\times$  TbY_aux))))));
END

POST
((TaX = TaY)  $\wedge$  (TbX = TbY))

```

17 Example0133: All paths via odd/even paths: linear versus non-linear program (two compiled loops)

Category	synthetic (syntactic generation from the classic Datalog programs); family: Obstructed.
Kind	equivalence.
Contributors	Fangzhu Shen.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	<code>linear-vs-nonlinear</code> <code>mutual-recursion</code> <code>nonlinear-recursion</code> <code>prophecy</code> .
Input	Benchmark/Example0133/Input.lean (canonical notation).
Summary	All paths through odd and even halves, computed twice: by a linear program and by a non-linear one.
Sources	legacy source record v2__equiv_pairs__all_paths_via_even_odd__linear_vs_nonlinear__naive of the retired classic-source collection classic Datalog program all_paths_via_even_odd of the retired generator suite classic Datalog program all_paths_via_even_odd_nonlinear of the retired generator suite
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).
Note	Kept after the duplicate review at Leo's decision: it is 0132 plus the union outputs and an implied conjunct, but Val's Part 1 review kept both and the all-paths program is a classic program in its own right.

Datalog program(s). *programX — Odd and even paths and their union, linearly.* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
TbX(x1, y1) :- E(x1, y1).
TbX(x1, y1) :- E(x1, z1), TaX(z1, y1).
TaX(x1, y1) :- E(x1, z1), TbX(z1, y1).
TcX(x1, y1) :- TaX(x1, y1).
TcX(x1, y1) :- TbX(x1, y1).
```

programY — Odd and even paths and their union, non-linearly. — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
TaY(x1, y1) :- E(x1, z1), E(z1, y1).
TaY(x1, y1) :- TaY(x1, z1), TaY(z1, y1).
TaY(x1, y1) :- TbY(x1, z1), TbY(z1, y1).
TbY(x1, y1) :- E(x1, y1).
TbY(x1, y1) :- TbY(x1, z1), TaY(z1, y1).
TbY(x1, y1) :- TaY(x1, z1), TbY(z1, y1).
TcY(x1, y1) :- TaY(x1, y1).
TcY(x1, y1) :- TbY(x1, y1).
```

Reading. The command runs two independently compiled Datalog programs in sequence — a linear even/odd program producing TaX, TbX and their union TcX, and a non-linear even/odd program producing TaY, TbY and their union TcY — merged by the preprocessor into a single verified loop. The postcondition states that all three pairs of relations agree, TaX=TaY, TbX=TbY and TcX=TcY, so the Hoare triple is an equivalence claim between the linear and non-linear implementations of the even/odd path decomposition and its union (all paths of length one or more), both reaching the same least fixpoints.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\top$

The precondition is simply true, imposing no constraint on the input relations.

POST: $(\forall x y. (TaX(x,y) \leftrightarrow TaY(x,y))) \wedge (\forall z u. (TbX(z,u) \leftrightarrow TbY(z,u))) \wedge \forall v w. (TcX(v,w) \leftrightarrow TcY(v,w))$

The postcondition asserts that the linear program's three outputs equal the non-linear program's corresponding outputs: $TaX = TaY$, $TbX = TbY$, and $TcX = TcY$.

Whiel encoding (Hoare triple).

```

SCHEMA
  E/2, TaX/2, TbX/2, TcX/2, TaY/2, TbY/2, TcY/2, TaX_aux/2, TbX_aux/2, TcX_aux/2, TaY_aux/2, TbY_aux/2, TcY_aux/2

PRE
  true

CMD
  TbX_aux := {}[2];
  TaX_aux := {}[2];
  TcX_aux := {}[2];
  TbX :=
    (E ∪ (π[0, 3] (σ[#1 = #2] (E × TaX_aux)))));
  TaX := (π[0, 3] (σ[#1 = #2] (E × TbX_aux)));
  TcX := (TaX_aux ∪ TbX_aux);
  WHILE
    (¬ ((TbX = TbX_aux) ∧
      (TaX = TaX_aux) ∧ (TcX = TcX_aux))) DO
    TbX_aux := TbX;
    TaX_aux := TaX;
    TcX_aux := TcX;
    TbX :=
      (TbX ∪
       (E ∪
        (π[0, 3]
         (σ[#1 = #2] (E × TaX_aux)))));
    TaX :=
      (TaX ∪
       (π[0, 3] (σ[#1 = #2] (E × TbX_aux))));
    TcX := (TcX ∪ (TaX_aux ∪ TbX_aux));
  END;
  TaY_aux := {}[2];
  TbY_aux := {}[2];
  TcY_aux := {}[2];
  TaY :=
    ((π[0, 3] (σ[#1 = #2] (E × E))) ∪
     (π[0, 3]
      (σ[#1 = #2] (TaY_aux × TaY_aux))) ∪
     (π[0, 3]
      (σ[#1 = #2] (TbY_aux × TbY_aux))));
  TbY :=
    (E ∪
     (π[0, 3]
      (σ[#1 = #2] (TbY_aux × TaY_aux))) ∪
     (π[0, 3]
      (σ[#1 = #2] (TaY_aux × TbY_aux))));
  TcY := (TaY_aux ∪ TbY_aux);
  WHILE
    (¬ ((TaY = TaY_aux) ∧
      (TbY = TbY_aux) ∧ (TcY = TcY_aux))) DO
    TaY_aux := TaY;
    TbY_aux := TbY;
    TcY_aux := TcY;
    TaY :=
      (TaY ∪
       ((π[0, 3] (σ[#1 = #2] (E × E))) ∪
        (π[0, 3]
         (σ[#1 = #2]
          (TaY_aux × TaY_aux))) ∪
         (π[0, 3]
          (σ[#1 = #2]
           (TbY_aux × TbY_aux))))));
    TbY :=
      (TbY ∪
       (E ∪
        (π[0, 3]
         (σ[#1 = #2]
          (TbY_aux × TaY_aux))) ∪
         (π[0, 3]
          (σ[#1 = #2]
           (TaY_aux × TbY_aux))))));
    TcY := (TcY ∪ (TaY_aux ∪ TbY_aux));
  END
  POST
    ((TaX = TaY) ∧ ((TbX = TbY) ∧ (TcX = TcY)))

```

18 Example0134: Red/blue alternating paths: linear versus non-linear program (two compiled loops)

Category	synthetic (syntactic generation from the classic Datalog programs); family: Obstructed.
Kind	equivalence.
Contributors	Fangzhu Shen.
Status	certified valid (certificate checked outside the library build) (Certificate/Valid.lean, written by the certificate emitter); expected valid. Placement note: a proof module imported by <code>Certificate/Valid.lean</code> does not build under the library build's 4194304 KB limit; it builds under 12582912 KB (peak 1705 MB)
Tags	<code>linear-vs-nonlinear</code> <code>mutual-recursion</code> <code>nonlinear-recursion</code> <code>prophecy</code> .
Input	<code>Benchmark/Example0134/Input.lean</code> (canonical notation).
Summary	Alternating red and blue walks, computed twice: by a linear program and by a non-linear one.
Sources	legacy source record <code>v2__equiv_pairs__red_blue__linear_vs_nonlinear__naive</code> of the retired classic-source collection classic Datalog program <code>red_blue_linear</code> of the retired generator suite classic Datalog program <code>red_blue_nonlinear</code> of the retired generator suite
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).

Datalog program(s). *programX* — *Colour-alternating walks, linearly.* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
TaX(x1, y1) :- B(x1, y1).
TaX(x1, y1) :- B(x1, z1), TcX(z1, y1).
TbX(x1, y1) :- TaX(x1, z1), R(z1, y1).
TcX(x1, y1) :- TdX(x1, z1), B(z1, y1).
TdX(x1, y1) :- R(x1, y1).
TdX(x1, y1) :- R(x1, z1), TbX(z1, y1).
```

programY — *Colour-alternating walks, non-linearly.* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
TaY(x1, y1) :- B(x1, y1).
TaY(x1, y1) :- TaY(x1, z1), TcY(z1, y1).
TbY(x1, y1) :- B(x1, z1), R(z1, y1).
TbY(x1, y1) :- TaY(x1, z1), TdY(z1, y1).
TcY(x1, y1) :- R(x1, z1), B(z1, y1).
TcY(x1, y1) :- TdY(x1, z1), TaY(z1, y1).
TdY(x1, y1) :- R(x1, y1).
TdY(x1, y1) :- TdY(x1, z1), TbY(z1, y1).
```

Reading. The command runs two independently compiled Datalog programs in sequence, each computing the four families of colour-alternating walks in a red/blue-edged graph (walks that start and end blue, start blue end red, start red end blue, and start and end red) — one program linear, the other non-linear — merged by the preprocessor into a single verified loop. The postcondition states that all four output relations from the linear program equal the corresponding relations from the non-linear program, so the Hoare triple is an equivalence claim between two implementations of colour-alternating reachability, both converging to the same least fixpoints.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\top$

The precondition is simply true, imposing no constraint on the input relations.

POST: $(\forall x \ y. (TaX(x,y) \leftrightarrow TaY(x,y))) \wedge (\forall z \ u. (TbX(z,u) \leftrightarrow TbY(z,u))) \wedge (\forall v \ w. (TcX(v,w) \leftrightarrow TcY(v,w))) \wedge \forall x1 \ x2. (TdX(x1,x2) \leftrightarrow TdY(x1,x2))$

The postcondition asserts that all four linear-program outputs equal the corresponding non-linear-program outputs: $TaX=TaY$, $TbX=TbY$, $TcX=TcY$, and $TdX=TdY$.

Whiel encoding (Hoare triple).

```

SCHEMA
  B/2, R/2, TaX/2, TbX/2, TcX/2, TdX/2, TaY/2, TbY/2, TcY/2, TdY/2, TaX_aux/2, TbX_aux/2, TcX_aux/2, TdX_aux/2, TaY_aux/2, TbY_aux/2, TcY_aux/2, TdY_aux/2

PRE
  true

CMD
  TaX_aux := {}[2];
  TbX_aux := {}[2];
  TcX_aux := {}[2];
  TdX_aux := {}[2];
  TaX :=
    (B  $\cup$  ( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (B  $\times$  TcX_aux))));
  TbX := ( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (TaX_aux  $\times$  R)));
  TcX := ( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (TdX_aux  $\times$  B)));
  TdX :=
    (R  $\cup$  ( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (R  $\times$  TbX_aux))));
  WHILE
    ( $\neg$  ((TaX = TaX_aux)  $\wedge$ 
      ((TbX = TbX_aux)  $\wedge$ 
        ((TcX = TcX_aux)  $\wedge$ 
          (TdX = TdX_aux))))) DO
    TaX_aux := TaX;
    TbX_aux := TbX;
    TcX_aux := TcX;
    TdX_aux := TdX;
    TaX :=
      (TaX  $\cup$ 
        (B  $\cup$ 
          ( $\pi$ [0, 3]
            ( $\sigma$ [#1 = #2] (B  $\times$  TcX_aux))));
    TbX :=
      (TbX  $\cup$ 
        ( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (TaX_aux  $\times$  R))));
    TcX :=
      (TcX  $\cup$ 
        ( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (TdX_aux  $\times$  B))));
    TdX :=
      (TdX  $\cup$ 
        (R  $\cup$ 
          ( $\pi$ [0, 3]
            ( $\sigma$ [#1 = #2] (R  $\times$  TbX_aux))));
  END;
  TaY_aux := {}[2];
  TbY_aux := {}[2];
  TcY_aux := {}[2];
  TdY_aux := {}[2];
  TaY :=
    (B  $\cup$ 
      ( $\pi$ [0, 3]
        ( $\sigma$ [#1 = #2] (TaY_aux  $\times$  TcY_aux))));
  TbY :=
    (( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (B  $\times$  R)))  $\cup$ 
      ( $\pi$ [0, 3]
        ( $\sigma$ [#1 = #2] (TaY_aux  $\times$  TdY_aux))));
  TcY :=
    (( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (R  $\times$  B)))  $\cup$ 
      ( $\pi$ [0, 3]
        ( $\sigma$ [#1 = #2] (TdY_aux  $\times$  TaY_aux))));
  TdY :=
    (R  $\cup$ 
      ( $\pi$ [0, 3]
        ( $\sigma$ [#1 = #2] (TdY_aux  $\times$  TbY_aux))));
  WHILE
    ( $\neg$  ((TaY = TaY_aux)  $\wedge$ 
      ((TbY = TbY_aux)  $\wedge$ 
        ((TcY = TcY_aux)  $\wedge$ 
          (TdY = TdY_aux))))) DO
    TaY_aux := TaY;
    TbY_aux := TbY;
    TcY_aux := TcY;
    TdY_aux := TdY;
    TaY :=
      (TaY  $\cup$ 
        (B  $\cup$ 
          ( $\pi$ [0, 3]
            ( $\sigma$ [#1 = #2]
              (TaY_aux  $\times$  TcY_aux))));
    TbY :=
      (TbY  $\cup$ 
        (( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (B  $\times$  R)))  $\cup$ 
          ( $\pi$ [0, 3]
            ( $\sigma$ [#1 = #2]
              (TaY_aux  $\times$  TdY_aux))));
    TcY :=
      (TcY  $\cup$ 
        (( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (R  $\times$  B)))  $\cup$ 
          ( $\pi$ [0, 3]
            ( $\sigma$ [#1 = #2]
              (TdY_aux  $\times$  TaY_aux))));
    TdY :=
      (TdY  $\cup$ 
        (R  $\cup$ 
          ( $\pi$ [0, 3]
            ( $\sigma$ [#1 = #2]
              (TdY_aux  $\times$  TbY_aux))));
  END
POST
  ((TaX = TaY)  $\wedge$ 
    ((TbX = TbY)  $\wedge$ 
      ((TcX = TcY)  $\wedge$ 
        (TdX = TdY)))

```

$$((\text{TxX} = \text{TbY}) \wedge ((\text{TcX} = \text{TcY}) \wedge (\text{TdX} = \text{TdY})))$$

19 Example0136: Right-linear transitive closure versus the non-linear odd/even path program (two compiled loops)

Category	synthetic (syntactic generation from the classic Datalog programs); family: Obstructed.
Kind	equivalence.
Contributors	Fangzhu Shen.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	<code>linear-vs-nonlinear</code> <code>mutual-recursion</code> <code>nonlinear-recursion</code> <code>prophecy</code> .
Input	Benchmark/Example0136/Input.lean (canonical notation).
Summary	Transitive closure computed right-linearly, against all paths computed through non-linear odd and even halves.
Sources	legacy source record v2__equiv_pairs__tc_left_vs_all_paths_via_eo_nonlinear__naive of the retired classic-source collection
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).

Datalog program(s). *programX* — *Transitive closure, right-linearly.* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
TX(x1, y1) :- E(x1, y1).
TX(x1, y1) :- E(x1, z1), TX(z1, y1).
```

programY — *Odd and even paths and their union, non-linearly.* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
TaY(x1, y1) :- E(x1, z1), E(z1, y1).
TaY(x1, y1) :- TaY(x1, z1), TaY(z1, y1).
TaY(x1, y1) :- TbY(x1, z1), TbY(z1, y1).
TbY(x1, y1) :- E(x1, y1).
TbY(x1, y1) :- TbY(x1, z1), TaY(z1, y1).
TbY(x1, y1) :- TaY(x1, z1), TbY(z1, y1).
TcY(x1, y1) :- TaY(x1, y1).
TcY(x1, y1) :- TbY(x1, y1).
```

Reading. The command runs two independently compiled Datalog programs in sequence — a right-linear transitive-closure program producing TX, and a non-linear even/odd program producing TaY, TbY and their union TcY — merged by the preprocessor into a single verified loop. The postcondition TX=TcY is an equivalence claim: the ordinary transitive closure of E equals the union of the even-length and odd-length path relations, since every nonempty path has odd or even length.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in Input.lean remains the authority.

PRE: $\top$

The precondition is simply true, imposing no constraint on the input relations.

POST: $\forall x y. (\text{TX}(x,y) \leftrightarrow \text{TcY}(x,y))$

The postcondition asserts that the transitive closure TX computed by the first program equals TcY, the union of even- and odd-length paths computed by the second.

Whiel encoding (Hoare triple).

```

SCHEMA
  E/2, TX/2, TaY/2, TbY/2, TcY/2, TX_aux/2, TaY_aux/2, TbY_aux/2, TcY_aux/2

PRE
  true

CMD
  TX_aux :=  $\emptyset[2]$ ;
  TX :=
    ( $E \cup (\pi[0, 3] (\sigma[\#1 = \#2] (E \times TX\_aux))))$ );
  WHILE ( $\neg (TX = TX\_aux)$ ) DO
    TX_aux := TX;
    TX :=
      (TX  $\cup$ 
       ( $E \cup$ 
        ( $\pi[0, 3] (\sigma[\#1 = \#2] (E \times TX\_aux))))$ ))
  END;
  TaY_aux :=  $\emptyset[2]$ ;
  TbY_aux :=  $\emptyset[2]$ ;
  TcY_aux :=  $\emptyset[2]$ ;
  TaY :=
    (( $\pi[0, 3] (\sigma[\#1 = \#2] (E \times E))$ )  $\cup$ 
     ( $\pi[0, 3] (\sigma[\#1 = \#2] (TaY\_aux \times TaY\_aux))$ )  $\cup$ 
     ( $\pi[0, 3] (\sigma[\#1 = \#2] (TbY\_aux \times TbY\_aux))$ )));
  TbY :=
    ( $E \cup$ 
     ( $\pi[0, 3] (\sigma[\#1 = \#2] (TbY\_aux \times TaY\_aux))$ )  $\cup$ 
     ( $\pi[0, 3] (\sigma[\#1 = \#2] (TaY\_aux \times TbY\_aux))$ )));
  TcY := (TaY_aux  $\cup$  TbY_aux);
  WHILE
    ( $\neg ((TaY = TaY\_aux) \wedge (TbY = TbY\_aux) \wedge (TcY = TcY\_aux))$ ) DO
    TaY_aux := TaY;
    TbY_aux := TbY;
    TcY_aux := TcY;
    TaY :=
      (TaY  $\cup$ 
       ( $\pi[0, 3] (\sigma[\#1 = \#2] (E \times E))$ )  $\cup$ 
       ( $\pi[0, 3] (\sigma[\#1 = \#2] (TaY\_aux \times TaY\_aux))$ )  $\cup$ 
       ( $\pi[0, 3] (\sigma[\#1 = \#2] (TbY\_aux \times TbY\_aux))$ )));
    TbY :=
      (TbY  $\cup$ 
       ( $E \cup$ 
        ( $\pi[0, 3] (\sigma[\#1 = \#2] (TbY\_aux \times TaY\_aux))$ )  $\cup$ 
        ( $\pi[0, 3] (\sigma[\#1 = \#2] (TaY\_aux \times TbY\_aux))$ ))));
    TcY := (TcY  $\cup$  (TaY_aux  $\cup$  TbY_aux))
  END

POST
  (TX = TcY)

```

20 Example0137: Non-linear transitive closure versus the linear odd/even path program (two compiled loops)

Category	synthetic (syntactic generation from the classic Datalog programs); family: Obstructed.
Kind	equivalence.
Contributors	Fangzhu Shen.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	<code>linear-vs-nonlinear</code> <code>mutual-recursion</code> <code>nonlinear-recursion</code> <code>prophecy</code> .
Input	<code>Benchmark/Example0137/Input.lean</code> (canonical notation).
Summary	Transitive closure computed non-linearly, against all paths computed through linear odd and even halves.
Sources	legacy source record <code>v2__equiv_pairs__tc_nonlin_vs_all_paths_via_eo_linear__naive</code> of the retired classic-source collection classic Datalog program <code>all_paths_via_even_odd</code> of the retired generator suite classic Datalog program <code>tc-nonlinear</code> of the retired generator suite
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).

Datalog program(s). *programX* — *Transitive closure, non-linearly.* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
TX(x1, y1) :- E(x1, y1).
TX(x1, y1) :- TX(x1, z1), TX(z1, y1).
```

programY — *Odd and even paths and their union, linearly.* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
TbY(x1, y1) :- E(x1, y1).
TbY(x1, y1) :- E(x1, z1), TaY(z1, y1).
TaY(x1, y1) :- E(x1, z1), TbY(z1, y1).
TcY(x1, y1) :- TaY(x1, y1).
TcY(x1, y1) :- TbY(x1, y1).
```

Reading. The command runs two independently compiled Datalog programs in sequence — a non-linear transitive-closure program producing TX by squaring, and a linear even/odd program producing TaY, TbY and their union TcY — merged by the preprocessor into a single verified loop. The postcondition TX=TcY is an equivalence claim: the non-linearly computed transitive closure of E equals the union of the even-length and odd-length path relations computed linearly, since every nonempty path has odd or even length.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\top$

The precondition is simply true, imposing no constraint on the input relations.

POST: $\forall x y. (TX(x,y) \leftrightarrow TcY(x,y))$

The postcondition asserts that the transitive closure TX computed by the first program equals TcY, the union of even- and odd-length paths computed by the second.

Whiel encoding (Hoare triple).

```

SCHEMA
E/2, TX/2, TaY/2, TbY/2, TcY/2, TX_aux/2, TaY_aux/2, TbY_aux/2, TcY_aux/2

PRE
true

CMD
TX_aux :=  $\emptyset[2]$ ;
TX :=
  (E  $\cup$ 
   ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  ( $TX\_aux \times TX\_aux$ )))));
WHILE ( $\neg$  (TX = TX_aux)) DO
  TX_aux := TX;
  TX :=
    (TX  $\cup$ 
     (E  $\cup$ 
      ( $\pi[0, 3]$ 
       ( $\sigma[\#1 = \#2]$  ( $TX\_aux \times TX\_aux$ ))))));
END;
TbY_aux :=  $\emptyset[2]$ ;
TaY_aux :=  $\emptyset[2]$ ;
TcY_aux :=  $\emptyset[2]$ ;
TbY :=
  (E  $\cup$  ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (E  $\times$  TaY_aux)))));
TaY := ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (E  $\times$  TbY_aux)));
TcY := (TaY_aux  $\cup$  TbY_aux);
WHILE
  ( $\neg$  ((TbY = TbY_aux)  $\wedge$ 
        (TaY = TaY_aux)  $\wedge$  (TcY = TcY_aux))) DO
  TbY_aux := TbY;
  TaY_aux := TaY;
  TcY_aux := TcY;
  TbY :=
    (TbY  $\cup$ 
     (E  $\cup$ 
      ( $\pi[0, 3]$ 
       ( $\sigma[\#1 = \#2]$  (E  $\times$  TaY_aux))))));
  TaY :=
    (TaY  $\cup$ 
     ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (E  $\times$  TbY_aux))));
  TcY := (TcY  $\cup$  (TaY_aux  $\cup$  TbY_aux))
END
POST
(TX = TcY)

```

21 Example0138: Joint-loop equivalence: tc_nonlinear vs all_paths_via_even_odd (non-linear, naive)

Category	synthetic (syntactic generation from the classic Datalog programs); family: Synthetic.
Kind	equivalence.
Contributors	Fangzhu Shen, Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	mutual-recursion nonlinear-recursion prophecy .
Input	Benchmark/Example0138/Input.lean (canonical notation).
Summary	Transitive closure and the union of odd and even paths, both computed non-linearly, compared.
Sources	legacy source record v2__equiv_pairs__tc_nonlin_vs_all_paths_via_eo_nonlinear__naive of the retired classic-source collection classic Datalog program all_paths_via_even_odd_nonlinear of the retired generator suite classic Datalog program tc-nonlinear of the retired generator suite
Prophecy	accepted invariant with a prophecy clause above level zero (Core.json); no certificate built from it yet.

Datalog program(s). *programX* — *Transitive closure, non-linearly.* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```

TX(x1, y1) :- E(x1, y1).
TX(x1, y1) :- TX(x1, z1), TX(z1, y1).

```

programY — *Odd and even paths and their union, non-linearly.* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
TaY(x1, y1) :- E(x1, z1), E(z1, y1).
TaY(x1, y1) :- TaY(x1, z1), TaY(z1, y1).
TaY(x1, y1) :- TbY(x1, z1), TbY(z1, y1).
TbY(x1, y1) :- E(x1, y1).
TbY(x1, y1) :- TbY(x1, z1), TaY(z1, y1).
TbY(x1, y1) :- TaY(x1, z1), TbY(z1, y1).
TcY(x1, y1) :- TaY(x1, y1).
TcY(x1, y1) :- TbY(x1, y1).
```

Reading. The command runs two independently compiled Datalog programs in sequence — a non-linear transitive-closure program producing TX by squaring, and a non-linear even/odd program producing TaY, TbY and their union TcY, also by squaring — merged by the preprocessor into a single verified loop. The postcondition TX=TcY is an equivalence claim: the transitive closure of E equals the union of the even-length and odd-length path relations, both computed non-linearly, since every nonempty path has odd or even length; the header notes this re-encodes a retired hand-written lockstep-loop version of the same case.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\top$

The precondition is simply true, imposing no constraint on the input relations.

POST: $\forall x y. (TX(x,y) \leftrightarrow TcY(x,y))$

The postcondition asserts that the transitive closure TX computed by the first program equals TcY, the union of even- and odd-length paths computed by the second.

Whiel encoding (Hoare triple).

```
SCHEMA
  E/2, TX/2, TaY/2, TbY/2, TcY/2, TX_aux/2, TaY_aux/2, TbY_aux/2, TcY_aux/2

PRE
  true

CMD
  TX_aux :=  $\emptyset[2]$ ;
  TX := (E  $\cup$   $\pi[0,3]$  ( $\sigma[\#1 = \#2]$  ((TX_aux  $\times$  TX_aux))));
  WHILE ( $\neg$ ((TX = TX_aux))) DO
    TX_aux := TX;
    TX := (TX  $\cup$  (E  $\cup$   $\pi[0,3]$  ( $\sigma[\#1 = \#2]$  ((TX_aux  $\times$  TX_aux))));
  END;
  TaY_aux :=  $\emptyset[2]$ ;
  TbY_aux :=  $\emptyset[2]$ ;
  TcY_aux :=  $\emptyset[2]$ ;
  TaY := ( $\pi[0,3]$  ( $\sigma[\#1 = \#2]$  ((E  $\times$  E)))  $\cup$  ( $\pi[0,3]$  ( $\sigma[\#1 = \#2]$  ((TaY_aux  $\times$  TaY_aux)))  $\cup$   $\pi[0,3]$  ( $\sigma[\#1 = \#2]$  ((TbY_aux  $\times$  TbY_aux))));
  TbY := (E  $\cup$  ( $\pi[0,3]$  ( $\sigma[\#1 = \#2]$  ((TbY_aux  $\times$  TaY_aux)))  $\cup$   $\pi[0,3]$  ( $\sigma[\#1 = \#2]$  ((TaY_aux  $\times$  TbY_aux))));
  TcY := (TaY  $\cup$  TbY);
  WHILE ( $\neg$ ((TaY = TaY_aux)  $\wedge$  ((TbY = TbY_aux)  $\wedge$  (TcY = TcY_aux)))) DO
    TaY_aux := TaY;
    TbY_aux := TbY;
    TcY_aux := TcY;
    TaY := (TaY  $\cup$  ( $\pi[0,3]$  ( $\sigma[\#1 = \#2]$  ((E  $\times$  E)))  $\cup$  ( $\pi[0,3]$  ( $\sigma[\#1 = \#2]$  ((TaY_aux  $\times$  TaY_aux)))  $\cup$   $\pi[0,3]$  ( $\sigma[\#1 = \#2]$  ((TbY_aux  $\times$  TbY_aux))));
    TbY := (TbY  $\cup$  (E  $\cup$  ( $\pi[0,3]$  ( $\sigma[\#1 = \#2]$  ((TbY_aux  $\times$  TaY_aux)))  $\cup$   $\pi[0,3]$  ( $\sigma[\#1 = \#2]$  ((TaY_aux  $\times$  TbY_aux))));
    TcY := (TcY  $\cup$  (TaY_aux  $\cup$  TbY_aux));
  END;

POST
  (TX = TcY)
```

22 Example0162: Two-player game reachability mixing existential and universal successor steps

Category	synthetic (syntactic generation from the classic Datalog programs); family: Synthetic.
Kind	equivalence (with a least fixpoint, via a pre-fixpoint witness).
Contributors	Fangzhu Shen, Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	fixpoint-witness negation.
Input	Benchmark/Example0162/Input.lean (canonical notation).
Summary	Canonical form of the earlier single-level encoding of this case: the same schema, precondition, command and postcondition.
Sources	legacy source record hard-game-reach of the retired classic-source collection

Datalog program(s). *Datalog reading of the game reachability (written for this report; the operator is monotone, T is its least fixpoint; the universal step is spelled with a negated escape relation) — hand-written reading (this report).*

```

T(x) :- Target(x).
T(x) :- Pa(x), E(x, y), T(y).           % player-A vertex with some successor in T
T(x) :- V(x), not Pa(x), not Escape(x). % player-B vertex all of whose successors are in T
Escape(x) :- E(x, y), not T(y).

```

Reading. The loop computes the attractor set of a two-player reachability game on a graph E : each vertex belongs to the existential player (the Pa vertices) or the universal player (V minus Pa). A Pa -vertex is winning if it has some successor already known to be winning; a non- Pa vertex in V is winning if all of its successors that lie in V are already winning (so it has no escape from the winning set). T is the least fixpoint of this one-step operator starting from $Target$: the set of positions from which the existential player can force play to reach $Target$. The precondition says Rc contains $Target$ and is closed under the same one-step operator (a pre-fixpoint); the postcondition says T is itself a fixpoint of that operator and is contained in Rc — the standard least-fixpoint-correctness statement, with Rc a universally quantified dummy standing in for every pre-fixpoint.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $(\forall x. (Target(x) \vee Pa(x) \wedge (\exists y. E(x,y) \wedge Rc(y)) \vee V(x) \wedge \neg Pa(x) \wedge (\neg \exists z. E(x,z) \wedge V(z) \wedge \neg Rc(z)) \rightarrow Rc(x))) \wedge \forall u. (Target(u) \rightarrow Rc(u))$

PRE says Rc contains every $Target$ vertex and is closed under the attractor step: any Pa -vertex with an Rc -winning successor, or any non- Pa V -vertex all of whose V -successors are Rc -winning, is itself in Rc .

POST: $(\forall x. (T(x) \leftrightarrow Target(x) \vee Pa(x) \wedge (\exists y. E(x,y) \wedge T(y)) \vee V(x) \wedge \neg Pa(x) \wedge \neg \exists z. E(x,z) \wedge V(z) \wedge \neg T(z))) \wedge \forall u. (T(u) \rightarrow Rc(u))$

POST says T equals the result of one attractor step applied to T itself (T is a fixpoint of the operator), and that T is contained in Rc .

Whiel encoding (Hoare triple).

```

SCHEMA
  Target/1, V/1, Pa/1, Rc/1, T/1, S/1, E/2

PRE
  (((((Target  $\cup$  ( $\pi$ [0] ( $\sigma$ [#0 = #1] (Pa  $\times$  ( $\pi$ [0] ( $\sigma$ [#1 = #2] (E  $\times$  Rc))))))  $\cup$  ((V  $\setminus$  Pa)  $\setminus$  ( $\pi$ [0] ( $\sigma$ [#1 = #2] (E  $\times$  (V  $\setminus$  Rc))))))  $\subseteq$ 
 $\hookrightarrow$  Rc)  $\wedge$  (Target  $\subseteq$  Rc))

CMD

```

```

T := ∅;
S := ((Target ∪ (π[0] (σ[#0 = #1] (Pa × (π[0] (σ[#1 = #2] (E × T)))))) ∪ ((V \ Pa) \ (π[0] (σ[#1 = #2] (E × (V \ T))))));
WHILE (S ≠ T) DO
  T := S;
  S := ((Target ∪ (π[0] (σ[#0 = #1] (Pa × (π[0] (σ[#1 = #2] (E × T)))))) ∪ ((V \ Pa) \ (π[0] (σ[#1 = #2] (E × (V \ T))))));
END

POST
((T = ((Target ∪ (π[0] (σ[#0 = #1] (Pa × (π[0] (σ[#1 = #2] (E × T)))))) ∪ ((V \ Pa) \ (π[0] (σ[#1 = #2] (E × (V \ T))))))
↪ ∧ (T ⊆ Rc))

```

23 Example0163: Greatest-fixpoint safe set: shrinking iteration bounded below by Rb

Category	synthetic (syntactic generation from the classic Datalog programs); family: Synthetic.
Kind	equivalence (with a greatest fixpoint, via a post-fixpoint witness).
Contributors	Fangzhu Shen, Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	fixpoint-witness greatest-fixpoint negation .
Input	Benchmark/Example0163/Input.lean (canonical notation).
Summary	Canonical form of the earlier single-level encoding of this case: the same schema, precondition, command and postcondition.
Sources	legacy source record hard-gfp-safeset of the retired classic-source collection

Datalog program(s). *Datalog reading of the greatest-fixpoint safe set (written for this report; complement formulation: $T = V \setminus \text{Unsafe}$)* — hand-written reading (this report).

```

Unsafe(x) :- Bad(x).
Unsafe(x) :- E(x, y), Unsafe(y).
T(x) :- V(x), not Unsafe(x).

```

Reading. The loop computes a greatest-fixpoint safe set over a graph E with vertex set V and designated bad vertices Bad : starting from $T := V$ and iterating $T := (V \setminus \text{Bad}) \setminus \{x : \text{some } E\text{-successor of } x \text{ lies outside the current } T\}$, the decreasing iteration converges to the largest set of non-bad vertices all of whose successors also stay in the set — the standard safety/invariant computation used in model checking or as the safe region of a reachability game, equivalently the Datalog reading $T(x) :- V(x), \text{not Unsafe}(x)$ where Unsafe is the least fixpoint of reachability to Bad . The Hoare triple is the coinduction (greatest-fixpoint elimination) obligation: given any Rb that is already a post-fixpoint of the safe-set operator, the computed T must itself be an exact fixpoint of the operator and dominate Rb from above, establishing that the naive decreasing iteration computes the greatest fixpoint.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\forall x. (Rb(x) \rightarrow V(x) \wedge \neg \text{Bad}(x) \wedge \neg \exists y. E(x, y) \wedge V(y) \wedge \neg Rb(y))$

The precondition states that Rb is a post-fixpoint of the safe-set operator: $Rb \subseteq (V \setminus \text{Bad}) \setminus \{x : E(x, y) \text{ for some } y \notin Rb\}$, written $Rb \subseteq ((V \setminus \text{Bad}) \setminus \pi[0](\sigma[\#1 = \#2](E \times (V \setminus Rb))))$.

POST: $(\forall x. (T(x) \leftrightarrow V(x) \wedge \neg \text{Bad}(x) \wedge \neg \exists y. E(x, y) \wedge V(y) \wedge \neg T(y))) \wedge \forall z. (Rb(z) \rightarrow T(z))$

The postcondition states that the computed T is an exact fixpoint of the safe-set operator, $T = (V \setminus \text{Bad}) \setminus \{x : E(x, y) \text{ for some } y \notin T\}$, and that $Rb \subseteq T$.

Whiel encoding (Hoare triple).

```

SCHEMA
  Bad/1, V/1, Rb/1, T/1, S/1, E/2

PRE
  (Rb ⊆ ((V \ Bad) \ (π[0] (σ[#1 = #2] (E × (V \ Rb))))))

CMD
  T := V;
  S := (V \ Bad);
  WHILE (S ≠ T) DO
    T := S;
    S := ((V \ Bad) \ (π[0] (σ[#1 = #2] (E × (V \ T)))))
  END

POST
  ((T = ((V \ Bad) \ (π[0] (σ[#1 = #2] (E × (V \ T))))) ∧ (Rb ⊆ T))

```

24 Example1030: Foreign-key support core for sharded materialized rows

Category	LLM-generated (GPT/Claude benchmark programs); family: LLM.
Kind	equivalence (with a greatest fixpoint, via a post-fixpoint witness).
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid (expected; strengthened claim).
Tags	<code>fixpoint-witness</code> <code>greatest-fixpoint</code> <code>integrity-constraints</code> <code>negation</code> .
Input	Benchmark/Example1030/Input.lean (canonical notation).
Summary	Rows are pairs (id, shard) in Base; Req(id, ref) says row id needs row ref to exist in the same shard. The loop repeatedly removes rows with an unmet requirement (Bad) until none remain, so Keep is the greatest subset of Base in which every requirement is satisfied within the subset: the greatest fixpoint of $G(X) = X \setminus \text{Bad}(X)$.
Sources	retired-corpus source record llm_30 of the LLM family retired-corpus input record llm_30 of the LLM family

Datalog program(s). *Datalog reading of the greatest-fixpoint integrity core (written for this report; complement formulation: $\text{Keep} = \text{Base} \setminus \text{Removed}$, a row is removed when a requirement points outside the kept rows of its shard) — hand-written reading (this report).*

```

Removed(id, s) :- Base(id, s), Req(id, ref), not Base(ref, s).
Removed(id, s) :- Base(id, s), Req(id, ref), Removed(ref, s).
Keep(id, s) :- Base(id, s), not Removed(id, s).

```

Reading. The Datalog program computes a referential-integrity closure over sharded materialized rows: Removed accumulates ids whose foreign-key requirement (Req) points outside Base, or that depend transitively on another already-removed row, and $\text{Keep} = \text{Base} \setminus \text{Removed}$ is dually the greatest fixpoint of the operator that deletes a row whenever its requirement is unmet within the same shard. The Hoare triple states the greatest-fixpoint correctness result in its maximality form: Rb is a free relation the program never touches, ranging over every subset of Base that is internally self-consistent (a post-fixpoint of the delete operator), and the postcondition asserts that Keep itself is self-consistent ($\text{Bad} = \emptyset$) and contains every such Rb, i.e. Keep is the greatest self-consistent subset of Base.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $(\forall x y. (Rb(x,y) \rightarrow Base(x,y))) \wedge \forall z u. (Rb(z,u) \rightarrow Rb(z,u) \wedge \neg \exists v. Rb(z,u) \wedge Req(z,v) \wedge \neg(Rb(z,u) \wedge Req(z,v) \wedge Rb(v,u)))$

Rb is a subset of Base such that no row of Rb has an unmet foreign-key requirement within Rb itself.

POST: $(\forall x y. (Keep(x,y) \rightarrow Base(x,y))) \wedge (\neg \exists u. Bad(z,u)) \wedge \forall v w. (Rb(v,w) \rightarrow Keep(v,w))$

Keep is a subset of Base in which every row's requirement is satisfied ($Bad = \emptyset$), and Keep contains every self-consistent subset Rb of Base, i.e. Keep is the greatest such subset.

Whiel encoding (Hoare triple).

```

SCHEMA
  Base/2, Req/2, Keep/2, Keep_aux/2, Bad/2, Rb/2, Need/3, Good/3, BadNeed/3

PRE
  (Rb ⊆ Base) ∧ (Rb ⊆ (Rb \ (π[0, 1] ((π[0, 1, 3] (σ[#0 = #2] (Rb × Req))) \ (π[0, 1, 2] (σ[(#2 = #3) ∧ (#1 = #4)] ((π[0, 1,
  ↪ 3] (σ[#0 = #2] (Rb × Req))) × Rb))))))

CMD
  Keep_aux := Base;
  Need := (π[0, 1, 3] (σ[#0 = #2] (Keep_aux × Req)));
  Good := (π[0, 1, 2] (σ[(#2 = #3) ∧ (#1 = #4)] (Need × Keep_aux)));
  BadNeed := (Need \ Good);
  Bad := (π[0, 1] BadNeed);
  Keep := (Keep_aux \ Bad);
  WHILE Keep ≠ Keep_aux DO
  Keep_aux := Keep;
  Need := (π[0, 1, 3] (σ[#0 = #2] (Keep_aux × Req)));
  Good := (π[0, 1, 2] (σ[(#2 = #3) ∧ (#1 = #4)] (Need × Keep_aux)));
  BadNeed := (Need \ Good);
  Bad := (π[0, 1] BadNeed);
  Keep := (Keep_aux \ Bad)
  END

POST
  ((Keep ⊆ Base) ∧ (Bad = ∅[2])) ∧ (Rb ⊆ Keep)

```

25 Example1061: Equivalence of computed same-generation with lfp of the same-generation rules (pre-fixpoint)

Category	LLM-generated (GPT/Claude benchmark programs); family: LLM.
Kind	equivalence (with a least fixpoint, via a pre-fixpoint witness).
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	<code>fixpoint-witness</code> .
Input	<code>Benchmark/Example1061/Input.lean</code> (canonical notation).
Summary	Canonical form of the earlier single-level encoding of this case: the same schema, precondition, command and postcondition; the snapshot relation <code>SG_2</code> is now <code>SG_aux</code> .
Sources	retired-corpus source record <code>llm_61</code> of the LLM family retired-corpus input record <code>llm_61</code> of the LLM family

Datalog program(s). *prog_p* — production check (the hand-written loop equals this program's naive compilation on the check instances).

```

SGC(x1, y1) :- Flat(x1, y1).
SGC(x1, y1) :- Up(x1, z1), SGC(z1, x2), Down(x2, y1).

```

Reading. The Datalog program computes the classical same-generation query over a hierarchy: SGC holds directly related Flat pairs, plus pairs reached by going Up one level, recursively finding a same-generation pair, then coming Down one level. The Hoare triple is the standard least-fixpoint correctness statement: K is any pre-fixpoint of the same-generation immediate-consequence operator, and the postcondition shows the loop's computed SG is itself a pre-fixpoint of that operator and is contained in every such K, so the loop computes exactly the least fixpoint of the same-generation rules.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\forall x y. (\text{Flat}(x,y) \vee (\exists z. \text{Up}(x,z) \wedge \exists u. K(z,u) \wedge \text{Down}(u,y)) \rightarrow K(x,y))$

K contains every Flat pair and is closed under composing an Up-edge, a pair already in K, and a Down-edge.

POST: $(\forall x y. (\text{Flat}(x,y) \rightarrow \text{SG}(x,y))) \wedge (\forall z u. ((\exists v. \text{Up}(z,v) \wedge \exists w. \text{SG}(v,w) \wedge \text{Down}(w,u)) \rightarrow \text{SG}(z,u))) \wedge \forall x1 x2. (\text{SG}(x1,x2) \rightarrow K(x1,x2))$

SG contains every Flat pair and is closed under one more Up/SG/Down composition step, and SG is contained in K, together pinning SG to the least fixpoint of the same-generation rules.

Whiel encoding (Hoare triple).

```

SCHEMA
  Flat/2, Up/2, Down/2, SG/2, SG_aux/2, K/2

PRE
  ((Flat  $\cup$  ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  ( $\text{Up} \times (\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  ( $K \times \text{Down}$ )))))))  $\subseteq$  K)

CMD
  SG_aux :=  $\emptyset$ ;
  SG := Flat;
  WHILE SG  $\neq$  SG_aux DO
    SG_aux := SG;
    SG := (Flat  $\cup$  ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  ( $\text{Up} \times (\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  ( $\text{SG\_aux} \times \text{Down}$ )))))))
  END

POST
  (((Flat  $\subseteq$  SG)  $\wedge$  (( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  ( $\text{Up} \times (\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  ( $\text{SG} \times \text{Down}$ )))))))  $\subseteq$  SG))  $\wedge$  (SG  $\subseteq$  K))

```

26 Example1081: Audited delegation contained in broad authorization closure

Category	LLM-generated (GPT/Claude benchmark programs); family: two programs under TGD constraints (LLM-generated, GPT-5.6; legacy raw_new pipeline).
Kind	containment.
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	dependencies mutual-recursion nonlinear-recursion prophecy.
Input	Benchmark/Example1081/Input.lean (canonical notation).
Summary	Audited delegation contained in broad authorization closure.
Sides	naive compiler: program C1 of the retired corpus case Example1081 (output Grant) naive compiler: program C2 of the retired corpus case Example1081 (output Permit)
Sources	LLM-generated (GPT-5.6), legacy raw_new TGD pipeline this repository the retired corpus case Example1081 (programs C1, C2 and the TGD list tgd0..) TGD pipeline run raw-new-tgd-0002-product: TGDs translated to relational algebra by RelCalc.TGD.listToAssertExpr; programs compiled with the certified naive compiler (read 2026-09-15) method this repository the equivalence-under-constraints paper draft (technical appendix: reduction of containment/equivalence under TGDs to a Hoare triple) (read 2026-09-15) legacy source record raw_tgd_gpt56_w_inv.json legacy source record Benchmark/Example1081/Input.lean Datalog source noted in the legacy inventory of Example1081
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).
Note	Legacy raw_new TGD case (GPT-5.6); one representative of the specialisation-in-generalisation template (1083, 1085, 1087, 1089 are its siblings, not included).

Datalog program(s). *prog_c1* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
Grant(x1, y1) :- Trusted(x1, y1).
Delegated(x1, y1) :- Special(x1, y1).
Audited(x1, y1) :- Composite(x1, y1).
Grant(x1, z1) :- Delegated(x1, y1), Audited(y1, z1).
Delegated(x1, z1) :- Grant(x1, y1), Trusted(y1, z1).
Audited(x1, z1) :- Grant(x1, y1), Evidence(y1, z1, x2), Valid(x2).
Grant(x1, z1) :- Grant(x1, y1), Audited(y1, z1).
```

prog_c2 — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
Permit(x1, y1) :- Broad(x1, y1).
Trace(x1, y1) :- Broad(x1, y1).
Permit(x1, y1) :- Evidence(x1, y1, z1), Valid(z1).
Permit(x1, y1) :- Trace(x1, y1).
Trace(x1, z1) :- Permit(x1, y1), Permit(y1, z1).
Trace(x1, z1) :- Permit(x1, y1), Trace(y1, z1).
Trace(x1, z1) :- Trace(x1, y1), Permit(y1, z1).
Trace(x1, z1) :- Trace(x1, y1), Trace(y1, z1).
Trace(x1, z1) :- Permit(x1, y1), Evidence(y1, z1, x2), Valid(x2).
```

Reading. This is a query-containment-under-dependencies problem, the setting of the relational chase: program C1 computes Grant, the least fixpoint of a delegation program built from Trusted pairs directly and from Delegated/Audited intermediate relations that compose earlier grants (Composite grants backed by an evidence witness); program C2 computes Permit, the least fixpoint of a broader authorization closure built from Broad pairs, evidence-backed pairs, and a transitive Trace relation. The precondition states the four tuple-generating dependencies (one of them existential) relating the two programs' input relations, and the postcondition claims that whenever those dependencies hold, every pair C1's program grants is also permitted by C2's program: Grant is contained in Permit.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $(\forall x y. (\text{Trusted}(x,y) \rightarrow \text{Broad}(x,y))) \wedge (\forall z u. (\text{Special}(z,u) \rightarrow \text{Broad}(z,u))) \wedge (\forall v w. (\text{Composite}(v,w) \rightarrow \exists x1. \text{Evidence}(v,w,x1) \wedge \text{Valid}(x1))) \wedge \forall x2 x3. ((\exists x4. \text{Evidence}(x2,x3,x4) \wedge \text{Valid}(x4)) \rightarrow \text{Broad}(x2,x3))$

PRE says every Trusted or Special pair is a Broad pair, every Composite pair has some Evidence witness marked Valid, and every pair with a Valid Evidence witness is itself a Broad pair.

POST: $\forall x y. (\text{Grant}(x,y) \rightarrow \text{Permit}(x,y))$

POST says every pair the delegation program's Grant relation derives is also derived by the authorization program's Permit relation.

Whiel encoding (Hoare triple).

```
SCHEMA
Valid/1, Trusted/2, Special/2, Composite/2, Broad/2, Grant/2, Delegated/2, Audited/2, Permit/2, Trace/2, Grant_aux/2,
  ↳ Delegated_aux/2, Audited_aux/2, Permit_aux/2, Trace_aux/2, Evidence/3

PRE
((Trusted ⊆ Broad) ∧ ((Special ⊆ Broad) ∧ ((Composite ⊆ π[0,1] (σ[#2 = #3] ((Evidence × Valid)))) ∧ (π[0,1] (σ[#2 = #3]
  ↳ ((Evidence × Valid))) ⊆ Broad))))

CMD
Grant_aux := ∅[2];
Delegated_aux := ∅[2];
Audited_aux := ∅[2];
Grant := (Trusted ∪ (π[0,3] (σ[#1 = #2] ((Delegated_aux × Audited_aux))) ∪ π[0,3] (σ[#1 = #2] ((Grant_aux × Audited_aux)))));
Delegated := (Special ∪ π[0,3] (σ[#1 = #2] ((Grant_aux × Trusted))));
Audited := (Composite ∪ π[0,3] (σ[#1 = #2 ∧ #4 = #5] ((Grant_aux × (Evidence × Valid)))));
WHILE (¬(((Grant = Grant_aux) ∧ ((Delegated = Delegated_aux) ∧ (Audited = Audited_aux))))) DO
  Grant_aux := Grant;
  Delegated_aux := Delegated;
  Audited_aux := Audited;
  Grant := (Grant ∪ (Trusted ∪ (π[0,3] (σ[#1 = #2] ((Delegated_aux × Audited_aux))) ∪ π[0,3] (σ[#1 = #2] ((Grant_aux ×
    ↳ Audited_aux)))));
  Delegated := (Delegated ∪ (Special ∪ π[0,3] (σ[#1 = #2] ((Grant_aux × Trusted))));
  Audited := (Audited ∪ (Composite ∪ π[0,3] (σ[#1 = #2 ∧ #4 = #5] ((Grant_aux × (Evidence × Valid))))));
END;
Permit_aux := ∅[2];
Trace_aux := ∅[2];
Permit := (Broad ∪ (π[0,1] (σ[#2 = #3] ((Evidence × Valid))) ∪ Trace_aux));
Trace := (Broad ∪ (π[0,3] (σ[#1 = #2] ((Permit_aux × Permit_aux))) ∪ (π[0,3] (σ[#1 = #2] ((Permit_aux × Trace_aux))) ∪
  ↳ (π[0,3] (σ[#1 = #2] ((Trace_aux × Permit_aux))) ∪ (π[0,3] (σ[#1 = #2] ((Trace_aux × Trace_aux))) ∪ π[0,3] (σ[#1 = #2 ∧
    ↳ #4 = #5] ((Permit_aux × (Evidence × Valid))))))));
WHILE (¬(((Permit = Permit_aux) ∧ (Trace = Trace_aux)))) DO
  Permit_aux := Permit;
  Trace_aux := Trace;
  Permit := (Permit ∪ (Broad ∪ (π[0,1] (σ[#2 = #3] ((Evidence × Valid))) ∪ Trace_aux));
  Trace := (Trace ∪ (Broad ∪ (π[0,3] (σ[#1 = #2] ((Permit_aux × Permit_aux))) ∪ (π[0,3] (σ[#1 = #2] ((Permit_aux ×
    ↳ Trace_aux))) ∪ (π[0,3] (σ[#1 = #2] ((Trace_aux × Permit_aux))) ∪ (π[0,3] (σ[#1 = #2] ((Trace_aux × Trace_aux))) ∪
    ↳ π[0,3] (σ[#1 = #2 ∧ #4 = #5] ((Permit_aux × (Evidence × Valid))))))));
END

POST
(Grant ⊆ Permit)
```

27 Example1088: Equivalent forward dependency closure and transposed reverse resolution

Category	LLM-generated (GPT/Claude benchmark programs); family: two programs under TGD constraints (LLM-generated, GPT-5.6; legacy raw_new pipeline).
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	dependencies mutual-recursion nonlinear-recursion.
Input	Benchmark/Example1088/Input.lean (canonical notation).
Summary	Equivalent forward dependency closure and transposed reverse resolution.
Sides	naive compiler: program C1 of the retired corpus case Example1088 (output ForwardReach) naive compiler: program C2 of the retired corpus case Example1088 (output ResolvedPair)
Sources	LLM-generated (GPT-5.6), legacy raw_new TGD pipeline this repository the retired corpus case Example1088 (programs C1, C2 and the TGD list tgd0..) TGD pipeline run raw-new-tgd-0002-product: TGDs translated to relational algebra by RelCalc.TGD.listToAssertExpr; programs compiled with the certified naive compiler (read 2026-09-15) method this repository the equivalence-under-constraints paper draft (technical appendix: reduction of containment/equivalence under TGDs to a Hoare triple) (read 2026-09-15) legacy source record raw_tgd_gpt56_w_inv.json legacy source record Benchmark/Example1088/Input.lean Datalog source noted in the legacy inventory of Example1088
Obstruction note	from the metadata: anchored: both sides are naive closures at the same rate and the legacy product loop was certified with a quantifier-free invariant; no prophecy level expected
Note	Legacy raw_new TGD case (GPT-5.6): forward closure equals the transposed reverse resolution under inverse seed views and bridge-proof witnesses.

Datalog program(s). *prog_c1* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
ForwardReach(x1, y1) :- ForwardSeed(x1, y1).
ForwardStep(x1, y1) :- ForwardEdge(x1, y1).
ForwardReach(x1, z1) :- ForwardReach(x1, y1), ForwardStep(y1, z1).
ForwardStep(x1, z1) :- ForwardReach(x1, y1), BridgeProof(y1, z1, x2), ValidBridge(x2).
ForwardReach(x1, z1) :- ForwardStep(x1, y1), ForwardReach(y1, z1).
```

prog_c2 — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
ResolvedPair(x1, y1) :- BackwardSeed(y1, x1).
ReverseStep(y1, x1) :- BackwardEdge(y1, x1).
ResolvedPair(x1, z1) :- ResolvedPair(x1, y1), ReverseStep(z1, y1).
ReverseStep(z1, x1) :- ResolvedPair(x1, y1), BridgeProof(y1, z1, x2), ValidBridge(x2).
ResolvedPair(x1, z1) :- ReverseStep(y1, x1), ResolvedPair(y1, z1).
```

Reading. This is a query-equivalence-under-dependencies problem, the setting of the relational chase: program C1 computes `ForwardReach`, the transitive closure of a forward step relation (direct `ForwardEdge` pairs, plus pairs backed by a valid bridge proof) starting from `ForwardSeed`; program C2 computes `ResolvedPair`, the same kind of closure computed instead over the transposed backward relations (`BackwardSeed`, `BackwardEdge`) with its own reverse step relation. The precondition's six tuple-generating dependencies (two of them existential) say the forward and backward seed relations are transposes of each other, that `ForwardEdge` equals the pairs backed by a valid bridge proof, and that `BackwardEdge` equals the transpose of that same relation; under those dependencies the postcondition claims the two closures agree, `ForwardReach = ResolvedPair`.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in `Input.lean`. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $(\forall x y. (\text{ForwardSeed}(x,y) \rightarrow \text{BackwardSeed}(y,x))) \wedge (\forall z u. (\text{BackwardSeed}(z,u) \rightarrow \text{ForwardSeed}(u,z))) \wedge (\forall v w. (\text{ForwardEdge}(v,w) \rightarrow \exists x1. \text{BridgeProof}(v,w,x1) \wedge \text{ValidBridge}(x1))) \wedge (\forall x2 x3. ((\exists x4. \text{BridgeProof}(x2,x3,x4) \wedge \text{ValidBridge}(x4)) \rightarrow \text{ForwardEdge}(x2,x3))) \wedge (\forall x5 x6. ((\exists x7. \text{BridgeProof}(x5,x6,x7) \wedge \text{ValidBridge}(x7)) \rightarrow \text{BackwardEdge}(x6,x5))) \wedge \forall x8 x9. (\text{BackwardEdge}(x8,x9) \rightarrow \exists x10. \text{BridgeProof}(x9,x8,x10) \wedge \text{ValidBridge}(x10))$

PRE says `ForwardSeed` and `BackwardSeed` are transposes of each other, `ForwardEdge` equals the pairs backed by a valid bridge proof, and `BackwardEdge` equals the transpose of that same bridge-proof relation.

POST: $\forall x y. (\text{ForwardReach}(x,y) \leftrightarrow \text{ResolvedPair}(x,y))$

POST says the forward-computed reachability relation `ForwardReach` equals the reverse-computed relation `ResolvedPair`.

Whiel encoding (Hoare triple).

```
SCHEMA
ValidBridge/1, ForwardSeed/2, BackwardSeed/2, ForwardEdge/2, BackwardEdge/2, ForwardReach/2, ForwardStep/2, ResolvedPair/2,
  ↳ ReverseStep/2, ForwardReach_aux/2, ForwardStep_aux/2, ResolvedPair_aux/2, ReverseStep_aux/2, BridgeProof/3

PRE
((ForwardSeed ⊆ π[1,0] (BackwardSeed)) ∧ ((BackwardSeed ⊆ π[1,0] (ForwardSeed)) ∧ ((ForwardEdge ⊆ π[0,1] (σ[#2 = #3]
  ↳ ((BridgeProof × ValidBridge)))) ∧ ((π[0,1] (σ[#2 = #3] ((BridgeProof × ValidBridge))) ⊆ ForwardEdge) ∧ ((π[0,1] (σ[#2 =
  ↳ #3] ((BridgeProof × ValidBridge))) ⊆ π[1,0] (BackwardEdge)) ∧ (BackwardEdge ⊆ π[1,0] (σ[#2 = #3] ((BridgeProof ×
  ↳ ValidBridge)))))))

CMD
ForwardReach_aux := ∅[2];
ForwardStep_aux := ∅[2];
ForwardReach := (ForwardSeed ∪ (π[0,3] (σ[#1 = #2] ((ForwardReach_aux × ForwardStep_aux))) ∪ π[0,3] (σ[#1 = #2]
  ↳ ((ForwardStep_aux × ForwardReach_aux)))));
ForwardStep := (ForwardEdge ∪ π[0,3] (σ[#1 = #2 ∧ #4 = #5] ((ForwardReach_aux × (BridgeProof × ValidBridge)))));
WHILE (¬(((ForwardReach = ForwardReach_aux) ∧ (ForwardStep = ForwardStep_aux)))) DO
  ForwardReach_aux := ForwardReach;
  ForwardStep_aux := ForwardStep;
  ForwardReach := (ForwardReach ∪ (ForwardSeed ∪ (π[0,3] (σ[#1 = #2] ((ForwardReach_aux × ForwardStep_aux))) ∪ π[0,3] (σ[#1 =
  ↳ #2] ((ForwardStep_aux × ForwardReach_aux)))));
  ForwardStep := (ForwardStep ∪ (ForwardEdge ∪ π[0,3] (σ[#1 = #2 ∧ #4 = #5] ((ForwardReach_aux × (BridgeProof ×
  ↳ ValidBridge)))));
END;
ResolvedPair_aux := ∅[2];
ReverseStep_aux := ∅[2];
ResolvedPair := (π[1,0] (BackwardSeed) ∪ (π[0,2] (σ[#1 = #3] ((ResolvedPair_aux × ReverseStep_aux))) ∪ π[1,3] (σ[#0 = #2]
  ↳ ((ReverseStep_aux × ResolvedPair_aux)))));
ReverseStep := (BackwardEdge ∪ π[3,0] (σ[#1 = #2 ∧ #4 = #5] ((ResolvedPair_aux × (BridgeProof × ValidBridge)))));
WHILE (¬(((ResolvedPair = ResolvedPair_aux) ∧ (ReverseStep = ReverseStep_aux)))) DO
  ResolvedPair_aux := ResolvedPair;
  ReverseStep_aux := ReverseStep;
  ResolvedPair := (ResolvedPair ∪ (π[1,0] (BackwardSeed) ∪ (π[0,2] (σ[#1 = #3] ((ResolvedPair_aux × ReverseStep_aux))) ∪
  ↳ π[1,3] (σ[#0 = #2] ((ReverseStep_aux × ResolvedPair_aux)))));
  ReverseStep := (ReverseStep ∪ (BackwardEdge ∪ π[3,0] (σ[#1 = #2 ∧ #4 = #5] ((ResolvedPair_aux × (BridgeProof ×
  ↳ ValidBridge)))));
END

POST
(ForwardReach = ResolvedPair)
```

28 Example1090: Equivalent taint reachability with ternary modes and projected frontiers

Category	LLM-generated (GPT/Claude benchmark programs); family: two programs under TGD constraints (LLM-generated, GPT-5.6; legacy raw_new pipeline).
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	dependencies mutual-recursion nonlinear-recursion.
Input	Benchmark/Example1090/Input.lean (canonical notation).
Summary	Equivalent taint reachability with ternary modes and projected frontiers.
Sides	naive compiler: program C1 of the retired corpus case Example1090 (output TaintReach) naive compiler: program C2 of the retired corpus case Example1090 (output AnalysisReach)
Sources	LLM-generated (GPT-5.6), legacy raw_new TGD pipeline this repository the retired corpus case Example1090 (programs C1, C2 and the TGD list tgd0..) TGD pipeline run raw-new-tgd-0002-product: TGDs translated to relational algebra by RelCalc.TGD.listToAssertExpr; programs compiled with the certified naive compiler (read 2026-09-15) method this repository the equivalence-under-constraints paper draft (technical appendix: reduction of containment/equivalence under TGDs to a Hoare triple) (read 2026-09-15) legacy source record raw_tgd_gpt56_w_inv.json legacy source record Benchmark/Example1090/Input.lean Datalog source noted in the legacy inventory of Example1090
Obstruction note	from the metadata: anchored: both sides are naive closures at the same rate and the legacy product loop was certified with a quantifier-free invariant; no prophecy level expected
Note	Legacy raw_new TGD case (GPT-5.6): a ternary mode-carrying state equals a binary projected frontier; exact only because every node has a mode (existential dependency).

Datalog program(s). *prog_c1* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
TaintReach(x1, y1) :- SeedPair(x1, y1).
TaintState(x1, y1, z1) :- SeedPair(x1, y1), ModeTag(y1, z1).
TaintReach(x1, z1) :- TaintState(x1, y1, x2), FlowEdge(y1, z1).
TaintState(x1, z1, x2) :- TaintReach(x1, y1), FlowEdge(y1, z1), ModeTag(z1, x2).
TaintState(x1, z1, y2) :- TaintState(x1, y1, x2), FlowEdge(y1, z1), ModeTag(z1, y2).
TaintReach(x1, z1) :- TaintReach(x1, y1), TaintReach(y1, z1).
```

prog_c2 — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
AnalysisReach(x1, y1) :- InputPair(x1, y1).
FrontierState(x1, y1) :- InputPair(x1, y1).
AnalysisReach(x1, z1) :- FrontierState(x1, y1), ViewEdge(y1, z1).
FrontierState(x1, z1) :- AnalysisReach(x1, y1), ViewEdge(y1, z1).
FrontierState(x1, z1) :- FrontierState(x1, y1), StateWitness(y1, z1, x2), EnabledMode(x2).
AnalysisReach(x1, z1) :- AnalysisReach(x1, y1), AnalysisReach(y1, z1).
```

Reading. This is a query-equivalence-under-dependencies problem, the setting of the relational chase: program C1 computes TaintReach, an interprocedural-taint-style reachability relation computed with the help of an auxiliary ternary TaintState that additionally records a mode witness at each frontier vertex; program C2 computes AnalysisReach, the same reachability relation computed with a simpler binary FrontierState that drops the mode tag. The precondition's eight tuple-generating dependencies (three of them existential) say SeedPair equals InputPair, FlowEdge equals ViewEdge, every vertex reached by SeedPair or FlowEdge carries some ModeTag, and FlowEdge coincides exactly with the pairs backed by a valid StateWitness; under these, the postcondition claims the two closures compute the same relation, TaintReach = AnalysisReach — the mode bookkeeping in C1 turns out not to change what gets reached, once the dependencies guarantee a mode witness always exists.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in Input.lean remains the authority.

PRE: $(\forall x y. (\text{SeedPair}(x,y) \rightarrow \text{InputPair}(x,y))) \wedge (\forall z u. (\text{InputPair}(z,u) \rightarrow \text{SeedPair}(z,u))) \wedge (\forall v w. (\text{FlowEdge}(v,w) \rightarrow \text{ViewEdge}(v,w))) \wedge (\forall x_1 x_2. (\text{ViewEdge}(x_1,x_2) \rightarrow \text{FlowEdge}(x_1,x_2))) \wedge (\forall x_3. ((\exists x_4. \text{SeedPair}(x_4,x_3)) \rightarrow \exists x_5. \text{ModeTag}(x_3,x_5))) \wedge (\forall x_6. ((\exists x_7. \text{FlowEdge}(x_7,x_6)) \rightarrow \exists x_8. \text{ModeTag}(x_6,x_8))) \wedge (\forall x_9 x_{10}. (\text{FlowEdge}(x_9,x_{10}) \rightarrow \exists x_{11}. \text{StateWitness}(x_9,x_{10},x_{11}) \wedge \text{EnabledMode}(x_{11}))) \wedge \forall x_{12} x_{13}. ((\exists x_{14}. \text{StateWitness}(x_{12},x_{13},x_{14}) \wedge \text{EnabledMode}(x_{14})) \rightarrow \text{FlowEdge}(x_{12},x_{13})))$

PRE says SeedPair and InputPair are the same relation, FlowEdge and ViewEdge are the same relation, every vertex reached by SeedPair or FlowEdge carries some ModeTag, and FlowEdge coincides exactly with the pairs backed by a valid StateWitness.

POST: $\forall x y. (\text{TaintReach}(x,y) \leftrightarrow \text{AnalysisReach}(x,y))$

POST says the ternary mode-carrying reachability relation TaintReach and the binary frontier relation AnalysisReach compute the same pairs.

Whiel encoding (Hoare triple).

```
SCHEMA
  EnabledMode/1, SeedPair/2, InputPair/2, FlowEdge/2, ViewEdge/2, ModeTag/2, TaintReach/2, AnalysisReach/2, FrontierState/2,
  ↪ TaintReach_aux/2, AnalysisReach_aux/2, FrontierState_aux/2, StateWitness/3, TaintState/3, TaintState_aux/3

PRE
  ((SeedPair ⊆ InputPair) ∧ ((InputPair ⊆ SeedPair) ∧ ((FlowEdge ⊆ ViewEdge) ∧ ((ViewEdge ⊆ FlowEdge) ∧ ((π[1] (SeedPair) ⊆
  ↪ π[0] (ModeTag)) ∧ ((π[1] (FlowEdge) ⊆ π[0] (ModeTag)) ∧ ((FlowEdge ⊆ π[0,1] (σ[#2 = #3] ((StateWitness × EnabledMode))))
  ↪ ∧ (π[0,1] (σ[#2 = #3] ((StateWitness × EnabledMode))) ⊆ FlowEdge)))))))))

CMD
  TaintReach_aux := ∅[2];
  TaintState_aux := ∅[3];
  TaintReach := (SeedPair ∪ (π[0,4] (σ[#1 = #3] ((TaintState_aux × FlowEdge))) ∪ π[0,3] (σ[#1 = #2] ((TaintReach_aux ×
  ↪ TaintReach_aux)))));
  TaintState := (π[0,1,3] (σ[#1 = #2] ((SeedPair × ModeTag))) ∪ (π[0,3,5] (σ[(#1 = #2 ∧ #3 = #4)] ((TaintReach_aux × (FlowEdge ×
  ↪ ModeTag)))) ∪ π[0,4,6] (σ[(#1 = #3 ∧ #4 = #5)] ((TaintState_aux × (FlowEdge × ModeTag)))));
  WHILE (¬(((TaintReach = TaintReach_aux) ∧ (TaintState = TaintState_aux)))) DO
    TaintReach_aux := TaintReach;
    TaintState_aux := TaintState;
    TaintReach := (TaintReach ∪ (SeedPair ∪ (π[0,4] (σ[#1 = #3] ((TaintState_aux × FlowEdge))) ∪ π[0,3] (σ[#1 = #2]
    ↪ ((TaintReach_aux × TaintReach_aux)))));
    TaintState := (TaintState ∪ (π[0,1,3] (σ[#1 = #2] ((SeedPair × ModeTag))) ∪ (π[0,3,5] (σ[(#1 = #2 ∧ #3 = #4)]
    ↪ ((TaintReach_aux × (FlowEdge × ModeTag)))) ∪ π[0,4,6] (σ[(#1 = #3 ∧ #4 = #5)] ((TaintState_aux × (FlowEdge ×
    ↪ ModeTag))))));
  END;
  AnalysisReach_aux := ∅[2];
  FrontierState_aux := ∅[2];
  AnalysisReach := (InputPair ∪ (π[0,3] (σ[#1 = #2] ((FrontierState_aux × ViewEdge))) ∪ π[0,3] (σ[#1 = #2] ((AnalysisReach_aux ×
  ↪ AnalysisReach_aux)))));
  FrontierState := (InputPair ∪ (π[0,3] (σ[#1 = #2] ((AnalysisReach_aux × ViewEdge))) ∪ π[0,3] (σ[(#1 = #2 ∧ #4 = #5)]
  ↪ ((FrontierState_aux × (StateWitness × EnabledMode)))));
  WHILE (¬(((AnalysisReach = AnalysisReach_aux) ∧ (FrontierState = FrontierState_aux)))) DO
    AnalysisReach_aux := AnalysisReach;
    FrontierState_aux := FrontierState;
    AnalysisReach := (AnalysisReach ∪ (InputPair ∪ (π[0,3] (σ[#1 = #2] ((FrontierState_aux × ViewEdge))) ∪ π[0,3] (σ[#1 = #2]
    ↪ ((AnalysisReach_aux × AnalysisReach_aux)))));
    FrontierState := (FrontierState ∪ (InputPair ∪ (π[0,3] (σ[#1 = #2] ((AnalysisReach_aux × ViewEdge))) ∪ π[0,3] (σ[(#1 = #2 ∧
    ↪ #4 = #5)] ((FrontierState_aux × (StateWitness × EnabledMode))))));
  END

POST
  (TaintReach = AnalysisReach)
```

29 Example1091: Delegation reachability bounded by a transitively closed policy view

Category	LLM-generated (GPT/Claude benchmark programs); family: two programs under TGD constraints (LLM-generated, Claude Opus; legacy raw_new pipeline).
Kind	containment.
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	dependencies mutual-recursion nonlinear-recursion.
Input	Benchmark/Example1091/Input.lean (canonical notation).
Summary	Delegation reachability bounded by a transitively closed policy view.
Sides	naive compiler: program C1 of the retired corpus case Example1091 (output P) naive compiler: program C2 of the retired corpus case Example1091 (output T)
Sources	LLM-generated (Claude Opus), legacy raw_new TGD pipeline this repository the retired corpus case Example1091 (programs C1, C2 and the TGD list tgd0..) TGD pipeline run raw-new-tgd-0002-product: TGDs translated to relational algebra by RelCalc.TGD.listToAssertExpr; programs compiled with the certified naive compiler (read 2026-09-15) method this repository the equivalence-under-constraints paper draft (technical appendix: reduction of containment/equivalence under TGDs to a Hoare triple) (read 2026-09-15) legacy source record raw_tgd_opus_w_inv.json legacy source record Benchmark/Example1091/Input.lean Datalog source noted in the legacy inventory of Example1091
Obstruction note	from the metadata: anchored: both sides are naive closures at the same rate and the legacy product loop was certified with a quantifier-free invariant; no prophecy level expected
Note	Legacy raw_new TGD case (Claude Opus): every link has a midpoint (existential dependency); containment in a constraint-closed policy view.

Datalog program(s). *prog_c1* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
P(x1, y1) :- Link(x1, y1).
P(x1, y1) :- Mid(x1, y1).
P(x1, z1) :- P(x1, y1), Q(y1, z1).
P(x1, x2) :- P(x1, y1), Mid(y1, z1), P(z1, x2).
Q(x1, y1) :- Tri(x1, z1, y1).
Q(x1, z1) :- Q(x1, y1), P(y1, z1).
```

prog_c2 — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
T(x1, y1) :- BaseTwo(x1, y1).
G(x1, y1) :- Jump(x1, y1).
G(x1, z1) :- G(x1, y1), Jump(y1, z1).
T(x1, z1) :- T(x1, y1), G(y1, z1).
T(x1, z1) :- T(x1, y1), T(y1, z1).
```

Reading. Program C1 recursively derives grantable accesses P via direct delegation (Link, Mid) and composition with Q, where Q is built from ternary relay facts Tri and further composed with P, a mutually recursive delegation/role-composition propagation typical of access-control reachability analysis. Program C2 recomputes reachability T as the transitive closure of BaseTwo composed with a separately closed relation G built from Jump edges. The Hoare triple is a containment claim under tuple-generating dependencies (the classical chase setting): the precondition encodes five TGDs forcing C to be transitively closed, to contain Mid and the first/third coordinates of every Tri triple, to guarantee every Link edge a two-hop Mid-Mid witness, and to force BaseTwo to contain all of C; under these constraints the postcondition claims $P \subseteq T$, expected valid because both programs are naive closures advancing at the same rate.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $(\forall x y. ((\exists z. C(x,z) \wedge C(z,y)) \rightarrow C(x,y))) \wedge (\forall u v. (Mid(u,v) \rightarrow C(u,v))) \wedge (\forall w x1. (Link(w,x1) \rightarrow \exists x2. Mid(w,x2) \wedge Mid(x2,x1))) \wedge (\forall x3 x4. ((\exists x5. Tri(x3,x5,x4)) \rightarrow C(x3,x4))) \wedge \forall x6 x7. (C(x6,x7) \rightarrow BaseTwo(x6,x7))$

The precondition is five TGDs: C is transitively closed and contains Mid and the first/third coordinates of every Tri triple; every Link edge has some two-hop Mid-Mid witness; and BaseTwo contains all of C.

POST: $\forall x y. (P(x,y) \rightarrow T(x,y))$

The postcondition states that every pair C1 derives into P is also derived by C2 into T, i.e. $P \subseteq T$.

Whiel encoding (Hoare triple).

```

SCHEMA
  C/2, Mid/2, Link/2, BaseTwo/2, Jump/2, P/2, Q/2, T/2, G/2, P_aux/2, Q_aux/2, T_aux/2, G_aux/2, Tri/3

PRE
  (( $\pi[0,3] (\sigma[\#1 = \#2] ((C \times C))) \subseteq C$ )  $\wedge$  ( $Mid \subseteq C$ )  $\wedge$  ( $Link \subseteq \pi[0,3] (\sigma[\#1 = \#2] ((Mid \times Mid)))$ )  $\wedge$  ( $(\pi[0,2] (Tri) \subseteq C) \wedge (C \hookrightarrow \subseteq BaseTwo)$ ))))))

CMD
  P_aux :=  $\emptyset[2]$ ;
  Q_aux :=  $\emptyset[2]$ ;
  P := (Link  $\cup$  (Mid  $\cup$  ( $\pi[0,3] (\sigma[\#1 = \#2] ((P\_aux \times Q\_aux))) \cup \pi[0,5] (\sigma[(\#1 = \#2 \wedge \#3 = \#4)] ((P\_aux \times (Mid \times P\_aux))))))$ );
  Q := ( $\pi[0,2] (Tri) \cup \pi[0,3] (\sigma[\#1 = \#2] ((Q\_aux \times P\_aux)))$ );
  WHILE ( $\neg((P = P\_aux) \wedge (Q = Q\_aux))$ ) DO
    P_aux := P;
    Q_aux := Q;
    P := (P  $\cup$  (Link  $\cup$  (Mid  $\cup$  ( $\pi[0,3] (\sigma[\#1 = \#2] ((P\_aux \times Q\_aux))) \cup \pi[0,5] (\sigma[(\#1 = \#2 \wedge \#3 = \#4)] ((P\_aux \times (Mid \times P\_aux))))$ )));
    Q := (Q  $\cup$  ( $\pi[0,2] (Tri) \cup \pi[0,3] (\sigma[\#1 = \#2] ((Q\_aux \times P\_aux)))$ ));
  END;
  T_aux :=  $\emptyset[2]$ ;
  G_aux :=  $\emptyset[2]$ ;
  T := (BaseTwo  $\cup$  ( $\pi[0,3] (\sigma[\#1 = \#2] ((T\_aux \times G\_aux))) \cup \pi[0,3] (\sigma[\#1 = \#2] ((T\_aux \times T\_aux))))$ );
  G := (Jump  $\cup$  ( $\pi[0,3] (\sigma[\#1 = \#2] ((G\_aux \times Jump)))$ );
  WHILE ( $\neg((T = T\_aux) \wedge (G = G\_aux))$ ) DO
    T_aux := T;
    G_aux := G;
    T := (T  $\cup$  (BaseTwo  $\cup$  ( $\pi[0,3] (\sigma[\#1 = \#2] ((T\_aux \times G\_aux))) \cup \pi[0,3] (\sigma[\#1 = \#2] ((T\_aux \times T\_aux))))$ );
    G := (G  $\cup$  (Jump  $\cup$  ( $\pi[0,3] (\sigma[\#1 = \#2] ((G\_aux \times Jump)))$ ));
  END

POST
  (P  $\subseteq$  T)

```

30 Example1093: Lineage triples contained in a closed ternary cover

Category	LLM-generated (GPT/Claude benchmark programs); family: two programs under TGD constraints (LLM-generated, Claude Opus; legacy raw_new pipeline).
Kind	containment.
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	dependencies mutual-recursion nonlinear-recursion.
Input	Benchmark/Example1093/Input.lean (canonical notation).
Summary	Lineage triples contained in a closed ternary cover.
Sides	naive compiler: program C1 of the retired corpus case Example1093 (output Lin) naive compiler: program C2 of the retired corpus case Example1093 (output Prov)
Sources	LLM-generated (Claude Opus), legacy raw_new TGD pipeline this repository the retired corpus case Example1093 (programs C1, C2 and the TGD list tgd0..) TGD pipeline run raw-new-tgd-0002-product: TGDs translated to relational algebra by RelCalc.TGD.listToAssertExpr; programs compiled with the certified naive compiler (read 2026-09-15) method this repository the equivalence-under-constraints paper draft (technical appendix: reduction of <code>containment/equivalence</code> under TGDs to a Hoare triple) (read 2026-09-15) legacy source record raw_tgd_opus_w_inv.json legacy source record Benchmark/Example1093/Input.lean Datalog source noted in the legacy inventory of Example1093
Obstruction note	from the metadata: anchored: both sides are naive closures at the same rate and the legacy product loop was certified with a quantifier-free invariant; no prophecy level expected
Note	Legacy raw_new TGD case (Claude Opus): the provenance cover is closed under propagation up to a re-projected coordinate (head-existential dependencies).

Datalog program(s). *prog_c1* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```

Reach(x1, y1) :- Edge(x1, y1).
Reach(x1, z1) :- Reach(x1, y1), Reach(y1, z1).
Reach(x1, y1) :- Lin(x1, y1, z1).
Lin(x1, y1, z1) :- Fact(x1, y1, z1).
Lin(x1, z1, z2) :- Lin(x1, y1, z2), Reach(y1, z1).
Lin(x2, y1, z1) :- Lin(x1, y1, z1), Gen(x1, x2).

```

prog_c2 — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```

Prov(x1, y1, z1) :- Cover(x1, y1, z1).
Trav(x1, y1) :- Edge(x1, y1).
Trav(x1, z1) :- Trav(x1, y1), Trav(y1, z1).
Prov(x1, z1, z2) :- Prov(x1, y1, z2), Trav(y1, z1).
Prov(x2, y1, z1) :- Prov(x1, y1, z1), Gen(x1, x2).

```

Reading. Program C1 derives ternary lineage/provenance triples `Lin(item, derived-item, source)` by propagating an initial `Fact` triple along a computed reachability relation `Reach` (built from graph edges `Edge` and fed back by `Lin`'s own second coordinate) and along generation edges `Gen`. Program C2 instead reads answers from `Prov`, defined via a ternary `Cover` relation the TGD constraints force to be closed under the same edge-traversal, generation-transfer,

and self-composition steps, so Cover is guaranteed to dominate any lineage C1's rules could derive. The claim is the containment $\text{Lin} \subseteq \text{Prov}$ under these dependencies, a chase-style containment check with an existential dependency (every Edge pair needs a Fact witness), expected valid.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $(\forall x y z. (\text{Fact}(x,y,z) \rightarrow \text{Cover}(x,y,z))) \wedge (\forall u v w. ((\exists x1. \text{Cover}(u,x1,v) \wedge \text{Edge}(x1,w)) \rightarrow \text{Cover}(u,w,v))) \wedge (\forall x2 x3 x4. ((\exists x5. \text{Cover}(x5,x2,x3) \wedge \text{Gen}(x5,x4)) \rightarrow \text{Cover}(x4,x2,x3))) \wedge (\forall x6 x7 x8. ((\exists x9 x10. \text{Cover}(x6,x9,x7) \wedge \text{Cover}(x9,x8,x10)) \rightarrow \text{Cover}(x6,x8,x7))) \wedge \forall x11 x12. (\text{Edge}(x11,x12) \rightarrow \exists x13. \text{Fact}(x11,x12,x13))$

The precondition is five TGDs: Fact triples are all in Cover; Cover is closed under extending its second coordinate along Edge, under moving its first coordinate along Gen, and under composing two Cover triples through a shared middle value; and every Edge pair has some Fact triple witnessing it.

POST: $\forall x y z. (\text{Lin}(x,y,z) \rightarrow \text{Prov}(x,y,z))$

The postcondition is the containment $\text{Lin} \subseteq \text{Prov}$: every lineage triple C1 derives is already present in C2's closed cover relation.

Whiel encoding (Hoare triple).

```
SCHEMA
  Edge/2, Gen/2, Reach/2, Trav/2, Reach_aux/2, Trav_aux/2, Fact/3, Cover/3, Lin/3, Prov/3, Lin_aux/3, Prov_aux/3

PRE
  ((Fact ⊆ Cover) ∧ ((π[0,2,4] (σ[#1 = #3] ((Cover × Edge))) ⊆ π[0,2,1] (Cover)) ∧ ((π[1,2,4] (σ[#0 = #3] ((Cover × Gen))) ⊆
  ↪ π[1,2,0] (Cover)) ∧ ((π[0,2,4] (σ[#1 = #3] ((Cover × Cover))) ⊆ π[0,2,1] (Cover)) ∧ (Edge ⊆ π[0,1] (Fact)))))

CMD
  Reach_aux := ∅[2];
  Lin_aux := ∅[3];
  Reach := (Edge ∪ (π[0,3] (σ[#1 = #2] ((Reach_aux × Reach_aux))) ∪ π[0,1] (Lin_aux)));
  Lin := (Fact ∪ (π[0,4,2] (σ[#1 = #3] ((Lin_aux × Reach_aux))) ∪ π[4,1,2] (σ[#0 = #3] ((Lin_aux × Gen))));
  WHILE (¬((Reach = Reach_aux) ∧ (Lin = Lin_aux))) DO
    Reach_aux := Reach;
    Lin_aux := Lin;
    Reach := (Reach ∪ (Edge ∪ (π[0,3] (σ[#1 = #2] ((Reach_aux × Reach_aux))) ∪ π[0,1] (Lin_aux)));
    Lin := (Lin ∪ (Fact ∪ (π[0,4,2] (σ[#1 = #3] ((Lin_aux × Reach_aux))) ∪ π[4,1,2] (σ[#0 = #3] ((Lin_aux × Gen))));
  END;
  Prov_aux := ∅[3];
  Trav_aux := ∅[2];
  Prov := (Cover ∪ (π[0,4,2] (σ[#1 = #3] ((Prov_aux × Trav_aux))) ∪ π[4,1,2] (σ[#0 = #3] ((Prov_aux × Gen))));
  Trav := (Edge ∪ (π[0,3] (σ[#1 = #2] ((Trav_aux × Trav_aux))));
  WHILE (¬((Prov = Prov_aux) ∧ (Trav = Trav_aux))) DO
    Prov_aux := Prov;
    Trav_aux := Trav;
    Prov := (Prov ∪ (Cover ∪ (π[0,4,2] (σ[#1 = #3] ((Prov_aux × Trav_aux))) ∪ π[4,1,2] (σ[#0 = #3] ((Prov_aux × Gen))));
    Trav := (Trav ∪ (Edge ∪ (π[0,3] (σ[#1 = #2] ((Trav_aux × Trav_aux))));
  END

POST
  (Lin ⊆ Prov)
```

31 Example1094: Routing table contained in an administratively closed distance view

Category	LLM-generated (GPT/Claude benchmark programs); family: two programs under TGD constraints (LLM-generated, Claude Opus; legacy raw_new pipeline).
Kind	containment.
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	dependencies mutual-recursion nonlinear-recursion.
Input	Benchmark/Example1094/Input.lean (canonical notation).
Summary	Routing table contained in an administratively closed distance view.
Sides	naive compiler: program C1 of the retired corpus case Example1094 (output Route) naive compiler: program C2 of the retired corpus case Example1094 (output Dist)
Sources	LLM-generated (Claude Opus), legacy raw_new TGD pipeline this repository the retired corpus case Example1094 (programs C1, C2 and the TGD list tgd0..) TGD pipeline run raw-new-tgd-0002-product: TGDs translated to relational algebra by RelCalc.TGD.listToAssertExpr; programs compiled with the certified naive compiler (read 2026-09-15) method this repository the equivalence-under-constraints paper draft (technical appendix: reduction of containment/equivalence under TGDs to a Hoare triple) (read 2026-09-15) legacy source record raw_tgd_opus_w_inv.json legacy source record Benchmark/Example1094/Input.lean Datalog source noted in the legacy inventory of Example1094
Obstruction note	from the metadata: anchored: both sides are naive closures at the same rate and the legacy product loop was certified with a quantifier-free invariant; no prophecy level expected
Note	Legacy raw_new TGD case (Claude Opus): every peer pair has a relay (existential dependency); containment in an administratively closed distance view.

Datalog program(s). *prog_c1* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
Route(x1, y1) :- Peer(x1, y1).
Hop(x1, y1) :- Adj(x1, y1).
Hop(x1, z1) :- Route(x1, y1), Hop(y1, z1).
Route(x1, z1) :- Route(x1, y1), Hop(y1, z1).
Route(x1, z1) :- Route(x1, y1), Relay(y1, x2, z1).
Route(x1, z1) :- Route(x1, y1), Route(y1, z1).
```

prog_c2 — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
Dist(x1, y1) :- Admin(x1, y1).
Span(x1, y1) :- Adj(x1, y1).
Span(x1, z1) :- Span(x1, y1), Span(y1, z1).
Dist(x1, z1) :- Dist(x1, y1), Span(y1, z1).
Dist(x1, z1) :- Dist(x1, y1), Dist(y1, z1).
```

Reading. Program C1 builds a routing table *Route* from direct peer links (*Peer*), adjacency hops composed through *Hop*, ternary relay legs (*Relay*), and transitive composition of *Route* with itself, a multi-hop route computation typical of network routing. Program C2 recomputes distances *Dist* by reading an administrative relation *Admin* that the TGD constraints force to contain *Peer*, *Adj*, and the endpoints of every *Relay* triple, and to be transitively closed, then closes *Dist* under *Adj*-based spans and self-composition. The claim is the containment $\text{Route} \subseteq \text{Dist}$ under these dependencies (an existential dependency guarantees every *Peer* edge a *Relay* witness), expected valid because both sides are naive closures advancing at the same rate.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in `Input.lean`. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $(\forall x y. (\text{Peer}(x,y) \rightarrow \text{Admin}(x,y))) \wedge (\forall z u. (\text{Adj}(z,u) \rightarrow \text{Admin}(z,u))) \wedge (\forall v w. ((\exists x1. \text{Admin}(v,x1) \wedge \text{Admin}(x1,w)) \rightarrow \text{Admin}(v,w))) \wedge (\forall x2 x3. ((\exists x4. \text{Relay}(x2,x4,x3)) \rightarrow \text{Admin}(x2,x3))) \wedge \forall x5 x6. (\text{Peer}(x5,x6) \rightarrow \exists x7. \text{Relay}(x5,x7,x6))$

The precondition is five TGDs: *Peer* and *Adj* are contained in *Admin*, *Admin* is transitively closed, the endpoints of every *Relay* triple are in *Admin*, and every *Peer* edge has some *Relay* triple witnessing it.

POST: $\forall x y. (\text{Route}(x,y) \rightarrow \text{Dist}(x,y))$

The postcondition is the containment $\text{Route} \subseteq \text{Dist}$: every route pair C1 computes is also a distance pair C2 computes.

Whiel encoding (Hoare triple).

```

SCHEMA
  Peer/2, Adj/2, Admin/2, Route/2, Hop/2, Dist/2, Span/2, Route_aux/2, Hop_aux/2, Dist_aux/2, Span_aux/2, Relay/3

PRE
  ((Peer ⊆ Admin) ∧ ((Adj ⊆ Admin) ∧ ((π[0,3] (σ[#1 = #2] ((Admin × Admin))) ⊆ Admin) ∧ ((π[0,2] (Relay) ⊆ Admin) ∧ (Peer ⊆
  ↪ π[0,2] (Relay)))))

CMD
  Route_aux := ∅[2];
  Hop_aux := ∅[2];
  Route := (Peer ∪ (π[0,3] (σ[#1 = #2] ((Route_aux × Hop_aux))) ∪ (π[0,4] (σ[#1 = #2] ((Route_aux × Relay))) ∪ π[0,3] (σ[#1 =
  ↪ #2] ((Route_aux × Route_aux))))));
  Hop := (Adj ∪ π[0,3] (σ[#1 = #2] ((Route_aux × Hop_aux))));
  WHILE (¬(((Route = Route_aux) ∧ (Hop = Hop_aux)))) DO
    Route_aux := Route;
    Hop_aux := Hop;
    Route := (Route ∪ (Peer ∪ (π[0,3] (σ[#1 = #2] ((Route_aux × Hop_aux))) ∪ (π[0,4] (σ[#1 = #2] ((Route_aux × Relay))) ∪
    ↪ π[0,3] (σ[#1 = #2] ((Route_aux × Route_aux))))));
    Hop := (Hop ∪ (Adj ∪ π[0,3] (σ[#1 = #2] ((Route_aux × Hop_aux))));
  END;
  Dist_aux := ∅[2];
  Span_aux := ∅[2];
  Dist := (Admin ∪ (π[0,3] (σ[#1 = #2] ((Dist_aux × Span_aux))) ∪ π[0,3] (σ[#1 = #2] ((Dist_aux × Dist_aux)))));
  Span := (Adj ∪ π[0,3] (σ[#1 = #2] ((Span_aux × Span_aux))));
  WHILE (¬(((Dist = Dist_aux) ∧ (Span = Span_aux)))) DO
    Dist_aux := Dist;
    Span_aux := Span;
    Dist := (Dist ∪ (Admin ∪ (π[0,3] (σ[#1 = #2] ((Dist_aux × Span_aux))) ∪ π[0,3] (σ[#1 = #2] ((Dist_aux × Dist_aux)))));
    Span := (Span ∪ (Adj ∪ π[0,3] (σ[#1 = #2] ((Span_aux × Span_aux))));
  END

POST
  (Route ⊆ Dist)

```

32 Example2004: Magic-set transformation: demanded answers equal the original program's answers (Soufflé)

Category	production (derived from a production system's source or documentation); family: Obstructed.
Kind	equivalence (stated as mutual containments).
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	database-engine magic-sets prophecy.
Input	Benchmark/Example2004/Input.lean (canonical notation).
Summary	A magic-set transformation of a right-linear path program, against the program it was derived from.
Sources	Soufflé magic-set transformation (documentation) the equivalence-and-containment paper draft (the pre-fixpoint witness method for equivalence and containment) retired-corpus source record production_4 of the Production family retired-corpus input record production_4 of the Production family Datalog source noted in the legacy inventory <code>reports/evidence/gen_examples.py</code>
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).
Note	The canonical re-encoding of the legacy obstructed case; the query seed Root is pinned to {"root"} by the precondition.

Datalog program(s). *programM* — *The magic-set transformation: demand and answers.* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
MPath(x1) :- Root(x1).
MPath(y1) :- MPath(x1), Edge(x1, y1).
PathBF(x1, y1) :- MPath(x1), Edge(x1, y1).
PathBF(x1, x2) :- MPath(x1), Edge(x1, y1), PathBF(y1, x2).
```

programP — *The original right-linear path program.* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
Path(x1, y1) :- Edge(x1, y1).
Path(x1, x2) :- Edge(x1, y1), Path(y1, x2).
```

Reading. This case checks a magic-set transformation of a right-linear reachability program: the transformed program seeds a demand set `MPath` from `Root` and propagates it forward along `Edge`, then computes `PathBF` only for pairs whose source is in the demand set, while the original program `Path` computes unrestricted transitive closure of `Edge`. This is the standard magic-set/demand-driven query optimization used to avoid computing answers a Datalog engine will not need. The postcondition states both directions of correctness for the transformation restricted to the demand set: every original-program answer whose source is demanded is also produced by the transformed program, and every transformed-program answer is also an original-program answer; each relation starts empty and monotonically accumulates until it stops changing, so both hold the least fixpoints of their respective programs.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\forall x. (\text{Root}(x) \leftrightarrow x = \text{"root"})$

The precondition pins the query's demand seed to the single source "root", i.e. $\text{Root} = \{\text{"root"}\}$.

POST: $(\forall x y. (\text{MPath}(x) \wedge \text{Path}(x,y) \rightarrow \text{PathBF}(x,y))) \wedge \forall z u. (\text{PathBF}(z,u) \rightarrow \text{Path}(z,u))$

The postcondition asserts both completeness and soundness of the transformation on the demanded sources: every Path pair whose source is in MPath is also a PathBF pair, and every PathBF pair is also a Path pair.

Whiel encoding (Hoare triple).

```

SCHEMA
  Root/1, MPath/1, MPath_aux/1, Edge/2, PathBF/2, PathBF_aux/2, Path/2, Path_aux/2

PRE
  (Root = { "root" })

CMD
  MPath_aux :=  $\emptyset[1]$ ;
  PathBF_aux :=  $\emptyset[2]$ ;
  MPath :=
    (Root  $\sqcup$ 
     ( $\pi[2]$  ( $\sigma[\#0 = \#1]$  (MPath_aux  $\times$  Edge))));
  PathBF :=
    (( $\pi[0, 2]$ 
     ( $\sigma[\#0 = \#1]$  (MPath_aux  $\times$  Edge)))  $\sqcup$ 
     ( $\pi[0, 4]$ 
      ( $\sigma[(\#0 = \#1) \wedge (\#2 = \#3)]$ 
       (MPath_aux  $\times$  (Edge  $\times$  PathBF_aux)))));
  WHILE
    ( $\neg$  ((MPath = MPath_aux)  $\wedge$ 
          (PathBF = PathBF_aux))) DO
    MPath_aux := MPath;
    PathBF_aux := PathBF;
    MPath :=
      (MPath  $\sqcup$ 
       (Root  $\sqcup$ 
        ( $\pi[2]$ 
         ( $\sigma[\#0 = \#1]$  (MPath_aux  $\times$  Edge))));
    PathBF :=
      (PathBF  $\sqcup$ 
       (( $\pi[0, 2]$ 
        ( $\sigma[\#0 = \#1]$  (MPath_aux  $\times$  Edge)))  $\sqcup$ 
        ( $\pi[0, 4]$ 
         ( $\sigma[(\#0 = \#1) \wedge (\#2 = \#3)]$ 
          (MPath_aux  $\times$ 
           (Edge  $\times$  PathBF_aux))))));
  END;
  Path_aux :=  $\emptyset[2]$ ;
  Path :=
    (Edge  $\sqcup$ 
     ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (Edge  $\times$  Path_aux))));
  WHILE ( $\neg$  (Path = Path_aux)) DO
    Path_aux := Path;
    Path :=
      (Path  $\sqcup$ 
       (Edge  $\sqcup$ 
        ( $\pi[0, 3]$ 
         ( $\sigma[\#1 = \#2]$  (Edge  $\times$  Path_aux))));
  END

POST
  (( $\pi[1, 2]$  ( $\sigma[\#0 = \#1]$  (MPath  $\times$  Path)))
    $\subseteq$  PathBF)  $\wedge$ 
  (PathBF  $\subseteq$  Path)

```

33 Example2005: Restricted-edge reachability is contained in full-edge reachability (edge-monotonicity)

Category	production (derived from a production system's source or documentation); family: Production.
Kind	containment (via a pre-fixpoint witness).
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	fixpoint-witness negation.
Input	Benchmark/Example2005/Input.lean (canonical notation).
Summary	containment_03: Restricted-edge reachability is contained in full-edge reachability (edge-monotonicity).
Sources	npm/arborist lib/calc-dep-flags.js the equivalence-and-containment paper draft (the pre-fixpoint witness method for equivalence and containment) retired-corpus source record production_5 of the Production family retired-corpus input record production_5 of the Production family

Datalog program(s). *prog_p* — production check (the hand-written loop equals this program's naive compilation on the check instances).

```
NonOptC(x1) :- RootN(x1).
NonOptC(y1) :- NonOptC(x1), EdgeHardC(x1, y1).
```

Reading. The program computes graph reachability from a root set RootN restricted to "hard" (non-optional) edges only, modeling the analysis a dependency resolver like npm's Arborist uses to decide which packages keep a required (non-optional) dependency path from the root. The Hoare triple is a containment claim rather than full equivalence: the precondition makes RBound a pre-fixpoint of the reachability operator over the unrestricted edge set EdgeAll, and the postcondition asserts the hard-edge-only reachable set NonOpt is contained in RBound, i.e. contained in full-edge reachability, the standard edge-monotonicity fact that removing edges (here, optional ones) can only shrink what is reachable.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in Input.lean remains the authority.

PRE: $\forall x. (\text{RootN}(x) \vee (\exists y. \text{EdgeAll}(y, x) \wedge \text{RBound}(y)) \rightarrow \text{RBound}(x))$

The precondition asserts RBound contains RootN and is closed under one more step of reachability through the full edge set EdgeAll, i.e. RBound is a pre-fixpoint of full-edge reachability.

POST: $\forall x. (\text{NonOpt}(x) \rightarrow \text{RBound}(x))$

The postcondition asserts the hard-edge-restricted reachable set NonOpt is contained in RBound.

Whiel encoding (Hoare triple).

```
SCHEMA
  RootN/1, NonOpt/1, NonOpt_aux/1, RBound/1, EdgeAll/2, EdgeOpt/2, EdgeHard/2

PRE
  ((RootN  $\cup$  ( $\pi$ [1] ( $\sigma$ [#0 = #2] (EdgeAll  $\times$  RBound))))  $\subseteq$  RBound)

CMD
  NonOpt_aux :=  $\emptyset$ ;
  EdgeHard := (EdgeAll  $\setminus$  EdgeOpt);
  NonOpt := RootN;
```

```

WHILE NonOpt ≠ NonOpt_aux DO
  NonOpt_aux := NonOpt;
  NonOpt := (RootN ∪
    (π[1] (σ[#0 = #2] (EdgeHard × NonOpt_aux))))
END

POST
  (NonOpt ⊆ RBound)

```

34 Example2006: Policy-filtered connectivity is contained in raw topological connectivity

Category	production (derived from a production system's source or documentation); family: Production.
Kind	containment (via a pre-fixpoint witness).
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	<code>fixpoint-witness negation</code> .
Input	<code>Benchmark/Example2006/Input.lean</code> (canonical notation).
Summary	containment_04: Policy-filtered connectivity is contained in raw topological connectivity.
Sources	Cilium network policy (documentation) batfish/batfish (network configuration analysis) the equivalence-and-containment paper draft (the pre-fixpoint witness method for equivalence and containment) retired-corpus source record production_6 of the Production family retired-corpus input record production_6 of the Production family

Datalog program(s). *prog_p* — production check (the hand-written loop equals this program's naive compilation on the check instances).

```

ConnC(x1, y1) :- AllowedC(x1, y1).
ConnC(x1, z1) :- ConnC(x1, y1), AllowedC(y1, z1).

```

Reading. The program computes reachability (Conn) over an edge set filtered by removing denied links (LinkE \ Denied), modeling how a network-policy engine like Cilium restricts which nodes remain connected under an access policy, contrasted with the unrestricted topological reachability a tool like Batfish would compute over the full link set LinkE. The Hoare triple is a containment claim: the precondition makes RBound a pre-fixpoint of the reachability operator over the full, unfiltered edge set LinkE, and the postcondition asserts the policy-filtered connectivity Conn is contained in RBound, i.e. contained in unrestricted reachability, since network policies can only remove connectivity, never add it.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\forall x y. (\text{LinkE}(x,y) \vee (\exists z. \text{RBound}(x,z) \wedge \text{LinkE}(z,y)) \rightarrow \text{RBound}(x,y))$

The precondition asserts RBound contains LinkE and is closed under one more step of composing itself with LinkE, i.e. RBound is a pre-fixpoint of full-topology reachability.

POST: $\forall x y. (\text{Conn}(x,y) \rightarrow \text{RBound}(x,y))$

The postcondition asserts the policy-filtered connectivity relation Conn is contained in RBound.

Whiel encoding (Hoare triple).

```

SCHEMA
  LinkE/2, Denied/2, AllowedE/2, Conn/2, Conn_aux/2, RBound/2

PRE
  ((LinkE  $\cup$  ( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (RBound  $\times$  LinkE))))  $\subseteq$  RBound)

CMD
  Conn_aux :=  $\emptyset$ ;
  AllowedE := (LinkE  $\setminus$  Denied);
  Conn := (LinkE  $\setminus$  Denied);
  WHILE Conn  $\neq$  Conn_aux DO
    Conn_aux := Conn;
    Conn := (AllowedE  $\cup$ 
      ( $\pi$ [0, 3] ( $\sigma$ [#1 = #2] (Conn_aux  $\times$  AllowedE))))
  END

POST
  (Conn  $\subseteq$  RBound)

```

35 Example2007: $E[f \text{ U } g]$ is contained in $EF \text{ } g$

Category	production (derived from a production system's source or documentation); family: Production.
Kind	containment (via a pre-fixpoint witness).
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	fixpoint-witness semi-naive.
Input	Benchmark/Example2007/Input.lean (canonical notation).
Summary	containment_05: $E[f \text{ U } g]$ is contained in $EF \text{ } g$.
Sources	NuSMV 2.6.0 core/mc/mcMc.c, eu() (hklarner/NuSMV-a mirror) NuSMV user manual, CTL model checking the equivalence-and-containment paper draft (the pre-fixpoint witness method for equivalence and containment) retired-corpus source record production_7 of the Production family retired-corpus input record production_7 of the Production family

Datalog program(s). *prog_p* — production check (the hand-written loop equals this program's naive compilation on the check instances).

```

YC(x1) :- SatG(x1).
YC(x1) :- SatF(x1), Trans(x1, y1), YC(y1).

```

Reading. The loop computes a CTL-style backward-reachability fixpoint: starting from states satisfying g (SatG) and repeatedly adding predecessors along Trans that also satisfy f (SatF), it computes the states satisfying the until formula $E[f \text{ U } g]$, mirroring the frontier-based eu() computation in a model checker such as NuSMV. The Hoare triple is a containment claim: the precondition makes RBound a pre-fixpoint of the unconstrained backward-reachability operator that computes $EF \text{ } g$ (reachability with no f -restriction), and the postcondition asserts the $E[f \text{ U } g]$ result Y is contained in RBound, the textbook CTL fact that any until-witness for g is in particular an eventually-witness for g .

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in `Input.lean`. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\forall x. (\text{SatG}(x) \vee (\exists y. \text{Trans}(x,y) \wedge \text{RBound}(y)) \rightarrow \text{RBound}(x))$

The precondition asserts `RBound` contains `SatG` and is closed under taking one more step of predecessors through `Trans`, i.e. `RBound` is a pre-fixpoint of the operator computing `EF g`.

POST: $\forall x. (Y(x) \rightarrow \text{RBound}(x))$

The postcondition asserts the computed until-set `Y` is contained in `RBound`.

Whiel encoding (Hoare triple).

```
SCHEMA
  SatF/1, SatG/1, Y/1, Yold/1, NewS/1, PreS/1, RBound/1, Trans/2

PRE
  ((SatG  $\cup$  ( $\pi$ [0] ( $\sigma$ [#1 = #2] (Trans  $\times$  RBound))))  $\subseteq$  RBound)

CMD
  Y := SatG;
  NewS := SatG;
  WHILE NewS  $\neq$   $\emptyset$  DO
    Yold := Y;
    PreS := ( $\pi$ [0] ( $\sigma$ [#1 = #2] (Trans  $\times$  NewS)));
    Y := (Y  $\cup$  (SatF  $\setminus$  (SatF  $\setminus$  PreS)));
    NewS := (Y  $\setminus$  Yold)
  END

POST
  (Y  $\subseteq$  RBound)
```

36 Example2010: Semi-naive evaluation $\equiv$ naive/declarative transitive closure

Category	production (derived from a production system's source or documentation); family: Production.
Kind	equivalence (with a least fixpoint, via a pre-fixpoint witness).
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	database-engine evaluation-strategies fixpoint-witness semi-naive.
Input	Benchmark/Example2010/Input.lean (canonical notation).
Summary	equivalence_01: Semi-naive evaluation $\equiv$ naive/declarative transitive closure.
Sources	postgres/postgres src/backend/executor/nodeRecursiveunion.c (ExecRecursiveUnion) Soufflé translation to C++ / semi-naive loop (documentation) the equivalence-and-containment paper draft (the pre-fixpoint witness method for equivalence and containment) retired-corpus source record production_10 of the Production family retired-corpus input record production_10 of the Production family

Datalog program(s). *prog_p* — production check (the hand-written loop equals this program's naive compilation on the check instances).

```
ResC(x1, y1) :- Edge(x1, y1).
ResC(x1, z1) :- ResC(x1, y1), Edge(y1, z1).
```

Reading. The loop computes transitive closure of Edge using a semi-naive evaluation strategy: it maintains an accumulated result Res and a frontier WorkT of newly derived pairs, extends WorkT through Edge, keeps only pairs not already in Res, and terminates when the frontier is empty, the standard semi-naive optimization of naive bottom-up Datalog evaluation, which would otherwise recompute the full join every round. The Hoare triple is the least-fixpoint correctness statement: the precondition makes RBound a pre-fixpoint of the naive transitive-closure operator, and the postcondition shows Res is itself a pre-fixpoint of that operator and is contained in RBound, which together pin Res to the least fixpoint, i.e. semi-naive evaluation computes exactly the same transitive closure as naive evaluation.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\forall x y. (\text{Edge}(x,y) \vee (\exists z. \text{RBound}(x,z) \wedge \text{Edge}(z,y)) \rightarrow \text{RBound}(x,y))$

The precondition asserts RBound contains Edge and is closed under one more step of composing itself with Edge, i.e. RBound is a pre-fixpoint of the naive transitive-closure operator.

POST: $(\forall x y. (\text{Edge}(x,y) \vee (\exists z. \text{Res}(x,z) \wedge \text{Edge}(z,y)) \rightarrow \text{Res}(x,y))) \wedge \forall u v. (\text{Res}(u,v) \rightarrow \text{RBound}(u,v))$

The postcondition asserts the semi-naive result Res is itself a pre-fixpoint of the same operator (contains Edge and is closed under composing with Edge) and is contained in RBound, together forcing Res to equal the least fixpoint.

Whiel encoding (Hoare triple).

```

SCHEMA
  Edge/2, Res/2, WorkT/2, Inter/2, RBound/2

PRE
  ((Edge  $\cup$  ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (RBound  $\times$  Edge))))  $\subseteq$  RBound)

CMD
  Res := Edge;
  WorkT := Edge;
  WHILE WorkT  $\neq$   $\emptyset$  DO
    Inter := (( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (WorkT  $\times$  Edge)))  $\setminus$  Res);
    Res := (Res  $\cup$  Inter);
    WorkT := Inter
  END

POST
  ((Edge  $\cup$  ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (Res  $\times$  Edge))))  $\subseteq$  Res)  $\wedge$ 
  (Res  $\subseteq$  RBound)

```

37 Example2011: Incremental view maintenance $\equiv$ from-scratch re-computation

Category	production (derived from a production system's source or documentation); family: Production.
Kind	equivalence (with a least fixpoint, via a pre-fixpoint witness).
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	evaluation-strategies fixpoint-witness view-maintenance.
Input	Benchmark/Example2011/Input.lean (canonical notation).
Summary	equivalence_02: Incremental view maintenance $\equiv$ from-scratch recomputation.
Sources	TimelyDataflow/differential-dataflow Materialize arrangements / incremental dataflows (documentation) the equivalence-and-containment paper draft (the pre-fixpoint witness method for equivalence and containment) retired-corpus source record production_11 of the Production family retired-corpus input record production_11 of the Production family
Note	Models insert-only re-evaluation restarted from the maintained view; Differential Dataflow/Materialize are the motivation, not the mechanism.

Datalog program(s). *prog_p* — production check (the hand-written loop equals this program's naive compilation on the check instances).

```

ReachC(x1) :- Roots(x1).
ReachC(x1) :- ReachOld(x1).
ReachC(y1) :- ReachC(x1), EBase(x1, y1).
ReachC(y1) :- ReachC(x1), EDelta(x1, y1).

```

Reading. The loop computes reachability incrementally: it restarts from a previously maintained view *ReachOld* and re-propagates from *Roots* and *ReachOld* through the updated edge set $EBase \cup EDelta$ until stable, modeling insert-only incremental view maintenance in a streaming or materialized-view system (the case notes this is simplified insert-only re-evaluation, not full timestamped differential-dataflow propagation, though the claim being checked is the guarantee that differential dataflow systems provide). The Hoare triple is the least-fixpoint correctness statement: the precondition constrains *ReachOld* to be a pre-fixpoint of the old edge set's operator and *RBound* to be a pre-fixpoint of the updated operator with $ReachOld \subseteq RBound$ standing in for *ReachOld*'s minimality, and the postcondition shows the incrementally recomputed *Reach* is itself a pre-fixpoint of the updated operator and is contained in *RBound*, together pinning *Reach* to the least fixpoint over $EBase \cup EDelta$, i.e. the incremental update yields exactly what a from-scratch recomputation would.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in *Input.lean* remains the authority.

PRE: $(\forall x. (Roots(x) \vee (\exists y. EBase(y,x) \wedge ReachOld(y)) \rightarrow ReachOld(x))) \wedge (\forall z. (Roots(z) \vee (\exists u. (EBase(u,z) \vee EDelta(u,z)) \wedge RBound(u)) \rightarrow RBound(z))) \wedge \forall v. (ReachOld(v) \rightarrow RBound(v))$

The precondition asserts *ReachOld* is a pre-fixpoint of reachability over *EBase* alone, and that *RBound* is a pre-fixpoint of reachability over the updated edge set $EBase \cup EDelta$ and contains *ReachOld*.

POST: $(\forall x. (Roots(x) \vee (\exists y. (EBase(y,x) \vee EDelta(y,x)) \wedge Reach(y)) \rightarrow Reach(x))) \wedge \forall z. (Reach(z) \rightarrow RBound(z))$

The postcondition asserts the incrementally updated *Reach* is itself a pre-fixpoint of reachability over $EBase \cup EDelta$ and is contained in *RBound*, together forcing *Reach* to equal the least fixpoint over the updated edges.

Whiel encoding (Hoare triple).

```

SCHEMA
  Roots/1, ReachOld/1, Reach/1, Reach_aux/1, RBound/1, EBase/2, EDelta/2

PRE
  ((Roots  $\cup$  ( $\pi[1]$  ( $\sigma[\#0 = \#2]$  ( $EBase \times ReachOld$ ))))  $\subseteq$  ReachOld)  $\wedge$ 
  (((Roots  $\cup$  ( $\pi[1]$  ( $\sigma[\#0 = \#2]$  ( $(EBase \cup EDelta) \times RBound$ ))))  $\subseteq$  RBound)  $\wedge$ 
  (ReachOld  $\subseteq$  RBound))

CMD
  Reach_aux :=  $\emptyset$ ;
  Reach := (ReachOld  $\cup$  Roots);
  WHILE Reach  $\neq$  Reach_aux DO
    Reach_aux := Reach;
    Reach := ((Roots  $\cup$  ReachOld)  $\cup$ 
      ( $\pi[1]$  ( $\sigma[\#0 = \#2]$  ( $(EBase \cup EDelta) \times Reach_aux$ ))))
  END

POST
  ((Roots  $\cup$  ( $\pi[1]$  ( $\sigma[\#0 = \#2]$  ( $(EBase \cup EDelta) \times Reach$ ))))  $\subseteq$  Reach)  $\wedge$ 
  (Reach  $\subseteq$  RBound)

```

38 Example2012: Frontier (grey-set) marking $\equiv$ naive mark propagation

Category	production (derived from a production system's source or documentation); family: Production.
Kind	equivalence (with a least fixpoint, via a pre-fixpoint witness).
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	evaluation-strategies fixpoint-witness semi-naive.
Input	Benchmark/Example2012/Input.lean (canonical notation).
Summary	equivalence_03: Frontier (grey-set) marking $\equiv$ naive mark propagation.
Sources	golang/go src/runtime/mgcmark.go (gcDrain, scanobject, greyobject) the equivalence-and-containment paper draft (the pre-fixpoint witness method for equivalence and containment) retired-corpus source record production_12 of the Production family retired-corpus input record production_12 of the Production family

Datalog program(s). *prog_p* — production check (the hand-written loop equals this program's naive compilation on the check instances).

```

MarkedC(x1) :- RootPtr(x1).
MarkedC(y1) :- MarkedC(x1), HeapPtr(x1, y1).

```

Reading. The loop computes reachability from root pointers `RootPtr` through heap pointers `HeapPtr` using an explicit grey frontier `Grey` that is repeatedly advanced and deduplicated against the accumulated `Marked` set, the tri-color/frontier marking strategy used by a garbage collector's mark phase (e.g. a Go-runtime-style `gcDrain`), contrasted with naively re-propagating over the whole marked set each round. The Hoare triple is the least-fixpoint correctness statement: the precondition makes `RBound` a pre-fixpoint of the naive mark-propagation operator, and the postcondition shows the frontier-computed `Marked` set is itself a pre-fixpoint of that operator and is contained in `RBound`, together pinning `Marked` to the least fixpoint, i.e. frontier-based marking with its already-marked dedup marks exactly the same objects naive re-scanning would.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\forall x. (\text{RootPtr}(x) \vee (\exists y. \text{HeapPtr}(y,x) \wedge \text{RBound}(y)) \rightarrow \text{RBound}(x))$

The precondition asserts `RBound` contains `RootPtr` and is closed under one more step of composing itself with `HeapPtr`, i.e. `RBound` is a pre-fixpoint of the naive mark-propagation operator.

POST: $(\forall x. (\text{RootPtr}(x) \vee (\exists y. \text{HeapPtr}(y,x) \wedge \text{Marked}(y)) \rightarrow \text{Marked}(x))) \wedge \forall z. (\text{Marked}(z) \rightarrow \text{RBound}(z))$

The postcondition asserts the frontier-computed `Marked` set is itself a pre-fixpoint of the same operator (contains `RootPtr` and is closed under `HeapPtr`) and is contained in `RBound`, together forcing `Marked` to equal the least fixpoint.

Whiel encoding (Hoare triple).

```

SCHEMA
  RootPtr/1, Marked/1, Grey/1, NewG/1, RBound/1, HeapPtr/2

PRE
  ((RootPtr  $\cup$  ( $\pi[1]$  ( $\sigma[\#0 = \#2]$  ( $\text{HeapPtr} \times \text{RBound}$ ))))  $\subseteq$  RBound)

CMD
  Marked := RootPtr;
  Grey := RootPtr;
  WHILE Grey  $\neq$   $\emptyset$  DO
    NewG := (( $\pi[1]$  ( $\sigma[\#0 = \#2]$  ( $\text{HeapPtr} \times \text{Grey}$ ))))  $\setminus$  Marked;
    Marked := (Marked  $\cup$  NewG);
    Grey := NewG
  END

POST
  ((RootPtr  $\cup$  ( $\pi[1]$  ( $\sigma[\#0 = \#2]$  ( $\text{HeapPtr} \times \text{Marked}$ ))))  $\subseteq$  Marked)  $\wedge$ 
  (Marked  $\subseteq$  RBound)

```

39 Example3001: Audited delegation alternation loses an uncertified reviewed hop

Category	LLM-generated (GPT/Claude benchmark programs); family: two programs under TGD constraints (LLM-generated, GPT (raw_invalid_gpt batch); legacy raw_new pipeline), invalid.
Kind	containment.
Contributors	Leo Zhang.
Status	certified invalid (Certificate/Invalid.lean, written by the certificate emitter); expected invalid.
Tags	dependencies mutual-recursion nonlinear-recursion.
Input	Benchmark/Example3001/Input.lean (canonical notation).
Summary	Audited delegation alternation loses an uncertified reviewed hop.
Sides	naive compiler: program C1 of the retired corpus case Example3001 (output AccessOne) naive compiler: program C2 of the retired corpus case Example3001 (output AccessTwo)
Sources	LLM-generated (GPT (raw_invalid_gpt batch)), legacy raw_new TGD pipeline this repository the retired corpus case Example3001 (programs C1, C2 and the TGD list tgd0..) TGD pipeline run raw-new-tgd-0002-product: TGDs translated to relational algebra by RelCalc.TGD.listToAssertExpr; programs compiled with the certified naive compiler (read 2026-09-15) method this repository the equivalence-under-constraints paper draft (technical appendix: reduction of containment/equivalence under TGDs to a Hoare triple) (read 2026-09-15) legacy source record raw_invalid_gpt.json legacy source record Benchmark/Example3001/Input.lean Datalog source noted in the legacy inventory of Example3001

Obstruction note

from the metadata: invalid (legacy kernel counterexample)

Note

Legacy raw_new invalid TGD case (GPT): the second program demands a certificate on reviewed hops that the dependencies do not guarantee.

Datalog program(s). *prog_c1* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
AccessOne(x1) :- Root(x1).
CertStateOne(y1) :- AccessOne(x1), Certified(x1, y1).
ReviewStateOne(y1) :- AccessOne(x1), Review(x1, y1, z1), Reviewer(z1).
AccessOne(y1) :- CertStateOne(x1), Review(x1, y1, z1), Reviewer(z1).
AccessOne(y1) :- ReviewStateOne(x1), Certified(x1, y1).
AccessOne(y1) :- CertStateOne(y1), ReviewStateOne(y1), Delegates(x1, y1).
```

prog_c2 — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
AccessTwo(x1) :- Root(x1).
CertStateTwo(y1) :- AccessTwo(x1), Certified(x1, y1).
ReviewStateTwo(y1) :- AccessTwo(x1), Review(x1, y1, z1), Reviewer(z1).
JointStateTwo(y1) :- CertStateTwo(x1), Review(x1, y1, z1), Reviewer(z1), Certified(x1, y1).
AccessTwo(y1) :- JointStateTwo(y1), Delegates(x1, y1).
AccessTwo(y1) :- CertStateTwo(y1), ReviewStateTwo(y1), Delegates(x1, y1).
```

Reading. Program C1 computes an access set *AccessOne* via delegation that alternates between a certified-state check and a reviewer-backed review state, allowing a step whenever either state alone combines with a background *Delegates* edge, and additionally whenever a certified hop and a reviewed hop chain together (certified-then-reviewed or reviewed-then-certified). Program C2 computes *AccessTwo* with an analogous structure but replaces that alternation with a stricter *JointStateTwo* requiring a reviewed hop to also be independently certified before it can combine with *Delegates*. This models an authorization engine comparing an alternating certified/reviewed delegation policy against a stricter one that insists a reviewed transition also be certified. The Hoare triple claims containment $\text{AccessOne} \subseteq \text{AccessTwo}$ under five TGDs relating *Certified*, *Review*, *Reviewer*, and *Delegates*, but the claim fails: the case records an explicit witness instance on which C1's alternating chaining reaches a principal that C2's stricter jointly-certified-and-reviewed requirement cannot reach, because C2 loses a deeply authorized principal that is only reachable through an uncertified reviewed hop.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in *Input.lean*. The relational-algebra assertion in *Input.lean* remains the authority.

PRE: $(\forall x y. (\text{Certified}(x,y) \rightarrow \text{Delegates}(x,y))) \wedge (\forall z u. ((\exists v. \text{Review}(z,u,v)) \rightarrow \text{Delegates}(z,u))) \wedge (\forall w. ((\exists x1 x2. \text{Review}(x1,x2,w)) \rightarrow \text{Reviewer}(w))) \wedge (\forall x3 x4. (\text{Certified}(x3,x4) \rightarrow \exists x5. \text{Review}(x3,x4,x5) \wedge \text{Reviewer}(x5))) \wedge \forall x6 x7. ((\exists x8 x9. \text{Certified}(x6,x8) \wedge \text{Review}(x8,x7,x9)) \rightarrow \text{Delegates}(x6,x7))$

The precondition is five TGDs: *Certified* and the first two coordinates of *Review* are contained in *Delegates*, the third coordinate of *Review* is a *Reviewer*, every *Certified* pair has a *Review/Reviewer* witness, and composing a *Certified* pair with a *Review* triple through their shared value lands in *Delegates*.

POST: $\forall x. (\text{AccessOne}(x) \rightarrow \text{AccessTwo}(x))$

The postcondition claims $\text{AccessOne} \subseteq \text{AccessTwo}$, but the case is invalid: the recorded witness instance shows a principal that C1's alternating certified/reviewed delegation reaches while C2's stricter jointly-certified delegation does not.

Whiel encoding (Hoare triple).

```
SCHEMA
  Root/1, Reviewer/1, AccessOne/1, CertStateOne/1, ReviewStateOne/1, AccessTwo/1, CertStateTwo/1, ReviewStateTwo/1, JointStateTwo/1, AccessOne_aux/1, CertStateOne_aux/1,
  ReviewStateOne_aux/1, AccessTwo_aux/1, CertStateTwo_aux/1, ReviewStateTwo_aux/1, JointStateTwo_aux/1, Certified/2, Delegates/2, Review/3
```

```

PRE
  ((Certified  $\subseteq$  Delegates)  $\wedge$  (( $\pi[0,1]$  (Review)  $\subseteq$  Delegates)  $\wedge$  (( $\pi[2]$  (Review)  $\subseteq$  Reviewer)  $\wedge$  ((Certified  $\subseteq$   $\pi[0,1]$  ( $\sigma[\#2 = \#3]$  ((Review  $\times$  Reviewer))))  $\wedge$  ( $\pi[0,3]$  ( $\sigma[\#1 = \#2]$   $\hookrightarrow$  ((Certified  $\times$  Review)))  $\subseteq$  Delegates))))))

CMD
  AccessOne_aux :=  $\emptyset[1]$ ;
  CertStateOne_aux :=  $\emptyset[1]$ ;
  ReviewStateOne_aux :=  $\emptyset[1]$ ;
  AccessOne := (Root  $\cup$  ( $\pi[2]$  ( $\sigma[\#0 = \#1 \wedge \#3 = \#4]$  ((CertStateOne_aux  $\times$  (Review  $\times$  Reviewer))))  $\cup$  ( $\pi[2]$  ( $\sigma[\#0 = \#1]$  ((ReviewStateOne_aux  $\times$  Certified)))  $\cup$  ( $\pi[0]$  ( $\sigma[\#0 = \#1 \wedge \#0 = \#3]$  ((CertStateOne_aux  $\times$  (ReviewStateOne_aux  $\times$  Delegates))))))));
  CertStateOne :=  $\pi[2]$  ( $\sigma[\#0 = \#1]$  ((AccessOne_aux  $\times$  Certified)));
  ReviewStateOne :=  $\pi[2]$  ( $\sigma[\#0 = \#1 \wedge \#3 = \#4]$  ((AccessOne_aux  $\times$  (Review  $\times$  Reviewer))));
  WHILE ( $\neg$ ((AccessOne = AccessOne_aux)  $\wedge$  ((CertStateOne = CertStateOne_aux)  $\wedge$  (ReviewStateOne = ReviewStateOne_aux)))) DO
    AccessOne_aux := AccessOne;
    CertStateOne_aux := CertStateOne;
    ReviewStateOne_aux := ReviewStateOne;
    AccessOne := (AccessOne  $\cup$  (Root  $\cup$  ( $\pi[2]$  ( $\sigma[\#0 = \#1 \wedge \#3 = \#4]$  ((CertStateOne_aux  $\times$  (Review  $\times$  Reviewer))))  $\cup$  ( $\pi[2]$  ( $\sigma[\#0 = \#1]$  ((ReviewStateOne_aux  $\times$  Certified)))  $\cup$  ( $\pi[0]$  ( $\sigma[\#0 = \#1 \wedge \#0 = \#3]$  ((CertStateOne_aux  $\times$  (ReviewStateOne_aux  $\times$  Delegates))))))));
    CertStateOne := (CertStateOne  $\cup$   $\pi[2]$  ( $\sigma[\#0 = \#1]$  ((AccessOne_aux  $\times$  Certified)));
    ReviewStateOne := (ReviewStateOne  $\cup$   $\pi[2]$  ( $\sigma[\#0 = \#1 \wedge \#3 = \#4]$  ((AccessOne_aux  $\times$  (Review  $\times$  Reviewer)))));
  END;
  AccessTwo_aux :=  $\emptyset[1]$ ;
  CertStateTwo_aux :=  $\emptyset[1]$ ;
  ReviewStateTwo_aux :=  $\emptyset[1]$ ;
  JointStateTwo_aux :=  $\emptyset[1]$ ;
  AccessTwo := (Root  $\cup$  ( $\pi[0]$  ( $\sigma[\#0 = \#2]$  ((JointStateTwo_aux  $\times$  Delegates)))  $\cup$  ( $\pi[0]$  ( $\sigma[\#0 = \#1 \wedge \#0 = \#3]$  ((CertStateTwo_aux  $\times$  (ReviewStateTwo_aux  $\times$  Delegates))))));
  CertStateTwo :=  $\pi[2]$  ( $\sigma[\#0 = \#1]$  ((AccessTwo_aux  $\times$  Certified)));
  ReviewStateTwo :=  $\pi[2]$  ( $\sigma[\#0 = \#1 \wedge \#3 = \#4]$  ((AccessTwo_aux  $\times$  (Review  $\times$  Reviewer))));
  JointStateTwo :=  $\pi[2]$  ( $\sigma[\#0 = \#1 \wedge \#3 = \#4 \wedge \#0 = \#5 \wedge \#2 = \#6]$  ((CertStateTwo_aux  $\times$  (Review  $\times$  (Reviewer  $\times$  Certified)))));
  WHILE ( $\neg$ ((AccessTwo = AccessTwo_aux)  $\wedge$  ((CertStateTwo = CertStateTwo_aux)  $\wedge$  (ReviewStateTwo = ReviewStateTwo_aux)  $\wedge$  (JointStateTwo = JointStateTwo_aux)))) DO
    AccessTwo_aux := AccessTwo;
    CertStateTwo_aux := CertStateTwo;
    ReviewStateTwo_aux := ReviewStateTwo;
    JointStateTwo_aux := JointStateTwo;
    AccessTwo := (AccessTwo  $\cup$  (Root  $\cup$  ( $\pi[0]$  ( $\sigma[\#0 = \#2]$  ((JointStateTwo_aux  $\times$  Delegates)))  $\cup$  ( $\pi[0]$  ( $\sigma[\#0 = \#1 \wedge \#0 = \#3]$  ((CertStateTwo_aux  $\times$  (ReviewStateTwo_aux  $\times$  Delegates))))));
    CertStateTwo := (CertStateTwo  $\cup$   $\pi[2]$  ( $\sigma[\#0 = \#1]$  ((AccessTwo_aux  $\times$  Certified)));
    ReviewStateTwo := (ReviewStateTwo  $\cup$   $\pi[2]$  ( $\sigma[\#0 = \#1 \wedge \#3 = \#4]$  ((AccessTwo_aux  $\times$  (Review  $\times$  Reviewer)))));
    JointStateTwo := (JointStateTwo  $\cup$   $\pi[2]$  ( $\sigma[\#0 = \#1 \wedge \#3 = \#4 \wedge \#0 = \#5 \wedge \#2 = \#6]$  ((CertStateTwo_aux  $\times$  (Review  $\times$  (Reviewer  $\times$  Certified)))));
  END;

POST
  (AccessOne  $\subseteq$  AccessTwo)

```

40 Example3009: Certified delegation closure exceeds audit-bound authorization

Category	LLM-generated (GPT/Claude benchmark programs); family: two programs under TGD constraints (LLM-generated, GPT (raw_invalid_gpt batch); legacy raw_new pipeline), invalid.
Kind	containment.
Contributors	Leo Zhang.
Status	certified invalid (Certificate/Invalid.lean, written by the certificate emitter); expected invalid.
Tags	dependencies mutual-recursion nonlinear-recursion.
Input	Benchmark/Example3009/Input.lean (canonical notation).
Summary	Certified delegation closure exceeds audit-bound authorization.
Sides	naive compiler: program C1 of the retired corpus case Example3009 (output GrantA) naive compiler: program C2 of the retired corpus case Example3009 (output AllowB)
Sources	LLM-generated (GPT (raw_invalid_gpt batch)), legacy raw_new TGD pipeline this repository the retired corpus case Example3009 (programs C1, C2 and the TGD list tgd0..) TGD pipeline run raw-new-tgd-0002-product: TGDs translated to relational algebra by RelCalc.TGD.listToAssertExpr; programs compiled with the certified naive compiler (read 2026-09-15) method this repository the equivalence-under-constraints paper draft (technical appendix: reduction of containment/equivalence under TGDs to a Hoare triple) (read 2026-09-15) legacy source record raw_invalid_gpt_2.json legacy source record Benchmark/Example3009/Input.lean Datalog source noted in the legacy inventory of Example3009
Obstruction note	from the metadata: invalid (legacy kernel counterexample)
Note	Legacy raw_new invalid TGD case (GPT): the second program demands a matching audit tuple per promotion.

Datalog program(s). *prog_c1* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
GrantA(x1) :- Root(x1).
PendingA(y1) :- GrantA(x1), Delegate(x1, y1).
CheckedA(y1) :- PendingA(y1), Certified(x1, y1), Trust(x1).
GrantA(y1) :- CheckedA(y1).
PendingA(y1) :- CheckedA(x1), GrantA(z1), Review(x1, y1).
```

prog_c2 — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
AllowB(x1) :- Root(x1).
QueueB(y1) :- AllowB(x1), Delegate(x1, y1).
VettedB(y1) :- QueueB(y1), Certified(x1, y1), Trust(x1).
AllowB(y1) :- VettedB(y1), Audit(x1, y1, z1), AllowB(z1).
QueueB(y1) :- VettedB(x1), Review(x1, y1).
```

Reading. Program C1 computes a grant set GrantA in which a pending principal, once checked against only a Certified/Trust pair, is promoted, and can then reopen pending status via a further Review hop even while relying on a previously certified check. Program C2 computes AllowB with an analogous pending/vetted structure but requires a matching Audit triple, linking the certified pair to a trusted endpoint, each time a vetted principal is promoted via Delegate/Audit composition, rather than only a Review hop. This models staged authorization delegation compared against a version that insists on a matching audit record for every promotion. The Hoare triple claims containment $\text{GrantA} \subseteq \text{AllowB}$ under four TGDs, including an existential one guaranteeing every root an audited delegate, but the claim fails: the recorded witness instance exhibits a principal that C1 grants via a reviewed hop with no matching audit tuple, which C2's audit-bound promotion rule fails to allow.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $(\forall x. (\text{Root}(x) \rightarrow \exists y. \text{Delegate}(x,y) \wedge \text{Audit}(x,y,x))) \wedge (\forall z u v. (\text{Audit}(z,u,v) \rightarrow \text{Certified}(z,u) \wedge \text{Trust}(v))) \wedge (\forall w x1. (\text{Delegate}(w,x1) \wedge \text{Certified}(w,x1) \rightarrow \text{Review}(w,x1))) \wedge \forall x2 x3. ((\exists x4. \text{Review}(x4,x2) \wedge \text{Audit}(x4,x2,x3)) \rightarrow \text{Delegate}(x3,x2)))$

The precondition is four TGDs: every Root element has some Delegate/Audit witness pairing it with itself, Audit triples are backed by a Certified pair and a Trust element, every certified Delegate pair is also a Review pair, and composing a Review pair with an Audit triple through shared coordinates yields a Delegate pair.

POST: $\forall x. (\text{GrantA}(x) \rightarrow \text{AllowB}(x))$

The postcondition claims $\text{GrantA} \subseteq \text{AllowB}$, but the case is invalid: the recorded witness instance shows a principal in GrantA that is not in AllowB, because C2 requires an audit tuple that C1's reviewed promotion does not guarantee.

Whiel encoding (Hoare triple).

```
SCHEMA
Root/1, Trust/1, GrantA/1, PendingA/1, CheckedA/1, AllowB/1, QueueB/1, VettedB/1, GrantA_aux/1, PendingA_aux/1, CheckedA_aux/1, AllowB_aux/1, QueueB_aux/1, VettedB_aux/1, Delegate/2,
  ↳ Certified/2, Review/2, Audit/3

PRE
((Root ⊆ π[0] (σ[(#0 = #2 ∧ #1 = #3) ∧ #0 = #4]) ((Delegate × Audit)))) ∧ ((Audit ⊆ (Certified × Trust)) ∧ ((π[0,1] (σ[(#0 = #2 ∧ #1 = #3]) ((Delegate × Certified)))) ⊆
  ↳ Review) ∧ (π[1,4] (σ[(#0 = #2 ∧ #1 = #3]) ((Review × Audit)))) ⊆ π[1,0] (Delegate))))

CMD
GrantA_aux := ∅[1];
PendingA_aux := ∅[1];
CheckedA_aux := ∅[1];
GrantA := (Root ∪ CheckedA_aux);
PendingA := (π[2] (σ[#0 = #1] ((GrantA_aux × Delegate))) ∪ π[3] (σ[#0 = #2] ((CheckedA_aux × (GrantA_aux × Review)))));
CheckedA := (π[0] (σ[(#0 = #2 ∧ #1 = #3]) ((PendingA_aux × (Certified × Trust)))));
WHILE (¬(((GrantA = GrantA_aux) ∧ ((PendingA = PendingA_aux) ∧ (CheckedA = CheckedA_aux)))) DO
  GrantA_aux := GrantA;
  PendingA_aux := PendingA;
  CheckedA_aux := CheckedA;
  GrantA := (GrantA ∪ (Root ∪ CheckedA_aux));
  PendingA := (PendingA ∪ (π[2] (σ[#0 = #1] ((GrantA_aux × Delegate))) ∪ π[3] (σ[#0 = #2] ((CheckedA_aux × (GrantA_aux × Review))))));
  CheckedA := (CheckedA ∪ π[0] (σ[(#0 = #2 ∧ #1 = #3]) ((PendingA_aux × (Certified × Trust)))));
END;
```

```

AllowB_aux := 0[1];
QueueB_aux := 0[1];
VettedB_aux := 0[1];
AllowB := (Root ∪ π[0] (σ[#0 = #2 ∧ #3 = #4] ((VettedB_aux × (Audit × AllowB_aux)))));
QueueB := (π[2] (σ[#0 = #1] ((AllowB_aux × Delegate))) ∪ π[2] (σ[#0 = #1] ((VettedB_aux × Review))));
VettedB := π[0] (σ[#0 = #2 ∧ #1 = #3] ((QueueB_aux × (Certified × Trust))));
WHILE (¬(((AllowB = AllowB_aux) ∧ ((QueueB = QueueB_aux) ∧ (VettedB = VettedB_aux))))) DO
  AllowB_aux := AllowB;
  QueueB_aux := QueueB;
  VettedB_aux := VettedB;
  AllowB := (AllowB ∪ (Root ∪ π[0] (σ[#0 = #2 ∧ #3 = #4] ((VettedB_aux × (Audit × AllowB_aux))))));
  QueueB := (QueueB ∪ (π[2] (σ[#0 = #1] ((AllowB_aux × Delegate))) ∪ π[2] (σ[#0 = #1] ((VettedB_aux × Review))));
  VettedB := (VettedB ∪ π[0] (σ[#0 = #2 ∧ #1 = #3] ((QueueB_aux × (Certified × Trust)))))
END

POST
  (GrantA ⊆ AllowB)

```

41 Example3014: Auditor trust diverges from resource trust under recursive elevation

Category	LLM-generated (GPT/Claude benchmark programs); family: two programs under TGD constraints (LLM-generated, GPT (raw_invalid_gpt batch); legacy raw_new pipeline), invalid.
Kind	containment.
Contributors	Leo Zhang.
Status	certified invalid (Certificate/Invalid.lean, written by the certificate emitter); expected invalid.
Tags	dependencies mutual-recursion nonlinear-recursion.
Input	Benchmark/Example3014/Input.lean (canonical notation).
Summary	Auditor trust diverges from resource trust under recursive elevation.
Sides	naive compiler: program C1 of the retired corpus case Example3014 (output AccessA) naive compiler: program C2 of the retired corpus case Example3014 (output AccessB)
Sources	LLM-generated (GPT (raw_invalid_gpt batch)), legacy raw_new TGD pipeline this repository the retired corpus case Example3014 (programs C1, C2 and the TGD list tgd0..) TGD pipeline run raw-new-tgd-0002-product: TGDs translated to relational algebra by RelCalc.TGD.listToAssertExpr; programs compiled with the certified naive compiler (read 2026-09-15) method this repository the equivalence-under-constraints paper draft (technical appendix: reduction of containment/equivalence under TGDs to a Hoare triple) (read 2026-09-15) legacy source record raw_invalid_gpt_2.json legacy source record Benchmark/Example3014/Input.lean Datalog source noted in the legacy inventory of Example3014
Obstruction note	from the metadata: invalid (legacy kernel counterexample)
Note	Legacy raw_new invalid TGD case (GPT): the proof is checked on the resource coordinate instead of the auditor coordinate.

Datalog program(s). *prog_c1* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```

AccessA(x1, y1) :- Request(x1, y1), Permit(x1, y1).
ElevatedA(x1, z1) :- AccessA(x1, y1), Grant(y1, z1), Scope(z1).
CheckedA(x1, z1) :- ElevatedA(x1, y1), Override(y1, z1), Proof(x1, z1, x2), Trusted(x2).
AccessA(x1, z1) :- CheckedA(x1, z1).
ElevatedA(x1, z1) :- AccessA(x1, y1), AccessA(x1, x2), Proof(y1, z1, x2).

```

prog_c2 — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```

AccessB(x1, y1) :- Request(x1, y1), Permit(x1, y1).
ElevatedB(x1, z1) :- AccessB(x1, y1), Grant(y1, z1), Scope(z1).
AuditB(x1, z1, x2) :- ElevatedB(x1, y1), Override(y1, z1), Proof(x1, z1, x2).
AccessB(x1, z1) :- AuditB(x1, z1, x2), Trusted(z1).
ElevatedB(x1, z1) :- AccessB(x1, y1), AccessB(x1, x2), Proof(y1, z1, x2).

```

Reading. Program C1 computes an access relation AccessA that elevates a principal's access to a new resource via $\text{Grant}/\text{Scope}$, re-checks that elevation by requiring a Proof triple whose auditor coordinate (the proof's third argument) is Trusted , and further allows elevation whenever two already-accessible resources are linked by a Proof triple. Program C2 computes AccessB with the analogous elevation structure but tests Trusted against the proof's resource coordinate rather than its auditor coordinate. This models recursive policy elevation comparing which coordinate of a proof triple, auditor versus resource, a trust check should apply to. The Hoare triple claims containment $\text{AccessA} \subseteq \text{AccessB}$ under five TGDs, including an existential one guaranteeing every request a trusted proof witness, but the claim fails: the recorded witness instance shows the two programs agree on the first trusted end-point but diverge at the next audited elevation, since trusting the auditor coordinate versus the resource coordinate of the same Proof triple picks out different principals.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in `Input.lean`. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $(\forall x y. (\text{Request}(x,y) \rightarrow \exists z. \text{Proof}(x,y,z) \wedge \text{Trusted}(z))) \wedge (\forall u v. ((\exists w. \text{Proof}(u,v,w)) \rightarrow \text{Permit}(u,v))) \wedge (\forall x1 x2. (\text{Grant}(x1,x2) \wedge \text{Scope}(x2) \rightarrow \text{Permit}(x1,x2))) \wedge (\forall x3. ((\exists x4 x5 x6. \text{Override}(x3,x5) \wedge \text{Proof}(x4,x5,x6)) \rightarrow \text{Scope}(x3))) \wedge \forall x7. ((\exists x8 x9. \text{Permit}(x8,x9) \wedge \text{Grant}(x9,x7)) \rightarrow \text{Scope}(x7)))$

The precondition is five TGDs: every Request pair has a Proof triple whose third coordinate is Trusted , the first two coordinates of every Proof triple are a Permit pair, every Grant pair whose target is in Scope is a Permit pair, the source of an Override pair with a matching Proof witness is in Scope , and the target of a Permit pair composed with a Grant pair is in Scope .

POST: $\forall x y. (\text{AccessA}(x,y) \rightarrow \text{AccessB}(x,y))$

The postcondition claims $\text{AccessA} \subseteq \text{AccessB}$, but the case is invalid: the recorded witness instance shows a resource in AccessA that is not in AccessB .

Whiel encoding (Hoare triple).

```

SCHEMA
Scope/1, Trusted/1, Request/2, Permit/2, Grant/2, Override/2, AccessA/2, ElevatedA/2, CheckedA/2, AccessB/2, ElevatedB/2, AccessA_aux/2, ElevatedA_aux/2, CheckedA_aux/2,
  AccessB_aux/2, ElevatedB_aux/2, Proof/3, AuditB/3, AuditB_aux/3

PRE
((Request ⊆ π[0,1] (σ[#2 = #3] ((Proof × Trusted)))) ∧ ((π[0,1] (Proof) ⊆ Permit) ∧ ((π[0,1] (σ[#1 = #2] ((Grant × Scope))) ⊆ Permit) ∧ ((π[0] (σ[#1 = #3] ((Override ×
  Proof))) ⊆ Scope) ∧ (π[3] (σ[#1 = #2] ((Permit × Grant))) ⊆ Scope))))))

CMD
AccessA_aux := ∅[2];
ElevatedA_aux := ∅[2];
CheckedA_aux := ∅[2];
AccessA := (π[0,1] (σ[(#0 = #2 ∧ #1 = #3)] ((Request × Permit))) ∪ CheckedA_aux);
ElevatedA := (π[0,3] (σ[(#1 = #2 ∧ #3 = #4)] ((AccessA_aux × (Grant × Scope)))) ∪ π[0,5] (σ[(#0 = #2 ∧ #1 = #4) ∧ #3 = #6] ((AccessA_aux × (AccessA_aux × Proof)))));
CheckedA := π[0,3] (σ[(#1 = #2 ∧ #0 = #4) ∧ #3 = #5] ∧ #6 = #7] ((ElevatedA_aux × (Override × (Proof × Trusted)))));
WHILE (¬(((AccessA = AccessA_aux) ∧ ((ElevatedA = ElevatedA_aux) ∧ (CheckedA = CheckedA_aux)))) DO
  AccessA_aux := AccessA;
  ElevatedA_aux := ElevatedA;
  CheckedA_aux := CheckedA;
  AccessA := (AccessA ∪ (π[0,1] (σ[(#0 = #2 ∧ #1 = #3)] ((Request × Permit))) ∪ CheckedA_aux));
  ElevatedA := (ElevatedA ∪ (π[0,3] (σ[(#1 = #2 ∧ #3 = #4)] ((AccessA_aux × (Grant × Scope)))) ∪ π[0,5] (σ[(#0 = #2 ∧ #1 = #4) ∧ #3 = #6] ((AccessA_aux × (AccessA_aux ×
    Proof))))));
  CheckedA := (CheckedA ∪ π[0,3] (σ[(#1 = #2 ∧ #0 = #4) ∧ #3 = #5] ∧ #6 = #7] ((ElevatedA_aux × (Override × (Proof × Trusted)))));
END;
AccessB_aux := ∅[2];
ElevatedB_aux := ∅[2];
AuditB_aux := ∅[3];
AccessB := (π[0,1] (σ[(#0 = #2 ∧ #1 = #3)] ((Request × Permit))) ∪ π[0,1] (σ[#1 = #3] ((AuditB_aux × Trusted)))));
ElevatedB := (π[0,3] (σ[(#1 = #2 ∧ #3 = #4)] ((AccessB_aux × (Grant × Scope)))) ∪ π[0,5] (σ[(#0 = #2 ∧ #1 = #4) ∧ #3 = #6] ((AccessB_aux × (AccessB_aux × Proof)))));
AuditB := π[0,3,6] (σ[(#1 = #2 ∧ #0 = #4) ∧ #3 = #5] ∧ #6 = #7] ((ElevatedB_aux × (Override × Proof)));
WHILE (¬(((AccessB = AccessB_aux) ∧ ((ElevatedB = ElevatedB_aux) ∧ (AuditB = AuditB_aux)))) DO
  AccessB_aux := AccessB;
  ElevatedB_aux := ElevatedB;
  AuditB_aux := AuditB;
  AccessB := (AccessB ∪ (π[0,1] (σ[(#0 = #2 ∧ #1 = #3)] ((Request × Permit))) ∪ π[0,1] (σ[#1 = #3] ((AuditB_aux × Trusted)))));
  ElevatedB := (ElevatedB ∪ (π[0,3] (σ[(#1 = #2 ∧ #3 = #4)] ((AccessB_aux × (Grant × Scope)))) ∪ π[0,5] (σ[(#0 = #2 ∧ #1 = #4) ∧ #3 = #6] ((AccessB_aux × (AccessB_aux ×
    Proof))))));
  AuditB := (AuditB ∪ π[0,3,6] (σ[(#1 = #2 ∧ #0 = #4) ∧ #3 = #5] ((ElevatedB_aux × (Override × Proof)))));
END

POST
(AccessA ⊆ AccessB)

```

42 Example4001: Directed triangle query versus its path-view reformulation

Category	verification task (VerificationTasks.pdf, section 10.1); family: Core.
Kind	equivalence.
Contributors	Leo Zhang, Val Tannen.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	views.
Input	Benchmark/Example4001/Input.lean (canonical notation).
Summary	A directed-triangle query and its reformulation over a length-two path view.
Sources	VerificationTasks.pdf, section 10.1 (Val Tannen; reports/VerificationTasks.pdf) the reformulations task document legacy source record Benchmark/Example4001/Input.lean legacy source record of the reformulations task document

Datalog program(s). *globalProgram* — ‘ $Q\ x :- Exy, Eyz, Ezx$ ’. — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```
Q(x1) :- E(x1, y1), E(y1, z1), E(z1, x1).
```

localProgram — ‘ $Q\uparrow\ x :- V\ xyz, V\ yzx$ ’. — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```
QUp(x1) :- V(x1, y1, z1), V(y1, z1, x1).
```

Reading. The two Datalog programs compute the directed-triangle query $Q(x) :- E(x,y), E(y,z), E(z,x)$ over an edge relation E , and its reformulation $Q\uparrow(x) :- V(x,y,z), V(y,z,x)$ over a length-two path view V . Under the view constraint that V records exactly the length-two paths of E together with their midpoint ($V = E \circ E$), the Hoare triple claims the two queries compute equal results, $Q = Q\uparrow$; this is an equivalence claim. The claim is certified valid: joining two copies of the view on overlapping variables reconstructs the directed-triangle detection under set semantics.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\forall x\ y\ z. (V(x,y,z) \leftrightarrow E(x,y) \wedge E(y,z))$

The precondition states that V equals the set of length-two paths through E : V is E composed with E , projected onto the start, midpoint and end columns with the middle columns equated.

POST: $\forall x. (Q(x) \leftrightarrow QUp(x))$

The postcondition asserts the two queries agree exactly, $Q = QUp$.

Whiel encoding (Hoare triple).

```
SCHEMA
  Q/1, QUp/1, Q_aux/1, QUp_aux/1, E/2, V/3
PRE
  (V = (π [0, 1, 3] (σ [#1 = #2] (E × E))))
CMD
  Q_aux := ∅ [1];
```

```

Q :=
  (π[0]
   (σ[[(#1 = #2 ∧ #3 = #4) ∧ #0 = #5]]
    (E × (E × E))));
  WHILE (¬ (Q = Q_aux)) DO
    Q_aux := Q;
    Q :=
      (Q ∪
       (π[0]
        (σ[[(#1 = #2 ∧ #3 = #4) ∧ #0 = #5]]
         (E × (E × E)))))
  END;
QUp_aux := ∅[1];
QUp :=
  (π[0]
   (σ[[(#1 = #3 ∧ #2 = #4) ∧ #0 = #5]]
    (V × V)));
  WHILE (¬ (QUp = QUp_aux)) DO
    QUp_aux := QUp;
    QUp :=
      (QUp ∪
       (π[0]
        (σ[[(#1 = #3 ∧ #2 = #4) ∧ #0 = #5]]
         (V × V))))
  END
POST
  (Q = QUp)

```

43 Example4002: Transitive closure versus its length-two path-view reformulation

Category	verification task (VerificationTasks.pdf, section 10.1); family: Core.
Kind	equivalence.
Contributors	Leo Zhang, Val Tannen.
Status	certified invalid (Certificate/Invalid.lean, written by the certificate emitter); expected invalid.
Tags	views.
Input	Benchmark/Example4002/Input.lean (canonical notation).
Summary	Right-linear transitive closure and a reformulation over a length-two path view, tested for equality.
Sources	VerificationTasks.pdf, section 10.1 (Val Tannen; reports/VerificationTasks.pdf) the reformulations task document legacy source record Benchmark/Example4002/Input.lean legacy source record of the reformulations task document

Datalog program(s). *globalProgram* — ‘ $T\ xy \text{ :- } Exy$ ’ and ‘ $T\ xy \text{ :- } Exz, T\ zy$ ’. — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```

T(x1, y1) :- E(x1, y1).
T(x1, y1) :- E(x1, z1), T(z1, y1).

```

localProgram — ‘ $T^\uparrow xy \text{ :- } V\ xwy$ ’ and ‘ $T^\uparrow xz \text{ :- } V\ xwy, T^\uparrow yz$ ’. — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```

TUp(x1, y1) :- V(x1, z1, y1).
TUp(x1, x2) :- V(x1, z1, y1), TUp(y1, x2).

```

Reading. The global program T is the standard right-linear transitive closure of edge relation E ; the local reformulation TUp chains the length-two path view V ($V = E \circ E$, keeping the midpoint) recursively: $TUp(x,y) \text{ :- } V(x,w,y)$, and $TUp(x,z) \text{ :- } V(x,w,y), TUp(y,z)$. The Hoare triple claims equality, $T = TUp$ — an equivalence claim between transitive closure and the view-chaining reformulation. This claim is invalid, certified by a kernel-checked counterexample: because every TUp derivation composes at least one V -step, TUp collects exactly the paths of even length at least two, so a single edge (e.g. $E = \{(0,1)\}$ with everything else empty) lies in T but not in TUp .

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\forall x y z. (V(x,y,z) \leftrightarrow E(x,y) \wedge E(y,z))$

The precondition states that V equals the set of length-two paths through E , i.e. $V = E \circ E$ with the midpoint kept.

POST: $\forall x y. (T(x,y) \leftrightarrow \text{TUp}(x,y))$

The postcondition asserts $T = \text{TUp}$, that transitive closure and the view-chaining reformulation compute the same relation — a claim that fails.

Whiel encoding (Hoare triple).

```

SCHEMA
  E/2, T/2, TUp/2, T_aux/2, TUp_aux/2, V/3

PRE
  (V = (π[0, 1, 3] (σ[#1 = #2] (E × E))))

CMD
  T_aux := ∅[2];
  T :=
    (E ∪ (π[0, 3] (σ[#1 = #2] (E × T_aux))));
  WHILE (¬ (T = T_aux)) DO
    T_aux := T;
    T :=
      (T ∪
       (E ∪
        (π[0, 3] (σ[#1 = #2] (E × T_aux)))))
  END;
  TUp_aux := ∅[2];
  TUp :=
    ((π[0, 2] V) ∪
     (π[0, 4] (σ[#2 = #3] (V × TUp_aux))));
  WHILE (¬ (TUp = TUp_aux)) DO
    TUp_aux := TUp;
    TUp :=
      (TUp ∪
       ((π[0, 2] V) ∪
        (π[0, 4]
         (σ[#2 = #3] (V × TUp_aux)))))
  END

POST
  (T = TUp)

```

44 Example4003: Containment of the path-view reformulation in transitive closure

Category	verification task (VerificationTasks.pdf, section 10.1); family: Core.
Kind	containment.
Contributors	Leo Zhang, Val Tannen.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	<code>prophecy views</code> .
Input	Benchmark/Example4003/Input.lean (canonical notation).
Summary	Right-linear transitive closure and a reformulation over a length-two path view, tested for containment.
Sources	VerificationTasks.pdf, section 10.1 (Val Tannen; reports/VerificationTasks.pdf) the reformulations task document legacy source record Benchmark/Example4003/Input.lean legacy source record of the reformulations task document
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).

Datalog program(s). *globalProgram* — ‘ $T\ xy \text{ :- } Exy$ ’ and ‘ $T\ xy \text{ :- } Exz, T\ zy$ ’. — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```
T(x1, y1) :- E(x1, y1).
T(x1, y1) :- E(x1, z1), T(z1, y1).
```

localProgram — ‘ $T^\uparrow\ xy \text{ :- } V\ xwy$ ’ and ‘ $T^\uparrow\ xz \text{ :- } V\ xwy, T^\uparrow\ yz$ ’. — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```
TUp(x1, y1) :- V(x1, z1, y1).
TUp(x1, x2) :- V(x1, z1, y1), TUp(y1, x2).
```

program — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```
T(x1, y1) :- E(x1, y1).
T(x1, y1) :- E(x1, z1), E(z1, y1).
T(x1, y1) :- E(x1, z1), T(z1, y1).
T(x1, y1) :- E(x1, z1), E(z1, x2), T(x2, y1).
```

Reading. This case uses the same global program (right-linear transitive closure T of E) and the same local reformulation (TUp , chaining the length-two path view $V = E \circ E$) as Example4002, but weakens the postcondition from equality to a one-directional (forward) containment, $TUp \subseteq T$: every pair the view-chaining reformulation derives should also be a transitive-closure pair. The header states the expected verdict is valid, since TUp collects exactly the even-length-at-least-two paths, all of which lie in T ; no certificate exists yet for this pass, so the classification stays unknown.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\forall x\ y\ z. (V(x,y,z) \leftrightarrow E(x,y) \wedge E(y,z))$

The precondition states that V equals the set of length-two paths through E , i.e. $V = E \circ E$ with the midpoint kept.

POST: $\forall x y. (\text{TUp}(x,y) \rightarrow \text{T}(x,y))$

The postcondition asserts $\text{TUp} \subseteq \text{T}$, that every pair produced by the view-chaining reformulation is also produced by transitive closure.

Whiel encoding (Hoare triple).

```

SCHEMA
  E/2, T/2, TUp/2, T_aux/2, TUp_aux/2, V/3

PRE
  (V = ( $\pi[0, 1, 3]$  ( $\sigma[\#1 = \#2]$  ( $E \times E$ ))))

CMD
  T_aux :=  $\emptyset[2]$ ;
  T :=
    ( $E \cup (\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  ( $E \times T_{\text{aux}}$ )))));
  WHILE ( $\neg (T = T_{\text{aux}})$ ) DO
    T_aux := T;
    T :=
      ( $T \cup$ 
        ( $E \cup$ 
          ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  ( $E \times T_{\text{aux}}$ ))))))
  END;
  TUp_aux :=  $\emptyset[2]$ ;
  TUp :=
    (( $\pi[0, 2]$  V)  $\cup$ 
     ( $\pi[0, 4]$  ( $\sigma[\#2 = \#3]$  ( $V \times T_{\text{Up\_aux}}$ )))));
  WHILE ( $\neg (T_{\text{Up}} = T_{\text{Up\_aux}})$ ) DO
    TUp_aux := TUp;
    TUp :=
      ( $T_{\text{Up}} \cup$ 
        (( $\pi[0, 2]$  V)  $\cup$ 
         ( $\pi[0, 4]$ 
          ( $\sigma[\#2 = \#3]$  ( $V \times T_{\text{Up\_aux}}$ ))))))
  END

POST
  ( $T_{\text{Up}} \subseteq T$ )

```

45 Example4004: Path-view reformulation versus the even-length path query

Category	verification task (VerificationTasks.pdf, section 10.1); family: Core.
Kind	equivalence.
Contributors	Leo Zhang, Val Tannen.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	mutual-recursion prophecy views.
Input	Benchmark/Example4004/Input.lean (canonical notation).
Summary	An odd/even path program and a reformulation over a length-two path view, tested for equality against the even half.
Sources	VerificationTasks.pdf, section 10.1 (Val Tannen; reports/VerificationTasks.pdf) the reformulations task document legacy source record Benchmark/Example4004/Input.lean legacy source record of the reformulations task document
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).

Datalog program(s). *globalProgram* — *The odd/even program, with output IDB ‘Te’.* — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```

To(x1, y1) :- E(x1, y1).
To(x1, y1) :- E(x1, z1), Te(z1, y1).
Te(x1, y1) :- E(x1, z1), To(z1, y1).

```

localProgram — ‘ $T\uparrow xy :- V\ xwy$ ’ and ‘ $T\uparrow xz :- V\ xwy, T\uparrow yz$ ’ — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```

TUp(x1, y1) :- V(x1, z1, y1).
TUp(x1, x2) :- V(x1, z1, y1), TUp(y1, x2).

```

program — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```

To(x1, y1) :- E(x1, y1).
To(x1, y1) :- E(x1, z1), Te(z1, y1).
Te(x1, y1) :- E(x1, z1), E(z1, y1).
Te(x1, y1) :- E(x1, z1), E(z1, x2), Te(x2, y1).

```

Reading. The global program computes paths of odd length (To) and even length (Te) over E by mutual recursion — $To(x,y):-E(x,y)$, $To(x,y):-E(x,z),Te(z,y)$, $Te(x,y):-E(x,z),To(z,y)$ — with Te as the relevant output. The local reformulation TUp chains the same length-two path view $V = E \circ E$ used in the neighboring cases. The Hoare triple claims equality, $TUp = Te$: that the view-chaining reformulation exactly recovers the even-length-path relation from the odd/even decomposition of transitive closure. The header states the expected verdict is valid, since Te is exactly the paths of even length at least two, which is what TUp collects; no certificate exists yet, so the classification stays unknown.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\forall x\ y\ z. (V(x,y,z) \leftrightarrow E(x,y) \wedge E(y,z))$

The precondition states that V equals the set of length-two paths through E, i.e. $V = E \circ E$ with the midpoint kept.

POST: $\forall x\ y. (TUp(x,y) \leftrightarrow Te(x,y))$

The postcondition asserts $TUp = Te$, that the view-chaining reformulation equals the even-length-path relation from the odd/even decomposition.

Whiel encoding (Hoare triple).

```

SCHEMA
  E/2, To/2, Te/2, TUp/2, To_aux/2, Te_aux/2, TUp_aux/2, V/3

PRE
  (V = (π [0, 1, 3] (σ [#1 = #2] (E × E))))

CMD
  To_aux := ∅ [2];
  Te_aux := ∅ [2];
  To :=
    (E ∪ (π [0, 3] (σ [#1 = #2] (E × Te_aux)))));
  Te := (π [0, 3] (σ [#1 = #2] (E × To_aux)));
  WHILE (¬ ((To = To_aux) ∧ (Te = Te_aux))) DO
    To_aux := To;
    Te_aux := Te;
    To :=
      (To ∪
       (E ∪
        (π [0, 3] (σ [#1 = #2] (E × Te_aux)))));
    Te :=
      (Te ∪
       (π [0, 3] (σ [#1 = #2] (E × To_aux)))));
  END;
  TUp_aux := ∅ [2];
  TUp :=
    ((π [0, 2] V) ∪
     (π [0, 4] (σ [#2 = #3] (V × TUp_aux)))));
  WHILE (¬ (TUp = TUp_aux)) DO
    TUp_aux := TUp;
    TUp :=
      (TUp ∪
       (TUp × TUp));
  END;

```

```

      (( $\pi[0, 2]$   $V$ )  $\cup$ 
       ( $\pi[0, 4]$ 
        ( $\sigma[\#2 = \#3]$  ( $V \times \text{TUp\_aux}$ ))))))
END
POST
  ( $\text{TUp} = \text{Te}$ )

```

46 Example4034: Containment of a reformulation over sound odd/even views in transitive closure

Category	verification task (VerificationTasks.pdf, section 10.1); family: Core.
Kind	conditional claim (implication).
Contributors	Leo Zhang, Val Tannen.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	mutual-recursion prophecy views.
Input	Benchmark/Example4034/Input.lean (canonical notation).
Summary	Transitive closure and a reformulation read off two sound odd/even views.
Sources	VerificationTasks.pdf, section 10.1 (Val Tannen; reports/VerificationTasks.pdf) the reformulations task document
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).

Datalog program(s). *localProgram* — ‘ $T^\dagger xy \text{ :- } Vo\ xy$ ’ and ‘ $T^\dagger xy \text{ :- } Ve\ xy$ ’. — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```

TUp(x1, y1) :- Vo(x1, y1).
TUp(x1, y1) :- Ve(x1, y1).

```

viewProgram — *The odd/even program defining the two views.* — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```

To(x1, y1) :- E(x1, y1).
To(x1, y1) :- E(x1, z1), Te(z1, y1).
Te(x1, y1) :- E(x1, z1), To(z1, y1).

```

globalProgram — ‘ $T\ xy \text{ :- } Exy$ ’ and ‘ $T\ xy \text{ :- } Exz, T\ zy$ ’. — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```

T(x1, y1) :- E(x1, y1).
T(x1, y1) :- E(x1, z1), T(z1, y1).

```

Reading. The global program T is the standard right-linear transitive closure of E . The local reformulation is loop-free: TUp is simply the union of two source relations Vo and Ve , which stand for odd- and even-length path views computed by a separate odd/even mutual-recursion program over E , but are only assumed sound ($\text{Vo} \subseteq \text{To}$, $\text{Ve} \subseteq \text{Te}$) rather than exact. Because that soundness hypothesis refers to the least fixpoints the loop itself computes, it cannot be phrased as a precondition, so it is folded into the postcondition as an implication written with $\neg$ and $\vee$: if Vo and Ve are sound approximations of the odd/even path relations, then the reformulation is contained in transitive closure, $\text{TUp} \subseteq T$. This is a conditional (soundness-guarded) containment claim. The claim is valid, since $\text{Vo} \cup \text{Ve} \subseteq \text{To} \cup \text{Te} = T$ whenever the hypothesis holds.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\top$

The precondition is simply true; no constraint holds at entry.

POST: $\neg((\forall x y. (Vo(x,y) \rightarrow To(x,y))) \wedge \forall z u. (Ve(z,u) \rightarrow Te(z,u))) \vee \forall v w. (TUp(v,w) \rightarrow T(v,w))$

The postcondition states that if Vo is contained in the odd-length-path relation To and Ve is contained in the even-length-path relation Te , then the reformulation TUp is contained in the transitive closure T .

Whiel encoding (Hoare triple).

```
SCHEMA
  E/2, Vo/2, Ve/2, T/2, To/2, Te/2, TUp/2, T_aux/2, To_aux/2, Te_aux/2, TUp_aux/2

PRE
  true

CMD
  TUp_aux :=  $\emptyset[2]$ ;
  TUp :=  $(Vo \cup Ve)$ ;
  WHILE  $(\neg (TUp = TUp\_aux))$  DO
    TUp_aux := TUp;
    TUp :=  $(TUp \cup (Vo \cup Ve))$ 
  END;
  To_aux :=  $\emptyset[2]$ ;
  Te_aux :=  $\emptyset[2]$ ;
  To :=
     $(E \cup (\pi[0, 3] (\sigma[\#1 = \#2] (E \times Te\_aux))))$ ;
  Te :=  $(\pi[0, 3] (\sigma[\#1 = \#2] (E \times To\_aux)))$ ;
  WHILE  $(\neg ((To = To\_aux) \wedge (Te = Te\_aux)))$  DO
    To_aux := To;
    Te_aux := Te;
    To :=
       $(To \cup$ 
         $(E \cup$ 
           $(\pi[0, 3] (\sigma[\#1 = \#2] (E \times Te\_aux))))))$ ;
    Te :=
       $(Te \cup$ 
         $(\pi[0, 3] (\sigma[\#1 = \#2] (E \times To\_aux))))$ 
  END;
  T_aux :=  $\emptyset[2]$ ;
  T :=
     $(E \cup (\pi[0, 3] (\sigma[\#1 = \#2] (E \times T\_aux))))$ ;
  WHILE  $(\neg (T = T\_aux))$  DO
    T_aux := T;
    T :=
       $(T \cup$ 
         $(E \cup$ 
           $(\pi[0, 3] (\sigma[\#1 = \#2] (E \times T\_aux))))$ 
  END

POST
   $((\neg ((Vo \subseteq To) \wedge (Ve \subseteq Te))) \vee (TUp \subseteq T))$ 
```

47 Example4035: Good-interior path query versus its begin-good and end-good edge-view reformulation

Category	verification task (VerificationTasks.pdf, section 10.1); family: Core.
Kind	equivalence.
Contributors	Leo Zhang, Val Tannen.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	prophecy views.
Input	Benchmark/Example4035/Input.lean (canonical notation).
Summary	Walks through good vertices and a reformulation over two conjunctive views.
Sources	VerificationTasks.pdf, section 10.1 (Val Tannen; reports/VerificationTasks.pdf) the reformulations task document
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).

Datalog program(s). *globalProgram* — ‘ $T\ xy \text{ :- } Exz, Gz, Ezy$ ’ and ‘ $T\ xy \text{ :- } Exz, Gz, T\ zy$ ’ — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```
T(x1, y1) :- E(x1, z1), G(z1), E(z1, y1).
T(x1, y1) :- E(x1, z1), G(z1), T(z1, y1).
```

localProgram — ‘ $T^\uparrow\ xy \text{ :- } Ve\ xz, Vb\ zy$ ’ and ‘ $T^\uparrow\ xy \text{ :- } Ve\ xz, T^\uparrow\ zy$ ’ — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```
TUp(x1, y1) :- Ve(x1, z1), Vb(z1, y1).
TUp(x1, y1) :- Ve(x1, z1), TUp(z1, y1).
```

Reading. The global program T computes walks of length at least two through digraph E whose interior vertices are all "good" (marked by unary relation G): $T(x,y) \text{ :- } E(x,z), G(z), E(z,y)$, and $T(x,y) \text{ :- } E(x,z), G(z), T(z,y)$ — a good-interior-vertex reachability query. The local reformulation TUp is defined over two exact conjunctive views, Vb (begin-good edges, $G(x) \wedge E(x,y)$) and Ve (end-good edges, $E(x,y) \wedge G(y)$): $TUp(x,y) \text{ :- } Ve(x,z), Vb(z,y)$, and $TUp(x,y) \text{ :- } Ve(x,z), TUp(z,y)$. The Hoare triple claims the two programs compute the same relation, $T = TUp$ — an equivalence claim that follows from substituting the view definitions into each TUp rule to recover the matching T rule. Expected valid; no certificate exists yet, so the classification stays unknown.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $(\forall x\ y. (Vb(x,y) \leftrightarrow G(x) \wedge E(x,y))) \wedge \forall z\ u. (Ve(z,u) \leftrightarrow E(z,u) \wedge G(u))$

The precondition states the two view equations exactly: Vb is the join of G and E giving good-start edges, and Ve is the join of E and G giving good-end edges.

POST: $\forall x\ y. (T(x,y) \leftrightarrow TUp(x,y))$

The postcondition asserts $T = TUp$, that the good-interior path query and its two-view reformulation agree.

Whiel encoding (Hoare triple).

```
SCHEMA
  E/2, T/2, Vb/2, Ve/2, TUp/2, T_aux/2, TUp_aux/2, G/1

PRE
  ((Vb = (π[1, 2] (σ[#0 = #1] (G × E)))) ∧
   (Ve = (π[0, 1] (σ[#1 = #2] (E × G)))))
```

```

CMD
  T_aux :=  $\emptyset[2]$ ;
  T :=
    (( $\pi[0, 4]$ 
      ( $\sigma[\#1 = \#2 \wedge \#1 = \#3]$  ( $E \times (G \times E)$ )))  $\cup$ 
      ( $\pi[0, 4]$ 
        ( $\sigma[\#1 = \#2 \wedge \#1 = \#3]$ 
          ( $E \times (G \times T\_aux)$ )))));
  WHILE ( $\neg (T = T\_aux)$ ) DO
    T_aux := T;
    T :=
      (T  $\cup$ 
        ( $\pi[0, 4]$ 
          ( $\sigma[\#1 = \#2 \wedge \#1 = \#3]$ 
            ( $E \times (G \times E)$ )))  $\cup$ 
          ( $\pi[0, 4]$ 
            ( $\sigma[\#1 = \#2 \wedge \#1 = \#3]$ 
              ( $E \times (G \times T\_aux)$ )))));
  END;
  Tup_aux :=  $\emptyset[2]$ ;
  Tup :=
    (( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  ( $Ve \times Vb$ )))  $\cup$ 
      ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  ( $Ve \times Tup\_aux$ ))));
  WHILE ( $\neg (Tup = Tup\_aux)$ ) DO
    Tup_aux := Tup;
    Tup :=
      (Tup  $\cup$ 
        ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  ( $Ve \times Vb$ )))  $\cup$ 
          ( $\pi[0, 3]$ 
            ( $\sigma[\#1 = \#2]$  ( $Ve \times Tup\_aux$ ))));
  END
POST
  (T = Tup)

```

48 Example4036: Airline reachability query versus its three-view reformulation, forward containment

Category	verification task (VerificationTasks.pdf, section 10.1); family: Core.
Kind	containment.
Contributors	Leo Zhang, Val Tannen.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	prophecy views.
Input	Benchmark/Example4036/Input.lean (canonical notation).
Summary	Single-airline reachability and a reformulation over three flight views, tested for containment.
Sources	VerificationTasks.pdf, section 10.1 (Val Tannen; reports/VerificationTasks.pdf) the reformulations task document
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).
Note	In an earlier (rate-matched) encoding this containment had a quantifier-free certificate; whether the compiled-sequence encoding needs a prophecy level has not been checked.

Datalog program(s). *globalProgram* — ‘ $T\ xyu :- Fxyu$ ’, ‘ $T\ xyu :- Fxz u, T\ zyu$ ’, and ‘ $Qy :- T\ byu$ ’. — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```

T(x1, y1, x2) :- F(x1, y1, x2).
T(x1, y1, x2) :- F(x1, z1, x2), T(z1, y1, x2).
Q(y1) :- T("b", y1, x2).

```

localProgram — ‘ $Q\uparrow y :- Va\ by$ ’, ‘ $Q\uparrow z :- Vb\ yc, Vc\ yz$ ’, and ‘ $Q\uparrow y :- Vb\ yu$ ’. — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```

QUp(y1) :- Va("b", y1).
QUp(z1) :- Vb(y1, "c"), Vc(y1, z1).
QUp(y1) :- Vb(y1, x2).

```

Reading. The global program computes per-airline reachability T over a ternary flight relation $F(x,y,u)$ (non-stop flight from x to y on airline u), then restricts to $Q(y) \text{:-} T(\text{"b"},y,u)$, the airports reachable from "b" while staying on a single airline. The local reformulation QUp is built, without recursion, from three conjunctive views: Va (direct flights on airline "a"), Vb (flights out of "b"), and Vc (two-hop connections on airline "c"), combined by three rules. The Hoare triple claims a one-directional (forward) containment, $QUp \subseteq Q$: every airport the reformulation derives is genuinely reachable from "b" on one airline. Expected valid, since each QUp rule unfolds to a genuine same-airline itinerary out of "b"; no certificate exists yet, so the classification stays unknown. This case is the twin of Example4037, which tests the reverse direction of the same containment.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $(\forall x y. (Va(x,y) \leftrightarrow F(x,y,"a"))) \wedge (\forall z u. (Vb(z,u) \leftrightarrow F("b",z,u))) \wedge \forall v w. (Vc(v,w) \leftrightarrow \exists x1. F(v,x1,"c") \wedge F(x1,w,"c"))$

The precondition states the three view equations: Va is F restricted to airline "a", Vb is F restricted to flights departing "b", and Vc is the two-hop composition of two airline-"c" flights.

POST: $\forall x. (QUp(x) \rightarrow Q(x))$

The postcondition asserts $QUp \subseteq Q$, that every airport reached by the three-view reformulation is reached by the global per-airline query.

Whiel encoding (Hoare triple).

```
SCHEMA
  Q/1, QUp/1, Q_aux/1, QUp_aux/1, Va/2, Vb/2, Vc/2, F/3, T/3, T_aux/3

PRE
  ((Va = (π[0, 1] (σ[#2 = "a"] F))) ∧
   ((Vb = (π[1, 2] (σ[#0 = "b"] F))) ∧
    (Vc =
      (π[0, 4]
       (σ[(#2 = "c" ∧ #1 = #3) ∧ #5 = "c"]
        (F × F)))))))

CMD
  T_aux := ∅[3];
  Q_aux := ∅[1];
  T :=
    (F ∪
     (π[0, 4, 2]
      (σ[(#1 = #3 ∧ #2 = #5)] (F × T_aux))));
  Q := (π[1] (σ[#0 = "b"] T_aux));
  WHILE (¬ ((T = T_aux) ∧ (Q = Q_aux))) DO
    T_aux := T;
    Q_aux := Q;
    T :=
      (T ∪
       (F ∪
        (π[0, 4, 2]
         (σ[(#1 = #3 ∧ #2 = #5)]
          (F × T_aux)))));
  Q := (Q ∪ (π[1] (σ[#0 = "b"] T_aux)));
END;
QUp_aux := ∅[1];
QUp :=
  ((π[1] (σ[#0 = "b"] Va)) ∪
   ((π[3]
    (σ[(#1 = "c" ∧ #0 = #2)] (Vb × Vc))) ∪
    (π[0] Vb)));
  WHILE (¬ (QUp = QUp_aux)) DO
    QUp_aux := QUp;
    QUp :=
      (QUp ∪
       ((π[1] (σ[#0 = "b"] Va)) ∪
        ((π[3]
         (σ[(#1 = "c" ∧ #0 = #2)]
          (Vb × Vc))) ∪
         (π[0] Vb)))));
  END
END

POST
  (QUp ⊆ Q)
```

49 Example4037: Airline reachability query versus its three-view reformulation, reverse containment

Category	verification task (VerificationTasks.pdf, section 10.1); family: Core.
Kind	containment.
Contributors	Leo Zhang, Val Tannen.
Status	certified invalid (Certificate/Invalid.lean, written by the certificate emitter); expected invalid.
Tags	views.
Input	Benchmark/Example4037/Input.lean (canonical notation).
Summary	Single-airline reachability and a reformulation over three flight views, tested for the reverse containment.
Sources	VerificationTasks.pdf, section 10.1 (Val Tannen; reports/VerificationTasks.pdf) the reformulations task document

Datalog program(s). *globalProgram* — ‘ $T\ xyu :- Fxyu$ ’, ‘ $T\ xyu :- Fxzu, T\ zyu$ ’, and ‘ $Qy :- T\ byu$ ’. — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```
T(x1, y1, x2) :- F(x1, y1, x2).
T(x1, y1, x2) :- F(x1, z1, x2), T(z1, y1, x2).
Q(y1) :- T("b", y1, x2).
```

localProgram — ‘ $Q\uparrow y :- Va\ by$ ’, ‘ $Q\uparrow z :- Vb\ yc, Vc\ yz$ ’, and ‘ $Q\uparrow y :- Vb\ yu$ ’. — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```
QUp(y1) :- Va("b", y1).
QUp(z1) :- Vb(y1, "c"), Vc(y1, z1).
QUp(y1) :- Vb(y1, x2).
```

Reading. This case uses the same global and local programs, views, and precondition as Example4036 (per-airline reachability T restricted to departures from "b" for Q , versus the three-view reformulation QUp), but tests the opposite direction of containment, $Q \subseteq QUp$: that every airport genuinely reachable from "b" on a single airline is also captured by the reformulation. This claim is invalid, certified by a kernel-checked counterexample: the reformulation is only maximally (forward-) contained in Q , not surjective onto it, so a two-hop route out of "b" on an airline other than "a" or "c" is invisible to all three views. The witness is $F = \{("b", "x", "d"), ("x", "y", "d")\}$, where at exit $Q = \{"x", "y"\}$ but $QUp = \{"x"\}$.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $(\forall x\ y. (Va(x,y) \leftrightarrow F(x,y,"a"))) \wedge (\forall z\ u. (Vb(z,u) \leftrightarrow F("b",z,u))) \wedge \forall v\ w. (Vc(v,w) \leftrightarrow \exists x1. F(v,x1,"c") \wedge F(x1,w,"c"))$

The precondition states the same three view equations as Example4036: Va is F restricted to airline "a", Vb is F restricted to flights departing "b", and Vc is the two-hop composition of two airline-"c" flights.

POST: $\forall x. (Q(x) \rightarrow QUp(x))$

The postcondition asserts $Q \subseteq QUp$, that every airport genuinely reachable from "b" on one airline is captured by the reformulation — a claim that fails.

Whiel encoding (Hoare triple).

```

SCHEMA
  Q/1, QUp/1, Q_aux/1, QUp_aux/1, Va/2, Vb/2, Vc/2, F/3, T/3, T_aux/3

PRE
  ((Va = (π[0, 1] (σ[#2 = "a"] F))) ∧
   ((Vb = (π[1, 2] (σ[#0 = "b"] F))) ∧
    (Vc =
      (π[0, 4]
       (σ[(#2 = "c" ∧ #1 = #3) ∧ #5 = "c"]
        (F × F)))))))

CMD
  T_aux := ∅[3];
  Q_aux := ∅[1];
  T :=
    (F ∪
     (π[0, 4, 2]
      (σ[(#1 = #3 ∧ #2 = #5)] (F × T_aux))));
  Q := (π[1] (σ[#0 = "b"] T_aux));
  WHILE (¬ ((T = T_aux) ∧ (Q = Q_aux))) DO
    T_aux := T;
    Q_aux := Q;
    T :=
      (T ∪
       (F ∪
        (π[0, 4, 2]
         (σ[(#1 = #3 ∧ #2 = #5)]
          (F × T_aux)))));
    Q := (Q ∪ (π[1] (σ[#0 = "b"] T_aux)));
  END;
  QUp_aux := ∅[1];
  QUp :=
    ((π[1] (σ[#0 = "b"] Va)) ∪
     ((π[3]
      (σ[(#1 = "c" ∧ #0 = #2)] (Vb × Vc))) ∪
      (π[0] Vb)));
  WHILE (¬ (QUp = QUp_aux)) DO
    QUp_aux := QUp;
    QUp :=
      (QUp ∪
       ((π[1] (σ[#0 = "b"] Va)) ∪
        ((π[3]
         (σ[(#1 = "c" ∧ #0 = #2)]
          (Vb × Vc))) ∪
         (π[0] Vb))));
  END

POST
  (Q ⊆ QUp)

```

50 Example4040: Closure of red and blue edges versus its reformulation over Datalog path views

Category	verification task (VerificationTasks.pdf, section 10.1); family: Core.
Kind	equivalence.
Contributors	Leo Zhang, Val Tannen.
Status	certified valid (certificate checked outside the library build) (Certificate/Valid.lean, written by the certificate emitter); expected valid. Placement note: a proof module imported by Certificate/Valid.lean does not build under the library build's 4194304 KB limit; it builds under 12582912 KB (peak 1651 MB)
Tags	linear-vs-nonlinear nonlinear-recursion prophecy views.
Input	Benchmark/Example4040/Input.lean (canonical notation).
Summary	Two-colour reachability and a reformulation read off two recursive path views.
Sources	VerificationTasks.pdf, section 10.1 (Val Tannen; reports/VerificationTasks.pdf) the reformulations task document
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).

Datalog program(s). *globalProgram* — *The transitive closure of $R \cup B'$, left-linearly.* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```

T(x1, y1) :- R(x1, y1).
T(x1, y1) :- B(x1, y1).
T(x1, y1) :- R(x1, z1), T(z1, y1).
T(x1, y1) :- B(x1, z1), T(z1, y1).

```

viewProgram — *The red and the blue monochrome path views.* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
Vr(x1, y1) :- R(x1, y1).
Vr(x1, y1) :- R(x1, z1), Vr(z1, y1).
Vb(x1, y1) :- B(x1, y1).
Vb(x1, y1) :- B(x1, z1), Vb(z1, y1).
```

reformProgram — *The reformulation over the two path views.* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
TUp(x1, y1) :- Vr(x1, y1).
TUp(x1, y1) :- Vb(x1, y1).
TUp(x1, y1) :- TUp(x1, z1), TUp(z1, y1).
```

program — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
T(x1, y1) :- R(x1, y1).
T(x1, y1) :- B(x1, y1).
T(x1, y1) :- R(x1, z1), T(z1, y1).
T(x1, y1) :- B(x1, z1), T(z1, y1).
T(x1, y1) :- T(x1, z1), T(z1, y1).
```

Reading. The global program T computes the transitive closure of $R \cup B$ (red and blue edges of a digraph combined), advancing one edge at a time. The local reformulation is itself recursive Datalog: it first computes two monochrome path views, Vr (red-only paths) and Vb (blue-only paths), each by its own right-linear recursion over R or B , then defines $TUp(x,y) :- Vr(x,y)$, $TUp(x,y) :- Vb(x,y)$, and the nonlinear rule $TUp(x,y) :- TUp(x,z), TUp(z,y)$, composing whole monochrome walks together. The Hoare triple claims equality, $T = TUp$: both compute the same transitive closure of $R \cup B$, one by single-edge steps and the other by self-composing entire same-colour walks. Certified valid.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in `Input.lean`. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\top$

The precondition is simply true; there is no separate view hypothesis, since the views Vr and Vb are themselves computed by Datalog programs within the command.

POST: $\forall x y. (T(x,y) \leftrightarrow TUp(x,y))$

The postcondition asserts $T = TUp$, that the closure of the combined red/blue edges equals the path-view-composition reformulation.

Whiel encoding (Hoare triple).

```
SCHEMA
  R/2, B/2, T/2, Vr/2, Vb/2, TUp/2, T_aux/2, Vr_aux/2, Vb_aux/2, TUp_aux/2

PRE
  true

CMD
  T_aux :=  $\emptyset[2]$ ;
  T :=
    (R  $\cup$ 
     (B  $\cup$ 
      (( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  ( $R \times T\_aux$ )))  $\cup$ 
       ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  ( $B \times T\_aux$ ))))));
  WHILE ( $\neg (T = T\_aux)$ ) DO
    T_aux := T;
    T :=
      (R  $\cup$ 
       (B  $\cup$ 
        (( $\pi[0, 3]$ 
          ( $\sigma[\#1 = \#2]$  ( $R \times T\_aux$ )))  $\cup$ 
         ( $\pi[0, 3]$ 
          ( $\sigma[\#1 = \#2]$  ( $B \times T\_aux$ ))))))
```

```

      (σ[#1 = #2] (B × T_aux))))))
END;
Vr_aux := ∅[2];
Vb_aux := ∅[2];
Vr :=
  (R ∪ (π[0, 3] (σ[#1 = #2] (R × Vr_aux))));
Vb :=
  (B ∪ (π[0, 3] (σ[#1 = #2] (B × Vb_aux))));
WHILE (¬ ((Vr = Vr_aux) ∧ (Vb = Vb_aux))) DO
  Vr_aux := Vr;
  Vb_aux := Vb;
  Vr :=
    (Vr ∪
     (R ∪
      (π[0, 3] (σ[#1 = #2] (R × Vr_aux)))));
  Vb :=
    (Vb ∪
     (B ∪
      (π[0, 3] (σ[#1 = #2] (B × Vb_aux)))));
END;
TUp_aux := ∅[2];
TUp :=
  (Vr ∪
   (Vb ∪
    (π[0, 3]
     (σ[#1 = #2] (TUp_aux × TUp_aux)))));
WHILE (¬ (TUp = TUp_aux)) DO
  TUp_aux := TUp;
  TUp :=
    (TUp ∪
     (Vr ∪
      (Vb ∪
       (π[0, 3]
        (σ[#1 = #2]
         (TUp_aux × TUp_aux))))));
END
POST
  (T = TUp)

```

51 Example4041: Transitive closure versus its reformulation over exact odd/even views

Category	verification task (VerificationTasks.pdf, section 10.1); family: Core.
Kind	equivalence.
Contributors	Jesse Comer, Val Tannen.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	mutual-recursion prophecy views .
Input	Benchmark/Example4041/Input.lean (canonical notation).
Summary	Transitive closure and a reformulation read off two exact odd/even views.
Sources	VerificationTasks.pdf, section 10.1 (Val Tannen; reports/VerificationTasks.pdf) the reformulations task document
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).

Datalog program(s). *globalProgram* — ‘ $T \ xy \text{ :- } Exy$ ’ and ‘ $T \ xy \text{ :- } Exz, T \ zy$ ’. — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```

T(x1, y1) :- E(x1, y1).
T(x1, y1) :- E(x1, z1), T(z1, y1).

```

viewProgram — *The odd/even program that computes the two views.* — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```

To(x1, y1) :- E(x1, y1).
To(x1, y1) :- E(x1, z1), Te(z1, y1).
Te(x1, y1) :- E(x1, z1), To(z1, y1).

```

reformProgram — *The reformulation ‘ $T \uparrow \text{ :- } To$ ’, ‘ $T \uparrow \text{ :- } Te$ ’.* — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```

TUp(x1, y1) :- To(x1, y1).
TUp(x1, y1) :- Te(x1, y1).

```

Reading. The global program T is the standard right-linear transitive closure of E . The local reformulation computes the odd/even path decomposition of E by mutual recursion (To for odd-length paths, Te for even-length paths) and then unions them, $TUp(x,y):-To(x,y)$, $TUp(x,y):-Te(x,y)$. Because the views are exact — computed within the same command rather than assumed as free source tables — the precondition is trivially true and correctness is carried entirely by the computation itself. The Hoare triple claims equality, $TUp = T$: the union of the odd- and even-length-path relations is exactly the transitive closure. Certified valid, via an eleven-clause loop invariant (eight clauses at level 0, three at level 1) closing all 23 verification obligations.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in `Input.lean`. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\top$

The precondition is simply true.

POST: $\forall x y. (TUp(x,y) \leftrightarrow T(x,y))$

The postcondition asserts $TUp = T$, that the union of the odd- and even-length path views equals the transitive closure.

Whiel encoding (Hoare triple).

```

SCHEMA
  E/2, T/2, To/2, Te/2, TUp/2, T_aux/2, To_aux/2, Te_aux/2, TUp_aux/2

PRE
  true

CMD
  T_aux :=  $\emptyset[2]$ ;
  T :=
    (E  $\cup$  ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (E  $\times$  T_aux))));
  WHILE ( $\neg$  (T = T_aux)) DO
    T_aux := T;
    T :=
      (T  $\cup$ 
       (E  $\cup$ 
        ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (E  $\times$  T_aux)))))
  END;
  To_aux :=  $\emptyset[2]$ ;
  Te_aux :=  $\emptyset[2]$ ;
  To :=
    (E  $\cup$  ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (E  $\times$  Te_aux))));
  Te := ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (E  $\times$  To_aux)));
  WHILE ( $\neg$  ((To = To_aux)  $\wedge$  (Te = Te_aux))) DO
    To_aux := To;
    Te_aux := Te;
    To :=
      (To  $\cup$ 
       (E  $\cup$ 
        ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (E  $\times$  Te_aux)))))
    Te :=
      (Te  $\cup$ 
       ( $\pi[0, 3]$  ( $\sigma[\#1 = \#2]$  (E  $\times$  To_aux))))
  END;
  TUp_aux :=  $\emptyset[2]$ ;
  TUp := (To  $\cup$  Te);
  WHILE ( $\neg$  (TUp = TUp_aux)) DO
    TUp_aux := TUp;
    TUp := (TUp  $\cup$  (To  $\cup$  Te))
  END

POST
  (TUp = T)

```

52 Example5001: PostgreSQL recursive UNION versus naive seeded closure

Category	production pair (two algorithms from production sources or legacy production cases); family: direct-equality pair.
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	database-engine evaluation-strategies prophecy semi-naive.
Input	Benchmark/Example5001/Input.lean (canonical notation).
Summary	PostgreSQL's WITH RECURSIVE executor against the naive evaluation of the same recursive query.
Sides	legacy Example2037 (PostgreSQL nodeRecursiveunion.c model) naive compiler on the seeded closure program
Sources	postgres/postgres src/backend/executor/nodeRecursiveunion.c (ExecRecursiveUnion) PostgreSQL documentation, WITH queries (recursive)
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).

Datalog program(s). *prog_p* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
Tcl(x1, y1) :- Base(x1, y1).
Tcl(x1, y1) :- Tcl(x1, z1), Edge(z1, y1).
```

Reading. Both programs compute $\text{Base} \circ \text{Edge}^*$, the rows reachable from a base relation by following zero or more Edge steps — a standard recursive-transitive-closure computation of the kind PostgreSQL's WITH RECURSIVE executor performs. Side 1 models that executor: it maintains a working table WorkT holding only the previous round's newly derived rows, joins WorkT with Edge, subtracts rows already in the accumulated result Res, and halts once nothing new is produced — a frontier/delta-based fixpoint evaluation. Side 2 is the standard naive evaluation of $\text{Tcl}(x,y) \text{ :- Base}(x,y)$, $\text{Tcl}(x,y) \text{ :- Tcl}(x,z), \text{Edge}(z,y)$, recomputing the whole relation each round. Run one after the other as independent programs, the Hoare triple claims the two agree, $\text{Res} = \text{Tcl}$ — the standard correctness statement that a delta-restricted (semi-naive-style) executor computes the same least fixpoint as full naive evaluation.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in Input.lean remains the authority.

PRE: $\top$

The precondition is simply true.

POST: $\forall x y. (\text{Res}(x,y) \leftrightarrow \text{Tcl}(x,y))$

The postcondition asserts $\text{Res} = \text{Tcl}$, that the PostgreSQL-style recursive-union executor and naive evaluation compute the same closure.

Whiel encoding (Hoare triple).

```
SCHEMA
  Base/2, Edge/2, Res/2, WorkT/2, Inter/2, Tcl/2, Tcl_aux/2

PRE
```

```

true

CMD
Res := Base;
WorkT := Base;
WHILE (WorkT ≠ ∅) DO
  Inter := ((π[0, 3] (σ[#1 = #2] (WorkT × Edge))) \ Res);
  Res := (Res ∪ Inter);
  WorkT := Inter
END;
Tcl_aux := ∅[2];
Tcl := (Base ∪ π[0,3] (σ[#1 = #2] ((Tcl_aux × Edge))));
WHILE (¬(Tcl = Tcl_aux)) DO
  Tcl_aux := Tcl;
  Tcl := (Tcl ∪ (Base ∪ π[0,3] (σ[#1 = #2] ((Tcl_aux × Edge)))))
END

POST
(Res = Tcl)

```

53 Example5002: Datafrog semi-naive delta evaluation versus naive evaluation

Category	production pair (two algorithms from production sources or legacy production cases); family: direct-equality pair.
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	database-engine evaluation-strategies prophecy semi-naive.
Input	Benchmark/Example5002/Input.lean (canonical notation).
Summary	Datafrog's semi-naive delta evaluation of a reachability query against the naive evaluation of the same query.
Sides	legacy Example2039 (Datafrog / Soufflé semi-naive delta loop) naive compiler on the transposed reachability program
Sources	rust-lang/datafrog (README example, src/lib.rs Variable/from_join/changed)
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).
Note	Kept after the duplicate review: the loop coincides with 5001 up to transposition and seeding, but it is the corpus's only Datafrog case (2039 was retired).

Datalog program(s). *prog_p* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```

Reach(y1, x1) :- EdgeF(x1, y1).
Reach(z1, x1) :- Reach(y1, x1), EdgeF(y1, z1).

```

Reading. Both sides compute the transpose of the transitive closure of EdgeF: Nodes/Reach(y,x) holds when y is reachable from x by one or more EdgeF steps. Side 1 models Datafrog's (and similarly Soufflé's) semi-naive delta evaluation: it maintains only the newly derived facts DeltaN from the previous round, joins them with EdgeF, subtracts what is already known, and halts when the delta becomes empty. Side 2 is naive evaluation of Reach(y,x):-EdgeF(x,y), Reach(z,x):-Reach(y,x),EdgeF(y,z), recomputing the full relation every round. The Hoare triple, running the two independent programs side by side, claims equality, Nodes = Reach — the standard semi-naive-evaluation correctness statement, that restricting joins to newly derived facts each round yields the same least fixpoint as full naive evaluation.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in `Input.lean`. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\top$

The precondition is simply true.

POST: $\forall x y. (\text{Nodes}(x,y) \leftrightarrow \text{Reach}(x,y))$

The postcondition asserts $\text{Nodes} = \text{Reach}$, that the semi-naive delta evaluation and naive evaluation compute the same reachability relation.

Whiel encoding (Hoare triple).

```

SCHEMA
  EdgeF/2, Nodes/2, DeltaN/2, NewN/2, Reach/2, Reach_aux/2

PRE
  true

CMD
  Nodes := ( $\pi[1, 0]$  EdgeF);
  DeltaN := ( $\pi[1, 0]$  EdgeF);
  WHILE (DeltaN  $\neq$   $\emptyset$ ) DO
    NewN := (( $\pi[3, 1]$  ( $\sigma[\#0 = \#2]$  (DeltaN  $\times$  EdgeF))) \ Nodes);
    Nodes := (Nodes  $\cup$  NewN);
    DeltaN := NewN
  END;
  Reach_aux :=  $\emptyset[2]$ ;
  Reach := ( $\pi[1,0]$  (EdgeF)  $\cup$   $\pi[3,1]$  ( $\sigma[\#0 = \#2]$  ((Reach_aux  $\times$  EdgeF))));
  WHILE ( $\neg$ ((Reach = Reach_aux))) DO
    Reach_aux := Reach;
    Reach := (Reach  $\cup$  ( $\pi[1,0]$  (EdgeF)  $\cup$   $\pi[3,1]$  ( $\sigma[\#0 = \#2]$  ((Reach_aux  $\times$  EdgeF)))))
  END

POST
  (Nodes = Reach)

```

54 Example5003: Worklist breadth-first reachability versus naive evaluation

Category	production pair (two algorithms from production sources or legacy production cases); family: direct-equality pair.
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	evaluation-strategies prophecy semi-naive .
Input	Benchmark/Example5003/Input.lean (canonical notation).
Summary	A worklist breadth-first search against the naive evaluation of the reachability query it implements.
Sides	hard case Example0161 (worklist BFS with explicit frontier) naive compiler on the reachability program
Sources	hard-case source records of the retired classic-source collection (hard-*.json)
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).

Datalog program(s). *prog_p* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```

Reach(x1) :- Src(x1).
Reach(y1) :- Reach(x1), E(x1, y1).

```

Reading. Both sides compute the set of vertices reachable from a source set Src over edge relation E . Side 1 is an explicit worklist/frontier breadth-first search: R is the visited set and W the current frontier; each round computes the newly discovered vertices N as E -successors of W not already in R , adds them to R , and advances the frontier $W := N$ — the classic BFS-as-fixpoint-iteration pattern. Side 2 is naive Datalog evaluation of $\text{Reach}(x) :- \text{Src}(x)$, $\text{Reach}(y) :- \text{Reach}(x), E(x, y)$. The Hoare triple, running the two independent programs side by side, claims equality, $R = \text{Reach}$ — that the explicit worklist BFS and full naive evaluation compute the same reachable set.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in `Input.lean`. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\top$

The precondition is simply true.

POST: $\forall x. (R(x) \leftrightarrow \text{Reach}(x))$

The postcondition asserts $R = \text{Reach}$, that worklist BFS and naive Datalog evaluation compute the same reachable set.

Whiel encoding (Hoare triple).

```
SCHEMA
  Src/1, R/1, W/1, N/1, Reach/1, Reach_aux/1, E/2

PRE
  true

CMD
  R := Src;
  W := Src;
  N := Src;
  WHILE (W ≠ ∅) DO
    N := ((π[2] (σ[#0 = #1] (W × E))) \ R);
    R := (R ∪ N);
    W := N
  END;
  Reach_aux := ∅[1];
  Reach := (Src ∪ π[2] (σ[#0 = #1] ((Reach_aux × E))));
  WHILE (¬((Reach = Reach_aux))) DO
    Reach_aux := Reach;
    Reach := (Reach ∪ (Src ∪ π[2] (σ[#0 = #1] ((Reach_aux × E)))))
  END

POST
  (R = Reach)
```

55 Example5004: Materialized role-hierarchy closure with a join versus a recursive has-role query

Category	production pair (two algorithms from production sources or legacy production cases); family: direct-equality pair.
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	access-control evaluation-strategies prophecy.
Input	Benchmark/Example5004/Input.lean (canonical notation).
Summary	Two ways access-control engines answer "does user u hold role r": materialize the role hierarchy's closure and join once (Casbin's role manager), or recurse over the hierarchy from the user's direct roles (OPA/Zanzibar style, legacy Example2035 without its violation query).
Sides	naive compiler: role closure then join (Casbin RoleManager model) naive compiler: recursive has-role (legacy Example2035 program)
Sources	casbin/casbin rbac/default-role-manager/role_manager.go (HasLink, hasLinkHelper) casbin/casbin enforcer.go (maxHierarchyLevel = 10)
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).

Datalog program(s). *prog_pa* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
Inh(x1, y1) :- RoleInh(x1, y1).
Inh(x1, z1) :- Inh(x1, y1), RoleInh(y1, z1).
```

prog_pb — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
HasM(x1, y1) :- UserRole(x1, y1).
HasM(x1, z1) :- UserRole(x1, y1), Inh(y1, z1).
```

prog_pc — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
Has(x1, y1) :- UserRole(x1, y1).
Has(x1, z1) :- Has(x1, y1), RoleInh(y1, z1).
```

Reading. Both sides answer "does user u hold role r" via role-inheritance closure over UserRole and RoleInh. Side 1 models a materialize-then-join strategy (as in Casbin's default role manager): it first computes the full, unbounded role-inheritance closure Inh by recursion over RoleInh, then joins it once against UserRole to get HasM — with the join program reading the closure program's output, making the sequence dependent. Side 2 models an OPA/Zanzibar-style engine that instead recurses directly from a user's own roles without ever materializing the hierarchy: Has(u,r):-UserRole(u,r), Has(u,t):-Has(u,r),RoleInh(r,t). The Hoare triple claims the two access-control strategies agree, HasM = Has — both compute UserRole ◦ RoleInh*, the standard role-hierarchy-closure correctness statement, even though the two loops converge at different rates since the join side must wait for the closure computation to finish first.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in Input.lean remains the authority.

PRE: $\top$

The precondition is simply true.

POST: $\forall x y. (\text{HasM}(x,y) \leftrightarrow \text{Has}(x,y))$

The postcondition asserts HasM = Has, that materialize-then-join role resolution and direct recursive role resolution compute the same has-role relation.

Whiel encoding (Hoare triple).

```

SCHEMA
  UserRole/2, RoleInh/2, Inh/2, Inh_aux/2, HasM/2, HasM_aux/2, Has/2, Has_aux/2

PRE
  true

CMD
  Inh_aux :=  $\emptyset[2]$ ;
  Inh :=  $(\text{RoleInh} \cup \pi[0,3] (\sigma[\#1 = \#2] ((\text{Inh\_aux} \times \text{RoleInh}))))$ ;
  WHILE ( $\neg((\text{Inh} = \text{Inh\_aux}))$ ) DO
    Inh_aux := Inh;
    Inh :=  $(\text{Inh} \cup (\text{RoleInh} \cup \pi[0,3] (\sigma[\#1 = \#2] ((\text{Inh\_aux} \times \text{RoleInh}))))$ 
  END;
  HasM_aux :=  $\emptyset[2]$ ;
  HasM :=  $(\text{UserRole} \cup \pi[0,3] (\sigma[\#1 = \#2] ((\text{UserRole} \times \text{Inh}))))$ ;
  WHILE ( $\neg((\text{HasM} = \text{HasM\_aux}))$ ) DO
    HasM_aux := HasM;
    HasM :=  $(\text{HasM} \cup (\text{UserRole} \cup \pi[0,3] (\sigma[\#1 = \#2] ((\text{UserRole} \times \text{Inh}))))$ 
  END;
  Has_aux :=  $\emptyset[2]$ ;
  Has :=  $(\text{UserRole} \cup \pi[0,3] (\sigma[\#1 = \#2] ((\text{Has\_aux} \times \text{RoleInh}))))$ ;
  WHILE ( $\neg((\text{Has} = \text{Has\_aux}))$ ) DO
    Has_aux := Has;
    Has :=  $(\text{Has} \cup (\text{UserRole} \cup \pi[0,3] (\sigma[\#1 = \#2] ((\text{Has\_aux} \times \text{RoleInh}))))$ 
  END

POST
  (HasM = Has)

```

56 Example5005: Incremental maintenance of a reachability view versus recomputation from scratch

Category	production pair (two algorithms from production sources or legacy production cases); family: direct-equality pair.
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	evaluation-strategies prophecy view-maintenance.
Input	Benchmark/Example5005/Input.lean (canonical notation).
Summary	Incremental view maintenance against recomputation, the direct form of legacy Example2011.
Sides	naive compiler: old view legacy Example2025 (incremental maintainer) naive compiler: recomputation on EBase $\cup$ EDelta
Sources	TimelyDataflow/differential-dataflow Materialize arrangements / incremental dataflows (documentation)
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).

Datalog program(s). *prog_pa* — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```
ReachOld(x1) :- Roots(x1).
ReachOld(y1) :- ReachOld(x1), EBase(x1, y1).
```

prog_pc — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```
ReachNew(x1) :- Roots(x1).
ReachNew(y1) :- ReachNew(x1), EBase(x1, y1).
ReachNew(y1) :- ReachNew(x1), EDelta(x1, y1).
```

Reading. All three programs concern reachability from Roots over an edge relation that is updated by an insertion EDelta. Program 1 (compiled) computes the old view ReachOld using only the base edges EBase. Program 2 is an incremental-view-maintenance algorithm, in the style used by systems like Differential Dataflow or Materialize: it restarts from ReachOld $\cup$ Roots and iterates over the combined edges EBase $\cup$ EDelta, an insert-only re-evaluation seeded from the previously materialized view rather than computed from scratch; it reads program 1’s output, so the sequence is dependent. Program 3 (compiled) recomputes ReachNew from scratch directly over the updated graph EBase $\cup$ EDelta. The Hoare triple claims the incrementally maintained result equals full recomputation, Reach = ReachNew — the standard correctness statement for incremental view maintenance under an insertion-only update.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in Input.lean remains the authority.

PRE: $\top$

The precondition is simply true.

POST: $\forall x. (\text{Reach}(x) \leftrightarrow \text{ReachNew}(x))$

The postcondition asserts Reach = ReachNew, that incrementally maintaining the reachability view from the old result equals recomputing it from scratch on the updated graph.

Whiel encoding (Hoare triple).

```

SCHEMA
  Roots/1, ReachOld/1, ReachOld_aux/1, Reach/1, Reach_2/1, ReachNew/1, ReachNew_aux/1, EBase/2, EDelta/2

PRE
  true

CMD
  ReachOld_aux := ∅[1];
  ReachOld := (Roots ∪ π[2] (σ[#0 = #1] ((ReachOld_aux × EBase))));
  WHILE (¬((ReachOld = ReachOld_aux))) DO
    ReachOld_aux := ReachOld;
    ReachOld := (ReachOld ∪ (Roots ∪ π[2] (σ[#0 = #1] ((ReachOld_aux × EBase)))))
  END;
  Reach_2 := ∅[1];
  Reach := (ReachOld ∪ Roots);
  WHILE (Reach ≠ Reach_2) DO
    Reach_2 := Reach;
    Reach := ((Roots ∪ ReachOld) ∪ (π[1] (σ[#0 = #2] ((EBase ∪ EDelta) × Reach_2))))
  END;
  ReachNew_aux := ∅[1];
  ReachNew := (Roots ∪ (π[2] (σ[#0 = #1] ((ReachNew_aux × EBase))) ∪ π[2] (σ[#0 = #1] ((ReachNew_aux × EDelta)))));
  WHILE (¬((ReachNew = ReachNew_aux))) DO
    ReachNew_aux := ReachNew;
    ReachNew := (ReachNew ∪ (Roots ∪ (π[2] (σ[#0 = #1] ((ReachNew_aux × EBase))) ∪ π[2] (σ[#0 = #1] ((ReachNew_aux ×
      ↪ EDelta))))))
  END

POST
  (Reach = ReachNew)

```

57 Example5006: Semi-naive closure with an appended delta versus a prepended delta

Category	production pair (two algorithms from production sources or legacy production cases); family: direct-equality pair.
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	evaluation-strategies linear-vs-nonlinear prophecy semi-naive.
Input	Benchmark/Example5006/Input.lean (canonical notation).
Summary	Two semi-naive evaluations of transitive closure that differ in where the delta is joined: appended ('Delta ∘ E', legacy Example1042) or prepended ('E ∘ Delta', legacy Example1058).
Sides	legacy Example1042 (append delta) legacy Example1058 (prepend delta)
Sources	LLM benchmark source records of the retired corpus (GPT and Claude generated; no external source)
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).

Datalog program(s). *Side 1, appended delta (legacy Example1042); side 2, prepended delta (legacy Example1058); written for this report — hand-written reading (this report).*

```

AccA(x, y) :- E(x, y).
AccA(x, z) :- AccA(x, y), E(y, z).

AccP(x, y) :- E(x, y).
AccP(x, z) :- E(x, y), AccP(y, z).

```

Reading. Both sides compute the transitive closure of E via semi-naive delta evaluation, differing only in which side of the join the growing frontier is placed on: side A appends the delta (joining the frontier on the left of E , $\Delta \circ E$) while side P prepends it (joining E onto the frontier from the left, $E \circ \Delta$), each round adding the frontier of walks one edge longer than the previous round. The Hoare triple, running the two independent programs side by side, claims the two evaluation orders produce equal results, $\text{AccA} = \text{AccP}$ — since both frontiers hold the same set of walks of length $n+1$ after round n regardless of which side of the join the new edge is added on, the fixpoints coincide.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in `Input.lean`. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\top$

The precondition is simply true.

POST: $\forall x y. (\text{AccA}(x,y) \leftrightarrow \text{AccP}(x,y))$

The postcondition asserts $\text{AccA} = \text{AccP}$, that the append-delta and prepend-delta semi-naive evaluations of transitive closure compute the same relation.

Whiel encoding (Hoare triple).

```

SCHEMA
  E/2, AccA/2, DeltaA/2, AccP/2, DeltaP/2

PRE
  true

CMD
  AccA := E;
  DeltaA := E;
  WHILE (DeltaA ≠ ∅) DO
    DeltaA := ((π[0, 3] (σ[#1 = #2] (DeltaA × E))) \ AccA);
    AccA := (AccA ∪ DeltaA)
  END;
  AccP := E;
  DeltaP := E;
  WHILE (DeltaP ≠ ∅) DO
    DeltaP := ((π[0, 3] (σ[#1 = #2] (E × DeltaP))) \ AccP);
    AccP := (AccP ∪ DeltaP)
  END

POST
  (AccA = AccP)

```

58 Example5007: Transitive closure by repeated squaring versus linear closure

Category	production pair (two algorithms from production sources or legacy production cases); family: direct-equality pair.
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	evaluation-strategies linear-vs-nonlinear nonlinear-recursion prophecy.
Input	Benchmark/Example5007/Input.lean (canonical notation).
Summary	Closure by repeated squaring (legacy Example1056) against the naive evaluation of the linear closure program.
Sides	legacy Example1056 (closure by squaring) naive compiler on the linear closure program
Sources	LLM benchmark source records of the retired corpus (GPT and Claude generated; no external source)
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).

Datalog program(s). *prog_p* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
T(x1, y1) :- E(x1, y1).
T(x1, y1) :- E(x1, z1), T(z1, y1).
```

Reading. Both sides compute the transitive closure of E. Side 1 computes it by repeated squaring: starting from E, each round unions in $TSq \circ TSq$ (the self-composition of the accumulated relation), doubling the walk length covered every iteration — a logarithmic-depth closure algorithm. Side 2 is standard linear naive evaluation, $T(x,y) :- E(x,y)$, $T(x,y) :- E(x,z), T(z,y)$, which extends the covered walk length by one edge per round. The Hoare triple claims the two algorithms converge to the same relation, $TSq = T$ — the same least fixpoint reached at very different rates, walks up to length $2^{(n+1)}$ after round n for the squaring side versus length $n+2$ for the linear side.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in *Input.lean* remains the authority.

PRE: $\top$

The precondition is simply true.

POST: $\forall x y. (TSq(x,y) \leftrightarrow T(x,y))$

The postcondition asserts $TSq = T$, that closure by repeated squaring and linear naive closure compute the same transitive-closure relation.

Whiel encoding (Hoare triple).

```
SCHEMA
  E/2, TSq_2/2, TSq_2/2, T/2, T_aux/2

PRE
  true

CMD
  TSq_2 :=  $\emptyset[2]$ ;
  TSq := E;
  WHILE (TSq  $\neq$  TSq_2) DO
```

```

    TSq_2 := TSq;
    TSq := (TSq ∪ (π[0, 3] (σ[#1 = #2] (TSq_2 × TSq_2))))
END;
T_aux := ∅[2];
T := (E ∪ π[0,3] (σ[#1 = #2] ((E × T_aux))));
WHILE (¬((T = T_aux))) DO
    T_aux := T;
    T := (T ∪ (E ∪ π[0,3] (σ[#1 = #2] ((E × T_aux)))))
END
POST
(TSq = T)

```

59 Example5008: Soufflé tc benchmark: naive versus semi-naive evaluation

Category	Soufflé benchmark program (naive versus semi-naive translation); family: Soufflé naive vs semi-naive.
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified valid (certificate checked outside the library build) (Certificate/Valid.lean, written by the certificate emitter); expected valid. Placement note: a proof module imported by <code>Certificate/Valid.lean</code> does not build under the library build's 4194304 KB limit; it builds under 12582912 KB (peak 1686 MB)
Tags	database-engine evaluation-strategies nonlinear-recursion prophecy semi-naive.
Input	Benchmark/Example5008/Input.lean (canonical notation).
Summary	Soufflé's 'tc' benchmark program evaluated naively and semi-naively, tested for equality of every output.
Sides	naive translation of tc.dl (Soufflé benchmark suite) semi-naive translation of tc.dl (Soufflé benchmark suite)
Sources	souffle-lang/benchmarks (the Soufflé benchmark programs) Soufflé translation to C++ / semi-naive loop (documentation) program source file tc.dl of the Soufflé benchmark suite (naive translation)
Prophecy	accepted invariant with a prophecy clause above level zero (Core.json); no certificate built from it yet.

Datalog program(s). *normalized Soufflé source (tc.dl of the Soufflé benchmark suite (naive translation))* — Soufflé benchmark program, normalized (both sides are script translations of it).

```

.decl tcl(v0:symbol, v1:symbol) output
.decl tcr(v0:symbol, v1:symbol) output
.decl base(v0:symbol, v1:symbol) input
.decl tc(v0:symbol, v1:symbol) output

tcl(X, Y) :- base(X, Y).
tcl(X, Y) :- tcl(X, Z), base(Z, Y).
tcr(X, Y) :- base(X, Y).
tcr(X, Y) :- base(X, Z), tcr(Z, Y).
tc(X, Y) :- base(X, Y).
tc(X, Y) :- tc(X, Z), tc(Z, Y).

```

Reading. Soufflé's tc.dl benchmark computes graph transitive closure via three intertwined recursive relations over an edge relation base: tc (the ordinary both-sides closure), tcl (built by left-extending tcl with base), and tcr (built by right-extending base with tcr) — a standard reachability computation. The Hoare triple compares

Soufflé’s naive fixpoint evaluation of these rules against classical semi-naive (delta-based) evaluation of the same rules, claiming the two strategies compute identical output relations, the standard naive/semi-naive evaluation equivalence for Datalog.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\top$

True; no precondition constrains the inputs.

POST: $(\forall x y. (tc(x,y) \leftrightarrow tcS(x,y))) \wedge (\forall z u. (tcl(z,u) \leftrightarrow tclS(z,u))) \wedge \forall v w. (tcr(v,w) \leftrightarrow tcrS(v,w))$

The naively evaluated `tc`, `tcl`, and `tcr` each equal the corresponding semi-naively evaluated relation (`tcS`, `tclS`, `tcrS`).

Whiel encoding (Hoare triple).

```
SCHEMA
  base/2, tc/2, tcl/2, tcr/2, tcNew/2, tcOld/2, tclNew/2, tclOld/2, tcrNew/2, tcrOld/2, tcS/2, tclS/2, tcrS/2, deltaTcS/2, deltaTcOldS/2, deltaTclS/2, deltaTclOldS/2, deltaTcrS/2,
  ↪ deltaTcrOldS/2

PRE
  true

CHD
  tcNew := (π[0, 1] base);
  tcOld := ∅[2];
  tclNew := (π[0, 1] base);
  tclOld := ∅[2];
  tcrNew := (π[0, 1] base);
  tcrOld := ∅[2];
  WHILE (((tcNew ≠ tcOld) ∨ (tclNew ≠ tclOld)) ∨ (tcrNew ≠ tcrOld)) DO
    tcOld := tcNew;
    tclOld := tclNew;
    tcrOld := tcrNew;
    tcNew := (tcNew ∪ (π[0, 3] (σ[#2 = #1] (tcOld × tcOld))));
    tclNew := (tclNew ∪ (π[0, 3] (σ[#2 = #1] (tclOld × base))));
    tcrNew := (tcrNew ∪ (π[0, 3] (σ[#2 = #1] (base × tcrOld))));
  END;
  tc := tcNew;
  tcl := tclNew;
  tcr := tcrNew;
  tcS := ∅[2];
  deltaTcS := (π[0, 1] base);
  deltaTcOldS := ∅[2];
  tclS := ∅[2];
  deltaTclS := (π[0, 1] base);
  deltaTclOldS := ∅[2];
  tcrS := ∅[2];
  deltaTcrS := (π[0, 1] base);
  deltaTcrOldS := ∅[2];
  WHILE (((deltaTcS ≠ ∅[2]) ∨ (deltaTclS ≠ ∅[2])) ∨ (deltaTcrS ≠ ∅[2])) DO
    deltaTcOldS := deltaTcS;
    deltaTclOldS := deltaTclS;
    deltaTcrOldS := deltaTcrS;
    tcS := (tcS ∪ deltaTcOldS);
    tclS := (tclS ∪ deltaTclOldS);
    tcrS := (tcrS ∪ deltaTcrOldS);
    deltaTcS := (((π[0, 3] (σ[#2 = #1] (deltaTcOldS × tcS))) ∪ (π[0, 3] (σ[#2 = #1] (tcS × deltaTcOldS)))) ∪ (π[0, 3] (σ[#2 = #1] (deltaTcOldS × deltaTcOldS)))) \ tcS;
    deltaTclS := ((π[0, 3] (σ[#2 = #1] (deltaTclOldS × base))) \ tclS);
    deltaTcrS := ((π[0, 3] (σ[#2 = #1] (base × deltaTcrOldS))) \ tcrS);
  END

POST
  ((tc = tcS) ∧ (tcl = tclS) ∧ (tcr = tcrS))
```

60 Example5009: Soufflé family benchmark: naive versus semi-naive evaluation

Category	Soufflé benchmark program (naive versus semi-naive translation); family: Soufflé naive vs semi-naive.
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified valid (certificate checked outside the library build) (Certificate/Valid.lean, written by the certificate emitter); expected valid. Placement note: a proof module imported by Certificate/Valid.lean does not build under the library build's 4194304 KB limit; it builds under 12582912 KB (peak 1668 MB)
Tags	database-engine evaluation-strategies negation prophecy semi-naive.
Input	Benchmark/Example5009/Input.lean (canonical notation).
Summary	Soufflé's 'family' benchmark program evaluated naively and semi-naively, tested for equality of every output.
Sides	naive translation of family.dl (Soufflé benchmark suite) semi-naive translation of family.dl (Soufflé benchmark suite)
Sources	souffle-lang/benchmarks (the Soufflé benchmark programs) Soufflé translation to C++ / semi-naive loop (documentation) program source file family.dl of the Soufflé benchmark suite (naive translation)
Prophecy	accepted invariant with a prophecy clause above level zero (Core.json); no certificate built from it yet.

Datalog program(s). *normalized Soufflé source (family.dl of the Soufflé benchmark suite (naive translation))*
— Soufflé benchmark program, normalized (both sides are script translations of it).

```
.decl father(v0:symbol, v1:symbol) input
.decl mother(v0:symbol, v1:symbol) input
.decl parent(v0:symbol, v1:symbol) output
.decl ancestor(v0:symbol, v1:symbol) output
.decl grandmother(v0:symbol, v1:symbol) output
.decl sibling(v0:symbol, v1:symbol) output
.decl cousin(v0:symbol, v1:symbol) output
.decl relative(v0:symbol, v1:symbol) output

parent(X, Y) :- father(X, Y).
parent(X, Y) :- mother(X, Y).
ancestor(X, Y) :- parent(X, Y).
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).
grandmother(X, Y) :- mother(X, Z), ancestor(Z, Y).
sibling(X, Y) :- parent(Z, X), parent(Z, Y), X != Y.
cousin(X, Y) :- ancestor(Z, X), ancestor(Z, Y), X != Y, !sibling(X, Y), !parent(X, Y).
relative(X, Y) :- sibling(X, Z), parent(Z, Y).
```

Reading. Soufflé's family.dl benchmark derives standard kinship relations — parent, the transitive-closure relation ancestor, sibling, cousin, grandmother, and relative — from father/mother facts, a typical family-relationships Datalog program with one recursive rule (ancestor) and several non-recursive relations built on top of it. The triple compares naive fixpoint evaluation of the whole program against semi-naive evaluation, claiming every one of the six output relations is computed identically by both strategies.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in Input.lean remains the authority.

PRE: $\top$

True; no precondition constrains the inputs.

POST: $(\forall x y. (\text{ancestor}(x,y) \leftrightarrow \text{ancestorS}(x,y))) \wedge (\forall z u. (\text{cousin}(z,u) \leftrightarrow \text{cousinS}(z,u))) \wedge (\forall v w. (\text{grandmother}(v,w) \leftrightarrow \text{grandmotherS}(v,w))) \wedge (\forall x_1 x_2. (\text{parent}(x_1,x_2) \leftrightarrow \text{parentS}(x_1,x_2))) \wedge (\forall x_3 x_4. (\text{relative}(x_3,x_4) \leftrightarrow \text{relativeS}(x_3,x_4))) \wedge \forall x_5 x_6. (\text{sibling}(x_5,x_6) \leftrightarrow \text{siblingS}(x_5,x_6))$

Each of the six output relations (ancestor, cousin, grandmother, parent, relative, sibling) computed by naive evaluation equals its semi-naive counterpart.

Whiel encoding (Hoare triple).

SCHEMA

father/2, mother/2, ancestor/2, cousin/2, grandmother/2, parent/2, relative/2, sibling/2, ancestorNew/2, ancestorOld/2,
 $\hookrightarrow$ ancestorS/2, cousinS/2, grandmotherS/2, parentS/2, relativeS/2, siblingS/2, deltaAncestorS/2

PRE

true

CMD

```
parent := (( $\pi[0, 1]$  father)  $\cup$  ( $\pi[0, 1]$  mother));
sibling := ( $\pi[1, 3]$  ( $\sigma[\#2 = \#0 \wedge \neg(\#1 = \#3)]$  (parent  $\times$  parent)));
relative := ( $\pi[0, 3]$  ( $\sigma[\#2 = \#1]$  (sibling  $\times$  parent)));
ancestorNew := ( $\pi[0, 1]$  parent);
ancestorOld :=  $\emptyset[2]$ ;
WHILE (ancestorNew  $\neq$  ancestorOld) DO
  ancestorOld := ancestorNew;
  ancestorNew := (ancestorNew  $\cup$  ( $\pi[0, 3]$  ( $\sigma[\#2 = \#1]$  (parent  $\times$  ancestorOld))));
END;
ancestor := ancestorNew;
cousin := ( $\pi[1, 3]$  ((( $\sigma[\#2 = \#0 \wedge \neg(\#1 = \#3)]$  (ancestor  $\times$  ancestor))  $\setminus$  ( $\pi[0, 1, 2, 3]$  ( $\sigma[\#4 = \#1 \wedge \#5 = \#3]$  (( $\sigma[\#2 = \#0 \wedge \neg(\#1 = \#3)]$  (ancestor  $\times$  ancestor))  $\times$  sibling))))  $\setminus$  ( $\pi[0, 1, 2, 3]$  ( $\sigma[\#4 = \#1 \wedge \#5 = \#3]$  ((( $\sigma[\#2 = \#0 \wedge \neg(\#1 = \#3)]$  (ancestor  $\times$  ancestor))  $\times$  sibling))))  $\times$  parent))));
grandmother := ( $\pi[0, 3]$  ( $\sigma[\#2 = \#1]$  (mother  $\times$  ancestor)));
parentS := (( $\pi[0, 1]$  father)  $\cup$  ( $\pi[0, 1]$  mother));
siblingS := ( $\pi[1, 3]$  ( $\sigma[\#2 = \#0 \wedge \neg(\#1 = \#3)]$  (parentS  $\times$  parentS)));
relativeS := ( $\pi[0, 3]$  ( $\sigma[\#2 = \#1]$  (siblingS  $\times$  parentS)));
ancestorS :=  $\emptyset[2]$ ;
deltaAncestorS := ( $\pi[0, 1]$  parentS);
WHILE (deltaAncestorS  $\neq$   $\emptyset[2]$ ) DO
  ancestorS := (ancestorS  $\cup$  deltaAncestorS);
  deltaAncestorS := (( $\pi[0, 3]$  ( $\sigma[\#2 = \#1]$  (parentS  $\times$  deltaAncestorS)))  $\setminus$  ancestorS);
END;
cousinS := ( $\pi[1, 3]$  ((( $\sigma[\#2 = \#0 \wedge \neg(\#1 = \#3)]$  (ancestorS  $\times$  ancestorS))  $\setminus$  ( $\pi[0, 1, 2, 3]$  ( $\sigma[\#4 = \#1 \wedge \#5 = \#3]$  (( $\sigma[\#2 = \#0 \wedge \neg(\#1 = \#3)]$  (ancestorS  $\times$  ancestorS))  $\times$  siblingS))))  $\setminus$  ( $\pi[0, 1, 2, 3]$  ( $\sigma[\#4 = \#1 \wedge \#5 = \#3]$  ((( $\sigma[\#2 = \#0 \wedge \neg(\#1 = \#3)]$  (ancestorS  $\times$  ancestorS))  $\times$  siblingS))))  $\times$  parentS))));
grandmotherS := ( $\pi[0, 3]$  ( $\sigma[\#2 = \#1]$  (mother  $\times$  ancestorS)));
```

POST

((ancestor = ancestorS) $\wedge$ (cousin = cousinS) $\wedge$ (grandmother = grandmotherS) $\wedge$ (parent = parentS) $\wedge$ (relative = relativeS) $\wedge$ (sibling = siblingS))

61 Example5010: Soufflé orbits benchmark: naive versus semi-naive evaluation

Category	Soufflé benchmark program (naive versus semi-naive translation); family: Soufflé naive vs semi-naive.
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	database-engine evaluation-strategies negation nonlinear-recursion prophecy semi-naive.
Input	Benchmark/Example5010/Input.lean (canonical notation).
Summary	Soufflé's 'orbits' benchmark program evaluated naively and semi-naively, tested for equality of every output.
Sides	naive translation of orbits.dl (Soufflé benchmark suite) semi-naive translation of orbits.dl (Soufflé benchmark suite)
Sources	souffle-lang/benchmarks (the Soufflé benchmark programs) Soufflé translation to C++ / semi-naive loop (documentation) program source file orbits.dl of the Soufflé benchmark suite (naive translation)
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).

Datalog program(s). *normalized Soufflé source (orbits.dl of the Soufflé benchmark suite (naive translation))*
 — Soufflé benchmark program, normalized (both sides are script translations of it).

```
.decl orbits_seed(v0:symbol, v1:symbol) input
.decl orbits(v0:symbol, v1:symbol) output
.decl intermediate(v0:symbol, v1:symbol) output
.decl satellite(v0:symbol, v1:symbol) output

orbits(V0, V1) :- orbits_seed(V0, V1).
orbits(X, Y) :- orbits(X, Z), orbits(Z, Y).
satellite(X, Y) :- orbits(X, Y), !intermediate(X, Y).
intermediate(X, Y) :- orbits(X, Z), orbits(Z, Y).
```

Reading. Soufflé's orbits.dl benchmark computes the transitive closure of a seed relation, orbits (the orbits of the seed relation under composition), together with intermediate (a two-step composition of orbits with itself) and satellite (orbit pairs not reachable by such an intermediate two-step path) built on top of it — a graph-composition closure computation. The triple compares naive versus semi-naive evaluation of the whole program, claiming all three output relations agree between the two strategies.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in Input.lean remains the authority.

PRE: $\top$

True; no precondition constrains the inputs.

POST: $(\forall x y. (\text{intermediate}(x,y) \leftrightarrow \text{intermediateS}(x,y))) \wedge (\forall z u. (\text{orbits}(z,u) \leftrightarrow \text{orbitsS}(z,u))) \wedge \forall v w. (\text{satellite}(v,w) \leftrightarrow \text{satelliteS}(v,w))$

The naively computed intermediate, orbits, and satellite relations each equal their semi-naive counterparts.

Whiel encoding (Hoare triple).

```

SCHEMA
  orbitsSeed/2, intermediate/2, orbits/2, satellite/2, orbitsNew/2, orbitsOld/2, intermediateS/2, orbitsS/2, satelliteS/2,
  ⇔ deltaOrbitsS/2

PRE
  true

CMD
  orbitsNew := (π[0, 1] orbitsSeed);
  orbitsOld := ∅[2];
  WHILE (orbitsNew ≠ orbitsOld) DO
    orbitsOld := orbitsNew;
    orbitsNew := (orbitsNew ∪ (π[0, 3] (σ[#2 = #1] (orbitsOld × orbitsOld))))
  END;
  orbits := orbitsNew;
  intermediate := (π[0, 3] (σ[#2 = #1] (orbits × orbits)));
  satellite := (π[0, 1] (orbits \ (π[0, 1] (σ[(#2 = #0 ∧ #3 = #1)] (orbits × intermediate)))));
  orbitsS := ∅[2];
  deltaOrbitsS := (π[0, 1] orbitsSeed);
  WHILE (deltaOrbitsS ≠ ∅[2]) DO
    orbitsS := (orbitsS ∪ deltaOrbitsS);
    deltaOrbitsS := (((π[0, 3] (σ[#2 = #1] (deltaOrbitsS × orbitsS))) ∪ (π[0, 3] (σ[#2 = #1] (orbitsS × deltaOrbitsS)))) ∪ (π[0,
    ⇔ 3] (σ[#2 = #1] (deltaOrbitsS × deltaOrbitsS)))) \ orbitsS)
  END;
  intermediateS := (π[0, 3] (σ[#2 = #1] (orbitsS × orbitsS)));
  satelliteS := (π[0, 1] (orbitsS \ (π[0, 1] (σ[(#2 = #0 ∧ #3 = #1)] (orbitsS × intermediateS)))));

POST
  ((intermediate = intermediateS) ∧ (orbits = orbitsS) ∧ (satellite = satelliteS))

```

62 Example5011: Soufflé topological_ordering benchmark: naive versus semi-naive evaluation

Category	Soufflé benchmark program (naive versus semi-naive translation); family: Soufflé naive vs semi-naive.
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified invalid (Certificate/Invalid.lean, written by the certificate emitter); expected invalid.
Tags	database-engine evaluation-strategies negation nonlinear-recursion semi-naive.
Input	Benchmark/Example5011/Input.lean (canonical notation).
Summary	Soufflé's 'topological_ordering' benchmark program evaluated naively and semi-naively, tested for equality of every output.
Sides	naive translation of topological_ordering.dl (Soufflé benchmark suite) semi-naive translation of topological_ordering.dl (Soufflé benchmark suite)
Sources	souffle-lang/benchmarks (the Soufflé benchmark programs) Soufflé translation to C++ / semi-naive loop (documentation) program source file topological_ordering.dl of the Soufflé benchmark suite (naive translation)

Datalog program(s). *normalized Soufflé source (topological_ordering.dl of the Soufflé benchmark suite (naive translation))* — Soufflé benchmark program, normalized (both sides are script translations of it).

```

.decl edge(v0:symbol, v1:symbol) input
.decl RankZero(v0:symbol) input
.decl RankOne(v0:symbol) input
.decl Succ(v0:symbol, v1:symbol) input
.decl vertex(v0:symbol) output
.decl is_before(v0:symbol, v1:symbol) output
.decl is_after(v0:symbol, v1:symbol) output
.decl isolated(v0:symbol) output

```

```

.decl source(v0:symbol) output
.decl indices(v0:symbol, v1:symbol) output
.decl has_successor_rank(v0:symbol, v1:symbol) output
.decl index(v0:symbol, v1:symbol) output

vertex(v) :- edge(v, __anon_1).
vertex(v) :- edge(__anon_2, v).
is_before(x, y) :- edge(x, y).
is_before(x, y) :- is_before(x, z), is_before(z, y).
is_after(x, y) :- edge(y, x).
is_after(x, y) :- is_after(z, x), is_after(y, z).
isolated(x) :- vertex(x), !edge(__anon_3, x), !edge(x, __anon_4).
source(x) :- vertex(x), !edge(__anon_5, x), edge(x, __anon_6).
indices(x, i) :- isolated(x), RankZero(i).
indices(x, i) :- source(x), RankOne(i).
indices(x, i1) :- is_after(x, y), indices(y, i), Succ(i, i1).
has_successor_rank(x, i) :- indices(x, i), Succ(i, i1), indices(x, i1).
index(x, i) :- indices(x, i), !has_successor_rank(x, i).

```

Reading. Soufflé's `topological_ordering.dl` benchmark computes a topological layering of a directed graph. `is_before` and `is_after` are the transitive closure of the edge relation and of its reverse; isolated vertices receive rank zero and source vertices rank one, and a vertex that comes after a vertex of rank `i` also receives the next rank (ranks are symbols ordered by the input relation `Succ`). `index` keeps, for each vertex, the rank that has no successor rank assigned to the same vertex, that is, its maximal rank: on an acyclic graph, one more than the length of the longest path reaching it from a source. The triple compares naive and semi-naive evaluation of this whole stratified program and claims that all eight output relations agree.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in `Input.lean`. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\top$

True; no precondition constrains the inputs.

POST: $(\forall x y. (\text{hasSuccessorRank}(x,y) \leftrightarrow \text{hasSuccessorRankS}(x,y))) \wedge (\forall z u. (\text{index}(z,u) \leftrightarrow \text{indexS}(z,u))) \wedge (\forall v w. (\text{indices}(v,w) \leftrightarrow \text{indicesS}(v,w))) \wedge (\forall x_1 x_2. (\text{isAfter}(x_1,x_2) \leftrightarrow \text{isAfterS}(x_1,x_2))) \wedge (\forall x_3 x_4. (\text{isBefore}(x_3,x_4) \leftrightarrow \text{isBeforeS}(x_3,x_4))) \wedge (\forall x_5. (\text{isolated}(x_5) \leftrightarrow \text{isolatedS}(x_5))) \wedge (\forall x_6. (\text{source}(x_6) \leftrightarrow \text{sourceS}(x_6))) \wedge \forall x_7. (\text{vertex}(x_7) \leftrightarrow \text{vertexS}(x_7))$

Each of the eight output relations (`hasSuccessorRank`, `index`, `indices`, `isAfter`, `isBefore`, `isolated`, `source`, `vertex`) computed naively equals its semi-naive counterpart.

Whiel encoding (Hoare triple).

```

SCHEMA
RankOne/1, RankZero/1, isolated/1, source/1, vertex/1, isolatedS/1, sourceS/1, vertexS/1, Succ/2, edge/2, hasSuccessorRank/2, index/2, indices/2, isAfter/2, isBefore/2, indicesNew/2,
↪ indicesOld/2, isAfterNew/2, isAfterOld/2, isBeforeNew/2, isBeforeOld/2, hasSuccessorRankS/2, indexS/2, indicesS/2, isAfterS/2, isBeforeS/2, deltaIndicesS/2, deltaIndicesOldS/2,
↪ deltaIsAfterS/2, deltaIsAfterOldS/2, deltaIsBeforeS/2, deltaIsBeforeOldS/2

PRE
true

CMD
vertex := ((π[0] edge) ∪ (π[1] edge));
isolated := (π[0] ((vertex \ (π[0] (σ[#2 = #0] (vertex × edge)))) \ (π[0] (σ[#1 = #0] ((vertex \ (π[0] (σ[#2 = #0] (vertex × edge)))) × edge)))));
source := (π[0] ((σ[#1 = #0] (vertex × edge)) \ (π[0, 1, 2] (σ[#4 = #0] ((σ[#1 = #0] (vertex × edge)) × edge)))));
indicesNew := ((π[0, 1] (isolated × RankZero)) ∪ (π[0, 1] (source × RankOne)));
indicesOld := ∅[2];
isAfterNew := (π[1, 0] edge);
isAfterOld := ∅[2];
isBeforeNew := (π[0, 1] edge);
isBeforeOld := ∅[2];
WHILE ((indicesNew ≠ indicesOld) ∨ (isAfterNew ≠ isAfterOld) ∨ (isBeforeNew ≠ isBeforeOld)) DO
indicesOld := indicesNew;
isAfterOld := isAfterNew;
isBeforeOld := isBeforeNew;
indicesNew := (indicesNew ∪ (π[0, 5] (σ[#2 = #1 ∧ #4 = #3] ((isAfterOld × indicesOld) × Succ)))));
isAfterNew := (isAfterNew ∪ (π[1, 2] (σ[#3 = #0] (isAfterOld × isAfterOld)))));
isBeforeNew := (isBeforeNew ∪ (π[0, 3] (σ[#2 = #1] (isBeforeOld × isBeforeOld)))));
END;
indices := indicesNew;
isAfter := isAfterNew;
isBefore := isBeforeNew;
hasSuccessorRank := (π[0, 1] (σ[(#2 = #1 ∧ #4 = #0) ∧ #5 = #3] ((indices × Succ) × indices)));
index := (π[0, 1] (indices \ (π[0, 1] (σ[(#2 = #0 ∧ #3 = #1] (indices × hasSuccessorRank)))));
vertexS := (π[0] edge) ∪ (π[1] edge);
isolatedS := (π[0] ((vertexS \ (π[0] (σ[#2 = #0] (vertexS × edge)))) \ (π[0] (σ[#1 = #0] ((vertexS \ (π[0] (σ[#2 = #0] (vertexS × edge)))) × edge)))));
sourceS := (π[0] ((σ[#1 = #0] (vertexS × edge)) \ (π[0, 1, 2] (σ[#4 = #0] ((σ[#1 = #0] (vertexS × edge)) × edge)))));
indicesS := ∅[2];
deltaIndicesS := ((π[0, 1] (isolatedS × RankZero)) ∪ (π[0, 1] (sourceS × RankOne)));
deltaIndicesOldS := ∅[2];
isAfterS := ∅[2];

```

```

deltaIsAfterS := (π[1, 0] edge);
deltaIsAfterOldS := 0[2];
isBeforeS := 0[2];
deltaIsBeforeS := (π[0, 1] edge);
deltaIsBeforeOldS := 0[2];
WHILE (((deltaIndicesS ≠ 0[2]) ∨ (deltaIsAfterS ≠ 0[2])) ∨ (deltaIsBeforeS ≠ 0[2])) DO
  deltaIndicesOldS := deltaIndicesS;
  deltaIsAfterOldS := deltaIsAfterS;
  deltaIsBeforeOldS := deltaIsBeforeS;
  indicesS := (indicesS ∪ deltaIndicesOldS);
  isAfterS := (isAfterS ∪ deltaIsAfterOldS);
  isBeforeS := (isBeforeS ∪ deltaIsBeforeOldS);
  deltaIndicesS := ((π[0, 5] (σ[(#2 = #1 ∧ #4 = #3]) ((isAfterS × deltaIndicesOldS × Succ))) \ indicesS);
  deltaIsAfterS := (((π[1, 2] (σ[#3 = #0] (deltaIsAfterOldS × isAfterS))) ∪ (π[1, 2] (σ[#3 = #0] (isAfterS × deltaIsAfterOldS)))) ∪ (π[1, 2] (σ[#3 = #0] (deltaIsAfterOldS ×
  ↪ deltaIsAfterOldS)))) \ isAfterS;
  deltaIsBeforeS := (((π[0, 3] (σ[#2 = #1] (deltaIsBeforeOldS × isBeforeS))) ∪ (π[0, 3] (σ[#2 = #1] (isBeforeS × deltaIsBeforeOldS))) ∪ (π[0, 3] (σ[#2 = #1] (deltaIsBeforeOldS
  ↪ × deltaIsBeforeOldS)))) \ isBeforeS);
END;
hasSuccessorRankS := (π[0, 1] (σ[(#2 = #1 ∧ #4 = #0) ∧ #5 = #3]) ((indicesS × Succ × indicesS)));
indexS := (π[0, 1] (indicesS \ (π[0, 1] (σ[(#2 = #0 ∧ #3 = #1]) (indicesS × hasSuccessorRankS))));
POST
((hasSuccessorRank = hasSuccessorRankS) ∧ (index = indexS) ∧ (indices = indicesS) ∧ (isAfter = isAfterS) ∧ (isBefore = isBeforeS) ∧ (isolated = isolatedS) ∧ (source = sourceS) ∧
  ↪ (vertex = vertexS))

```

63 Example5012: Casbin depth-bounded role resolution contained in the unbounded role closure

Category	production pair (two algorithms from production sources or legacy production cases); family: production discrepancy pair (Casbin depth bound).
Kind	containment.
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	<code>access-control</code> <code>prophecy recursion-limit</code> .
Input	Benchmark/Example5012/Input.lean (canonical notation).
Summary	Casbin's depth-bounded role resolution against the unbounded role-hierarchy closure: the containment direction.
Sides	Casbin HasLink with <code>maxHierarchyLevel = 10</code> , encoded as <code>StepA = UserRole</code> and ten distinct step relations <code>StepB..StepK</code> (<code>role_manager.go</code> , <code>enforcer.go</code>) naive compiler on the recursive has-role program (Example2035 / 5004)
Sources	<code>casbin/casbin rbac/default-role-manager/role_manager.go</code> (<code>HasLink</code> , <code>hasLinkHelper</code>) <code>casbin/casbin enforcer.go</code> (<code>maxHierarchyLevel = 10</code>)
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).

Datalog program(s). *prog_p* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```

Has(x1, y1) :- UserRole(x1, y1).
Has(x1, z1) :- Has(x1, y1), RoleInh(y1, z1).

```

Reading. Side 1 encodes Casbin's default role manager, which resolves role inheritance only up to a fixed depth bound (`maxHierarchyLevel = 10`) by unrolling ten loop-free steps of composition with `RoleInh` into `HasB`; side 2 is the naive-compiler translation of the unbounded recursive has-role Datalog program (`Has` derived from `UserRole` directly, and from `Has` composed with `RoleInh`) evaluated to its least fixpoint. The Hoare triple states that Casbin's depth-bounded role resolution is contained in the true unbounded transitive closure of role inheritance: every role grant the bounded lookup produces is also justified by the full closure.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in Input.lean remains the authority.

PRE: $\top$

True; no precondition constrains the inputs.

POST: $\forall x y. (\text{HasB}(x,y) \rightarrow \text{Has}(x,y))$

The depth-bounded role relation HasB is a subset of the unbounded role-closure relation Has.

Whiel encoding (Hoare triple).

```

SCHEMA
  UserRole/2, RoleInh/2, StepA/2, StepB/2, StepC/2, StepD/2, StepE/2, StepF/2, StepG/2, StepH/2, StepI/2, StepJ/2, StepK/2, HasB/2,
  ↪ Has/2, Has_aux/2

PRE
  true

CMD
  StepA := UserRole;
  StepB := (StepA ∪ (π[0, 3] (σ[#1 = #2] (StepA × RoleInh))));
  StepC := (StepB ∪ (π[0, 3] (σ[#1 = #2] (StepB × RoleInh))));
  StepD := (StepC ∪ (π[0, 3] (σ[#1 = #2] (StepC × RoleInh))));
  StepE := (StepD ∪ (π[0, 3] (σ[#1 = #2] (StepD × RoleInh))));
  StepF := (StepE ∪ (π[0, 3] (σ[#1 = #2] (StepE × RoleInh))));
  StepG := (StepF ∪ (π[0, 3] (σ[#1 = #2] (StepF × RoleInh))));
  StepH := (StepG ∪ (π[0, 3] (σ[#1 = #2] (StepG × RoleInh))));
  StepI := (StepH ∪ (π[0, 3] (σ[#1 = #2] (StepH × RoleInh))));
  StepJ := (StepI ∪ (π[0, 3] (σ[#1 = #2] (StepI × RoleInh))));
  StepK := (StepJ ∪ (π[0, 3] (σ[#1 = #2] (StepJ × RoleInh))));
  HasB := StepK;
  Has_aux := ∅[2];
  Has := (UserRole ∪ π[0,3] (σ[#1 = #2] ((Has_aux × RoleInh))));
  WHILE (¬(Has = Has_aux)) DO
    Has_aux := Has;
    Has := (Has ∪ (UserRole ∪ π[0,3] (σ[#1 = #2] ((Has_aux × RoleInh)))))
  END

POST
  (HasB ⊆ Has)

```

64 Example5013: Casbin depth-bounded role resolution versus the unbounded role closure

Category	production pair (two algorithms from production sources or legacy production cases); family: production discrepancy pair (Casbin depth bound).
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified invalid (certificate checked outside the library build) (Certificate/Invalid.lean, written by the certificate emitter); expected invalid. Placement note: peak resident memory 6935552 KB while checking <code>Certificate/Invalid.lean</code> exceeds the library build's 4194304 KB limit (peak 6773 MB)
Tags	<code>access-control recursion-limit</code> .
Input	<code>Benchmark/Example5013/Input.lean</code> (canonical notation).
Summary	Casbin's depth-bounded role resolution against the unbounded role-hierarchy closure: the equality, which fails.
Sides	Casbin HasLink with <code>maxHierarchyLevel = 10</code> , encoded as <code>StepA = UserRole</code> and ten distinct step relations <code>StepB..StepK</code> (<code>role_manager.go</code> , <code>enforcer.go</code>) naive compiler on the recursive has-role program (Example2035 / 5004)
Sources	<code>casbin/casbin rbac/default-role-manager/role_manager.go</code> (HasLink, hasLinkHelper) <code>casbin/casbin enforcer.go</code> (<code>maxHierarchyLevel = 10</code>)

Datalog program(s). *prog_p* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
Has(x1, y1) :- UserRole(x1, y1).
Has(x1, z1) :- Has(x1, y1), RoleInh(y1, z1).
```

Reading. Same two sides as Example5012 (Casbin's depth-10-bounded role resolution HasB versus the unbounded recursive-closure relation Has), but here the postcondition asserts full equality rather than containment. The expected verdict is invalid: because Casbin stops resolving inheritance after ten steps while the true closure has no such bound, an eleven-edge inheritance chain lets the unbounded closure grant a role that the bounded resolution denies, so HasB is a strict subset of Has, not equal to it.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\top$

True; no precondition constrains the inputs.

POST: $\forall x y. (\text{HasB}(x,y) \leftrightarrow \text{Has}(x,y))$

The claim (which fails) is that the depth-bounded role relation HasB equals the unbounded role-closure relation Has.

Whiel encoding (Hoare triple).

```
SCHEMA
  UserRole/2, RoleInh/2, StepA/2, StepB/2, StepC/2, StepD/2, StepE/2, StepF/2, StepG/2, StepH/2, StepI/2, StepJ/2, StepK/2, HasB/2,
   $\hookrightarrow$  Has/2, Has_aux/2

PRE
  true
```

```

CMD
StepA := UserRole;
StepB := (StepA ∪ (π[0, 3] (σ[#1 = #2] (StepA × RoleInh))));
StepC := (StepB ∪ (π[0, 3] (σ[#1 = #2] (StepB × RoleInh))));
StepD := (StepC ∪ (π[0, 3] (σ[#1 = #2] (StepC × RoleInh))));
StepE := (StepD ∪ (π[0, 3] (σ[#1 = #2] (StepD × RoleInh))));
StepF := (StepE ∪ (π[0, 3] (σ[#1 = #2] (StepE × RoleInh))));
StepG := (StepF ∪ (π[0, 3] (σ[#1 = #2] (StepF × RoleInh))));
StepH := (StepG ∪ (π[0, 3] (σ[#1 = #2] (StepG × RoleInh))));
StepI := (StepH ∪ (π[0, 3] (σ[#1 = #2] (StepH × RoleInh))));
StepJ := (StepI ∪ (π[0, 3] (σ[#1 = #2] (StepI × RoleInh))));
StepK := (StepJ ∪ (π[0, 3] (σ[#1 = #2] (StepJ × RoleInh))));
HasB := StepK;
Has_aux := ∅[2];
Has := (UserRole ∪ π[0,3] (σ[#1 = #2] ((Has_aux × RoleInh))));
WHILE (¬((Has = Has_aux))) DO
  Has_aux := Has;
  Has := (Has ∪ (UserRole ∪ π[0,3] (σ[#1 = #2] ((Has_aux × RoleInh))));
END

POST
(HasB = Has)

```

65 Example5014: W3C SPARQL 1.1 ALP evaluation of p^* and $p+$ versus the Datalog closure with reflexive pairs over nodes(G)

Category	specification (a standard or a paper's procedure versus a declarative program); family: specification versus declarative program.
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	evaluation-strategies semi-naive.
Input	Benchmark/Example5014/Input.lean (canonical notation).
Summary	SPARQL 1.1 property paths: the W3C evaluation procedure for ' $?x \ p^* \ ?y$ ' and ' $?x \ p+ \ ?y$ ' (the ALP function with a visited set) against the Datalog transitive closure, with the reflexive pairs that ' p^* ' adds over every term of the graph.
Sides	hand-written: W3C ALP procedure set-at-a-time, visited set indexed by start, seeded with the reflexive pairs over nodes(G); OneOrMorePath = $P \circ \text{Alp}$ stated in the postcondition naive compiler: $Tc = \text{closure of } P$; $\text{Star} = Tc \text{ plus reflexive pairs over nodes(G)}$
Sources	W3C SPARQL 1.1 Query Language, Recommendation 21 March 2013 (sections 9.1 (ZeroOrMorePath, OneOrMorePath), 9.4 (Arbitrary Length Path Matching, set semantics), 18.4 (Definition: Node set of a graph; Definition: Function ALP; Evaluation of ZeroOrMorePath; Evaluation of OneOrMorePath); read 2026-09-15)
Twin	Example5015 (invalid: reflexive pairs over the endpoints of P only).
Obstruction note	from the metadata: anchored (independent loops merged side by side, lockstep frontiers: $\text{Front} = \text{Star} \setminus \text{Star_aux}$; the $p+$ conjunct needs the closure inclusions $P \subseteq Tc$, $P \circ Tc \subseteq Tc$, $Tc \subseteq P \cup P \circ Tc$)

Datalog program(s). *prog_p* — ' p^* ' over nodes(G) and the closure ' $p+$ ', as Datalog. — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```

Tc(x1, y1) :- P(x1, y1).
Tc(x1, z1) :- Tc(x1, y1), P(y1, z1).
Star(x1, x1) :- G(x1, y1).
Star(y1, y1) :- G(x1, y1).
Star(x1, y1) :- Tc(x1, y1).

```

Reading. Side 1 implements the W3C SPARQL 1.1 ALP (arbitrary-length-path) evaluation procedure for property paths $?x \text{ p}^* ?y$ and $?x \text{ p}^+ ?y$, run set-at-a-time from every node of the graph via a frontier/visited-set loop; side 2 is the naive-compiler translation of the corresponding Datalog transitive-closure program (Tc, the closure of the predicate-specific relation P) together with a reflexive relation Star seeded over every node of the whole graph G. The triple states the specification's ALP procedure computes exactly the same result as this declarative closure formulation: the visited set Alp equals Star, and one P-step composed with Alp equals the closure Tc, the equivalence that licenses implementing p^*/p^+ by a recursive query instead of explicit graph traversal.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\forall x y. (P(x,y) \rightarrow G(x,y))$

The predicate-specific relation P (triples with the path's predicate) is a subset of the full graph relation G.

POST: $(\forall x y. (\text{Alp}(x,y) \leftrightarrow \text{Star}(x,y))) \wedge \forall z u. ((\exists v. P(z,v) \wedge \text{Alp}(v,u)) \leftrightarrow \text{Tc}(z,u))$

The ALP visited-set relation Alp equals Star (the closure of P plus reflexive pairs over every node of G), and composing one P-step with Alp equals the transitive-closure relation Tc.

Whiel encoding (Hoare triple).

```

SCHEMA
  G/2, P/2, Alp/2, Front/2, Next/2, Tc/2, Tc_aux/2, Star/2, Star_aux/2

PRE
  (P ⊆ G)

CMD
  Alp := (π[0, 0] G ∪ π[1, 1] G);
  Front := Alp;
  WHILE (Front ≠ ∅) DO
    Next := (π[0, 3] (σ[#1 = #2] (Front × P)) \ Alp);
    Alp := (Alp ∪ Next);
    Front := Next
  END;
  Tc_aux := ∅[2];
  Star_aux := ∅[2];
  Tc := (P ∪ π[0,3] (σ[#1 = #2] ((Tc_aux × P)))));
  Star := (π[0,0] (G) ∪ (π[1,1] (G) ∪ Tc_aux));
  WHILE (¬(((Tc = Tc_aux) ∧ (Star = Star_aux)))) DO
    Tc_aux := Tc;
    Star_aux := Star;
    Tc := (Tc ∪ (P ∪ π[0,3] (σ[#1 = #2] ((Tc_aux × P)))));
    Star := (Star ∪ (π[0,0] (G) ∪ (π[1,1] (G) ∪ Tc_aux)))
  END

POST
  ((Alp = Star) ∧ (π[0, 3] (σ[#1 = #2] (P × Alp)) = Tc))

```

66 Example5015: W3C SPARQL 1.1 ALP evaluation of p^* versus closure plus identity over the p endpoints (invalid twin of Example5014)

Category	specification (a standard or a paper's procedure versus a declarative program); family: specification versus declarative program (invalid twin).
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified invalid (Certificate/Invalid.lean, written by the certificate emitter); expected invalid.
Tags	evaluation-strategies semi-naive.
Input	Benchmark/Example5015/Input.lean (canonical notation).
Summary	SPARQL 1.1 property paths, the invalid twin of Example5014: the W3C procedure for ' $?x p^* ?y$ ' against the closure of the ' p ' triples plus the identity on their own endpoints, the common misreading of the zero-length case.
Sides	hand-written: W3C ALP procedure as in Example5014 naive compiler: $Tc = \text{closure of } P$; $Star = Tc \text{ plus reflexive pairs over the endpoints of } P$ (the misreading)
Sources	W3C SPARQL 1.1 Query Language, Recommendation 21 March 2013 (sections 9.1 (ZeroOrMorePath, OneOrMorePath), 9.4 (Arbitrary Length Path Matching, set semantics), 18.4 (Definition: Node set of a graph; Definition: Function ALP; Evaluation of ZeroOrMorePath; Evaluation of OneOrMorePath); read 2026-09-15)
Twin	Example5014 (valid).
Obstruction note	from the metadata: n/a (invalid; kernel counterexample)

Datalog program(s). *prog_p* — The closure ' p^+ ' and "closure plus identity" over the endpoints of ' p ' (the misreading of ' p^* '). — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
Tc(x1, y1) :- P(x1, y1).
Tc(x1, z1) :- Tc(x1, y1), P(y1, z1).
Star(x1, x1) :- P(x1, y1).
Star(y1, y1) :- P(x1, y1).
Star(x1, y1) :- Tc(x1, y1).
```

Reading. Same ALP procedure (side 1) as Example5014, but side 2's reflexive pairs are seeded only over the endpoints of P (the p -labeled triples) rather than over every node of the whole graph G — what one gets by naively translating p^* as closure-plus-identity restricted to the p -subgraph. The expected verdict is invalid: the W3C specification's node set ranges over all of G , so a node occurring only in triples with other predicates still satisfies $?x p^* ?x$ under the specification but is missing from this narrower $Star$ relation, breaking the claimed equality.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\forall x y. (P(x,y) \rightarrow G(x,y))$

The predicate-specific relation P is a subset of the full graph relation G .

POST: $(\forall x y. (Alp(x,y) \leftrightarrow Star(x,y))) \wedge \forall z u. ((\exists v. P(z,v) \wedge Alp(v,u)) \leftrightarrow Tc(z,u))$

The claim (which fails) is that the ALP visited-set relation Alp equals $Star$, here seeded reflexively only over the endpoints of P , and that one P -step composed with Alp equals Tc .

Whiel encoding (Hoare triple).

```

SCHEMA
  G/2, P/2, Alp/2, Front/2, Next/2, Tc/2, Tc_aux/2, Star/2, Star_aux/2

PRE
  (P ⊆ G)

CMD
  Alp := (π[0, 0] G ∪ π[1, 1] G);
  Front := Alp;
  WHILE (Front ≠ ∅) DO
    Next := (π[0, 3] (σ[#1 = #2] (Front × P)) \ Alp);
    Alp := (Alp ∪ Next);
    Front := Next
  END;
  Tc_aux := ∅[2];
  Star_aux := ∅[2];
  Tc := (P ∪ π[0,3] (σ[#1 = #2] ((Tc_aux × P)))));
  Star := (π[0,0] (P) ∪ (π[1,1] (P) ∪ Tc_aux));
  WHILE (¬((Tc = Tc_aux) ∧ (Star = Star_aux))) DO
    Tc_aux := Tc;
    Star_aux := Star;
    Tc := (Tc ∪ (P ∪ π[0,3] (σ[#1 = #2] ((Tc_aux × P)))));
    Star := (Star ∪ (π[0,0] (P) ∪ (π[1,1] (P) ∪ Tc_aux)))
  END

POST
  ((Alp = Star) ∧ (π[0, 3] (σ[#1 = #2] (P × Alp)) = Tc))

```

67 Example5016: DRed deletion maintenance of a materialised transitive closure versus recomputation

Category	specification (a standard or a paper's procedure versus a declarative program); family: maintenance algorithm versus recomputation.
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified valid (certificate checked outside the library build) (Certificate/Valid.lean, written by the certificate emitter); expected valid. Placement note: a proof module imported by Certificate/Valid.lean does not build under the library build's 4194304 KB limit; it builds under 12582912 KB (peak 1673 MB)
Tags	<code>evaluation-strategies negation prophecy semi-naive view-maintenance</code> .
Input	<code>Benchmark/Example5016/Input.lean</code> (canonical notation).
Summary	DRed (Delete/Rederive) maintenance of a materialised transitive closure under deletions against recomputation of the closure from the updated edges.
Sides	naive compiler: $T = \text{closure of } E$ (old materialisation) hand-written DRed after Motik et al. DR1-DR5 / GMS section 7 steps 1-2 for the closure program, deletion-only update naive compiler: $T_n = \text{closure of } EN = E \setminus \text{Del}$
Sources	A. Gupta, I. S. Mumick, V. S. Subrahmanian, Maintaining Views Incrementally, SIGMOD 1993 (sections section 7 (Incremental Maintenance of Recursive Views: steps 1-3, Theorem 7.1); read 2026-09-15) B. Motik, Y. Nenov, R. Piro, I. Horrocks, Incremental Update of Datalog Materialisation: the Backward/Forward Algorithm, AAAI 2015 (sections section 3 (The Delete/Rederive Algorithm, steps DR1-DR5, rules (5) and (6)); Algorithm 1 and Theorem 1 (B/F); read 2026-09-15)
Twin	Example5017 (invalid: overdeletion without rederivation).
Prophecy	accepted invariant with a prophecy clause above level zero (Core.json); no certificate built from it yet.

Datalog program(s). *prog_pa* — *The maintained program: the closure of the old edges.* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
T(x1, y1) :- E(x1, y1).
T(x1, z1) :- T(x1, y1), E(y1, z1).
```

prog_pc — *Recomputation: the closure of the updated edges.* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
Tn(x1, y1) :- EN(x1, y1).
Tn(x1, z1) :- Tn(x1, y1), EN(y1, z1).
```

rules as stated in the header comment of Input.lean — header comment.

```
T(x, y) :- E(x, y)
T(x, z) :- T(x, y), E(y, z)
T-(x, y) :- E-(x, y)
T-(x, z) :- T(x, y), E-(y, z)
T-(x, z) :- T-(x, y), E(y, z)
T+(x, y) :- T-(x, y), EN(x, y)
T+(x, z) :- T-(x, z), T(x, y), EN(y, z)
T+(x, z) :- T+(x, y), EN(y, z)
Tn(x, y) :- EN(x, y)
Tn(x, z) :- Tn(x, y), EN(y, z)
```

Reading. The maintained program is the transitive closure T of an edge relation E ; side 1 implements DRed (Delete/Rederive), the standard incremental-view-maintenance algorithm of Gupta, Mumick and Subrahmanian: it overdeletes closure facts that could depend on deleted edges, removes them, then rederives any of those facts that still have a surviving derivation on the updated edge set, propagating rederived facts back through the closure rule; side 2 recomputes the closure T_n from scratch on the updated edge set $EN = E$ minus Del . The triple states DRed's maintained closure equals the recomputed closure, the standard correctness result for incremental deletion maintenance of a recursive Datalog view.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\forall x y. (Del(x,y) \rightarrow E(x,y))$

The set of edges to delete, Del , is a subset of the edge relation E .

POST: $\forall x y. (T(x,y) \leftrightarrow T_n(x,y))$

The DRed-maintained closure relation T equals the freshly recomputed closure T_n on the edges remaining after deletion.

Whiel encoding (Hoare triple).

```
SCHEMA
  E/2, Del/2, EN/2, T/2, T_aux/2, Over/2, Cand/2, New/2, Tn/2, Tn_aux/2

PRE
  (Del  $\subseteq$  E)

CMD
  T_aux :=  $\emptyset[2]$ ;
  T := (E  $\cup \pi[0,3] (\sigma[\#1 = \#2] ((T\_aux \times E))))$ ;
  WHILE ( $\neg((T = T\_aux))$ ) DO
    T_aux := T;
    T := (T  $\cup (E \cup \pi[0,3] (\sigma[\#1 = \#2] ((T\_aux \times E))))$ )
  END;
  EN := (E  $\setminus$  Del);
  Over :=  $\emptyset[2]$ ;
  Cand := (Del  $\cup \pi[0, 3] (\sigma[\#1 = \#2] (T \times Del))$ );
  WHILE ((Cand  $\setminus$  Over)  $\neq \emptyset$ ) DO
    New := (Cand  $\setminus$  Over);
    Over := (Over  $\cup$  New);
    Cand :=  $\pi[0, 3] (\sigma[\#1 = \#2] (New \times E))$ 
  END;
  T := (T  $\setminus$  Over);
```

```

Cand := (Over \ (Over \ (EN ∪ π[0, 3] (σ[#1 = #2] (T × EN)))));
WHILE ((Cand \ T) ≠ ∅) DO
  New := (Cand \ T);
  T := (T ∪ New);
  Cand := π[0, 3] (σ[#1 = #2] (New × EN))
END;
Tn_aux := ∅[2];
Tn := (EN ∪ π[0,3] (σ[#1 = #2] ((Tn_aux × EN)))));
WHILE (¬((Tn = Tn_aux))) DO
  Tn_aux := Tn;
  Tn := (Tn ∪ (EN ∪ π[0,3] (σ[#1 = #2] ((Tn_aux × EN))))))
END
POST
(T = Tn)

```

68 Example5017: Overdeletion without rederivation versus recomputation (invalid twin of Example5016)

Category	specification (a standard or a paper's procedure versus a declarative program); family: maintenance algorithm versus recomputation (invalid twin).
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified invalid (Certificate/Invalid.lean, written by the certificate emitter); expected invalid.
Tags	evaluation-strategies negation semi-naive view-maintenance.
Input	Benchmark/Example5017/Input.lean (canonical notation).
Summary	Deletion by overdeletion alone (semi-naive propagation of the deleted edges through the closure, without DRed's rederivation phase) against recomputation: the invalid twin of Example5016.
Sides	naive compiler: $T = \text{closure of } E$ hand-written: DRed step 1 only (DR1-DR3: overdeletion, removal), no rederivation naive compiler: $Tn = \text{closure of } EN = E \setminus \text{Del}$
Sources	A. Gupta, I. S. Mumick, V. S. Subrahmanian, Maintaining Views Incrementally, SIGMOD 1993 (sections section 7 (Incremental Maintenance of Recursive Views: steps 1-3, Theorem 7.1); read 2026-09-15) B. Motik, Y. Nenov, R. Piro, I. Horrocks, Incremental Update of Datalog Materialisation: the Backward/Forward Algorithm, AAAI 2015 (sections section 3 (The Delete/Rederive Algorithm, steps DR1-DR5, rules (5) and (6)); Algorithm 1 and Theorem 1 (B/F); read 2026-09-15)
Twin	Example5016 (valid).
Obstruction note	from the metadata: n/a (invalid; kernel counterexample)

Datalog program(s). *prog_pa* — *The maintained program: the closure of the old edges.* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```

T(x1, y1) :- E(x1, y1).
T(x1, z1) :- T(x1, y1), E(y1, z1).

```

prog_pc — *Recomputation: the closure of the updated edges.* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```

Tn(x1, y1) :- EN(x1, y1).
Tn(x1, z1) :- Tn(x1, y1), EN(y1, z1).

```

rules as stated in the header comment of Input.lean — header comment.

```

T(x, y) :- E(x, y)
T(x, z) :- T(x, y), E(y, z)
T-(x, y) :- E-(x, y)
T-(x, z) :- T(x, y), E-(y, z)
T-(x, z) :- T-(x, y), E(y, z)
Tn(x, y) :- EN(x, y)
Tn(x, z) :- Tn(x, y), EN(y, z)

```

Reading. Same maintained closure and recomputation side as Example5016, but side 1 performs only the overdeletion phase of DRed (propagating deleted edges through the closure and removing every fact that could depend on them) without the rederivation phase that restores facts with alternative surviving derivations. The expected verdict is invalid: overdeletion is a genuine overestimate, so a closure fact reachable via a deleted edge but also via another surviving path is deleted and never restored, leaving the maintained closure T missing facts that the recomputed closure Tn still contains.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\forall x y. (\text{Del}(x,y) \rightarrow E(x,y))$

The set of edges to delete, Del, is a subset of the edge relation E.

POST: $\forall x y. (T(x,y) \leftrightarrow Tn(x,y))$

The claim (which fails) is that the overdeletion-only maintained closure T equals the recomputed closure Tn on the edges remaining after deletion.

Whiel encoding (Hoare triple).

```

SCHEMA
  E/2, Del/2, EN/2, T/2, T_aux/2, Over/2, Cand/2, New/2, Tn/2, Tn_aux/2

PRE
  (Del ⊆ E)

CMD
  T_aux := ∅[2];
  T := (E ∪ π[0,3] (σ[#1 = #2] ((T_aux × E))));
  WHILE (¬((T = T_aux))) DO
    T_aux := T;
    T := (T ∪ (E ∪ π[0,3] (σ[#1 = #2] ((T_aux × E)))))
  END;
  EN := (E \ Del);
  Over := ∅[2];
  Cand := (Del ∪ π[0,3] (σ[#1 = #2] (T × Del)));
  WHILE ((Cand \ Over) ≠ ∅) DO
    New := (Cand \ Over);
    Over := (Over ∪ New);
    Cand := π[0,3] (σ[#1 = #2] (New × E))
  END;
  T := (T \ Over);
  Tn_aux := ∅[2];
  Tn := (EN ∪ π[0,3] (σ[#1 = #2] ((Tn_aux × EN))));
  WHILE (¬((Tn = Tn_aux))) DO
    Tn_aux := Tn;
    Tn := (Tn ∪ (EN ∪ π[0,3] (σ[#1 = #2] ((Tn_aux × EN)))))
  END

POST
  (T = Tn)

```

69 Example5018: Chase order-independence on the full-TGD subset of ChaseBench correctness/tgds: dependency-at-a-time versus parallel chase

Category	specification (a standard or a paper’s procedure versus a declarative program); family: two evaluation orders of one chase.
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified valid (certificate checked outside the library build) (Certificate/Valid.lean, written by the certificate emitter); expected valid. Placement note: a proof module imported by Certificate/Valid.lean does not build under the library build’s 4194304 KB limit; it builds under 12582912 KB (peak 1692 MB)
Tags	<code>chase evaluation-strategies mutual-recursion</code> .
Input	Benchmark/Example5018/Input.lean (canonical notation).
Summary	Chase order-independence on a full-TGD scenario of ChaseBench: the parallel chase (every dependency applied to every active trigger in each round) against the dependency-at-a-time chase (one dependency chased to its own fixpoint, then the next, the sweep repeated until no dependency has an active trigger).
Sides	hand-written: s-t TGDs once, then target TGDs in file order each to its fixpoint (inner loops), sweep repeated (outer loop) naive compiler: the six full TGDs as a Datalog program (parallel chase)
Sources	ChaseBench (dbunibas/chasebench), scenario scenarios/correctness/tgds (commit 7427e1c1e3196769ee4ea04557465965ec26d02b (master, 2017-09-13); read 2026-09-15) M. Benedikt, G. Konstantinidis, G. Mecca, B. Motik, P. Papotti, D. Santoro, E. Tsamoura, Benchmarking the Chase, PODS 2017 (sections Section 2 (chase sequences, fairness), Section 3 (Algorithm 1, Chase variants), Section 8 (Restricted vs. unrestricted vs. parallel chase); read 2026-09-15) program source file ChaseBench scenarios/correctness/tgds (full TGDs; the existential t2 -> t3 dropped, t3 an input)
Twin	Example5019 (invalid: one pass in file order).
Obstruction note	from the metadata: rate mismatch (parallel round versus one dependency per flattened iteration; nested loops flattened under five flags beside the compiled loop)

Datalog program(s). *prog_p* — The full TGDs of ChaseBench scenarios/correctness/tgds as a Datalog program ($S = s$, $TA = t1$, $TB = t2$, $TC = t3$, $WA = w1$, $WB = w2$). — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```
TA(x1, y1, z1) :- S(x1, y1, z1).
WA(x1, y1) :- S(x1, y1, z1).
TB(x1, y1) :- TA(x1, y1, z1).
TB(y1, y1) :- TC(x1, y1, z1).
WB(x1, y1) :- WA(x1, y1).
WA(y1, y1) :- WB(x1, y1).
```

Reading. The program is the full-TGD subset of ChaseBench’s tgds correctness scenario, a small chain of six dependencies (renamed S/TA/TB/TC/WA/WB) relating source and target relations in a data-exchange setting; side 1 hand-encodes the dependency-at-a-time chase (each dependency chased to its own fixpoint in file order, the sweep over all dependencies repeated until nothing new is derived), while side 2 is the naive-compiler translation applying all dependencies in parallel each round. The triple states the two chase orders converge to the same universal solution, illustrating the chase’s order-independence for full (non-existential) dependencies.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in Input.lean remains the authority.

PRE: $\top$

True; no precondition constrains the inputs.

POST: $(\forall x y z. (TAQ(x,y,z) \leftrightarrow TA(x,y,z))) \wedge (\forall u v. (TBQ(u,v) \leftrightarrow TB(u,v))) \wedge (\forall w x1. (WAQ(w,x1) \leftrightarrow WA(w,x1))) \wedge \forall x2 x3. (WBQ(x2,x3) \leftrightarrow WB(x2,x3))$

Each target relation computed by the dependency-at-a-time chase (TAQ, TBQ, WAQ, WBQ) equals the corresponding relation computed by the parallel chase (TA, TB, WA, WB).

Whiel encoding (Hoare triple).

```

SCHEMA
  S/3, TA/3, TC/3, TA_aux/3, TAQ/3, TB/2, WA/2, WB/2, TB_aux/2, WA_aux/2, WB_aux/2, TBQ/2, WAQ/2, WBQ/2

PRE
  true

CMD
  TAQ := S;
  WAQ :=  $\pi[0, 1] S$ ;
  TBQ :=  $\emptyset[2]$ ;
  WBQ :=  $\emptyset[2]$ ;
  WHILE ( $\neg((\pi[0, 1] TAQ) \subseteq TBQ) \wedge ((\pi[1, 1] TC) \subseteq TBQ) \wedge ((WAQ \subseteq WBQ) \wedge ((\pi[1, 1] WBQ) \subseteq WAQ)))$ ) DO
    WHILE ( $\neg((\pi[0, 1] TAQ) \subseteq TBQ)$ ) DO
      TBQ :=  $(TBQ \cup \pi[0, 1] TAQ)$ 
    END;
    WHILE ( $\neg((\pi[1, 1] TC) \subseteq TBQ)$ ) DO
      TBQ :=  $(TBQ \cup \pi[1, 1] TC)$ 
    END;
    WHILE ( $\neg(WAQ \subseteq WBQ)$ ) DO
      WBQ :=  $(WBQ \cup WAQ)$ 
    END;
    WHILE ( $\neg((\pi[1, 1] WBQ) \subseteq WAQ)$ ) DO
      WAQ :=  $(WAQ \cup \pi[1, 1] WBQ)$ 
    END;
  END;
  TA_aux :=  $\emptyset[3]$ ;
  WA_aux :=  $\emptyset[2]$ ;
  TB_aux :=  $\emptyset[2]$ ;
  WB_aux :=  $\emptyset[2]$ ;
  TA := S;
  WA :=  $(\pi[0,1] (S) \cup \pi[1,1] (WB\_aux))$ ;
  TB :=  $(\pi[0,1] (TA\_aux) \cup \pi[1,1] (TC))$ ;
  WB := WA_aux;
  WHILE ( $\neg(((TA = TA\_aux) \wedge (WA = WA\_aux) \wedge (TB = TB\_aux) \wedge (WB = WB\_aux))))$ ) DO
    TA_aux := TA;
    WA_aux := WA;
    TB_aux := TB;
    WB_aux := WB;
    TA :=  $(TA \cup S)$ ;
    WA :=  $(WA \cup (\pi[0,1] (S) \cup \pi[1,1] (WB\_aux)))$ ;
    TB :=  $(TB \cup (\pi[0,1] (TA\_aux) \cup \pi[1,1] (TC)))$ ;
    WB :=  $(WB \cup WA\_aux)$ 
  END;

POST
   $((TAQ = TA) \wedge (TBQ = TB)) \wedge ((WAQ = WA) \wedge (WBQ = WB))$ 

```

70 Example5019: One pass over the dependencies versus the parallel chase on ChaseBench correctness/tgds (invalid twin of Example5018)

Category	specification (a standard or a paper’s procedure versus a declarative program); family: two evaluation orders of one chase (invalid twin).
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified invalid (Certificate/Invalid.lean, written by the certificate emitter); expected invalid.
Tags	chase evaluation-strategies mutual-recursion.
Input	Benchmark/Example5019/Input.lean (canonical notation).
Summary	One pass over the dependencies of a full-TGD ChaseBench scenario against the parallel chase: the invalid twin of Example5018.
Sides	hand-written: s-t TGDs once, then each target TGD to its fixpoint once in file order, no repetition (not a fair chase sequence) naive compiler: the six full TGDs as a Datalog program (parallel chase)
Sources	ChaseBench (dbunibas/chasebench), scenario scenarios/correctness/tgds (commit 7427e1c1e3196769ee4ea04557465965ec26d02b (master, 2017-09-13); read 2026-09-15) M. Benedikt, G. Konstantinidis, G. Mecca, B. Motik, P. Papotti, D. Santoro, E. Tsamoura, Benchmarking the Chase, PODS 2017 (sections Section 2 (chase sequences, fairness), Section 3 (Algorithm 1, Chase variants), Section 8 (Restricted vs. unrestricted vs. parallel chase); read 2026-09-15) program source file ChaseBench scenarios/correctness/tgds (full TGDs)
Twin	Example5018 (valid).
Obstruction note	from the metadata: n/a (invalid; kernel counterexample)

Datalog program(s). *prog_p* — The full TGDs of ChaseBench scenarios/correctness/tgds as a Datalog program ($S = s$, $TA = t1$, $TB = t2$, $TC = t3$, $WA = w1$, $WB = w2$). — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```
TA(x1, y1, z1) :- S(x1, y1, z1).
WA(x1, y1) :- S(x1, y1, z1).
TB(x1, y1) :- TA(x1, y1, z1).
TB(y1, y1) :- TC(x1, y1, z1).
WB(x1, y1) :- WA(x1, y1).
WA(y1, y1) :- WB(x1, y1).
```

Reading. Same scenario and parallel-chase side as Example5018, but side 1 applies the s-t dependencies once and then chases each target dependency to its own fixpoint exactly once in file order, without repeating the sweep over all dependencies. The expected verdict is invalid: that single pass is not a fair chase sequence, since a dependency whose trigger becomes newly active only after a later dependency fires is never re-applied, so the one-pass result falls short of the true least fixpoint the parallel chase computes.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in Input.lean remains the authority.

PRE: $\top$

True; no precondition constrains the inputs.

POST: $(\forall x y z. (TAQ(x,y,z) \leftrightarrow TA(x,y,z))) \wedge (\forall u v. (TBQ(u,v) \leftrightarrow TB(u,v))) \wedge (\forall w x1. (WAQ(w,x1) \leftrightarrow WA(w,x1))) \wedge \forall x2 x3. (WBQ(x2,x3) \leftrightarrow WB(x2,x3))$

The claim (which fails) is that the single-pass target relations (TAQ, TBQ, WAQ, WBQ) equal the parallel chase's fixpoint relations (TA, TB, WA, WB).

Whiel encoding (Hoare triple).

```
SCHEMA
  S/3, TA/3, TC/3, TA_aux/3, TAQ/3, TB/2, WA/2, WB/2, TB_aux/2, WA_aux/2, WB_aux/2, TBQ/2, WAQ/2, WBQ/2

PRE
  true

CMD
  TAQ := S;
  WAQ :=  $\pi[0, 1] S$ ;
  TBQ :=  $\emptyset[2]$ ;
  WBQ :=  $\emptyset[2]$ ;
  WHILE ( $\neg((\pi[0, 1] TAQ) \subseteq TBQ)$ ) DO
    TBQ :=  $(TBQ \cup \pi[0, 1] TAQ)$ 
  END;
  WHILE ( $\neg((\pi[1, 1] TC) \subseteq TBQ)$ ) DO
    TBQ :=  $(TBQ \cup \pi[1, 1] TC)$ 
  END;
  WHILE ( $\neg(WAQ \subseteq WBQ)$ ) DO
    WBQ :=  $(WBQ \cup WAQ)$ 
  END;
  WHILE ( $\neg((\pi[1, 1] WBQ) \subseteq WAQ)$ ) DO
    WAQ :=  $(WAQ \cup \pi[1, 1] WBQ)$ 
  END;
  TA_aux :=  $\emptyset[3]$ ;
  WA_aux :=  $\emptyset[2]$ ;
  TB_aux :=  $\emptyset[2]$ ;
  WB_aux :=  $\emptyset[2]$ ;
  TA := S;
  WA :=  $(\pi[0, 1] S) \cup \pi[1, 1] (WB\_aux)$ ;
  TB :=  $(\pi[0, 1] (TA\_aux) \cup \pi[1, 1] (TC))$ ;
  WB := WA_aux;
  WHILE ( $\neg(((TA = TA\_aux) \wedge ((WA = WA\_aux) \wedge ((TB = TB\_aux) \wedge (WB = WB\_aux))))))$  DO
    TA_aux := TA;
    WA_aux := WA;
    TB_aux := TB;
    WB_aux := WB;
    TA :=  $(TA \cup S)$ ;
    WA :=  $(WA \cup (\pi[0, 1] S) \cup \pi[1, 1] (WB\_aux))$ ;
    TB :=  $(TB \cup (\pi[0, 1] (TA\_aux) \cup \pi[1, 1] (TC)))$ ;
    WB :=  $(WB \cup WA\_aux)$ 
  END

POST
   $((TAQ = TA) \wedge (TBQ = TB)) \wedge ((WAQ = WA) \wedge (WBQ = WB))$ 
```

71 Example5020: RDFS closure by the rho-df rules versus the full W3C RDFS rule set: equal up to the reflexive subPropertyOf/subClassOf triples

Category	specification (a standard or a paper's procedure versus a declarative program); family: two rule sets for one semantics.
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified valid (certificate checked outside the library build) (Certificate/Valid.lean, written by the certificate emitter); expected valid. Placement note: a proof module imported by Certificate/Valid.lean does not build under the library build's 4194304 KB limit; it builds under 12582912 KB (peak 1753 MB)
Tags	<code>mutual-recursion</code> <code>nonlinear-recursion</code> .
Input	Benchmark/Example5020/Input.lean (canonical notation).
Summary	RDFS entailment computed by two rule sets: the minimal rho-df system of Munoz, Perez and Gutierrez against the full RDFS entailment patterns of the W3C, on the same input. The two closures agree on triples and on typing, and differ exactly by the reflexive subPropertyOf and subClassOf triples that RDFS derives for every property and class.
Sides	naive compiler: rho-df system $\vdash \text{mrdf}$, rules (2a),(2b),(3a),(3b),(4a),(4b) = rdfs5, rdfs7, rdfs11, rdfs9, rdfs2, rdfs3

naive compiler: W3C RDFS entailment patterns on mrdg-graphs (the six shared patterns, rdfs6, rdfs10, rdfsD2, and the typing forced by the RDFS axiomatic triples of the rho-df vocabulary, in IsProp/IsClass)

Sources S. Munoz, J. Perez, C. Gutierrez, Simple and Efficient Minimal RDFS, Journal of Web Semantics 7(3), 2009 (sections Definition 3 (rho-df vocabulary), Table 1 (rules (1)-(7)), Definition 15 (mrdg graphs, $\vdash$ mrdg), Proposition 17, Corollary 18; read 2026-09-15)

W3C RDF 1.1 Semantics, Recommendation 25 February 2014 (sections 8.1.1 (rdfsD2), 9 (RDFS axiomatic triples), 9.2.1 (rdfs1-rdfs13); read 2026-09-15)

Apache Jena, jena-core/src/main/resources/etc/rdfs.rules (commit f9eed353614c178b80b1e099ea2598e648014074 (main, 2026-09-14); read 2026-09-15)

Twin Example5021 (invalid: reflexive subClassOf triples forgotten).

Obstruction note

from the metadata: anchored (independent compiled loops merged side by side, one round of the same patterns per iteration; reflexive pairs lag one round: $\text{SubPropF} = \text{SubPropM} \cup \pi[0,0]$ IsProp_aux , $\text{SubClassF} = \text{SubClassM} \cup \pi[0,0]$ IsClass_aux)

Datalog program(s). *prog_pm* — rho-df, the system $\vdash$ mrdg: rules (2a), (2b), (3a), (3b), (4a), (4b) of Munoz, Perez, Gutierrez (= rdfs5, rdfs7, rdfs11, rdfs9, rdfs2, rdfs3). — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
SubPropM(x1, y1) :- SubProp(x1, y1).
SubPropM(x1, z1) :- SubPropM(x1, y1), SubPropM(y1, z1).
TripleM(x1, y1, z1) :- Triple(x1, y1, z1).
TripleM(x1, z1, y1) :- SubPropM(x2, z1), TripleM(x1, x2, y1).
SubClassM(x1, y1) :- SubClass(x1, y1).
SubClassM(x1, z1) :- SubClassM(x1, y1), SubClassM(y1, z1).
TypeM(x1, y1) :- TypeOf(x1, y1).
TypeM(x1, z1) :- SubClassM(y1, z1), TypeM(x1, y1).
TypeM(x1, z1) :- Dom(y1, z1), TripleM(x1, y1, x2).
TypeM(y1, z1) :- Range(x2, z1), TripleM(x1, x2, y1).
```

prog_pf — The W3C RDFS entailment patterns on mrdg-graphs: the six shared patterns, rdfs6, rdfs10, rdfsD2 and the typing forced by the RDFS axiomatic triples of the rho-df vocabulary ($\text{IsProp} = \text{rdf:type rdf:Property}$, $\text{IsClass} = \text{rdf:type rdf:Class}$). — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
SubPropF(x1, y1) :- SubProp(x1, y1).
SubPropF(x1, z1) :- SubPropF(x1, y1), SubPropF(y1, z1).
SubPropF(x1, x1) :- IsProp(x1).
TripleF(x1, y1, z1) :- Triple(x1, y1, z1).
TripleF(x1, z1, y1) :- SubPropF(x2, z1), TripleF(x1, x2, y1).
SubClassF(x1, y1) :- SubClass(x1, y1).
SubClassF(x1, z1) :- SubClassF(x1, y1), SubClassF(y1, z1).
SubClassF(x1, x1) :- IsClass(x1).
TypeF(x1, y1) :- TypeOf(x1, y1).
TypeF(x1, z1) :- SubClassF(y1, z1), TypeF(x1, y1).
TypeF(x1, z1) :- Dom(y1, z1), TripleF(x1, y1, x2).
TypeF(y1, z1) :- Range(x2, z1), TripleF(x1, x2, y1).
IsProp(y1) :- TripleF(x1, y1, z1).
IsProp(x1) :- SubPropF(x1, y1).
IsProp(y1) :- SubPropF(x1, y1).
IsProp(x1) :- Dom(x1, y1).
IsProp(x1) :- Range(x1, y1).
IsClass(y1) :- TypeF(x1, y1).
IsClass(x1) :- SubClassF(x1, y1).
IsClass(y1) :- SubClassF(x1, y1).
IsClass(y1) :- Dom(x1, y1).
IsClass(y1) :- Range(x1, y1).
```

Reading. Both sides compute RDFS entailment closures over ground triples using rho-df vocabulary relations (subclass, subproperty, domain, range, type, generic triples); side 1 implements the minimal rho-df rule set of Munoz, Perez and Gutierrez (subproperty/subclass transitivity plus type and triple propagation), while side 2 implements

the full W3C RDFS rule set restricted to the same vocabulary, additionally deriving that every recognized property or class is reflexively a subproperty/subclass of itself. The triple states the two closures agree exactly on triples and typing, and that the full closure's subproperty/subclass relations equal the minimal closure's relations unioned with the extra reflexive pairs forced by which terms are recognized as properties or classes — a precise account of what the dropped RDFS rules contribute.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in `Input.lean`. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\top$

True; no precondition constrains the inputs.

POST: $(\forall x \ y \ z. (\text{TripleM}(x,y,z) \leftrightarrow \text{TripleF}(x,y,z))) \wedge (\forall u \ v. (\text{TypeM}(u,v) \leftrightarrow \text{TypeF}(u,v))) \wedge (\forall w \ x1. (\text{SubPropM}(w,x1) \vee \text{IsProp}(w) \wedge w = x1 \leftrightarrow \text{SubPropF}(w,x1))) \wedge \forall x2 \ x3. (\text{SubClassM}(x2,x3) \vee \text{IsClass}(x2) \wedge x2 = x3 \leftrightarrow \text{SubClassF}(x2,x3))$

The two rule sets derive identical Triple and Type relations, and the full rule set's subproperty/subclass relations equal the minimal rule set's subproperty/subclass relations unioned with the reflexive pairs over the terms recognized as properties/classes.

Whiel encoding (Hoare triple).

```
SCHEMA
  TypeOf/2, SubClass/2, SubProp/2, Dom/2, Range/2, TypeM/2, SubClassM/2, SubPropM/2, TypeM_aux/2, SubClassM_aux/2, SubPropM_aux/2, TypeF/2, SubClassF/2, SubPropF/2, TypeF_aux/2,
  ↳ SubClassF_aux/2, SubPropF_aux/2, Triple/3, TripleM/3, TripleM_aux/3, TripleF/3, TripleF_aux/3, IsProp/1, IsClass/1, IsProp_aux/1, IsClass_aux/1

PRE
  true

CMD
  SubPropM_aux := ∅[2];
  TripleM_aux := ∅[3];
  SubClassM_aux := ∅[2];
  TypeM_aux := ∅[2];
  SubPropM := (SubProp ∪ π[0,3] (σ[#1 = #2] ((SubPropM_aux × SubPropM_aux)))));
  TripleM := (Triple ∪ π[2,1,4] (σ[#0 = #3] ((SubPropM_aux × TripleM_aux)))));
  SubClassM := (SubClass ∪ π[0,3] (σ[#1 = #2] ((SubClassM_aux × SubClassM_aux)))));
  TypeM := (TypeOf ∪ π[2,1] (σ[#0 = #3] ((SubClassM_aux × TypeM_aux))) ∪ π[2,1] (σ[#0 = #3] ((Dom × TripleM_aux))) ∪ π[4,1] (σ[#0 = #3] ((Range × TripleM_aux)))));
  WHILE (¬(((SubPropM = SubPropM_aux) ∧ (TripleM = TripleM_aux) ∧ (SubClassM = SubClassM_aux) ∧ (TypeM = TypeM_aux)))) DO
  SubPropM_aux := SubPropM;
  TripleM_aux := TripleM;
  SubClassM_aux := SubClassM;
  TypeM_aux := TypeM;
  SubPropM := (SubPropM ∪ (SubProp ∪ π[0,3] (σ[#1 = #2] ((SubPropM_aux × SubPropM_aux)))));
  TripleM := (TripleM ∪ (Triple ∪ π[2,1,4] (σ[#0 = #3] ((SubPropM_aux × TripleM_aux)))));
  SubClassM := (SubClassM ∪ (SubClass ∪ π[0,3] (σ[#1 = #2] ((SubClassM_aux × SubClassM_aux)))));
  TypeM := (TypeM ∪ (TypeOf ∪ π[2,1] (σ[#0 = #3] ((SubClassM_aux × TypeM_aux))) ∪ π[2,1] (σ[#0 = #3] ((Dom × TripleM_aux))) ∪ π[4,1] (σ[#0 = #3] ((Range × TripleM_aux)))));
  END;
  SubPropF_aux := ∅[2];
  TripleF_aux := ∅[3];
  SubClassF_aux := ∅[2];
  TypeF_aux := ∅[2];
  IsProp_aux := ∅[1];
  IsClass_aux := ∅[1];
  SubPropF := (SubProp ∪ π[0,3] (σ[#1 = #2] ((SubPropF_aux × SubPropF_aux))) ∪ π[0,0] (IsProp_aux));
  TripleF := (Triple ∪ π[2,1,4] (σ[#0 = #3] ((SubPropF_aux × TripleF_aux)))));
  SubClassF := (SubClass ∪ π[0,3] (σ[#1 = #2] ((SubClassF_aux × SubClassF_aux))) ∪ π[0,0] (IsClass_aux));
  TypeF := (TypeOf ∪ π[2,1] (σ[#0 = #3] ((SubClassF_aux × TypeF_aux))) ∪ π[2,1] (σ[#0 = #3] ((Dom × TripleF_aux))) ∪ π[4,1] (σ[#0 = #3] ((Range × TripleF_aux)))));
  IsProp := (π[1] (TripleF_aux) ∪ π[0] (SubPropF_aux) ∪ π[1] (SubPropF_aux) ∪ π[0] (Dom) ∪ π[0] (Range)));
  IsClass := (π[1] (TypeF_aux) ∪ π[0] (SubClassF_aux) ∪ π[1] (SubClassF_aux) ∪ π[1] (Dom) ∪ π[1] (Range)));
  WHILE (¬(((SubPropF = SubPropF_aux) ∧ (TripleF = TripleF_aux) ∧ (SubClassF = SubClassF_aux) ∧ (TypeF = TypeF_aux) ∧ (IsProp = IsProp_aux) ∧ (IsClass = IsClass_aux)))) DO
  SubPropF_aux := SubPropF;
  TripleF_aux := TripleF;
  SubClassF_aux := SubClassF;
  TypeF_aux := TypeF;
  IsProp_aux := IsProp;
  IsClass_aux := IsClass;
  SubPropF := (SubPropF ∪ (SubProp ∪ π[0,3] (σ[#1 = #2] ((SubPropF_aux × SubPropF_aux))) ∪ π[0,0] (IsProp_aux)));
  TripleF := (TripleF ∪ (Triple ∪ π[2,1,4] (σ[#0 = #3] ((SubPropF_aux × TripleF_aux)))));
  SubClassF := (SubClassF ∪ (SubClass ∪ π[0,3] (σ[#1 = #2] ((SubClassF_aux × SubClassF_aux))) ∪ π[0,0] (IsClass_aux)));
  TypeF := (TypeF ∪ (TypeOf ∪ π[2,1] (σ[#0 = #3] ((SubClassF_aux × TypeF_aux))) ∪ π[2,1] (σ[#0 = #3] ((Dom × TripleF_aux))) ∪ π[4,1] (σ[#0 = #3] ((Range × TripleF_aux)))));
  IsProp := (IsProp ∪ π[1] (TripleF_aux) ∪ π[0] (SubPropF_aux) ∪ π[1] (SubPropF_aux) ∪ π[0] (Dom) ∪ π[0] (Range)));
  IsClass := (IsClass ∪ π[1] (TypeF_aux) ∪ π[0] (SubClassF_aux) ∪ π[1] (SubClassF_aux) ∪ π[1] (Dom) ∪ π[1] (Range)));
  END

POST
  (((TripleM = TripleF) ∧ (TypeM = TypeF)) ∧
  (((SubPropM ∪ π[0, 0] IsProp) = SubPropF) ∧ ((SubClassM ∪ π[0, 0] IsClass) = SubClassF)))
```

72 Example5021: rho-df versus full RDFS with the reflexive subClassOf triples forgotten (invalid twin of Example5020)

Category	specification (a standard or a paper's procedure versus a declarative program); family: two rule sets for one semantics (invalid twin).
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified invalid (Certificate/Invalid.lean, written by the certificate emitter); expected invalid.
Tags	mutual-recursion nonlinear-recursion.
Input	Benchmark/Example5021/Input.lean (canonical notation).
Summary	RDFS entailment by the rho-df rules against the full W3C rule set, with the reflexive subClassOf triples forgotten: the invalid twin of Example5020.
Sides	naive compiler: rho-df system $\vdash \text{mrdf}$ (as Example5020) naive compiler: W3C RDFS patterns on mrdg-graphs (as Example5020)
Sources	S. Munoz, J. Perez, C. Gutierrez, Simple and Efficient Minimal RDFS, Journal of Web Semantics 7(3), 2009 (sections Definition 3 (rho-df vocabulary), Table 1 (rules (1)-(7)), Definition 15 (mrdg graphs, $\vdash \text{mrdf}$), Proposition 17, Corollary 18; read 2026-09-15) W3C RDF 1.1 Semantics, Recommendation 25 February 2014 (sections 8.1.1 (rdfD2), 9 (RDFS axiomatic triples), 9.2.1 (rdfs1-rdfs13); read 2026-09-15) Apache Jena, jena-core/src/main/resources/etc/rdfs.rules (commit f9eed353614c178b80b1e099ea2598e648014074 (main, 2026-09-14); read 2026-09-15)
Twin	Example5020 (valid).
Obstruction note	from the metadata: n/a (invalid; kernel counterexample)

Datalog program(s). *prog_pm* — rho-df, the system $\vdash \text{mrdf}$: rules (2a), (2b), (3a), (3b), (4a), (4b) of Munoz, Perez, Gutierrez (= rdfs5, rdfs7, rdfs11, rdfs9, rdfs2, rdfs3). — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```

SubPropM(x1, y1) :- SubProp(x1, y1).
SubPropM(x1, z1) :- SubPropM(x1, y1), SubPropM(y1, z1).
TripleM(x1, y1, z1) :- Triple(x1, y1, z1).
TripleM(x1, z1, y1) :- SubPropM(x2, z1), TripleM(x1, x2, y1).
SubClassM(x1, y1) :- SubClass(x1, y1).
SubClassM(x1, z1) :- SubClassM(x1, y1), SubClassM(y1, z1).
TypeM(x1, y1) :- TypeOf(x1, y1).
TypeM(x1, z1) :- SubClassM(y1, z1), TypeM(x1, y1).
TypeM(x1, z1) :- Dom(y1, z1), TripleM(x1, y1, x2).
TypeM(y1, z1) :- Range(x2, z1), TripleM(x1, x2, y1).

```

prog_pf — The W3C RDFS entailment patterns on mrdg-graphs: the six shared patterns, rdfs6, rdfs10, rdfD2 and the typing forced by the RDFS axiomatic triples of the rho-df vocabulary (*IsProp* = *rdf:type* *rdf:Property*, *IsClass* = *rdf:type* *rdfs:Class*). — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```

SubPropF(x1, y1) :- SubProp(x1, y1).
SubPropF(x1, z1) :- SubPropF(x1, y1), SubPropF(y1, z1).
SubPropF(x1, x1) :- IsProp(x1).
TripleF(x1, y1, z1) :- Triple(x1, y1, z1).
TripleF(x1, z1, y1) :- SubPropF(x2, z1), TripleF(x1, x2, y1).
SubClassF(x1, y1) :- SubClass(x1, y1).
SubClassF(x1, z1) :- SubClassF(x1, y1), SubClassF(y1, z1).
SubClassF(x1, x1) :- IsClass(x1).
TypeF(x1, y1) :- TypeOf(x1, y1).
TypeF(x1, z1) :- SubClassF(y1, z1), TypeF(x1, y1).
TypeF(x1, z1) :- Dom(y1, z1), TripleF(x1, y1, x2).

```

```

TypeF(y1, z1) :- Range(x2, z1), TripleF(x1, x2, y1).
IsProp(y1) :- TripleF(x1, y1, z1).
IsProp(x1) :- SubPropF(x1, y1).
IsProp(y1) :- SubPropF(x1, y1).
IsProp(x1) :- Dom(x1, y1).
IsProp(x1) :- Range(x1, y1).
IsClass(y1) :- TypeF(x1, y1).
IsClass(x1) :- SubClassF(x1, y1).
IsClass(y1) :- SubClassF(x1, y1).
IsClass(y1) :- Dom(x1, y1).
IsClass(y1) :- Range(x1, y1).

```

Reading. Same rho-df versus full-RDFS comparison as Example5020, but the claimed postcondition omits the reflexive-subclass correction: it accounts for the reflexive subproperty pairs (rdfs6) but asserts the subclass relations equal outright, with no correction for the reflexive subclass pairs (rdfs10). The expected verdict is invalid: rdfs10 makes every recognized class a subclass of itself under the full RDFS rules, a pair the minimal rho-df closure never derives, so the uncorrected equality fails whenever some class is recognized in the data.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in Input.lean remains the authority.

PRE: $\top$

True; no precondition constrains the inputs.

POST: $(\forall x \ y \ z. \ (\text{TripleM}(x,y,z) \leftrightarrow \text{TripleF}(x,y,z))) \wedge (\forall u \ v. \ (\text{TypeM}(u,v) \leftrightarrow \text{TypeF}(u,v))) \wedge (\forall w \ x1. \ (\text{SubPropM}(w,x1) \vee \text{IsProp}(w) \wedge w = x1 \leftrightarrow \text{SubPropF}(w,x1))) \wedge \forall x2 \ x3. \ (\text{SubClassM}(x2,x3) \leftrightarrow \text{SubClassF}(x2,x3))$

The claim (which fails) is that Triple and Type agree between the two rule sets, the subproperty relations agree once the reflexive property pairs are added back, but the subclass relations are asserted equal outright, with no reflexive-subclass correction.

Whiel encoding (Hoare triple).

```

SCHEMA
TypeOf/2, SubClass/2, SubProp/2, Dom/2, Range/2, TypeM/2, SubClassM/2, SubPropM/2, TypeM_aux/2, SubClassM_aux/2, SubPropM_aux/2, TypeF/2, SubClassF/2, SubPropF/2, TypeF_aux/2,
↪ SubClassF_aux/2, SubPropF_aux/2, Triple/3, TripleM/3, TripleM_aux/3, TripleF/3, TripleF_aux/3, IsProp/1, IsClass/1, IsProp_aux/1, IsClass_aux/1

PRE
true

CMD
SubPropM_aux := ∅[2];
TripleM_aux := ∅[3];
SubClassM_aux := ∅[2];
TypeM_aux := ∅[2];
SubPropM := (SubProp ∪ π[0,3] (σ[#1 = #2] ((SubPropM_aux × SubPropM_aux))));
TripleM := (Triple ∪ π[2,1,4] (σ[#0 = #3] ((SubPropM_aux × TripleM_aux))));
SubClassM := (SubClass ∪ π[0,3] (σ[#1 = #2] ((SubClassM_aux × SubClassM_aux))));
TypeM := (TypeOf ∪ π[2,1] (σ[#0 = #3] ((SubClassM_aux × TypeM_aux)) ∪ π[2,1] (σ[#0 = #3] ((Dom × TripleM_aux)) ∪ π[4,1] (σ[#0 = #3] ((Range × TripleM_aux))))));
WHILE ¬(((SubPropM = SubPropM_aux) ∧ ((TripleM = TripleM_aux) ∧ ((SubClassM = SubClassM_aux) ∧ (TypeM = TypeM_aux))))) DO
SubPropM_aux := SubPropM;
TripleM_aux := TripleM;
SubClassM_aux := SubClassM;
TypeM_aux := TypeM;
SubPropM := (SubProp ∪ π[0,3] (σ[#1 = #2] ((SubPropM_aux × SubPropM_aux))));
TripleM := (Triple ∪ π[2,1,4] (σ[#0 = #3] ((SubPropM_aux × TripleM_aux))));
SubClassM := (SubClass ∪ π[0,3] (σ[#1 = #2] ((SubClassM_aux × SubClassM_aux))));
TypeM := (TypeOf ∪ π[2,1] (σ[#0 = #3] ((SubClassM_aux × TypeM_aux)) ∪ π[2,1] (σ[#0 = #3] ((Dom × TripleM_aux)) ∪ π[4,1] (σ[#0 = #3] ((Range × TripleM_aux))))));
END;
SubPropF_aux := ∅[2];
TripleF_aux := ∅[3];
SubClassF_aux := ∅[2];
TypeF_aux := ∅[2];
IsProp_aux := ∅[1];
IsClass_aux := ∅[1];
SubPropF := (SubProp ∪ π[0,3] (σ[#1 = #2] ((SubPropF_aux × SubPropF_aux)) ∪ π[0,0] (IsProp_aux)));
TripleF := (Triple ∪ π[2,1,4] (σ[#0 = #3] ((SubPropF_aux × TripleF_aux))));
SubClassF := (SubClass ∪ π[0,3] (σ[#1 = #2] ((SubClassF_aux × SubClassF_aux)) ∪ π[0,0] (IsClass_aux)));
TypeF := (TypeOf ∪ π[2,1] (σ[#0 = #3] ((SubClassF_aux × TypeF_aux)) ∪ π[2,1] (σ[#0 = #3] ((Dom × TripleF_aux)) ∪ π[4,1] (σ[#0 = #3] ((Range × TripleF_aux))))));
IsProp := (π[1] (TripleF_aux) ∪ π[0] (SubPropF_aux) ∪ π[1] (SubPropF_aux) ∪ π[0] (Dom) ∪ π[0] (Range)));
IsClass := (π[1] (TypeF_aux) ∪ π[0] (SubClassF_aux) ∪ π[1] (SubClassF_aux) ∪ π[1] (Dom) ∪ π[1] (Range)));
WHILE ¬(((SubPropF = SubPropF_aux) ∧ ((TripleF = TripleF_aux) ∧ ((SubClassF = SubClassF_aux) ∧ (TypeF = TypeF_aux) ∧ ((IsProp = IsProp_aux) ∧ (IsClass = IsClass_aux)))))) DO
SubPropF_aux := SubPropF;
TripleF_aux := TripleF;
SubClassF_aux := SubClassF;
TypeF_aux := TypeF;
IsProp_aux := IsProp;
IsClass_aux := IsClass;
SubPropF := (SubProp ∪ π[0,3] (σ[#1 = #2] ((SubPropF_aux × SubPropF_aux)) ∪ π[0,0] (IsProp_aux)));
TripleF := (Triple ∪ π[2,1,4] (σ[#0 = #3] ((SubPropF_aux × TripleF_aux))));
SubClassF := (SubClass ∪ π[0,3] (σ[#1 = #2] ((SubClassF_aux × SubClassF_aux)) ∪ π[0,0] (IsClass_aux)));
TypeF := (TypeOf ∪ π[2,1] (σ[#0 = #3] ((SubClassF_aux × TypeF_aux)) ∪ π[2,1] (σ[#0 = #3] ((Dom × TripleF_aux)) ∪ π[4,1] (σ[#0 = #3] ((Range × TripleF_aux))))));
IsProp := (IsProp ∪ π[1] (TripleF_aux) ∪ π[0] (SubPropF_aux) ∪ π[1] (SubPropF_aux) ∪ π[0] (Dom) ∪ π[0] (Range)));
IsClass := (IsClass ∪ π[1] (TypeF_aux) ∪ π[0] (SubClassF_aux) ∪ π[1] (SubClassF_aux) ∪ π[1] (Dom) ∪ π[1] (Range)));
END

```

```

POST
(((TripleM = TripleF) ∧ (TypeM = TypeF)) ∧
  (((SubPropM ⊔ π[0, 0] IsProp) = SubPropF) ∧ (SubClassM = SubClassF))))

```

73 Example5022: SQL recursion limits (MySQL cte_max_recursion_depth, SQL Server MAXRECURSION): completion within the limit implies the closure

Category	specification (a standard or a paper's procedure versus a declarative program); family: engine semantics versus declarative program (conditional claim).
Kind	implication (disjunction in the postcondition).
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	database-engine prophecy recursion-limit.
Input	Benchmark/Example5022/Input.lean (canonical notation).
Summary	Recursion limits of SQL engines: a recursive CTE run under MySQL's cte_max_recursion_depth or SQL Server's MAXRECURSION against the Datalog closure it is meant to compute. Claim: when the engine does not abort, its result is the closure.
Sides	hand-written, loop-free: four semi-naive iterations of the recursive CTE under limit $k = 4$ (ResA = anchor; NewB..NewE new rows per iteration; ResB..ResE cumulative), MySQL UNION DISTINCT semantics; SQL Server's UNION ALL termination implies the modelled condition naive compiler: Cl = closure (the CTE's Datalog meaning)
Sources	MySQL 8.4 Reference Manual, WITH (Common Table Expressions): Recursive Common Table Expressions; Limiting Common Table Expression Recursion (read 2026-09-15) MySQL server source, share/messages_to_clients.txt (ER_CTE_MAX_RECURSION_DEPTH = 3636) (commit 99960bf74fa919347e4f4e3ca47672f333d6e91f (branch 8.4); read 2026-09-15) Microsoft SQL Server, WITH common_table_expression (Transact-SQL) (read 2026-09-15) Microsoft SQL Server, Hints (Transact-SQL) - Query (MAXRECURSION) (read 2026-09-15) Microsoft SQL Server, Database Engine events and errors (0 to 999) (error 530) (read 2026-09-15)
Prophecy	accepted invariant with a prophecy clause above level zero (Core.json); no certificate built from it yet.

Datalog program(s). *prog_p* — *The recursive CTE as Datalog.* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```

Cl(x1, y1) :- Base(x1, y1).
Cl(x1, z1) :- Cl(x1, y1), E(y1, z1).

```

Reading. Side 1 encodes, loop-free, four semi-naive iterations of a recursive common table expression under a fixed recursion-depth limit ($k = 4$), as implemented by MySQL's cte_max_recursion_depth or SQL Server's MAXRECURSION, tracking new rows and cumulative results per iteration; side 2 is the naive-compiler translation of the closure the CTE is meant to compute. The triple states a conditional correctness property for bounded-recursion SQL engines: if the engine's iteration limit is not exceeded (the final iteration produces no new rows), the truncated result actually equals the full transitive closure — a limited engine only ever silently under-approximates by aborting, never by returning a wrong-but-apparently-complete answer.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\top$

True; no precondition constrains the inputs.

POST: $(\exists x y. \text{NewE}(x,y)) \vee \forall z u. (\text{Cl}(z,u) \leftrightarrow \text{ResE}(z,u))$

Either the final iteration still produced new rows (the recursion-depth limit was exhausted), or the limited engine's result equals the true closure.

Whiel encoding (Hoare triple).

```

SCHEMA
  Base/2, E/2, ResA/2, NewB/2, ResB/2, NewC/2, ResC/2, NewD/2, ResD/2, NewE/2, ResE/2, Cl/2, Cl_aux/2

PRE
  true

CMD
  ResA := Base;
  NewB := ( $\pi[0, 3] (\sigma[\#1 = \#2] (\text{ResA} \times \text{E})) \setminus \text{ResA}$ );
  ResB := ( $\text{ResA} \cup \text{NewB}$ );
  NewC := ( $\pi[0, 3] (\sigma[\#1 = \#2] (\text{NewB} \times \text{E})) \setminus \text{ResB}$ );
  ResC := ( $\text{ResB} \cup \text{NewC}$ );
  NewD := ( $\pi[0, 3] (\sigma[\#1 = \#2] (\text{NewC} \times \text{E})) \setminus \text{ResC}$ );
  ResD := ( $\text{ResC} \cup \text{NewD}$ );
  NewE := ( $\pi[0, 3] (\sigma[\#1 = \#2] (\text{NewD} \times \text{E})) \setminus \text{ResD}$ );
  ResE := ( $\text{ResD} \cup \text{NewE}$ );
  Cl_aux :=  $\emptyset[2]$ ;
  Cl := ( $\text{Base} \cup \pi[0,3] (\sigma[\#1 = \#2] ((\text{Cl\_aux} \times \text{E})))$ );
  WHILE ( $\neg((\text{Cl} = \text{Cl\_aux}))$ ) DO
    Cl_aux := Cl;
  Cl := ( $\text{Cl} \cup (\text{Base} \cup \pi[0,3] (\sigma[\#1 = \#2] ((\text{Cl\_aux} \times \text{E})))$ );
  END

POST
   $((\text{NewE} \neq \emptyset) \vee (\text{Cl} = \text{ResE}))$ 

```

74 Example5023: Win-move: the alternating fixpoint of the well-founded semantics (nested loops) versus the attractor computation

Category	specification (a standard or a paper's procedure versus a declarative program); family: two computations of one semantics.
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified valid (certificate checked outside the library build) (<code>Certificate/Valid.lean</code> , written by the certificate emitter); expected valid. Placement note: a proof module imported by <code>Certificate/Valid.lean</code> does not build under the library build's 4194304 KB limit; it builds under 12582912 KB (peak 1672 MB)
Tags	<code>evaluation-strategies</code> <code>greatest-fixpoint</code> <code>negation</code> <code>prophecy</code> .
Input	<code>Benchmark/Example5023/Input.lean</code> (canonical notation).
Summary	Win-move under the well-founded semantics: the alternating fixpoint (Van Gelder) computed with nested loops against the attractor computation of Example0162 and legacy Example2019 (positions with a move to a losing position win; positions all of whose moves lead to winning positions lose).
Sides	hand-written: alternating fixpoint $A_0 = \emptyset$, $A_{i+1} = S_P(A_i)$ with Under (even) / Over (odd), each S_P an inner least-fixpoint loop in the naive compiler's form for <code>OverI(x) :- Move(x,y)</code> , <code>NotUnder(y)</code> and <code>UnderI(x) :- Move(x,y)</code> , <code>NotOver(y)</code> hand-written: attractor (Example0162 / legacy 2019 style, simultaneous update of Win and Lose)

Sources	A. Van Gelder, K. A. Ross, J. S. Schlipf, The Well-Founded Semantics for General Logic Programs, JACM 38(3), 1991 (read not retrievable 2026-09-15 (ACM DL 403 / bot check)) A. Van Gelder, The Alternating Fixpoint of Logic Programs with Negation, PODS 1989 (read not retrievable 2026-09-15 (ACM DL 403 / bot check)) A. Van Gelder, The Alternating Fixpoint of Logic Programs with Negation, JCSS 47(1), 1993 (read not retrievable 2026-09-15 (ScienceDirect 403, also in the in-app browser)) S. Abiteboul, R. Hull, V. Vianu, Foundations of Databases, chapter 15 (Example 15.3.1; 15.3 A Fixpoint Definition; Example 15.3.8(b); Theorem 15.3.9) (read 2026-09-15) Example0162 and the retired corpus case Example2019 (attractor side) (read 2026-09-15) program source file win(X) :- move(X,Y), not win(Y)
Twin	Example5024 (invalid: Over = Win, i.e. no draws).
Prophecy	accepted invariant with a prophecy clause above level zero (Core.json); no certificate built from it yet.

Datalog program(s). *prog_pO* — The positive programs $P^{\wedge I}$ of the two half-steps of the alternation (*not win(Y)* read as *NotUnder(Y)*; resp. *NotOver(Y)*); their naive compilation is the inner loop of each half-step. — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
OverI(x1) :- Move(x1, y1), NotUnder(y1).
```

prog_pU — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
UnderI(x1) :- Move(x1, y1), NotOver(y1).
```

rules as stated in the header comment of Input.lean — header comment.

```
win(X) :- move(X, Y), not win(Y)
OverI(x) :- Move(x, y), NotUnder(y)
UnderI(x) :- Move(x, y), NotOver(y)
```

Reading. Both sides compute the well-founded semantics of the classic win-move game program (a position wins if it has a move to a position that is not a win), whose model is three-valued (winning/losing/drawn); side 1 computes Van Gelder's alternating fixpoint directly, alternating an outer loop between an even sequence (Under) and odd sequence (Over), each obtained via an inner positive-program least-fixpoint loop — the first nested-loop case in this corpus — while side 2 computes the same sets via the game-theoretic attractor construction (simultaneously updating Win/Lose sets by one half-step each iteration), as in Example0162. The triple states the alternating fixpoint's even limit and the complement of its odd limit coincide exactly with the attractor's Win and Lose sets, the standard correspondence between well-founded semantics and attractor-based game solving.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\top$

True; no precondition constrains the inputs.

POST: $(\forall x. (\text{Under}(x) \leftrightarrow \text{Win}(x))) \wedge \forall y. (\text{Nodes}(y) \wedge \neg \text{Over}(y) \leftrightarrow \text{Lose}(y))$

The alternating fixpoint's even limit Under equals the attractor's Win set, and the complement of the odd limit Over (relative to all positions) equals the attractor's Lose set.

Whiel encoding (Hoare triple).

```

SCHEMA
  Move/2, Nodes/1, Under/1, Over/1, Prev/1, NotUnder/1, NotOver/1, OverI/1, OverI_aux/1, UnderI/1, UnderI_aux/1, Win/1, Lose/1, WinP/1, LoseP/1

PRE
  true

CMD
  Nodes := ( $\pi[0]$  Move  $\sqcup$   $\pi[1]$  Move);
  Under :=  $\emptyset[1]$ ;
  Over :=  $\emptyset[1]$ ;
  Prev := Nodes;
  WHILE ( $\neg$ ((Under = Prev))) DO
    Prev := Under;
    NotUnder := (Nodes \ Under);
    OverI_aux :=  $\emptyset[1]$ ;
    OverI :=  $\pi[0]$  ( $\sigma[\#1 = \#2]$  ((Move  $\times$  NotUnder)));
    WHILE ( $\neg$ ((OverI = OverI_aux))) DO
      OverI_aux := OverI;
      OverI := (OverI  $\sqcup$   $\pi[0]$  ( $\sigma[\#1 = \#2]$  ((Move  $\times$  NotUnder))));
    END;
    Over := OverI;
    NotOver := (Nodes \ Over);
    UnderI_aux :=  $\emptyset[1]$ ;
    UnderI :=  $\pi[0]$  ( $\sigma[\#1 = \#2]$  ((Move  $\times$  NotOver)));
    WHILE ( $\neg$ ((UnderI = UnderI_aux))) DO
      UnderI_aux := UnderI;
      UnderI := (UnderI  $\sqcup$   $\pi[0]$  ( $\sigma[\#1 = \#2]$  ((Move  $\times$  NotOver))));
    END;
    Under := UnderI
  END;
  WinP :=  $\emptyset[1]$ ;
  LoseP :=  $\emptyset[1]$ ;
  Win :=  $\pi[0]$  ( $\sigma[\#1 = \#2]$  (Move  $\times$  LoseP));
  Lose := (Nodes \  $\pi[0]$  (Move \  $\pi[0, 1]$  ( $\sigma[\#1 = \#2]$  (Move  $\times$  WinP))));
  WHILE ( $\neg$ ((Win = WinP)  $\wedge$  (Lose = LoseP))) DO
    WinP := Win;
    LoseP := Lose;
    Win :=  $\pi[0]$  ( $\sigma[\#1 = \#2]$  (Move  $\times$  LoseP));
    Lose := (Nodes \  $\pi[0]$  (Move \  $\pi[0, 1]$  ( $\sigma[\#1 = \#2]$  (Move  $\times$  WinP))));
  END

POST
  ((Under = Win)  $\wedge$  ((Nodes \ Over) = Lose))

```

75 Example5024: Win-move: alternating fixpoint versus attractor, claiming the odd limit is the win set (invalid twin of Example5023)

Category	specification (a standard or a paper's procedure versus a declarative program); family: two computations of one semantics (invalid twin).
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified invalid (Certificate/Invalid.lean, written by the certificate emitter); expected invalid.
Tags	evaluation-strategies greatest-fixpoint negation .
Input	Benchmark/Example5024/Input.lean (canonical notation).
Summary	Win-move under the well-founded semantics, the invalid twin of Example5023: the alternating fixpoint against the attractor, claiming in addition that the odd limit (the positions that are not false) is the attractor's win set, that is, that the well-founded model of win-move is total.
Sides	hand-written: alternating fixpoint as in Example5023 hand-written: attractor as in Example5023
Sources	A. Van Gelder, K. A. Ross, J. S. Schlipf, The Well-Founded Semantics for General Logic Programs, JACM 38(3), 1991 (read not retrievable 2026-09-15 (ACM DL 403 / bot check)) A. Van Gelder, The Alternating Fixpoint of Logic Programs with Negation, PODS 1989 (read not retrievable 2026-09-15 (ACM DL 403 / bot check)) A. Van Gelder, The Alternating Fixpoint of Logic Programs with Negation, JCSS 47(1), 1993 (read not retrievable 2026-09-15 (ScienceDirect 403, also in the in-app browser)) S. Abiteboul, R. Hull, V. Vianu, Foundations of Databases, chapter 15 (Example 15.3.1; 15.3 A Fixpoint Definition; Example 15.3.8(b); Theorem 15.3.9) (read 2026-09-15) Example0162 and the retired corpus case Example2019 (attractor side) (read 2026-09-15) program source file win(X) :- move(X,Y), not win(Y)

Twin Example5023 (valid).

Obstruction note

from the metadata: n/a (invalid; kernel counterexample)

Datalog program(s). *prog_pO* — The positive programs P^I of the two half-steps of the alternation (*‘not win(Y)’* read as *‘NotUnder(Y)’*, resp. *‘NotOver(Y)’*); their naive compilation is the inner loop of each half-step. — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```
OverI(x1) :- Move(x1, y1), NotUnder(y1).
```

prog_pU — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```
UnderI(x1) :- Move(x1, y1), NotOver(y1).
```

rules as stated in the header comment of *Input.lean* — header comment.

```
win(X) :- move(X, Y), not win(Y)
OverI(x) :- Move(x, y), NotUnder(y)
UnderI(x) :- Move(x, y), NotOver(y)
```

Reading. Same two computations as Example5023 (Van Gelder’s alternating fixpoint versus the game attractor for well-founded win-move), but the postcondition additionally claims Over equals Win directly — that the odd limit of the alternation coincides with the win set, equivalently that the well-founded model has no undefined (drawn) positions. The expected verdict is invalid: on a two-cycle where neither player can force a win, both the true win set and the attractor’s Win set are empty, but Over stabilizes at both positions (each has a move to a position not yet known to win), so those positions are genuinely undefined rather than losing, and Over does not equal Win.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in *Input.lean*. The relational-algebra assertion in *Input.lean* remains the authority.

PRE: $\top$

True; no precondition constrains the inputs.

POST: $(\forall x. (\text{Under}(x) \leftrightarrow \text{Win}(x))) \wedge \forall y. (\text{Over}(y) \leftrightarrow \text{Win}(y))$

The claim (which fails) is that both the even limit Under and the odd limit Over of the alternating fixpoint equal the attractor’s Win set.

Whiel encoding (Hoare triple).

```
SCHEMA
  Move/2, Nodes/1, Under/1, Over/1, Prev/1, NotUnder/1, NotOver/1, OverI/1, OverI_aux/1, UnderI/1, UnderI_aux/1, Win/1, Lose/1, WinP/1, LoseP/1

PRE
  true

CMD
  Nodes := (π[0] Move ∪ π[1] Move);
  Under := ∅[1];
  Over := ∅[1];
  Prev := Nodes;
  WHILE (¬((Under = Prev))) DO
    Prev := Under;
    NotUnder := (Nodes \ Under);
    OverI_aux := ∅[1];
    OverI := π[0] (σ[#1 = #2] ((Move × NotUnder)));
    WHILE (¬((OverI = OverI_aux))) DO
      OverI_aux := OverI;
      OverI := (OverI ∪ π[0] (σ[#1 = #2] ((Move × NotUnder))))
    END;
    Over := OverI;
    NotOver := (Nodes \ Over);
    UnderI_aux := ∅[1];
    UnderI := π[0] (σ[#1 = #2] ((Move × NotOver)));
    WHILE (¬((UnderI = UnderI_aux))) DO
```

```

UnderI_aux := UnderI;
UnderI := (UnderI ∪ π[0] (σ[#1 = #2] ((Move × NotOver))))
END;
Under := UnderI
END;
WinP := ∅[1];
LoseP := ∅[1];
Win := π[0] (σ[#1 = #2] (Move × LoseP));
Lose := (Nodes \ π[0] (Move \ π[0, 1] (σ[#1 = #2] (Move × WinP))));
WHILE (¬((Win = WinP) ∧ (Lose = LoseP))) DO
  WinP := Win;
  LoseP := Lose;
  Win := π[0] (σ[#1 = #2] (Move × LoseP));
  Lose := (Nodes \ π[0] (Move \ π[0, 1] (σ[#1 = #2] (Move × WinP))));
END
POST
((Under = Win) ∧ (Over = Win))

```

76 Example5030: NetworkX per-source transitive closure versus PostgreSQL recursive-union rounds

Category	production pair (two algorithms from production sources or legacy production cases); family: production algorithm pair (rate mismatch).
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified valid (certificate checked outside the library build) (Certificate/Valid.lean, written by the certificate emitter); expected valid. Placement note: a proof module imported by Certificate/Valid.lean does not build under the library build's 4194304 KB limit; it builds under 12582912 KB (peak 1679 MB)
Tags	<code>database-engine evaluation-strategies prophecy semi-naive</code> .
Input	Benchmark/Example5030/Input.lean (canonical notation).
Summary	NetworkX's per-source transitive closure against PostgreSQL's recursive-union rounds.
Sides	hand-written: NetworkX <code>transitive_closure(G, reflexive=False)</code> = for each node in Lt order, the edge BFS of <code>edge_bfs</code> set-at-a-time (nested loop) hand-written: PostgreSQL <code>ExecRecursiveUnion</code> working/intermediate-table rounds (legacy Example2037) seeded with <code>Edge</code>
Sources	NetworkX <code>networkx/networkx/networkx/algorithms/dag.py transitive_closure (reflexive=False branch: TC.add_edges_from((v, e[1]) for e in nx.edge_bfs(G, v) if e[1] not in TC[v]))</code> ; descendants (branch main; commit 4e74880b0da0 (2026-09-12); read 2026-09-15) NetworkX <code>networkx/networkx/networkx/algorithms/traversal/edgebfs.py edge_bfs (visited_nodes, visited_edges, deque of (node, edges_from(node)))</code> (branch main; commit 4e74880b0da0 (2026-09-12); read 2026-09-15) PostgreSQL <code>postgres/postgres/src/backend/executor/nodeRecursiveunion.c ExecRecursiveUnion</code> (header comment steps 2.1-2.6; 'Ignore tuple if already seen'; working/intermediate table swap) (branch master; commit 6e58d6356fcb (2026-09-15); read 2026-09-15; encoded as legacy Example2037 / Example5001 hand side with the base term equal to the edge relation)
Twin	Example5031 (invalid: per-source search with cutoff 2).
Prophecy	accepted invariant with a prophecy clause above level zero (Core.json); no certificate built from it yet.

Datalog program(s). *prog_T* — The Datalog program(*s*) of the header: compiled sides are their naive compilation; reference programs are compared with the hand-written sides on the check instances. — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```

T(x1, y1) :- Edge(x1, y1).
T(x1, y1) :- T(x1, z1), Edge(z1, y1).

```

Reading. The Datalog program is ordinary graph transitive closure, $T(x,y) :- \text{Edge}(x,y)$ and $T(x,y) :- T(x,z), \text{Edge}(z,y)$. Side 1 encodes NetworkX's `transitive_closure(reflexive=False)`: for each node taken in a fixed order it runs an edge-wise breadth-first search (`edge_bfs`) set-at-a-time, accumulating into `Tca` every head reached by one or more edges from that node. Side 2 encodes PostgreSQL's recursive-union executor (`ExecRecursiveUnion`): a working table of newly derived edges is repeatedly joined with `Edge` and folded into a result `Res`, already-seen rows dropped, until no new rows appear. The Hoare triple claims the two independently produced relations coincide, $Tca = Res$, i.e. the per-source edge-BFS search and PostgreSQL's round-based recursive union compute exactly the same transitive closure, for every strict total order used to sequence the per-source search.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in `Input.lean`. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $(\neg \exists x. Lt(x,x)) \wedge (\forall y z. ((\exists u. Lt(y,u) \wedge Lt(u,z)) \rightarrow Lt(y,z))) \wedge (\forall v w. (Lt(v,w) \rightarrow ((\exists x1. \text{Edge}(v,x1)) \vee \exists x2. \text{Edge}(x2,v)) \wedge ((\exists x3. \text{Edge}(w,x3)) \vee \exists x4. \text{Edge}(x4,w)))) \wedge \forall x5 x6. (((\exists x7. \text{Edge}(x5,x7)) \vee \exists x8. \text{Edge}(x8,x5)) \wedge ((\exists x9. \text{Edge}(x6,x9)) \vee \exists x10. \text{Edge}(x10,x6))) \wedge \neg x5 = x6 \rightarrow Lt(x5,x6) \vee Lt(x6,x5))$

Lt is a strict total order (irreflexive, transitive, total, and confined to the nodes that occur in `Edge`) over the nodes of the graph.

POST: $\forall x y. (Tca(x,y) \leftrightarrow Res(x,y))$

The relation `Tca`, built by NetworkX's per-source edge-BFS search taken node by node in Lt order, equals the relation `Res`, built by PostgreSQL's round-based recursive union.

Whiel encoding (Hoare triple).

```
SCHEMA
  Rest/1, Cur/1, Edge/2, Lt/2, Vis/2, Fr/2, Step/2, New/2, Tca/2, Res/2, WorkT/2, Inter/2

PRE
  (((((σ[#0 = #1] Lt) = ∅) ∧
    ((π[0, 3] (σ[#1 = #2] (Lt × Lt))) ⊆ Lt)) ∧
    ((Lt ⊆ ((π[0] Edge ∪ π[1] Edge) × (π[0] Edge ∪ π[1] Edge))) ∧
      ((σ[¬(#0 = #1)] ((π[0] Edge ∪ π[1] Edge) × (π[0] Edge ∪ π[1] Edge))) ⊆ (Lt ∪ π[1, 0] Lt))))))

CMD
  Tca := ∅[2];
  Rest := (π[0] Edge ∪ π[1] Edge);
  WHILE (Rest ≠ ∅) DO
    Cur := (Rest \ (π[1] (σ[#0 = #2] (Lt × Rest))));
    Vis := (π[0, 0] Cur);
    Fr := Vis;
    WHILE (Fr ≠ ∅) DO
      Step := (π[0, 3] (σ[#1 = #2] (Fr × Edge)));
      Tca := (Tca ∪ Step);
      New := (Step \ Vis);
      Vis := (Vis ∪ New);
      Fr := New;
    END;
    Rest := (Rest \ Cur);
  END;
  Res := Edge;
  WorkT := Edge;
  WHILE (WorkT ≠ ∅) DO
    Inter := ((π[0, 3] (σ[#1 = #2] (WorkT × Edge))) \ Res);
    Res := (Res ∪ Inter);
    WorkT := Inter;
  END;

POST
  (Tca = Res)
```

77 Example5031: NetworkX per-source search with cutoff 2 versus PostgreSQL recursive-union rounds (invalid twin of Example5030)

Category	production pair (two algorithms from production sources or legacy production cases); family: production algorithm pair (rate mismatch), invalid twin.
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified invalid (Certificate/Invalid.lean, written by the certificate emitter); expected invalid.
Tags	database-engine evaluation-strategies recursion-limit semi-naive.
Input	Benchmark/Example5031/Input.lean (canonical notation).
Summary	The invalid twin of Example5030: a per-source search with a depth cutoff against PostgreSQL's recursive-union rounds.
Sides	hand-written: for each node in Lt order, the levels 1 and 2 of single_source_shortest_path_length(G, v, cutoff=2) (loop-free per node) hand-written: PostgreSQL ExecRecursiveUnion rounds (legacy Example2037) seeded with Edge
Sources	NetworkX networkx/networkx networkx/algorithms/shortest_paths/unweighted.py single_source_shortest_path_length, _single_shortest_path_length (seen = set(firstlevel); while nextlevel and cutoff > level) (branch main; commit 4e74880b0da0 (2026-09-12); read 2026-09-15) PostgreSQL postgres/postgres src/backend/executor/nodeRecursiveunion.c ExecRecursiveUnion (header comment steps 2.1-2.6; 'Ignore tuple if already seen'; working/intermediate table swap) (branch master; commit 6e58d6356fcb (2026-09-15); read 2026-09-15; encoded as legacy Example2037 / Example5001 hand side with the base term equal to the edge relation)
Twin	Example5030 (valid: unbounded per-source closure).
Obstruction note	from the metadata: invalid (depth cutoff); rate mismatch as in Example5030

Datalog program(s). *prog_T* — The Datalog program(s) of the header: compiled sides are their naive compilation; reference programs are compared with the hand-written sides on the check instances. — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
T(x1, y1) :- Edge(x1, y1).
T(x1, y1) :- T(x1, z1), Edge(z1, y1).
```

Reading. This is the invalid twin of Example5030: side 2 is the same PostgreSQL recursive-union closure, but side 1 replaces the unbounded per-source search with NetworkX's single_source_shortest_path_length run with cutoff=2, so for each source it only collects the nodes reachable within two edges (levels 1 and 2), accumulated into TcB. The Hoare triple claims TcB = Res, i.e. that this depth-2-bounded per-source search still equals the full transitive closure computed by PostgreSQL's rounds. This fails because a depth cutoff silently drops longer paths: the case gives an explicit witness (a length-4 path under a compatible order) where the executor derives a pair the bounded search never reaches, so the claim is refuted by a kernel-checked counterexample.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in Input.lean remains the authority.

PRE: $(\neg \exists x. Lt(x, x)) \wedge (\forall y z. ((\exists u. Lt(y, u) \wedge Lt(u, z)) \rightarrow Lt(y, z))) \wedge (\forall v w. (Lt(v, w) \rightarrow ((\exists x1. Edge(v, x1)) \vee \exists x2. Edge(x2, v)) \wedge ((\exists x3. Edge(w, x3)) \vee \exists x4. Edge(x4, w)))) \wedge \forall x5 x6. (((\exists x7. Edge(x5, x7)) \vee \exists x8. Edge(x8, x5)) \wedge ((\exists x9. Edge(x6, x9)) \vee \exists x10. Edge(x10, x6)) \wedge \neg x5 = x6 \rightarrow Lt(x5, x6) \vee Lt(x6, x5))$

Lt is a strict total order over the nodes of Edge, exactly as in Example5030.

POST: $\forall x y. (\text{TcB}(x,y) \leftrightarrow \text{Res}(x,y))$

It is claimed that TcB , the union over sources of the nodes reachable within 2 edges (per NetworkX’s `single_source_shortest_path_length` with `cutoff=2`), equals Res , the full transitive closure computed by PostgreSQL’s recursive union — a claim the case shows to be false.

Whiel encoding (Hoare triple).

```

SCHEMA
  Rest/1, Cur/1, Edge/2, Lt/2, SeenA/2, LvlA/2, SeenB/2, LvlB/2, TcB/2, Res/2, WorkT/2, Inter/2

PRE
  ((( $(\sigma[\#0 = \#1] \text{ Lt}) = \emptyset) \wedge$ 
    ( $(\pi[0, 3] (\sigma[\#1 = \#2] (\text{ Lt } \times \text{ Lt}))) \subseteq \text{ Lt}) \wedge$ 
    ( $\text{ Lt } \subseteq ((\pi[0] \text{ Edge } \cup \pi[1] \text{ Edge}) \times (\pi[0] \text{ Edge } \cup \pi[1] \text{ Edge})) \wedge$ 
    ( $(\sigma[\neg(\#0 = \#1)] ((\pi[0] \text{ Edge } \cup \pi[1] \text{ Edge}) \times (\pi[0] \text{ Edge } \cup \pi[1] \text{ Edge})) \subseteq (\text{ Lt } \cup \pi[1, 0] \text{ Lt}))$ )))

CMD
  TcB :=  $\emptyset[2]$ ;
  Rest :=  $(\pi[0] \text{ Edge } \cup \pi[1] \text{ Edge})$ ;
  WHILE (Rest  $\neq \emptyset$ ) DO
    Cur := (Rest  $\setminus (\pi[1] (\sigma[\#0 = \#2] (\text{ Lt } \times \text{ Rest})))$ );
    SeenA :=  $(\pi[0, 0] \text{ Cur})$ ;
    LvlA :=  $((\pi[0, 3] (\sigma[\#1 = \#2] (\text{ SeenA } \times \text{ Edge}))) \setminus \text{ SeenA})$ ;
    SeenB :=  $(\text{ SeenA } \cup \text{ LvlA})$ ;
    LvlB :=  $((\pi[0, 3] (\sigma[\#1 = \#2] (\text{ LvlA } \times \text{ Edge}))) \setminus \text{ SeenB})$ ;
    TcB :=  $((\text{ TcB } \cup \text{ LvlA}) \cup \text{ LvlB})$ ;
    Rest := (Rest  $\setminus$  Cur)
  END;
  Res := Edge;
  WorkT := Edge;
  WHILE (WorkT  $\neq \emptyset$ ) DO
    Inter :=  $((\pi[0, 3] (\sigma[\#1 = \#2] (\text{ WorkT } \times \text{ Edge}))) \setminus \text{ Res})$ ;
    Res := (Res  $\cup$  Inter);
    WorkT := Inter
  END

POST
  (TcB = Res)

```

78 Example5032: Delta-driven incremental maintenance of a transitive closure under insertions versus PostgreSQL recomputation on the updated edges

Category	production pair (two algorithms from production sources or legacy production cases); family: production algorithm pair (dependent phases, rate mismatch).
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	database-engine evaluation-strategies prophecy semi-naive view-maintenance.
Input	Benchmark/Example5032/Input.lean (canonical notation).
Summary	Delta-driven incremental maintenance of a transitive closure under insertions against PostgreSQL’s recomputation on the updated edge set.
Sides	naive compiler: Told = closure of EBase (phase 1) hand-written: delta-driven maintainer (seed $\text{E}\Delta \cup \text{Told} \circ \text{E}\Delta$ minus Told, propagate $\Delta \circ (\text{EBase} \cup \text{E}\Delta)$ minus T until empty), the set-level delta rule of DBSP/Feldera and Materialize for an insert-only update hand-written: PostgreSQL ExecRecursiveUnion rounds (legacy Example2037) on $\text{EBase} \cup \text{E}\Delta$
Sources	Materialize MaterializeInc/materialize doc/user/content/sql/select/recursive-ctes.md WITH MUTUALLY RECURSIVE semantics (initialise empty, update bindings in sequence, stop when

no binding changes; 'work resulting only from the input changes for this iteration ... feeds back the resulting output changes') (branch main; commit 4e1db20efcc4 (2026-09-15); read 2026-09-15)

DBSP (sections 5.1 (recursion circuit, D yields the new facts of each iteration, semi-naive form 5.1), 6 (incremental recursive programs), 6.1 (transitive closure example $R(x,y) :- E(x,y); R(x,y) :- E(x,z), R(z,y)$), 6.2 (complexity); read 2026-09-15)

Feldera feldera/feldera crates/dbsp/src/operator/recursive.rs;
docs.feldera.com/docs/sql/recursion.mdx Circuit::recursive (doc: $\Delta x = y - x$ with $y = f(i+\Delta i, y)$; implicit distinct); SQL recursion semantics (least fixed point, DISTINCT, deletions recomputed incrementally) (branch main; commit 16c3f5324fa0 (2026-09-15); read 2026-09-15)

PostgreSQL postgres/postgres src/backend/executor/nodeRecursiveunion.c ExecRecursiveUnion (header comment steps 2.1-2.6; 'Ignore tuple if already seen'; working/intermediate table swap) (branch master; commit 6e58d6356fcb (2026-09-15); read 2026-09-15; encoded as legacy Example2037 / Example5001 hand side with the base term equal to the edge relation)

Prophecy accepted invariant with a prophecy clause above level zero (Core.json); no certificate built from it yet.

Datalog program(s). *prog_TOld* — The Datalog program(s) of the header: compiled sides are their naive compilation; reference programs are compared with the hand-written sides on the check instances. — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
TOld(x1, y1) :- EBase(x1, y1).
TOld(x1, y1) :- TOld(x1, z1), EBase(z1, y1).
```

prog_TNew — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
TNew(x1, y1) :- EBase(x1, y1).
TNew(x1, y1) :- EDelta(x1, y1).
TNew(x1, y1) :- TNew(x1, z1), EBase(z1, y1).
TNew(x1, y1) :- TNew(x1, z1), EDelta(z1, y1).
```

rules as stated in the header comment of Input.lean — header comment.

```
T(x, y) :- E(x, y)
T(x, y) :- T(x, z), E(z, y)
TOld(x, y) :- EBase(x, y)
TOld(x, y) :- TOld(x, z), EBase(z, y)
```

Reading. The case has three phases over an edge set that is split into a base part EBase and an inserted delta EDelta. Phase 1 is the naive fixpoint computation of TOld, the transitive closure of EBase alone. Phase 2 is a hand-written delta-driven maintainer modelled on DBSP/Feldera/Materialize-style incremental view maintenance: it seeds Delta with the consequences of the inserted edges against the already-known old closure (EDelta together with TOld composed with EDelta, minus what TOld already has) and then repeatedly extends only that delta by joining it with the whole new edge set $EBase \cup EDelta$, folding the result into T, until the delta is empty — the old view TOld is never recomputed. Phase 3 recomputes the closure Res of $EBase \cup EDelta$ from scratch via PostgreSQL's recursive-union rounds. The Hoare triple's claim $T = Res$ is the correctness statement for incremental view maintenance under an insertion: propagating only the change from the old fixpoint yields the same relation as recomputing the closure of the updated edge set from scratch.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in *Input.lean* remains the authority.

PRE: $\top$

The precondition is simply true; no constraints are placed on EBase, EDelta, or their relationship.

POST: $\forall x y. (T(x,y) \leftrightarrow \text{Res}(x,y))$

T, the closure obtained by seeding from the old closure TOld and propagating only the delta caused by the inserted edges EDelta, equals Res, the closure of EBase $\cup$ EDelta recomputed from scratch by PostgreSQL's recursive-union rounds.

Whiel encoding (Hoare triple).

```

SCHEMA
  EBase/2, EDelta/2, TOld/2, TOld_aux/2, Delta/2, T/2, Res/2, WorkT/2, Inter/2

PRE
  true

CMD
  TOld_aux :=  $\emptyset[2]$ ;
  TOld := (EBase  $\cup$   $\pi[0,3] (\sigma[\#1 = \#2] ((TOld\_aux \times EBase))))$ ;
  WHILE ( $\neg((TOld = TOld\_aux))$ ) DO
    TOld_aux := TOld;
    TOld := (TOld  $\cup$  (EBase  $\cup$   $\pi[0,3] (\sigma[\#1 = \#2] ((TOld\_aux \times EBase))))$ );
  END;
  Delta := ((EDelta  $\cup$  ( $\pi[0, 3] (\sigma[\#1 = \#2] (TOld \times EDelta)))) \setminus TOld$ );
  T := (TOld  $\cup$  Delta);
  WHILE (Delta  $\neq \emptyset$ ) DO
    Delta := (( $\pi[0, 3] (\sigma[\#1 = \#2] (Delta \times (EBase \cup EDelta)))) \setminus T$ );
    T := (T  $\cup$  Delta)
  END;
  Res := (EBase  $\cup$  EDelta);
  WorkT := (EBase  $\cup$  EDelta);
  WHILE (WorkT  $\neq \emptyset$ ) DO
    Inter := (( $\pi[0, 3] (\sigma[\#1 = \#2] (WorkT \times (EBase \cup EDelta)))) \setminus Res$ );
    Res := (Res  $\cup$  Inter);
    WorkT := Inter
  END

POST
  (T = Res)

```

79 Example5033: Soufflé magic-set demand-driven evaluation versus PostgreSQL bulk evaluation restricted to the demanded sources

Category	production pair (two algorithms from production sources or legacy production cases); family: production algorithm pair (rate mismatch).
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	database-engine evaluation-strategies magic-sets prophecy semi-naive.
Input	Benchmark/Example5033/Input.lean (canonical notation).
Summary	Soufflé's magic-set (demand-driven) evaluation of a right-linear path query with a bound source against PostgreSQL's bulk evaluation of the original query restricted to the demanded sources.
Sides	naive compiler: Soufflé MagicSetCoreTransformer output for Path with adornment bf (the program of Example2003/2004) hand-written: PostgreSQL ExecRecursiveUnion rounds (legacy Example2037) for the original right-linear query, joined with the demand set in the postcondition
Sources	Soufflé souffle-lang/souffle src/ast/transform/MagicSet.cpp MagicSetCoreTransformer::transform, createMagicClause, createMagicAtom, getMagicName (branch master; commit a1303be3c016 (2026-07-13); read 2026-09-15) Soufflé documentation souffle-lang/souffle-lang.github.io pages/docs/magicset.md The Algorithm: constraint normalisation, adornment (SIPS), the magic set transformation ('For all adorned

predicates ... Add the clause `magic(Ai_ai) :- magic(A_a), A1_a1, ..., A(i-1)_a(i-1)` ... Replace `C` with `A_a :- magic(A_a), T'` (read 2026-09-15)

PostgreSQL postgres/postgres src/backend/executor/nodeRecursiveunion.c ExecRecursiveUnion (header comment steps 2.1-2.6; 'Ignore tuple if already seen'; working/intermediate table swap) (branch master; commit 6e58d6356fcb (2026-09-15); read 2026-09-15; encoded as legacy Example2037 / Example5001 hand side with the base term equal to the edge relation)

Prophecy accepted invariant with a prophecy clause above level zero (Core.json); no certificate built from it yet.

Datalog program(s). *prog_magic* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
MPath(x1) :- Root(x1).
MPath(y1) :- MPath(x1), Edge(x1, y1).
PathBF(x1, y1) :- MPath(x1), Edge(x1, y1).
PathBF(x1, z1) :- MPath(x1), Edge(x1, y1), PathBF(y1, z1).
```

prog_PathR — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
PathR(x1, y1) :- Edge(x1, y1).
PathR(x1, z1) :- Edge(x1, y1), PathR(y1, z1).
```

rules as stated in the header comment of Input.lean — header comment.

```
Path(x, y) :- Edge(x, y)
Path(x, z) :- Edge(x, y), Path(y, z)
A_a :- A1_a1, ..., An_an
MPath(x) :- Root(x)
MPath(y) :- MPath(x), Edge(x, y)
PathBF(x, y) :- MPath(x), Edge(x, y)
PathBF(x, z) :- MPath(x), Edge(x, y), PathBF(y, z)
```

Reading. The underlying query is the right-linear path program `Path(x,y) :- Edge(x,y); Path(x,z) :- Edge(x,y), Path(y,z)`, asked with the first argument bound to a single constant. Side 1 encodes Soufflé's magic-set transformation for this adornment (bf): it introduces a demand relation `MPath` seeded from the bound source `Root` and propagated forward along edges, and an answer relation `PathBF` built only over pairs whose source is currently demanded — the demand-driven, top-down-restricted evaluation Soufflé's `-magic-transform` would produce. Side 2 encodes PostgreSQL's bulk recursive-union evaluation of the unrestricted query, producing the full path closure `Res` for every source, independent of any demand. The Hoare triple claims that the demand-driven answers `PathBF` equal the bulk answers `Res` restricted, by a join with `MPath`, to the pairs whose source is in the demand set — the standard correctness obligation of the magic-set transformation, stated directly against PostgreSQL's actual round-based executor rather than against an idealized reference program.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\forall x. (\text{Root}(x) \leftrightarrow x = \text{"root"})$

`Root` is fixed to the singleton set containing exactly the constant `"root"`, the single bound source of the query.

POST: $\forall x y. (\text{PathBF}(x,y) \leftrightarrow \text{MPath}(x) \wedge \text{Res}(x,y))$

`PathBF`, the magic-set/demand-driven answer relation, equals the projection of `MPath` $\times$ `Res` onto pairs whose source column matches a demanded node — i.e. the demand-driven answers equal the bulk-evaluated answers restricted to the demanded sources.

Whiel encoding (Hoare triple).

```

SCHEMA
  Root/1, MPath/1, MPath_aux/1, Edge/2, PathBF/2, PathBF_aux/2, Res/2, WorkT/2, Inter/2

PRE
  (Root = { "root" })

CMD
  MPath_aux := ∅[1];
  PathBF_aux := ∅[2];
  MPath := (Root ∪ π[2] (σ[#0 = #1] ((MPath_aux × Edge)))));
  PathBF := (π[0,2] (σ[#0 = #1] ((MPath_aux × Edge))) ∪ π[0,4] (σ[#0 = #1 ∧ #2 = #3] ((MPath_aux × (Edge × PathBF_aux)))));
  WHILE (¬((MPath = MPath_aux) ∧ (PathBF = PathBF_aux))) DO
    MPath_aux := MPath;
    PathBF_aux := PathBF;
    MPath := (MPath ∪ (Root ∪ π[2] (σ[#0 = #1] ((MPath_aux × Edge)))));
    PathBF := (PathBF ∪ (π[0,2] (σ[#0 = #1] ((MPath_aux × Edge))) ∪ π[0,4] (σ[#0 = #1 ∧ #2 = #3] ((MPath_aux × (Edge ×
      ↪ PathBF_aux))))));
  END;
  Res := Edge;
  WorkT := Edge;
  WHILE (WorkT ≠ ∅) DO
    Inter := ((π[0, 3] (σ[#1 = #2] (Edge × WorkT))) \ Res);
    Res := (Res ∪ Inter);
    WorkT := Inter
  END

POST
  (PathBF = (π[1, 2] (σ[#0 = #1] (MPath × Res))))

```

80 Example5034: Warshall's intermediate-vertex sweep (boolean Floyd-Warshall) versus PostgreSQL recursive-union rounds

Category	production pair (two algorithms from production sources or legacy production cases); family: production algorithm pair (rate mismatch).
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	database-engine evaluation-strategies prophecy semi-naive.
Input	Benchmark/Example5034/Input.lean (canonical notation).
Summary	Warshall's intermediate-vertex sweep, the boolean core of Floyd-Warshall, against PostgreSQL's recursive-union rounds.
Sides	hand-written: boolean Floyd-Warshall sweep over the vertices in Lt order (SciPy <code>_floyd_warshall</code> / NetworkX <code>floyd_warshall_predecessor_and_distance</code> , weights not modelled) hand-written: PostgreSQL <code>ExecRecursiveUnion</code> rounds (legacy Example2037) seeded with Edge
Sources	SciPy <code>scipy/scipy/scipy/sparse/csgraph/_shortest_path.pyx</code> <code>floyd_warshall</code> (wrapper, unweighted option sets finite entries to 1), <code>_floyd_warshall</code> (for <code>k</code> in <code>range(N)</code> : for <code>i</code> in <code>range(N)</code> : if <code>dist_matrix[i, k] == INFINITY</code> : continue; for <code>j</code> in <code>range(N)</code> : <code>d_ijk = dist_matrix[i, k] + dist_matrix[k, j]</code> ; if <code>d_ijk < dist_matrix[i, j]</code> : <code>dist_matrix[i, j] = d_ijk</code> ; comment 'In each loop, this finds the shortest path from point <code>i</code> to point <code>j</code> using intermediate nodes <code>0 ... k</code> ') (branch main; commit 884ac0b6fc53 (2026-09-15); read 2026-09-15) NetworkX <code>networkx/networkx/networkx/algorithms/shortest_paths/dense.py</code> <code>floyd_warshall_predecessor_and_distance</code> (for <code>w</code> in <code>G</code> : for <code>u</code> in <code>G</code> : for <code>v</code> in <code>G</code> : <code>d = dist_u[w] + dist_w[v]</code> ; if <code>dist_u[v] > d</code> : ...), <code>floyd_warshall</code> , <code>floyd_warshall_numpy</code> (for <code>i</code> in <code>range(n)</code> : <code>A = np.minimum(A, A[i, :][np.newaxis, :] + A[:, i][:, np.newaxis])</code>) (branch main; commit 4e74880b0da0 (2026-09-12); read 2026-09-15) PostgreSQL <code>postgres/postgres/src/backend/executor/nodeRecursiveunion.c</code> <code>ExecRecursiveUnion</code> (header comment steps 2.1-2.6; 'Ignore tuple if already seen'; working/intermediate table swap)

(branch master; commit 6e58d6356fcb (2026-09-15); read 2026-09-15; encoded as legacy Example2037 / Example5001 hand side with the base term equal to the edge relation)

Twin Example5035 (invalid: only the first two vertices of the order are swept).

Prophecy accepted invariant with a prophecy clause above level zero (Core.json); no certificate built from it yet.

Datalog program(s). *prog_T* — The Datalog program(s) of the header: compiled sides are their naive compilation; reference programs are compared with the hand-written sides on the check instances. — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
T(x1, y1) :- Edge(x1, y1).
T(x1, y1) :- T(x1, z1), Edge(z1, y1).
```

Reading. The Datalog program is again transitive closure, $T(x,y) :- \text{Edge}(x,y); T(x,y) :- T(x,z), \text{Edge}(z,y)$. Side 1 encodes the boolean core of Floyd-Warshall (SciPy's `_floyd_warshall` / NetworkX's `floyd_warshall_predecessor_and_distance` with weights ignored, i.e. Warshall's algorithm): for each intermediate vertex taken in a fixed order, it composes all pairs currently entering that vertex with all pairs currently leaving it and folds the result into TW, one vertex-sweep at a time. Side 2 is PostgreSQL's recursive-union rounds on Edge, as in the other cases in this batch. The Hoare triple claims $TW = \text{Res}$: Warshall's intermediate-vertex sweep and PostgreSQL's round-based recursive union compute the same transitive closure, for every strict total order used to sequence the sweep.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $(\neg \exists x. \text{Lt}(x,x)) \wedge (\forall y z. ((\exists u. \text{Lt}(y,u) \wedge \text{Lt}(u,z)) \rightarrow \text{Lt}(y,z))) \wedge (\forall v w. (\text{Lt}(v,w) \rightarrow ((\exists x1. \text{Edge}(v,x1)) \vee \exists x2. \text{Edge}(x2,v)) \wedge ((\exists x3. \text{Edge}(w,x3)) \vee \exists x4. \text{Edge}(x4,w)))) \wedge \forall x5 x6. (((\exists x7. \text{Edge}(x5,x7)) \vee \exists x8. \text{Edge}(x8,x5)) \wedge ((\exists x9. \text{Edge}(x6,x9)) \vee \exists x10. \text{Edge}(x10,x6))) \wedge \neg x5 = x6 \rightarrow \text{Lt}(x5,x6) \vee \text{Lt}(x6,x5))$

Lt is a strict total order over the nodes that occur in Edge.

POST: $\forall x y. (TW(x,y) \leftrightarrow \text{Res}(x,y))$

TW, the relation built by sweeping intermediate vertices in Lt order and composing incoming with outgoing pairs at each vertex, equals Res, built by PostgreSQL's recursive-union rounds.

Whiel encoding (Hoare triple).

```
SCHEMA
  Rest/1, Cur/1, Edge/2, Lt/2, TW/2, InK/2, OutK/2, Res/2, WorkT/2, Inter/2

PRE
  (((((σ[#0 = #1] Lt) = ∅) ∧
    ((π[0, 3] (σ[#1 = #2] (Lt × Lt))) ⊆ Lt)) ∧
    ((Lt ⊆ ((π[0] Edge ∪ π[1] Edge) × (π[0] Edge ∪ π[1] Edge))) ∧
    ((σ[¬(#0 = #1)] ((π[0] Edge ∪ π[1] Edge) × (π[0] Edge ∪ π[1] Edge))) ⊆ (Lt ∪ π[1, 0] Lt))))))

CMD
  TW := Edge;
  Rest := (π[0] Edge ∪ π[1] Edge);
  WHILE (Rest ≠ ∅) DO
    Cur := (Rest \ (π[1] (σ[#0 = #2] (Lt × Rest))));
    InK := (π[0, 1] (σ[#1 = #2] (TW × Cur)));
    OutK := (π[1, 2] (σ[#0 = #1] (Cur × TW)));
    TW := (TW ∪ (π[0, 3] (σ[#1 = #2] (InK × OutK))));
    Rest := (Rest \ Cur)
  END;
  Res := Edge;
  WorkT := Edge;
  WHILE (WorkT ≠ ∅) DO
    Inter := ((π[0, 3] (σ[#1 = #2] (WorkT × Edge))) \ Res);
    Res := (Res ∪ Inter);
    WorkT := Inter
  END
```

POST
(TW = Res)

81 Example5035: Warshall's sweep truncated to the first two vertices versus PostgreSQL recursive-union rounds (invalid twin of Example5034)

Category	production pair (two algorithms from production sources or legacy production cases); family: production algorithm pair (rate mismatch), invalid twin.
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified invalid (Certificate/Invalid.lean, written by the certificate emitter); expected invalid.
Tags	database-engine evaluation-strategies recursion-limit semi-naive.
Input	Benchmark/Example5035/Input.lean (canonical notation).
Summary	The invalid twin of Example5034: Warshall's sweep stopped after the first two vertices of the order, against PostgreSQL's recursive-union rounds.
Sides	hand-written: boolean Floyd-Warshall sweeps of the first two vertices of the order only (loop-free) hand-written: PostgreSQL ExecRecursiveUnion rounds (legacy Example2037) seeded with Edge
Sources	SciPy scipy/scipy scipy/sparse/csgraph/_shortest_path.pyx floyd_warshall (wrapper, unweighted option sets finite entries to 1), _floyd_warshall (for k in range(N): for i in range(N): if dist_matrix[i, k] == INFINITY: continue; for j in range(N): d_ijk = dist_matrix[i, k] + dist_matrix[k, j]; if d_ijk < dist_matrix[i, j]: dist_matrix[i, j] = d_ijk; comment 'In each loop, this finds the shortest path from point i to point j using intermediate nodes 0 ... k') (branch main; commit 884ac0b6fc53 (2026-09-15); read 2026-09-15) NetworkX networkx/networkx networkx/algorithms/shortest_paths/dense.py floyd_warshall_predecessor_and_distance (for w in G: for u in G: for v in G: d = dist_u[w] + dist_w[v]; if dist_u[v] > d: ...), floyd_warshall, floyd_warshall_numpy (for i in range(n): A = np.minimum(A, A[i, :][np.newaxis, :] + A[:, i][, np.newaxis])) (branch main; commit 4e74880b0da0 (2026-09-12); read 2026-09-15) PostgreSQL postgres/postgres src/backend/executor/nodeRecursiveunion.c ExecRecursiveUnion (header comment steps 2.1-2.6; 'Ignore tuple if already seen'; working/intermediate table swap) (branch master; commit 6e58d6356fcb (2026-09-15); read 2026-09-15; encoded as legacy Example2037 / Example5001 hand side with the base term equal to the edge relation)
Twin	Example5034 (valid: all vertices swept).

Obstruction note

from the metadata: invalid (bounded number of sweeps); the bounded side is loop-free and enters the precondition, the verified loop is the executor's

Datalog program(s). *prog_T* — The Datalog program(s) of the header: compiled sides are their naive compilation; reference programs are compared with the hand-written sides on the check instances. — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
T(x1, y1) :- Edge(x1, y1).
T(x1, y1) :- T(x1, z1), Edge(z1, y1).
```

Reading. This is the invalid twin of Example5034: side 2 is again PostgreSQL’s recursive-union closure, but side 1 truncates Warshall’s sweep to only the first two vertices of the order (a loop-free computation of TWa then TWb, each adding the pairs composed through one more intermediate vertex). The Hoare triple claims TWb = Res, i.e. that sweeping only two intermediate vertices already produces the full transitive closure. This is false in general: cutting a sweep short is not the same as bounding path length, since it misses every path whose intermediate vertices lie outside the swept prefix regardless of the path’s length, and the case gives an explicit witness graph (a 4-node path, and separately a 3-cycle) where the two relations differ, backed by a kernel-checked counterexample.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $(\neg \exists x. \text{Lt}(x,x)) \wedge (\forall y z. ((\exists u. \text{Lt}(y,u) \wedge \text{Lt}(u,z)) \rightarrow \text{Lt}(y,z))) \wedge (\forall v w. (\text{Lt}(v,w) \rightarrow ((\exists x1. \text{Edge}(v,x1)) \vee \exists x2. \text{Edge}(x2,v)) \wedge ((\exists x3. \text{Edge}(w,x3)) \vee \exists x4. \text{Edge}(x4,w)))))) \wedge \forall x5 x6. (((\exists x7. \text{Edge}(x5,x7)) \vee \exists x8. \text{Edge}(x8,x5)) \wedge ((\exists x9. \text{Edge}(x6,x9)) \vee \exists x10. \text{Edge}(x10,x6))) \wedge \neg x5 = x6 \rightarrow \text{Lt}(x5,x6) \vee \text{Lt}(x6,x5))$

Lt is a strict total order over the nodes of Edge, exactly as in Example5034.

POST: $\forall x y. (\text{TWb}(x,y) \leftrightarrow \text{Res}(x,y))$

It is claimed that TWb, the result of sweeping only the first two vertices of Lt as intermediate vertices, equals Res, the full transitive closure from PostgreSQL’s recursive union — a claim the case shows to be false.

Whiel encoding (Hoare triple).

```
SCHEMA
  KA/1, RestB/1, KB/1, Edge/2, Lt/2, InA/2, OutA/2, TWa/2, InB/2, OutB/2, TWb/2, Res/2, WorkT/2, Inter/2

PRE
  (((((σ[#0 = #1] Lt) = ∅) ∧
    ((π[0, 3] (σ[#1 = #2] (Lt × Lt))) ⊆ Lt)) ∧
    ((Lt ⊆ ((π[0] Edge ∪ π[1] Edge) × (π[0] Edge ∪ π[1] Edge))) ∧
      ((σ[¬(#0 = #1)] ((π[0] Edge ∪ π[1] Edge) × (π[0] Edge ∪ π[1] Edge))) ⊆ (Lt ∪ π[1, 0] Lt))))))

CMD
  KA := ((π[0] Edge ∪ π[1] Edge) \ (π[1] Lt));
  RestB := ((π[0] Edge ∪ π[1] Edge) \ KA);
  KB := (RestB \ (π[1] (σ[#0 = #2] (Lt × RestB))));
  InA := (π[0, 1] (σ[#1 = #2] (Edge × KA)));
  OutA := (π[1, 2] (σ[#0 = #1] (KA × Edge)));
  TWa := (Edge ∪ (π[0, 3] (σ[#1 = #2] (InA × OutA))));
  InB := (π[0, 1] (σ[#1 = #2] (TWa × KB)));
  OutB := (π[1, 2] (σ[#0 = #1] (KB × TWa)));
  TWb := (TWa ∪ (π[0, 3] (σ[#1 = #2] (InB × OutB))));
  Res := Edge;
  WorkT := Edge;
  WHILE (WorkT ≠ ∅) DO
    Inter := ((π[0, 3] (σ[#1 = #2] (WorkT × Edge))) \ Res);
    Res := (Res ∪ Inter);
    WorkT := Inter
  END

POST
  (TWb = Res)
```

82 Example5036: NetworkX DAG transitive closure by a reverse topological sweep versus PostgreSQL recursive-union rounds

Category	production pair (two algorithms from production sources or legacy production cases); family: production algorithm pair (rate mismatch).
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	database-engine evaluation-strategies prophecy semi-naive.
Input	Benchmark/Example5036/Input.lean (canonical notation).
Summary	NetworkX's transitive closure of a directed acyclic graph by a reverse topological sweep against PostgreSQL's recursive-union rounds.
Sides	hand-written: NetworkX transitive_closure_dag = for each node in reverse Lt order, add its descendants at distance 2 in the current TC (set-at-a-time: Row $\circ$ TC) hand-written: PostgreSQL ExecRecursiveUnion rounds (legacy Example2037) seeded with Edge
Sources	NetworkX networkx/networkx networkx/algorithms/dag.py transitive_closure_dag (TC = G.copy(); for v in reversed(topo_order): TC.add_edges_from((v, u) for u in nx.descendants_at_distance(TC, v, 2))) (branch main; commit 4e74880b0da0 (2026-09-12); read 2026-09-15) NetworkX networkx/networkx networkx/algorithms/traversal/breadth_first_search.py descendants_at_distance (layer 'distance' of bfs_layers(G, source)), bfs_layers (branch main; commit 4e74880b0da0 (2026-09-12); read 2026-09-15) PostgreSQL postgres/postgres src/backend/executor/nodeRecursiveunion.c ExecRecursiveUnion (header comment steps 2.1-2.6; 'Ignore tuple if already seen'; working/intermediate table swap) (branch master; commit 6e58d6356fcb (2026-09-15); read 2026-09-15; encoded as legacy Example2037 / Example5001 hand side with the base term equal to the edge relation)
Prophecy	accepted invariant with a prophecy clause above level zero (Core.json); no certificate built from it yet.

Datalog program(s). *prog_T* — The Datalog program(s) of the header: compiled sides are their naive compilation; reference programs are compared with the hand-written sides on the check instances. — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
T(x1, y1) :- Edge(x1, y1).
T(x1, y1) :- T(x1, z1), Edge(z1, y1).
```

Reading. The Datalog program is transitive closure, $T(x,y) :- \text{Edge}(x,y)$; $T(x,y) :- T(x,z), \text{Edge}(z,y)$, over a graph that the precondition forces to be a DAG (Lt is a topological order containing Edge). Side 1 encodes NetworkX's transitive_closure_dag: processing nodes in reverse topological order, it computes each node's full row of descendants in one step by unioning its current row with that row composed with the whole closure built so far, relying on the fact that every successor of the current node, coming earlier in the reverse sweep, already has a complete row. Side 2 is PostgreSQL's recursive-union rounds on Edge. The Hoare triple claims $\text{TC} = \text{Res}$: the reverse-topological-sweep algorithm, which is only correct because the order respects the DAG's edges, computes the same transitive closure as PostgreSQL's generic recursive union, which needs no such assumption.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in Input.lean remains the authority.

PRE: $(\neg \exists x. \text{Lt}(x,x)) \wedge (\forall y z. ((\exists u. \text{Lt}(y,u) \wedge \text{Lt}(u,z)) \rightarrow \text{Lt}(y,z))) \wedge (\forall v w. (\text{Lt}(v,w) \rightarrow ((\exists x1. \text{Edge}(v,x1)) \vee \exists x2. \text{Edge}(x2,v)) \wedge ((\exists x3. \text{Edge}(w,x3)) \vee \exists x4. \text{Edge}(x4,w)))) \wedge (\forall x5 x6. (((\exists x7. \text{Edge}(x5,x7)) \vee \exists x8. \text{Edge}(x8,x5)) \wedge$

$((\exists x9. \text{Edge}(x6, x9)) \vee \exists x10. \text{Edge}(x10, x6)) \wedge \neg x5 = x6 \rightarrow \text{Lt}(x5, x6) \vee \text{Lt}(x6, x5))) \wedge \forall x11 \ x12. (\text{Edge}(x11, x12) \rightarrow \text{Lt}(x11, x12))$

Lt is a strict total order over the nodes of Edge and additionally contains all of Edge, i.e. Lt is a topological order of the (necessarily acyclic) graph.

POST: $\forall x \ y. (\text{TC}(x, y) \leftrightarrow \text{Res}(x, y))$

TC, built by sweeping nodes in reverse topological (Lt) order and merging each node's row with the closure computed so far, equals Res, built by PostgreSQL's recursive-union rounds.

Whiel encoding (Hoare triple).

```

SCHEMA
  Rest/1, Cur/1, Edge/2, Lt/2, TC/2, Row/2, Res/2, WorkT/2, Inter/2

PRE
  (((((σ[#0 = #1] Lt) = ∅) ∧
    ((π[0, 3] (σ[#1 = #2] (Lt × Lt))) ⊆ Lt)) ∧
    ((Lt ⊆ ((π[0] Edge ∪ π[1] Edge) × (π[0] Edge ∪ π[1] Edge))) ∧
    ((σ[¬(#0 = #1)] ((π[0] Edge ∪ π[1] Edge) × (π[0] Edge ∪ π[1] Edge))) ⊆ (Lt ∪ π[1, 0] Lt)))))) ∧
    (Edge ⊆ Lt))

CMD
  TC := Edge;
  Rest := (π[0] Edge ∪ π[1] Edge);
  WHILE (Rest ≠ ∅) DO
    Cur := (Rest \ (π[0] (σ[#1 = #2] (Lt × Rest))));
    Row := (π[1, 2] (σ[#0 = #1] (Cur × TC)));
    TC := (TC ∪ (π[0, 3] (σ[#1 = #2] (Row × TC))));
    Rest := (Rest \ Cur)
  END;
  Res := Edge;
  WorkT := Edge;
  WHILE (WorkT ≠ ∅) DO
    Inter := ((π[0, 3] (σ[#1 = #2] (WorkT × Edge))) \ Res);
    Res := (Res ∪ Inter);
    WorkT := Inter
  END

POST
  (TC = Res)

```

83 Example5037: Go runtime frontier marking versus the materialised Soufflé closure of the pointer graph joined with the roots

Category	production pair (two algorithms from production sources or legacy production cases); family: production algorithm pair (dependent phases, schema link by view).
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	evaluation-strategies nonlinear-recursion prophecy semi-naive.
Input	Benchmark/Example5037/Input.lean (canonical notation).
Summary	Go's garbage-collector marking, a frontier traversal with a marked-bit check, against the materialised transitive closure of the pointer graph joined with the roots.
Sides	hand-written: Go gcDrain/scanObject/greyobject frontier marking (legacy Example2012 program) naive compiler: Soufflé tc (non-linear closure of HeapPtr), joined with RootPtr in the postcondition
Sources	Go runtime golang/go src/runtime/mgcmark.go gcDrain (root jobs via markroot, then the heap marking loop popping objects/spans and calling scanObject/scanSpan until no work), greyobject ('If marked we have nothing to do'; setMarked; noscan fast path; putObj) (branch master; commit 8f5d82065574 (2026-09-15); read 2026-09-15) Go runtime golang/go src/runtime/mgcmark_greenteagc.go; src/runtime/mgcmark_nogreenteagc.go scanObject (for each pointer slot: findObject then greyobject) (branch master; commit 8f5d82065574 (2026-09-15); read 2026-09-15)
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).

Datalog program(s). *prog_Tc* — *The Datalog program(s) of the header: compiled sides are their naive compilation; reference programs are compared with the hand-written sides on the check instances.* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
Tc(x1, y1) :- HeapPtr(x1, y1).
Tc(x1, z1) :- Tc(x1, y1), Tc(y1, z1).
```

prog_MarkedR — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
MarkedR(x1) :- RootPtr(x1).
MarkedR(y1) :- MarkedR(x1), HeapPtr(x1, y1).
```

rules as stated in the header comment of Input.lean — header comment.

```
Marked(x) :- RootPtr(x)
Marked(y) :- Marked(x), HeapPtr(x, y)
Tc(x, y) :- HeapPtr(x, y)
Tc(x, z) :- Tc(x, y), Tc(y, z)
```

Reading. Two distinct Datalog programs are involved: a unary marking program $\text{Marked}(x) :- \text{RootPtr}(x)$; $\text{Marked}(y) :- \text{Marked}(x), \text{HeapPtr}(x,y)$, and Soufflé's benchmark transitive-closure program $\text{Tc}(x,y) :- \text{HeapPtr}(x,y)$; $\text{Tc}(x,z) :- \text{Tc}(x,y), \text{Tc}(y,z)$ over the same pointer-graph edges HeapPtr . Side 1 encodes the Go runtime's garbage-collector marking (gcDrain/scanObject/greyobject): a worklist ('grey set') traversal that starts at the roots and, level by level, marks and enqueues every newly reached object, stopping when the queue is empty. Side 2 is the non-linear (doubling) naive fixpoint computation of Tc, the full pointer-graph reachability closure, independent of

any roots. The Hoare triple links the two different schemas by a view stated in the postcondition: it claims the GC's marked set equals the roots together with everything the materialized closure Tc says is reachable from some root — i.e. that frontier-by-frontier marking from the roots computes exactly the root-restricted transitive closure of the pointer graph.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in `Input.lean`. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\top$

The precondition is simply true; no constraints are placed on `RootPtr` or `HeapPtr`.

POST: $\forall x. (\text{Marked}(x) \leftrightarrow \text{RootPtr}(x) \vee \exists y. \text{RootPtr}(y) \wedge \text{Tc}(y,x))$

Marked, the set produced by the GC's root-seeded frontier marking traversal, equals `RootPtr` unioned with the objects reachable from some root according to the separately computed pointer-graph transitive closure Tc.

Whiel encoding (Hoare triple).

```
SCHEMA
  RootPtr/1, Marked/1, Grey/1, NewG/1, HeapPtr/2, Tc/2, Tc_aux/2

PRE
  true

CMD
  Marked := RootPtr;
  Grey := RootPtr;
  WHILE (Grey ≠ ∅) DO
    NewG := ((π[1] (σ[#0 = #2] (HeapPtr × Grey))) \ Marked);
    Marked := (Marked ∪ NewG);
    Grey := NewG
  END;
  Tc_aux := ∅[2];
  Tc := (HeapPtr ∪ π[0,3] (σ[#1 = #2] ((Tc_aux × Tc_aux))));
  WHILE (¬((Tc = Tc_aux))) DO
    Tc_aux := Tc;
    Tc := (Tc ∪ (HeapPtr ∪ π[0,3] (σ[#1 = #2] ((Tc_aux × Tc_aux)))))
  END

POST
  (Marked = (RootPtr ∪ (π[2] (σ[#0 = #1] (RootPtr × Tc)))))
```

84 Example5038: OpenFGA Check by top-down user set dispatch versus OpenFGA ListObjects by reverse expansion from the user

Category	production pair (two algorithms from production sources or legacy production cases); family: production algorithm pair (rate mismatch; two evaluation directions in one codebase; replaces legacy correctness case Example2032).
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	access-control evaluation-strategies magic-sets prophecy.
Input	Benchmark/Example5038/Input.lean (canonical notation).
Summary	OpenFGA's two evaluation directions for one question, membership of users in groups through nested user sets: Check, which descends from the object, and ListObjects, which ascends from the user by reverse expansion.
Sides	naive compiler: OpenFGA Check, default top-down user set dispatch (checkDirect / checkDirectUserSetTuples), as the demand-driven form of the membership program restricted to the queried users naive compiler: OpenFGA ListObjects, reverse expansion (readTuplesAndExecute / ReadStartingWithUser) as the membership closure seeded at the queried users
Sources	OpenFGA openfga/openfga internal/graph/check.go LocalChecker.checkDirect, checkDirectUserTuple, checkDirectUserSetTuples (default strategy defaultUserSet dispatches Check(user, h#member) for every user set tuple of the object), ResolveCheck (maxResolutionDepth, hasCycle) (branch main; commit 73591ef16ce5 (2026-09-08); read 2026-09-15 (the derivation of legacy Example2032, verified against the source)) OpenFGA openfga/openfga internal/graph/recursive_resolver.go recursiveFastPath, recursiveMatchUserUserSet, breadthFirstRecursiveMatch (the planner's alternative Check strategy for recursive user sets: user sets of the user loaded once, breadth-first from the object until a level meets them; not modelled) (branch main; commit 73591ef16ce5 (2026-09-08); read 2026-09-15) OpenFGA openfga/openfga pkg/server/commands/reverseexpand/reverse_expand.go ReverseExpandQuery.execute (edges from the type system, candidate emission by trySendCandidate with candidateObjectsMap, visitedUserSetsMap, resolveNodeLimit), reverseExpandDirect, readTuplesAndExecute (ReadStartingWithUser, each found object#relation becomes a new source) (branch main; commit 73591ef16ce5 (2026-09-08); read 2026-09-15) OpenFGA openfga/openfga pkg/server/commands/list_objects.go ListObjectsQuery.evaluate (delegates to ReverseExpand.Execute; candidates with NoFurtherEvalStatus are results, RequiresFurtherEvalStatus goes to Check) (branch main; commit 73591ef16ce5 (2026-09-08); read 2026-09-15)
Prophecy	accepted invariant with a prophecy clause above level zero (Core.json); no certificate built from it yet.
Note	Replaces the legacy correctness case Example2032, whose bottom-up loop is the reference program of the check; both sides are OpenFGA algorithms (Check by top-down dispatch, ListObjects by reverse expansion).

Datalog program(s). *prog_demand* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
Demand(x1) :- Query(x1).
Demand(y1) :- Demand(x1), SubGroup(y1, x1).
```

```

Ans(x1, y1) :- Demand(y1), QUser(x1), DirectMember(x1, y1).
Ans(x1, y1) :- Demand(y1), SubGroup(z1, y1), Ans(x1, z1).

```

prog_reach — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```

Reach(x1, y1) :- QUser(x1), DirectMember(x1, y1).
Reach(x1, z1) :- Reach(x1, y1), SubGroup(y1, z1).

```

prog_MemberR — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```

MemberR(x1, y1) :- DirectMember(x1, y1).
MemberR(x1, z1) :- MemberR(x1, y1), SubGroup(y1, z1).

```

rules as stated in the header comment of Input.lean — header comment.

```

Member(u, g) :- DirectMember(u, g)
Member(u, g) :- Member(u, h), SubGroup(h, g)
Demand(g) :- Query(g)
Demand(h) :- Demand(g), SubGroup(h, g)
Ans(u, g) :- Demand(g), QUser(u), DirectMember(u, g)
Ans(u, g) :- Demand(g), SubGroup(h, g), Ans(u, h)
Reach(u, g) :- QUser(u), DirectMember(u, g)
Reach(u, g) :- Reach(u, h), SubGroup(h, g)

```

Reading. The underlying query is group membership through nested usersets, $\text{Member}(u, g) :- \text{DirectMember}(u, g); \text{Member}(u, g) :- \text{Member}(u, h), \text{SubGroup}(h, g)$, the object-relationship-model membership relation used by systems like OpenFGA. Side 1 encodes OpenFGA’s Check API in its default top-down strategy (checkDirect/checkDirectUsersetTuples): it first pushes demand for a queried object down through subgroups (Demand), then builds membership answers Ans back up from the direct memberships of demanded groups. Side 2 encodes OpenFGA’s ListObjects API via reverse expansion (readTuplesAndExecute/ReadStartingWithUser): it grows a reachability relation Reach upward from the queried users’ direct group memberships through subgroups, with no notion of a queried object. The Hoare triple claims that, restricted to the queried users and queried objects, the two APIs’ answers agree — Check’s top-down, demand-restricted evaluation and ListObjects’ bottom-up reverse expansion compute the same membership relation on the questions actually asked.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in *Input.lean* remains the authority.

PRE: $\top$

The precondition is simply true; no constraints are placed on DirectMember, SubGroup, QUser, or Query.

POST: $\forall x y. (\text{Ans}(x, y) \wedge \text{Query}(y) \leftrightarrow \text{Reach}(x, y) \wedge \text{Query}(y))$

Restricted to pairs whose object is in Query, the answer relation Ans produced by OpenFGA’s top-down Check equals the answer relation Reach produced by OpenFGA’s reverse-expansion ListObjects.

Whiel encoding (Hoare triple).

```

SCHEMA
  Query/1, QUser/1, Demand/1, Demand_aux/1, DirectMember/2, SubGroup/2, Ans/2, Ans_aux/2, Reach/2, Reach_aux/2

PRE
  true

CMD
  Demand_aux :=  $\emptyset[1]$ ;
  Ans_aux :=  $\emptyset[2]$ ;

```

```

Demand := (Query  $\cup$   $\pi[1]$  ( $\sigma[\#0 = \#2]$  ((Demand_aux  $\times$  SubGroup))));
Ans := ( $\pi[1,0]$  ( $\sigma[\#1 = \#2 \wedge \#0 = \#3]$  ((Demand_aux  $\times$  (QUser  $\times$  DirectMember)))))  $\cup$   $\pi[3,0]$  ( $\sigma[\#0 = \#2 \wedge \#1 = \#4]$  ((Demand_aux
 $\hookrightarrow$   $\times$  (SubGroup  $\times$  Ans_aux)))));
WHILE ( $\neg$ ((Demand = Demand_aux)  $\wedge$  (Ans = Ans_aux))) DO
  Demand_aux := Demand;
  Ans_aux := Ans;
  Demand := (Demand  $\cup$   $\pi[1]$  ( $\sigma[\#0 = \#2]$  ((Demand_aux  $\times$  SubGroup))));
  Ans := (Ans  $\cup$  ( $\pi[1,0]$  ( $\sigma[\#1 = \#2 \wedge \#0 = \#3]$  ((Demand_aux  $\times$  (QUser  $\times$  DirectMember)))))  $\cup$   $\pi[3,0]$  ( $\sigma[\#0 = \#2 \wedge \#1 = \#4]$ 
 $\hookrightarrow$  ((Demand_aux  $\times$  (SubGroup  $\times$  Ans_aux)))));
END;
Reach_aux :=  $\emptyset[2]$ ;
Reach := ( $\pi[0,2]$  ( $\sigma[\#0 = \#1]$  ((QUser  $\times$  DirectMember)))  $\cup$   $\pi[0,3]$  ( $\sigma[\#1 = \#2]$  ((Reach_aux  $\times$  SubGroup))));
WHILE ( $\neg$ ((Reach = Reach_aux))) DO
  Reach_aux := Reach;
  Reach := (Reach  $\cup$  ( $\pi[0,2]$  ( $\sigma[\#0 = \#1]$  ((QUser  $\times$  DirectMember)))  $\cup$   $\pi[0,3]$  ( $\sigma[\#1 = \#2]$  ((Reach_aux  $\times$  SubGroup))));
END
POST
(( $\pi[0, 1]$  ( $\sigma[\#1 = \#2]$  (Ans  $\times$  Query))) = ( $\pi[0, 1]$  ( $\sigma[\#1 = \#2]$  (Reach  $\times$  Query))))

```

85 Example5039: Permission exclusion applied at every object (SpiceDB/OpenFGA) is contained in the exclusion applied once to the finished closure

Category	production pair (two algorithms from production sources or legacy production cases); family: engine semantics versus a rewriting (replaces legacy correctness case Example2036).
Kind	containment.
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	access-control negation.
Input	Benchmark/Example5039/Input.lean (canonical notation).
Summary	Where the exclusion of a permission is applied: at every object, as SpiceDB and OpenFGA do, or once to the finished closure.
Sides	hand-written: the permission closure with the exclusion applied at every object (legacy Example2036: SpiceDB (reader + parent->view) - banned, OpenFGA but-not) naive compiler: the unrestricted closure reader + parent->view, then the exclusion applied once to the finished closure (loop-free suffix)
Sources	SpiceDB authzed/spicedb (documentation) Schema language: permissions, union, arrow, exclusion permission view = (reader + parent->view) - banned: the exclusion applies to the permission computed at each object, and the arrow reads the parent's permission (read 2026-09-15) OpenFGA openfga/openfga internal/graph/check.go checkSetOperation with the exclusion handler (base $\wedge$ $\neg$ subtract, evaluated per object) and checkTTU (the arrow parent->view: the check is dispatched to the parent object) (branch main; read 2026-09-15) SpiceDB schema language: union, intersection, exclusion, arrows (documentation) openfga/openfga internal/graph/check.go (checkDirect, checkDirectUserSetTuples, checkTTU, checkSetOperation) OpenFGA usersets (documentation)
Twin	Example5040 (invalid: the two claimed equal).
Obstruction note	from the metadata: anchored: View $\subseteq$ ViewC and View $\cap$ Banned = $\emptyset$ hold round by round and the exclusion of side 2 is a loop-free suffix; no prophecy level expected
Note	Replaces the legacy correctness case Example2036 (its loop is side 1).

Datalog program(s). *prog_ViewC* — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```
ViewC(x1, y1) :- Reader(x1, y1).
ViewC(x1, y1) :- ParentRel(y1, z1), ViewC(x1, z1).
```

prog_ViewR — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```
ViewR(x1, y1) :- Reader(x1, y1), Allowed(x1, y1).
ViewR(x1, y1) :- ParentRel(y1, z1), ViewR(x1, z1), Allowed(x1, y1).
```

rules as stated in the header comment of Input.lean — header comment.

```
View(u, d) :- Reader(u, d), not Banned(u, d)
View(u, d) :- ParentRel(d, p), View(u, p), not Banned(u, d)
ViewC(u, d) :- Reader(u, d)
ViewC(u, d) :- ParentRel(d, p), ViewC(u, p)
```

Reading. The permission modelled is SpiceDB/OpenFGA-style exclusion over an inherited relation, permission view = (reader + parent->view) - banned: a user views a document if they read it directly or view its parent, except when banned on that document. Side 1 encodes how SpiceDB and OpenFGA’s checkSetOperation/checkTTU actually evaluate this, applying the exclusion at every object in the recursive closure, so a ban on a document also blocks the permission from propagating further down through its descendants. Side 2 computes the unrestricted inheritance closure ViewC by the naive compiler and then subtracts the banned pairs Banned only once, after the closure is complete, giving Filtered. The Hoare triple claims $\text{View} \subseteq \text{Filtered}$: applying the exclusion at every recursive step only ever removes pairs relative to the unrestricted closure and never produces a banned pair, so the per-step-filtered relation is necessarily contained in the once-filtered relation — though, as Example5040 shows, the two are not equal.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in *Input.lean* remains the authority.

PRE: $\top$

The precondition is simply true; no constraints are placed on Reader, Banned, or ParentRel.

POST: $\forall x y. (\text{View}(x,y) \rightarrow \text{Filtered}(x,y))$

View, computed with the ban exclusion applied at every object of the recursive permission definition, is contained in Filtered, computed as the unrestricted inheritance closure with the ban exclusion subtracted only once at the end.

Whiel encoding (Hoare triple).

```
SCHEMA
  Reader/2, Banned/2, ParentRel/2, View/2, View_aux/2, ViewC/2, ViewC_aux/2, Filtered/2

PRE
  true

CMD
  View_aux :=  $\emptyset$ [2];
  View := (Reader \ Banned);
  WHILE (View  $\neq$  View_aux) DO
    View_aux := View;
    View := ((Reader  $\cup$  ( $\pi$ [2, 0] ( $\sigma$ [#1 = #3] (ParentRel  $\times$  View_aux)))) \ Banned)
  END;
  ViewC_aux :=  $\emptyset$ [2];
  ViewC := (Reader  $\cup$   $\pi$ [2,0] ( $\sigma$ [#1 = #3] ((ParentRel  $\times$  ViewC_aux))));
  WHILE ( $\neg$ ((ViewC = ViewC_aux))) DO
    ViewC_aux := ViewC;
```

```

ViewC := (ViewC ∪ (Reader ∪ π[2,0] (σ[#1 = #3] ((ParentRel × ViewC_aux))))))
END;
Filtered := (ViewC \ Banned)

POST
  (View ⊆ Filtered)

```

86 Example5040: Permission exclusion applied at every object versus once to the finished closure, claimed equal (invalid twin of Example5039)

Category	production pair (two algorithms from production sources or legacy production cases); family: engine semantics versus a rewriting (invalid twin).
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified invalid (Certificate/Invalid.lean, written by the certificate emitter); expected invalid.
Tags	<code>access-control</code> <code>negation</code> .
Input	<code>Benchmark/Example5040/Input.lean</code> (canonical notation).
Summary	Where the exclusion of a permission is applied: at every object, as SpiceDB and OpenFGA do, or once to the finished closure.
Sides	hand-written: the permission closure with the exclusion applied at every object (legacy Example2036: SpiceDB (reader + parent->view) - banned, OpenFGA but-not) naive compiler: the unrestricted closure reader + parent->view, then the exclusion applied once to the finished closure (loop-free suffix)
Sources	SpiceDB authzed/spicedb (documentation) Schema language: permissions, union, arrow, exclusion permission view = (reader + parent->view) - banned: the exclusion applies to the permission computed at each object, and the arrow reads the parent's permission (read 2026-09-15) OpenFGA openfga/openfga internal/graph/check.go <code>checkSetOperation</code> with the exclusion handler (base $\wedge$ $\neg$subtract, evaluated per object) and <code>checkTTU</code> (the arrow parent->view: the check is dispatched to the parent object) (branch main; read 2026-09-15) SpiceDB schema language: union, intersection, exclusion, arrows (documentation) openfga/openfga internal/graph/check.go (<code>checkDirect</code>, <code>checkDirectUserSetTuples</code>, <code>checkTTU</code>, <code>checkSetOperation</code>) OpenFGA usersets (documentation)
Twin	Example5039 (valid: the containment).
Obstruction note	from the metadata: invalid (the exclusion inside the recursion blocks inheritance through a banned parent; filtering the closure once does not)

Datalog program(s). `prog_ViewC` — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```

ViewC(x1, y1) :- Reader(x1, y1).
ViewC(x1, y1) :- ParentRel(y1, z1), ViewC(x1, z1).

```

`prog_ViewR` — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
ViewR(x1, y1) :- Reader(x1, y1), Allowed(x1, y1).
ViewR(x1, y1) :- ParentRel(y1, z1), ViewR(x1, z1), Allowed(x1, y1).
```

rules as stated in the header comment of *Input.lean* — header comment.

```
View(u, d) :- Reader(u, d), not Banned(u, d)
View(u, d) :- ParentRel(d, p), View(u, p), not Banned(u, d)
ViewC(u, d) :- Reader(u, d)
ViewC(u, d) :- ParentRel(d, p), ViewC(u, p)
```

Reading. This is the invalid twin of Example5039, with identical inputs and identical two sides (the per-object-excluded View versus the once-filtered Filtered), but the Hoare triple here claims the stronger equality $\text{View} = \text{Filtered}$ instead of the containment. This is false: the case gives an explicit witness where a user reads a parent document but is banned on that parent, and a child document inherits from the parent. Applying the exclusion at every step blocks the permission from ever reaching the child through the banned parent, whereas filtering the unrestricted closure once at the end only removes the directly banned pair on the parent and leaves the inherited permission on the child intact, so the two relations differ — a kernel-checked counterexample confirms this, illustrating that pushing a negation out of a recursive definition is unsound.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in *Input.lean* remains the authority.

PRE: $\top$

The precondition is simply true; no constraints are placed on Reader, Banned, or ParentRel.

POST: $\forall x y. (\text{View}(x,y) \leftrightarrow \text{Filtered}(x,y))$

It is claimed that View (exclusion applied at every recursive step) equals Filtered (exclusion applied once, after the unrestricted closure is complete) — a claim the case shows to be false via a banned-parent witness.

Whiel encoding (Hoare triple).

```
SCHEMA
  Reader/2, Banned/2, ParentRel/2, View/2, View_aux/2, ViewC/2, ViewC_aux/2, Filtered/2

PRE
  true

CMD
  View_aux :=  $\emptyset[2]$ ;
  View := (Reader  $\setminus$  Banned);
  WHILE (View  $\neq$  View_aux) DO
    View_aux := View;
    View := ((Reader  $\cup$  ( $\pi[2, 0]$  ( $\sigma[\#1 = \#3]$  (ParentRel  $\times$  View_aux)))))  $\setminus$  Banned)
  END;
  ViewC_aux :=  $\emptyset[2]$ ;
  ViewC := (Reader  $\cup$   $\pi[2,0]$  ( $\sigma[\#1 = \#3]$  ((ParentRel  $\times$  ViewC_aux))));
  WHILE ( $\neg$ ((ViewC = ViewC_aux))) DO
    ViewC_aux := ViewC;
    ViewC := (ViewC  $\cup$  (Reader  $\cup$   $\pi[2,0]$  ( $\sigma[\#1 = \#3]$  ((ParentRel  $\times$  ViewC_aux)))))
  END;
  Filtered := (ViewC  $\setminus$  Banned)

POST
  (View = Filtered)
```

87 Example5041: PostgreSQL foreign-key cascade as a trigger worklist with the surviving-reference RESTRICT check versus the declarative delete closure

Category	production pair (two algorithms from production sources or legacy production cases); family: production algorithm pair (replaces legacy correctness case Example2024).
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified valid (Certificate/Valid.lean, written by the certificate emitter); expected valid.
Tags	database-engine evaluation-strategies integrity-constraints negation prophecy semi-naive.
Input	Benchmark/Example5041/Input.lean (canonical notation).
Summary	PostgreSQL's foreign-key cascade, as the trigger queue runs it, against the declarative delete closure, each followed by the RESTRICT check.
Sides	hand-written: PostgreSQL RI_FKey_cascade_del as a worklist of newly deleted rows, then ri_restrict as the surviving-reference check naive compiler: the delete closure, then the same check as loop-free code
Sources	PostgreSQL postgres/postgres src/backend/utils/adt/ri_triggers.c RI_FKey_cascade_del (DELETE FROM ONLY fktable WHERE \$1 = fkatt1, queued as an AFTER trigger per deleted row and re-fired transitively), ri_restrict via RI_FKey_restrict_del / RI_FKey_noaction_del (SELECT 1 FROM ONLY fktable x WHERE \$1 = fkatt1 FOR KEY SHARE OF x: a violation only if a referencing row still exists when the check runs) (branch master; read 2026-09-15) PostgreSQL documentation postgresql.org Chapter 5, Constraints, Foreign Keys (ON DELETE CASCADE, RESTRICT, NO ACTION) (read 2026-09-15) postgres/postgres src/backend/utils/adt/ri_triggers.c (RI_FKey_cascade_del, ri_restrict)
Twin	Example5042 (invalid: the over-approximating RESTRICT check of legacy Example2024).
Prophecy	kernel-checked certificate with a level-1 prophecy clause (Certificate/Valid.lean, Core.json).
Note	Replaces the legacy correctness case Example2024; its over-approximating RESTRICT check is the invalid twin Example5042.

Datalog program(s). *prog_DelC* — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
DelC(x1) :- SeedDel(x1).
DelC(x1) :- RefBy(x1, y1), DelC(y1).
```

prog_DelR — fidelity module (kernel-checked: the compiled sides of the command are this program's naive compilation).

```
DelR(x1) :- SeedDel(x1).
DelR(x1) :- RefBy(x1, y1), DelR(y1).
```

rules as stated in the header comment of Input.lean — header comment.

```
Del(k) :- SeedDel(k)
Del(r) :- RefBy(r, k), Del(k)
```

Reading. The Datalog program is a delete-reachability closure, $\text{Del}(k) :- \text{SeedDel}(k)$; $\text{Del}(r) :- \text{RefBy}(r,k), \text{Del}(k)$, modelling ON DELETE CASCADE foreign keys, together with a RESTRICT check for rows referenced through non-cascading (RESTRICT/NO ACTION) foreign keys. Side 1 encodes PostgreSQL’s actual trigger-based implementation (`ri_triggers.c`): `RI_FKey_cascade_del` deletes rows via a worklist that re-fires for every newly deleted row (`Del/Front/New`), and `ri_restrict` then reports a violation only for rows that still exist and reference a deleted key, i.e. rows not themselves cascaded away in the same statement. Side 2 computes the same delete closure `DelC` by the naive compiler and applies the identical surviving-reference RESTRICT check to it (`ViolC`). The Hoare triple claims $\text{Del} = \text{DelC}$ and $\text{Viol} = \text{ViolC}$: PostgreSQL’s row-level trigger worklist computes exactly the declarative delete closure, and, since both sides use the same surviving-reference check on equal relations, the reported RESTRICT violations agree too.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in `Input.lean`. The relational-algebra assertion in `Input.lean` remains the authority.

PRE: $\top$

The precondition is simply true; no constraints are placed on `SeedDel`, `RefBy`, or `RestrictRef`.

POST: $(\forall x. (\text{Del}(x) \leftrightarrow \text{DelC}(x))) \wedge \forall y. (\text{Viol}(y) \leftrightarrow \text{ViolC}(y))$

`Del`, the set of keys deleted by PostgreSQL’s cascading trigger worklist, equals `DelC`, the same set computed as a declarative closure, and `Viol`, the RESTRICT violations reported by the surviving-reference check on `Del`, equals `ViolC`, the same check applied to `DelC`.

Whiel encoding (Hoare triple).

```

SCHEMA
  SeedDel/1, Del/1, Front/1, New/1, Viol/1, DelC/1, DelC_aux/1, ViolC/1, RefBy/2, RestrictRef/2

PRE
  true

CMD
  Del := SeedDel;
  Front := SeedDel;
  WHILE (Front  $\neq$   $\emptyset$ ) DO
    New := (( $\pi$ [0] ( $\sigma$ [#1 = #2] (RefBy  $\times$  Front))) \ Del);
    Del := (Del  $\cup$  New);
    Front := New
  END;
  Viol := (( $\pi$ [0] ( $\sigma$ [#1 = #2] (RestrictRef  $\times$  Del))) \ Del);
  DelC_aux :=  $\emptyset$ [1];
  DelC := (SeedDel  $\cup$   $\pi$ [0] ( $\sigma$ [#1 = #2] ((RefBy  $\times$  DelC_aux))));
  WHILE ( $\neg$ ((DelC = DelC_aux))) DO
    DelC_aux := DelC;
    DelC := (DelC  $\cup$  (SeedDel  $\cup$   $\pi$ [0] ( $\sigma$ [#1 = #2] ((RefBy  $\times$  DelC_aux)))))
  END;
  ViolC := (( $\pi$ [0] ( $\sigma$ [#1 = #2] (RestrictRef  $\times$  DelC))) \ DelC)

POST
  ((Del = DelC)  $\wedge$  (Viol = ViolC))

```

88 Example5042: The over-approximating RESTRICt check of legacy Example2024 versus PostgreSQL’s surviving-reference check, claimed equal (invalid twin of Example5041)

Category	production pair (two algorithms from production sources or legacy production cases); family: production discrepancy pair (invalid twin).
Kind	equivalence.
Contributors	Leo Zhang.
Status	certified invalid (Certificate/Invalid.lean, written by the certificate emitter); expected invalid.
Tags	database-engine evaluation-strategies integrity-constraints negation semi-naive.
Input	Benchmark/Example5042/Input.lean (canonical notation).
Summary	PostgreSQL’s foreign-key cascade, as the trigger queue runs it, against the declarative delete closure, each followed by the RESTRICt check.
Sides	hand-written: the cascade worklist of Example5041 followed by the check of legacy Example2024, every row referencing a deleted key under RESTRICt (no \ Del) naive compiler: the delete closure, then PostgreSQL’s surviving-reference check
Sources	PostgreSQL postgres/postgres src/backend/utils/adt/ri_triggers.c RI_FKey_cascade_del (DELETE FROM ONLY fktable WHERE \$1 = fkatt1, queued as an AFTER trigger per deleted row and re-fired transitively), ri_restrict via RI_FKey_restrict_del / RI_FKey_noaction_del (SELECT 1 FROM ONLY fktable x WHERE \$1 = fkatt1 FOR KEY SHARE OF x: a violation only if a referencing row still exists when the check runs) (branch master; read 2026-09-15) PostgreSQL documentation postgresql.org Chapter 5, Constraints, Foreign Keys (ON DELETE CASCADE, RESTRICt, NO ACTION) (read 2026-09-15) postgres/postgres src/backend/utils/adt/ri_triggers.c (RI_FKey_cascade_del, ri_restrict)
Twin	Example5041 (valid: both sides use the surviving-reference check).
Obstruction note	from the metadata: invalid (the legacy check counts a row that the same statement cascades away; PostgreSQL’s ri_restrict does not)

Datalog program(s). *prog_DelC* — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```
DelC(x1) :- SeedDel(x1).
DelC(x1) :- RefBy(x1, y1), DelC(y1).
```

prog_DelR — fidelity module (kernel-checked: the compiled sides of the command are this program’s naive compilation).

```
DelR(x1) :- SeedDel(x1).
DelR(x1) :- RefBy(x1, y1), DelR(y1).
```

rules as stated in the header comment of Input.lean — header comment.

```
Del(k) :- SeedDel(k)
Del(r) :- RefBy(r, k), Del(k)
```

Reading. This is the invalid twin of Example5041: both sides compute the same cascade-delete closure (Del via the trigger worklist, DelC via the declarative closure), but side 1’s RESTRICT check is changed to the over-approximating check inherited from the retired legacy Example2024, Viol := the rows referencing any deleted key, without excluding the rows that the same delete statement already cascaded away, whereas side 2 keeps PostgreSQL’s faithful surviving-reference check (which subtracts Del). The Hoare triple still claims Del = DelC and Viol = ViolC, but this is now false for the Viol conjunct: the case gives an explicit witness where a row is deleted under CASCADE via one key and simultaneously referenced under RESTRICT via another deleted key, so PostgreSQL’s real check (which only flags rows still present when the check runs) reports no violation, while the legacy over-approximating check flags the already-deleted row as a violation, refuted by a kernel-checked counterexample.

First-order reading of PRE/POST, translated mechanically from the relational-algebra assertion in Input.lean. The relational-algebra assertion in Input.lean remains the authority.

PRE: $\top$

The precondition is simply true; no constraints are placed on SeedDel, RefBy, or RestrictRef.

POST: $(\forall x. (\text{Del}(x) \leftrightarrow \text{DelC}(x))) \wedge \forall y. (\text{Viol}(y) \leftrightarrow \text{ViolC}(y))$

It is claimed that Del equals DelC (true, as in Example5041) and that Viol, computed here without excluding rows already cascaded away, equals ViolC, computed with PostgreSQL’s surviving-reference exclusion — the Viol equality is false, refuted by a cascade-and-restrict witness.

Whiel encoding (Hoare triple).

```

SCHEMA
  SeedDel/1, Del/1, Front/1, New/1, Viol/1, DelC/1, DelC_aux/1, ViolC/1, RefBy/2, RestrictRef/2

PRE
  true

CMD
  Del := SeedDel;
  Front := SeedDel;
  WHILE (Front  $\neq$   $\emptyset$ ) DO
    New := (( $\pi$ [0] ( $\sigma$ [#1 = #2] (RefBy  $\times$  Front))) \ Del);
    Del := (Del  $\cup$  New);
    Front := New
  END;
  Viol := ( $\pi$ [0] ( $\sigma$ [#1 = #2] (RestrictRef  $\times$  Del)));
  DelC_aux :=  $\emptyset$ [1];
  DelC := (SeedDel  $\cup$   $\pi$ [0] ( $\sigma$ [#1 = #2] ((RefBy  $\times$  DelC_aux))));
  WHILE ( $\neg$ ((DelC = DelC_aux))) DO
    DelC_aux := DelC;
    DelC := (DelC  $\cup$  (SeedDel  $\cup$   $\pi$ [0] ( $\sigma$ [#1 = #2] ((RefBy  $\times$  DelC_aux)))))
  END;
  ViolC := (( $\pi$ [0] ( $\sigma$ [#1 = #2] (RestrictRef  $\times$  DelC))) \ DelC)

POST
  ((Del = DelC)  $\wedge$  (Viol = ViolC))

```